

Kurs C++

Materiały dla studentów

Paweł Kisielewicz pkisiel@gmail.com

Kraków, 20 czerwca 2026

Spis treści

1	Wstęp	28
1.1	Jak czytać	28
1.2	Wymagania techniczne	28
1.3	Uwaga o wersji	28
2	00 - Start	29
2.1	Cele segmentu	29
2.2	Kolejność lekcji	29
2.3	Status porządkowania	29
2.4	Efekt końcowy	29
2.5	Checklist dla studenta	30
3	Kompilator C++	31
3.1	Cel lekcji	31
3.2	Wymagania wstępne	31
3.3	Krótką teoria	31
3.4	Sprawdzenie kompilatora	31
3.5	Przykład praktyczny	32
3.6	Znaczenie flag kompilacji	32
3.7	Typowe problemy	32
3.7.1	Brak kompilatora	32
3.7.2	Błąd składni	33
3.7.3	Niepoprawna nazwa pliku	33
3.8	Zadania do wykonania	33
3.9	Checklist dla studenta	33
3.10	Kryteria zaliczenia	33
3.11	Źródła	33
4	Visual Studio Code dla C++	34
4.1	Cel lekcji	34
4.2	Wymagania wstępne	34
4.3	Krótką teoria	34
4.4	Zalecane rozszerzenia	34
4.5	Przykład praktyczny	35
4.6	Typowe problemy	35
4.6.1	Terminal otwiera się w złym katalogu	35
4.6.2	VS Code nie widzi kompilatora	35
4.6.3	Podkreślenia w edytorze nie zawsze są błędem kompilacji	35
4.7	Zadania do wykonania	35
4.8	Checklist dla studenta	36
4.9	Kryteria zaliczenia	36
4.10	Źródła	36
5	CMake - pierwsze wprowadzenie	37
5.1	Cel lekcji	37
5.2	Wymagania wstępne	37

5.3	Krótką teoria	37
5.4	Minimalny projekt	37
5.5	Budowanie projektu	38
5.6	Znaczenie poleceń	38
5.7	Typowe problemy	38
5.7.1	CMake nie jest kompilatorem	38
5.7.2	Literówka w nazwie pliku	38
5.7.3	Mieszanie plików źródłowych i build	39
5.8	Zadania do wykonania	39
5.9	Checklist dla studenta	39
5.10	Kryteria zaliczenia	39
6	Git - podstawy pracy z repozytorium	40
6.1	Cel lekcji	40
6.2	Wymagania wstępne	40
6.3	Krótką teoria	40
6.4	Podstawowe polecenia	40
6.5	Praca z branchami	41
6.6	Typowe problemy	41
6.6.1	Plik nie trafił do commita	41
6.6.2	Przypadkowo dodany plik	41
6.6.3	Zmiana adresu zdalnego repozytorium	41
6.7	Zadania do wykonania	41
6.8	Checklist dla studenta	42
6.9	Kryteria zaliczenia	42
7	01 - Podstawy C++	43
7.1	Cele segmentu	43
7.2	Docelowa kolejność lekcji	43
7.3	Status porządkowania	43
7.4	Format lekcji	43
7.5	Przykłady kodu	44
7.6	Testy	44
7.7	Katalogi pomocnicze	44
8	01 - Pierwszy program	45
8.1	Cel lekcji	45
8.2	Wymagania wstępne	45
8.3	Krótką teoria	45
8.4	Funkcja <code>main</code>	45
8.5	Przykład kodu	45
8.6	Kompilacja i uruchomienie	46
8.7	Komentarze	46
8.8	Zadania do wykonania	46
8.9	Checklist dla studenta	46
8.10	Kryteria zaliczenia	47
8.11	Pytania kontrolne	47
9	02 - Wejście i wyjście	48
9.1	Cel lekcji	48
9.2	Wymagania wstępne	48
9.3	Krótką teoria	48
9.3.1	Strumień wyjściowy <code>std::cout</code>	48
9.3.2	<code>std::endl</code> i <code>\n</code>	48
9.3.3	Strumień wejściowy <code>std::cin</code>	49
9.3.4	Wczytywanie całej linii	49
9.3.5	Argumenty programu	49
9.4	Zadania do wykonania	50
9.5	Checklist dla studenta	50

9.6	Kryteria zaliczenia	50
9.7	Pytania kontrolne	50
10	03 - Typy, zmienne i operatory	51
10.1	Cel lekcji	51
10.2	Wymagania wstępne	51
10.3	Krótką teoria	51
10.3.1	Typ danych	51
10.3.2	Zmienna	51
10.3.3	Nazwy zmiennych	52
10.3.4	Operatory arytmetyczne	52
10.3.5	Dzielenie całkowite i dzielenie przecinkowe	52
10.3.6	Konwersja typu	52
10.4	Przykład kodu: podstawowe typy	53
10.5	Przykład kodu: kalkulator sumy	53
10.6	Biblioteka <code>cmath</code>	53
10.7	Zadania do wykonania	54
10.8	Checklist dla studenta	54
10.9	Kryteria zaliczenia	54
10.10	Pytania kontrolne	54
11	04 - Formatowanie wyjścia	55
11.1	Cel lekcji	55
11.2	Wymagania wstępne	55
11.3	Krótką teoria	55
11.3.1	Biblioteka <code>io</code>	55
11.3.2	<code>std::setprecision</code>	55
11.3.3	<code>std::fixed</code>	56
11.3.4	<code>std::setw</code>	56
11.3.5	<code>std::setfill</code>	56
11.4	Przykład kodu: prosta tabela	56
11.5	Zadania do wykonania	56
11.6	Checklist dla studenta	57
11.7	Kryteria zaliczenia	57
11.8	Pytania kontrolne	57
12	05 - Zadania	58
12.1	Cel zestawu zadań	58
12.2	Zasady wykonania	58
12.3	Poziom 1 - Pierwsze programy	58
12.3.1	Zadanie 1. Powitanie	58
12.3.2	Zadanie 2. Trzy linie tekstu	59
12.3.3	Zadanie 3. Komentarze	59
12.3.4	Zadanie 4. Pierwsza wizytówka	59
12.4	Poziom 2 - Wejście i wyjście	59
12.4.1	Zadanie 5. Dane użytkownika	59
12.4.2	Zadanie 6. Imię i nazwisko	59
12.4.3	Zadanie 7. Krótki formularz	60
12.4.4	Zadanie 8. Argumenty programu	60
12.5	Poziom 3 - Typy, zmienne i operatory	60
12.5.1	Zadanie 9. Dwie liczby	60
12.5.2	Zadanie 10. Pole prostokąta	60
12.5.3	Zadanie 11. Pole i obwód koła	60
12.5.4	Zadanie 12. BMI	61
12.5.5	Zadanie 13. Pierwiastek i potęga	61
12.5.6	Zadanie 14. Czas w sekundach	61
12.6	Poziom 4 - Formatowanie wyjścia	61
12.6.1	Zadanie 15. Liczba PI	61
12.6.2	Zadanie 16. Prosta tabela	62

12.6.3	Zadanie 17. Numer z zerami	62
12.6.4	Zadanie 18. Paragon	62
12.7	Poziom 5 - Zadania dodatkowe	62
12.7.1	Zadanie 19. Palindrom tekstowy	62
12.7.2	Zadanie 20. Prosty kalkulator argumentów	63
12.7.3	Zadanie 21. Liczby skojarzone	63
12.8	Wariant minimum	63
12.9	Zadania przeniesione na później	63
12.10	Kryteria zaliczenia segmentu	63
12.11	Mini-sprawdzian segmentu	64
12.12	Checklista oddania	64
13	02 - Sterowanie i pętle	65
13.1	Cele segmentu	65
13.2	Docelowa kolejność lekcji	65
13.3	Status porządkowania	65
13.4	Format lekcji	65
13.5	Przykłady kodu	66
13.6	Testy	66
13.7	Katalogi pomocnicze	66
14	01 - Instrukcje warunkowe	67
14.1	Cel lekcji	67
14.2	Wymagania wstępne	67
14.3	Krótką teorią	67
14.3.1	Po co są instrukcje warunkowe	67
14.3.2	Operatory porównania	67
14.4	Przykład kodu: <code>if</code> i <code>else</code>	68
14.4.1	<code>else if</code>	68
14.4.2	Operatory logiczne	68
14.5	Typowy błąd: średnik po <code>if</code>	69
14.6	Zadania do wykonania	69
14.7	Ćwiczenia dodatkowe	69
14.8	Kryteria zaliczenia	69
14.9	Pytania kontrolne	69
15	02 - <code>switch</code> i operator warunkowy	70
15.1	Cel lekcji	70
15.2	Wymagania wstępne	70
15.3	Krótką teorią	70
15.3.1	Instrukcja <code>switch</code>	70
15.4	Przykład kodu: proste menu	71
15.4.1	Dlaczego <code>break</code> jest ważny	71
15.4.2	Kiedy użyć <code>switch</code> , a kiedy <code>if</code>	71
15.4.3	Operator warunkowy <code>?:</code>	72
15.5	Przykład kodu: parzystość liczby	72
15.6	Zadania do wykonania	72
15.7	Ćwiczenia dodatkowe	72
15.8	Kryteria zaliczenia	73
15.9	Pytania kontrolne	73
16	03 - Pętla <code>for</code>	74
16.1	Cel lekcji	74
16.2	Wymagania wstępne	74
16.3	Krótką teorią	74
16.3.1	Składnia pętli <code>for</code>	74
16.3.2	Kolejność działania	74
16.3.3	Pętla z innym krokiem	75
16.3.4	Pętla malejąca	75

16.4	Przykład kodu: suma liczb	75
16.5	Przykład kodu: pętla i warunek	75
16.6	Typowe błędy	76
16.6.1	Zły warunek końca	76
16.6.2	Brak zmiany licznika	76
16.6.3	Użycie licznika poza zakresem	76
16.7	Zadania do wykonania	76
16.8	Ćwiczenia dodatkowe	76
16.9	Kryteria zaliczenia	76
16.10	Pytania kontrolne	77
17 04	- Pętla while i do while	78
17.1	Cel lekcji	78
17.2	Wymagania wstępne	78
17.3	Krótką teorią	78
17.3.1	Pętla while	78
17.3.2	Kiedy używać while	78
17.4	Przykład kodu: suma do zera	79
17.4.1	Pętla do while	79
17.4.2	Różnica między while i do while	79
17.4.3	Pętla nieskończona	80
17.5	Zadania do wykonania	80
17.6	Ćwiczenia dodatkowe	80
17.7	Kryteria zaliczenia	80
17.8	Pytania kontrolne	80
18 05	- break, continue i pętla zagnieżdżone	82
18.1	Cel lekcji	82
18.2	Wymagania wstępne	82
18.3	Krótką teorią	82
18.3.1	Instrukcja break	82
18.4	Przykład kodu: szukanie liczby	83
18.4.1	Instrukcja continue	83
18.4.2	Kiedy używać break i continue	83
18.4.3	Pętla zagnieżdżone	83
18.5	Przykład kodu: tabliczka mnożenia	84
18.5.1	break w pętli zagnieżdżonej	84
18.6	Zadania do wykonania	84
18.7	Ćwiczenia dodatkowe	84
18.8	Kryteria zaliczenia	84
18.9	Pytania kontrolne	85
19 06	- Zadania	86
19.1	Cel zestawu zadań	86
19.2	Zasady wykonania	86
19.3	Poziom 1 - Instrukcje warunkowe	86
19.3.1	Zadanie 1. Znak liczby	86
19.3.2	Zadanie 2. Parzystość	87
19.3.3	Zadanie 3. Kolejność rosnąca	87
19.3.4	Zadanie 4. Przedziały	87
19.3.5	Zadanie 5. Klasyfikacja BMI	87
19.4	Poziom 2 - switch i operator ?:	87
19.4.1	Zadanie 6. Dzień tygodnia	87
19.4.2	Zadanie 7. Ocena opisowa	87
19.4.3	Zadanie 8. Prosty kalkulator	87
19.4.4	Zadanie 9. Większa liczba	88
19.4.5	Zadanie 10. Pełnoletniość	88
19.5	Poziom 3 - Pętla for	88
19.5.1	Zadanie 11. Liczby od 1 do 110	88

19.5.2	Zadanie 12. Liczby podzielne przez 3	88
19.5.3	Zadanie 13. Suma liczb	88
19.5.4	Zadanie 14. Suma liczb parzystych	88
19.5.5	Zadanie 15. Silnia	88
19.6	Poziom 4 - while i do while	89
19.6.1	Zadanie 16. Odliczanie w dół	89
19.6.2	Zadanie 17. Suma do zera	89
19.6.3	Zadanie 18. Iloczyn liczb	89
19.6.4	Zadanie 19. Menu w pętli	89
19.6.5	Zadanie 20. Potęgi dwójki	89
19.7	Poziom 5 - break , continue i zagnieżdżenia	89
19.7.1	Zadanie 21. Pierwsza liczba podzielna przez 17	89
19.7.2	Zadanie 22. Pomijanie liczb	90
19.7.3	Zadanie 23. Tabliczka mnożenia	90
19.7.4	Zadanie 24. Prostokąt z gwiazdek	90
19.7.5	Zadanie 25. Trójkąt z gwiazdek	90
19.8	Zadania dodatkowe	90
19.8.1	Zadanie 26. Liczba pierwsza	90
19.8.2	Zadanie 27. Minimum, maksimum i średnia	90
19.8.3	Zadanie 28. Liczby trójkątne	91
19.9	Wariant minimum	91
19.10	Zadania przeniesione na później	91
19.11	Kryteria zaliczenia segmentu	91
19.12	Mini-sprawdzian segmentu	91
19.13	Checklista oddania	92
20	07 - Ćwiczenie: minimum, maksimum i średnia	93
20.1	Cel ćwiczenia	93
20.2	Wymagania wstępne	93
20.3	Opis zadania	93
20.4	Przykład działania	93
20.5	Wymagania techniczne	94
20.6	Wariant podstawowy	94
20.7	Proponowany podział pracy	94
20.8	Zadania dodatkowe	94
20.9	Pytania kontrolne	95
20.10	Scenariusze sprawdzenia	95
20.11	Kryteria zaliczenia	95
20.12	Źródło	95
21	03 - Funkcje, tablice, napisy	96
21.1	Cele segmentu	96
21.2	Docelowa kolejność lekcji	96
21.3	Status porządkowania	96
21.4	Format lekcji	96
21.5	Przykłady kodu	97
21.6	Testy	97
21.7	Katalogi pomocnicze	97
22	01 - Funkcje: podstawy	98
22.1	Cel lekcji	98
22.2	Wymagania wstępne	98
22.3	Krótką teoria	98
22.3.1	Po co są funkcje	98
22.3.2	Definicja funkcji	98
22.4	Przykład kodu: wywołanie funkcji	99
22.5	Przykład kodu: funkcja bez wartości zwracanej	99
22.5.1	Deklaracja funkcji	99
22.5.2	Deklaracja a definicja	100

22.5.3	Nazwy funkcji	100
22.6	Zadania do wykonania	100
22.7	Ćwiczenia dodatkowe	100
22.8	Kryteria zaliczenia	100
22.9	Pytania kontrolne	100
23	02 - Parametry i zakres zmiennych	102
23.1	Cel lekcji	102
23.2	Wymagania wstępne	102
23.3	Krótką teorią	102
23.3.1	Parametr i argument	102
23.4	Przykład kodu: przekazywanie przez wartość	102
23.4.1	Zmienne lokalne	103
23.4.2	Zakres zmiennej	103
23.4.3	Nazwy parametrów w deklaracji	103
23.5	Przykład kodu: pole prostokąta	103
23.6	Przykład kodu: BMI	104
23.7	Zadania do wykonania	104
23.8	Ćwiczenia dodatkowe	104
23.9	Kryteria zaliczenia	104
23.10	Pytania kontrolne	104
24	03 - Tablice jednowymiarowe	105
24.1	Cel lekcji	105
24.2	Wymagania wstępne	105
24.3	Krótką teorią	105
24.3.1	Czym jest tablica	105
24.3.2	Indeksy	105
24.3.3	Deklaracja i inicjalizacja	106
24.4	Przykład kodu: iteracja po tablicy	106
24.4.1	Rozmiar tablicy	106
24.5	Przykład kodu: suma i średnia	106
24.6	Przykład kodu: minimum i maksimum	106
24.7	Typowy błąd: wyjście poza zakres tablicy	107
24.8	Zadania do wykonania	107
24.9	Ćwiczenia dodatkowe	107
24.10	Kryteria zaliczenia	107
24.11	Pytania kontrolne	108
25	04 - Tablice dwuwymiarowe	109
25.1	Cel lekcji	109
25.2	Wymagania wstępne	109
25.3	Krótką teorią	109
25.3.1	Czym jest tablica dwuwymiarowa	109
25.3.2	Indeksy	109
25.3.3	Deklaracja	110
25.4	Przykład kodu: iteracja po tablicy dwuwymiarowej	110
25.5	Przykład kodu: suma wszystkich elementów	110
25.6	Przykład kodu: suma wierszy	110
25.6.1	Tablica dwuwymiarowa jako plansza	111
25.7	Typowe błędy	111
25.7.1	Zamiana wiersza z kolumną	111
25.7.2	Wyjście poza zakres	111
25.8	Zadania do wykonania	111
25.9	Ćwiczenia dodatkowe	111
25.10	Kryteria zaliczenia	111
25.11	Pytania kontrolne	112
26	05 - Napisy <code>std::string</code>	113

26.1	Cel lekcji	113
26.2	Wymagania wstępne	113
26.3	Krótką teoria	113
26.3.1	Typ <code>std::string</code>	113
26.3.2	<code>std::cin</code> i <code>std::getline</code>	113
26.3.3	Długość napisu	113
26.3.4	Indeksowanie znaków	114
26.4	Przykład kodu: iteracja po znakach	114
26.4.1	Łączenie napisów	114
26.5	Przykład kodu: proste sprawdzanie palindromu	114
26.6	Uwaga o polskich znakach	115
26.7	Zadania do wykonania	115
26.8	Ćwiczenia dodatkowe	115
26.9	Kryteria zaliczenia	115
26.10	Pytania kontrolne	115
27	06 - Zadania	116
27.1	Cel zestawu zadań	116
27.2	Zasady wykonania	116
27.3	Poziom 1 - Funkcje	116
27.3.1	Zadanie 1. Podstawowe działania	116
27.3.2	Zadanie 2. Kwadrat i sześciąt	117
27.3.3	Zadanie 3. Parzystość	117
27.3.4	Zadanie 4. BMI	117
27.3.5	Zadanie 5. Maksimum	117
27.4	Poziom 2 - Tablice jednowymiarowe	117
27.4.1	Zadanie 6. Wypisywanie tablicy	117
27.4.2	Zadanie 7. Suma i średnia	117
27.4.3	Zadanie 8. Minimum i maksimum	118
27.4.4	Zadanie 9. Odwrotna kolejność	118
27.4.5	Zadanie 10. Liczby parzyste	118
27.5	Poziom 3 - Tablice dwuwymiarowe	118
27.5.1	Zadanie 11. Wypisywanie macierzy	118
27.5.2	Zadanie 12. Suma macierzy	118
27.5.3	Zadanie 13. Sumy wierszy i kolumn	118
27.5.4	Zadanie 14. Największy element	119
27.5.5	Zadanie 15. Plansza 3 x 3	119
27.6	Poziom 4 - Napisy	119
27.6.1	Zadanie 16. Długość napisu	119
27.6.2	Zadanie 17. Znaki napisu	119
27.6.3	Zadanie 18. Pierwszy i ostatni znak	119
27.6.4	Zadanie 19. Liczenie litery	119
27.6.5	Zadanie 20. Palindrom	120
27.7	Poziom 5 - Zadania dodatkowe	120
27.7.1	Zadanie 21. Kółko i krzyżyk - plansza	120
27.7.2	Zadanie 22. Najdłuższy wspólny podciąg	120
27.7.3	Zadanie 23. Liczby skojarzone z funkcjami	120
27.8	Wariant minimum	120
27.9	Kryteria zaliczenia segmentu	121
27.10	Mini-sprawdzian segmentu	121
27.11	Checklista oddania	121
27.12	Archiwum	121
28	07 - Ćwiczenie: szyfr GADERYPOLUKI	122
28.1	Cel ćwiczenia	122
28.2	Wymagania wstępne	122
28.3	Opis szyfru	122
28.4	Opis zadania	122
28.5	Wymagania techniczne	123

28.6	Wariant podstawowy	123
28.7	Proponowany podział pracy	123
28.8	Zadania dodatkowe	123
28.9	Pytania kontrolne	123
28.10	Scenariusze sprawdzenia	124
28.11	Kryteria zaliczenia	124
28.12	Źródło	124
29	04 - Wskaźniki, referencje, pamięć	125
29.1	Cele segmentu	125
29.2	Kolejność lekcji	125
29.3	Status porządkowania	125
29.4	Format lekcji	125
29.5	Przykłady kodu	126
29.6	Testy	126
29.7	Katalogi pomocnicze	126
30	01 - Adresy i wskaźniki	127
30.1	Cel lekcji	127
30.2	Wymagania wstępne	127
30.3	Krótką teorią	127
30.3.1	Wartość i adres	127
30.3.2	Czym jest wskaźnik	127
30.3.3	Deklarowanie wskaźników	128
30.4	Przykład kodu: dereferencja	128
30.4.1	<code>nullptr</code>	128
30.5	Przykład kodu: wskaźnik i adres wskaźnika	128
30.6	Typowe błędy	129
30.6.1	Błędny typ	129
30.6.2	Dereferencja pustego wskaźnika	129
30.7	Zadania do wykonania	129
30.8	Ćwiczenia dodatkowe	129
30.9	Kryteria zaliczenia	129
30.10	Pytania kontrolne	130
31	02 - Referencje	131
31.1	Cel lekcji	131
31.2	Wymagania wstępne	131
31.3	Krótką teorią	131
31.3.1	Czym jest referencja	131
31.3.2	Referencję trzeba zainicjalizować	131
31.3.3	Referencji nie przepinamy	131
31.3.4	Referencja a wskaźnik	132
31.4	Przykład kodu: referencje w funkcjach	132
31.5	Przykład kodu: zamiana wartości przez referencje	132
31.5.1	<code>const</code> referencja	132
31.6	Zadania do wykonania	133
31.7	Ćwiczenia dodatkowe	133
31.8	Kryteria zaliczenia	133
31.9	Pytania kontrolne	133
32	03 - Przekazywanie danych do funkcji	134
32.1	Cel lekcji	134
32.2	Wymagania wstępne	134
32.3	Krótką teorią	134
32.3.1	Przekazywanie przez wartość	134
32.3.2	Przekazywanie przez wskaźnik	134
32.3.3	Przekazywanie przez referencję	135
32.4	Przykład kodu: zamiana wartości przez wskaźniki	135

32.5	Przykład kodu: zamiana wartości przez referencje	135
32.6	Przykład kodu: zwracanie więcej niż jednego wyniku	135
32.6.1	Kiedy używać którego sposobu	136
32.7	Zadania do wykonania	136
32.8	Ćwiczenia dodatkowe	136
32.9	Kryteria zaliczenia	136
32.10	Pytania kontrolne	136
33	04 - Bezpieczeństwo pamięci	138
33.1	Cel lekcji	138
33.2	Wymagania wstępne	138
33.3	Krótką teorią	138
33.3.1	Zasada 1: inicjalizuj wskaźniki	138
33.3.2	Zasada 2: sprawdzaj <code>nullptr</code>	138
33.3.3	Zasada 3: nie zwracaj adresu zmiennej lokalnej	138
33.3.4	Zasada 4: nie wychodź poza zakres tablicy	139
33.3.5	Zasada 5: używaj referencji, gdy argument musi istnieć	139
33.3.6	Zasada 6: ograniczaj zakres zmiennych	139
33.4	Dobre praktyki	140
33.5	Zadania do wykonania	140
33.6	Ćwiczenia dodatkowe	140
33.7	Kryteria zaliczenia	140
33.8	Pytania kontrolne	140
34	05 - Zadania	141
34.1	Cel zestawu zadań	141
34.2	Zasady wykonania	141
34.3	Poziom 1 - Adresy i wskaźniki	141
34.3.1	Zadanie 1. Adres zmiennej	141
34.3.2	Zadanie 2. Zmiana przez wskaźnik	142
34.3.3	Zadanie 3. <code>nullptr</code>	142
34.3.4	Zadanie 4. Wskaźnik do elementu tablicy	142
34.3.5	Zadanie 5. Największa wartość przez wskaźnik	142
34.4	Poziom 2 - Referencje	142
34.4.1	Zadanie 6. Zmiana przez referencję	142
34.4.2	Zadanie 7. Zamiana wartości	142
34.4.3	Zadanie 8. Minimum i maksimum	142
34.4.4	Zadanie 9. Poprawianie wyniku	143
34.4.5	Zadanie 10. Referencja czy wskaźnik	143
34.5	Poziom 3 - Przekazywanie argumentów do funkcji	143
34.5.1	Zadanie 11. Przekazanie przez wartość	143
34.5.2	Zadanie 12. Przekazanie przez wskaźnik	143
34.5.3	Zadanie 13. Przekazanie przez referencję	143
34.5.4	Zadanie 14. Dwa wyniki z funkcji	143
34.5.5	Zadanie 15. Warunkowa zamiana	144
34.6	Poziom 4 - Bezpieczeństwo pamięci	144
34.6.1	Zadanie 16. Bezpieczne wypisywanie tablicy	144
34.6.2	Zadanie 17. Bezpieczna suma tablicy	144
34.6.3	Zadanie 18. Nie wychodź poza tablicę	144
34.6.4	Zadanie 19. Brak wskaźnika do zmiennej lokalnej	145
34.6.5	Zadanie 20. Wskaźnik opcjonalny	145
34.7	Zadania dodatkowe	145
34.7.1	Zadanie 21. Statystyki tablicy	145
34.7.2	Zadanie 22. Sortowanie dwóch liczb	145
34.7.3	Zadanie 23. Proste menu z funkcjami	145
34.8	Wariant minimum	145
34.9	Kryteria zaliczenia segmentu	146
34.10	Mini-sprawdzian segmentu	146
34.11	Checklista oddania	146

34.12	Archiwum	147
35	05 - Pliki i wyjątki	148
35.1	Cele segmentu	148
35.2	Kolejność lekcji	148
35.3	Status porządkowania	148
35.4	Przykłady kodu	148
35.5	Testy	149
35.6	Format lekcji	149
35.7	Katalogi pomocnicze	149
36	01 - Pliki tekstowe	150
36.1	Cel lekcji	150
36.2	Wymagania wstępne	150
36.3	Krótką teorią	150
36.3.1	Biblioteka <code><fstream></code>	150
36.4	Przykład kodu: zapis do pliku	150
36.4.1	Sprawdzanie otwarcia pliku	150
36.5	Przykład kodu: dopisywanie do pliku	151
36.6	Przykład kodu: odczyt z pliku	151
36.6.1	Odczyt liniami i odczyt słowami	151
36.6.2	Zamykanie pliku	151
36.7	Typowe błędy	152
36.7.1	Brak sprawdzenia otwarcia pliku	152
36.7.2	Nadpisanie pliku zamiast dopisania	152
36.7.3	Użycie <code>>></code> tam, gdzie potrzebna jest cała linia	152
36.8	Zadania do wykonania	152
36.9	Ćwiczenia dodatkowe	152
36.10	Kryteria zaliczenia	152
36.11	Pytania kontrolne	153
37	02 - Odczyt i walidacja	154
37.1	Cel lekcji	154
37.2	Wymagania wstępne	154
37.3	Krótką teorią	154
37.3.1	Czym jest walidacja	154
37.4	Przykład kodu: sprawdzanie otwarcia pliku	154
37.5	Przykład kodu: sprawdzanie odczytu liczby	155
37.5.1	Walidacja zakresu	155
37.6	Przykład kodu: odczyt linii i parsowanie	155
37.6.1	Stan strumienia	156
37.7	Typowe błędy	156
37.7.1	Użycie wartości po nieudanym odczycie	156
37.7.2	Sprawdzenie zakresu przed sprawdzeniem typu	157
37.7.3	Przerywanie programu przy pierwszej błędnej linii	157
37.8	Zadania do wykonania	157
37.9	Ćwiczenia dodatkowe	157
37.10	Kryteria zaliczenia	157
37.11	Pytania kontrolne	157
38	03 - Wyjątki	158
38.1	Cel lekcji	158
38.2	Wymagania wstępne	158
38.3	Krótką teorią	158
38.3.1	Po co są wyjątki	158
38.4	Przykład kodu: <code>try</code> , <code>throw</code> , <code>catch</code>	158
38.5	Przykład kodu: rzucanie wyjątku z funkcji	159
38.6	Przykład kodu: łapanie wyjątków standardowych	159
38.6.1	Kolejność bloków <code>catch</code>	159

38.6.2	catch (...)	160
38.6.3	Kiedy używać wyjątków	160
38.7	Typowe błędy	160
38.7.1	Łapanie wyjątku przez wartość	160
38.7.2	Puste catch	160
38.7.3	Używanie wyjątków do zwykłego sterowania programem	161
38.8	Zadania do wykonania	161
38.9	Ćwiczenia dodatkowe	161
38.10	Kryteria zaliczenia	161
38.11	Pytania kontrolne	161
39	04 - Zadanie: menu plikowe	162
39.1	Cel lekcji	162
39.2	Wymagania wstępne	162
39.3	Krótką teorią	162
39.3.1	Wymagane funkcje programu	162
39.4	Przykład kodu: dopisywanie wpisu	162
39.5	Przykład kodu: wyświetlanie wpisów	163
39.6	Przykład kodu: czyszczenie pliku	163
39.6.1	Walidacja wyboru	163
39.7	Przykład referencyjny	163
39.8	Zadania do wykonania	163
39.9	Ćwiczenia dodatkowe	164
39.10	Kryteria zaliczenia	164
39.11	Pytania kontrolne	164
40	05 - Zadania	165
40.1	Cel zestawu zadań	165
40.2	Zasady wykonania	165
40.3	Poziom 1 - Zapis i odczyt pliku	165
40.3.1	Zadanie 1. Pierwszy zapis	165
40.3.2	Zadanie 2. Odczyt liniami	165
40.3.3	Zadanie 3. Numerowanie linii	166
40.3.4	Zadanie 4. Dopisywanie do pliku	166
40.3.5	Zadanie 5. Liczba linii	166
40.4	Poziom 2 - Walidacja danych	166
40.4.1	Zadanie 6. Brak pliku	166
40.4.2	Zadanie 7. Suma liczb z pliku	166
40.4.3	Zadanie 8. Błędne dane w pliku	167
40.4.4	Zadanie 9. Odczyt i zakres	167
40.4.5	Zadanie 10. Pomijanie błędnych linii	167
40.5	Poziom 3 - Wyjątki	167
40.5.1	Zadanie 11. Dzielenie przez zero	167
40.5.2	Zadanie 12. Obsługa wyjątku	167
40.5.3	Zadanie 13. Sprawdzanie wieku	168
40.5.4	Zadanie 14. Indeks poza zakresem	168
40.5.5	Zadanie 15. Plik jako wyjątek	168
40.6	Poziom 4 - Menu plikowe	168
40.6.1	Zadanie 16. Proste menu	168
40.6.2	Zadanie 17. Czyszczenie pliku	168
40.6.3	Zadanie 18. Liczenie wpisów	168
40.6.4	Zadanie 19. Walidacja wyboru menu	169
40.6.5	Zadanie 20. Pusty wpis	169
40.7	Zadania dodatkowe	169
40.7.1	Zadanie 21. Lista zadań w pliku	169
40.7.2	Zadanie 22. Prosty dziennik	169
40.7.3	Zadanie 23. Raport z liczb	169
40.8	Wariant minimum	169
40.9	Kryteria zaliczenia segmentu	170

40.10	Mini-sprawdzian segmentu	170
40.11	Checklista oddania	170
40.12	Archiwum	171
41	06 - Programowanie obiektowe	172
41.1	Cele segmentu	172
41.2	Kolejność lekcji	172
41.3	Status porządkowania	172
41.4	Przykłady kodu	173
41.5	Testy	173
41.6	Format lekcji	173
41.7	Katalogi pomocnicze	173
42	01 - Klasy i obiekty	174
42.1	Cel lekcji	174
42.2	Wymagania wstępne	174
42.3	Krótką teoria	174
42.3.1	Klasa	174
42.3.2	Obiekt	174
42.4	Przykład kodu: pola klasy	175
42.5	Przykład kodu: metody klasy	175
42.5.1	<code>public</code>	175
42.5.2	Nazwy klas	176
42.6	Typowe błędy	176
42.6.1	Brak średnika po klasie	176
42.6.2	Mylenie klasy z obiektem	176
42.6.3	Zbyt techniczna nazwa klasy	176
42.7	Zadania do wykonania	177
42.8	Kryteria zaliczenia	177
42.9	Pytania kontrolne	177
43	02 - Konstruktory i enkapsulacja	178
43.1	Cel lekcji	178
43.2	Wymagania wstępne	178
43.3	Krótką teoria	178
43.3.1	Problem z polami publicznymi	178
43.3.2	<code>private</code>	178
43.4	Przykład kodu: konstruktor	179
43.4.1	Getter	179
43.4.2	Setter	179
43.5	Przykład kodu: enkapsulacja	179
43.5.1	Konstruktor domyślny i konstruktor z parametrami	180
43.6	Typowe błędy	180
43.6.1	Publiczne pola bez kontroli	180
43.6.2	Getter, który zmienia stan	181
43.6.3	Setter bez walidacji	181
43.7	Zadania do wykonania	181
43.8	Kryteria zaliczenia	181
43.9	Pytania kontrolne	181
44	03 - Metody <code>const</code> i <code>this</code>	182
44.1	Cel lekcji	182
44.2	Wymagania wstępne	182
44.3	Krótką teoria	182
44.3.1	Metoda <code>const</code>	182
44.3.2	Dlaczego <code>const</code> jest przydatne	182
44.4	Przykład kodu: metody odczytujące i modyfikujące	183
44.5	Przykład kodu: wskaźnik <code>this</code>	183
44.5.1	Kiedy używać <code>this</code>	183

44.5.2	Dobre nazwy w klasach	184
44.5.3	Konwencja w tym kursie	184
44.6	Typowe błędy	184
44.6.1	Brak <code>const</code> przy getterze	184
44.6.2	Próba zmiany pola w metodzie <code>const</code>	185
44.6.3	Niejasne nazwy	185
44.7	Zadania do wykonania	185
44.8	Kryteria zaliczenia	185
44.9	Pytania kontrolne	185
45	04 - Kopiowanie obiektów	186
45.1	Cel lekcji	186
45.2	Wymagania wstępne	186
45.3	Krótką teorię	186
45.3.1	Kiedy powstaje kopia	186
45.4	Przykład kodu: konstruktor kopiujący	186
45.4.1	Domyślne kopiowanie	187
45.5	Przykład kodu: przekazywanie przez wartość	187
45.5.1	Przekazywanie przez referencję <code>const</code>	187
45.5.2	Przekazywanie przez referencję bez <code>const</code>	188
45.5.3	Zwracanie obiektów z funkcji	188
45.6	Typowe błędy	188
45.6.1	Niepotrzebne kopiowanie dużych obiektów	188
45.6.2	Brak <code>const</code> przy referencji tylko do odczytu	188
45.6.3	Mylenie kopii z tym samym obiektem	188
45.7	Zadania do wykonania	189
45.8	Kryteria zaliczenia	189
45.9	Pytania kontrolne	189
46	05 - Dziedziczenie	190
46.1	Cel lekcji	190
46.2	Wymagania wstępne	190
46.3	Krótką teorię	190
46.3.1	Klasa bazowa i klasa pochodna	190
46.3.2	Dziedziczenie <code>public</code>	191
46.4	Przykład kodu: konstruktor klasy bazowej	191
46.4.1	<code>protected</code>	191
46.5	Przykład kodu: rozszerzanie zachowania	192
46.5.1	Kiedy dziedziczenie ma sens	192
46.6	Typowe błędy	192
46.6.1	Dziedziczenie zamiast pola	192
46.6.2	Bezpośredni dostęp do zbyt wielu pól	192
46.6.3	Brak wywołania konstruktora bazowego	192
46.7	Zadania do wykonania	193
46.8	Kryteria zaliczenia	193
46.9	Pytania kontrolne	193
47	06 - Przeciążanie operatorów	194
47.1	Cel lekcji	194
47.2	Wymagania wstępne	194
47.3	Krótką teorię	194
47.3.1	Czym jest przeciążanie operatorów	194
47.4	Przykład kodu: operator jako metoda	194
47.5	Przykład kodu: operator <code>==</code>	195
47.6	Przykład kodu: operator <code><<</code>	195
47.6.1	Operator jako funkcja zaprzyjaźniona	195
47.6.2	Kiedy przeciążanie operatorów ma sens	195
47.7	Typowe błędy	196
47.7.1	Operator robi coś zaskakującego	196

47.7.2	Brak <code>const</code>	196
47.7.3	Zwracanie referencji do obiektu lokalnego	196
47.8	Zadania do wykonania	196
47.9	Kryteria zaliczenia	196
47.10	Pytania kontrolne	197
48	07 - Zadania	198
48.1	Cel zestawu zadań	198
48.2	Zasady wykonania	198
48.3	Wymagania jakościowe	198
48.4	Poziom 1 - Klasy i obiekty	198
48.4.1	Zadanie 1. Punkt	198
48.4.2	Zadanie 2. Prostokąt	198
48.4.3	Zadanie 3. Osoba	199
48.4.4	Zadanie 4. Produkt	199
48.4.5	Zadanie 5. Konto	199
48.5	Poziom 2 - Konstruktory i enkapsulacja	199
48.5.1	Zadanie 6. Konto z konstruktorem	199
48.5.2	Zadanie 7. Wpłata i wypłata	200
48.5.3	Zadanie 8. Samochód	200
48.5.4	Zadanie 9. Walidacja settera	200
48.5.5	Zadanie 10. Klasa <code>Uczen</code>	200
48.6	Poziom 3 - <code>const</code> , <code>this</code> i kopiowanie	200
48.6.1	Zadanie 11. Gettery <code>const</code>	200
48.6.2	Zadanie 12. Wartość produktu	200
48.6.3	Zadanie 13. Konstruktor z <code>this</code>	201
48.6.4	Zadanie 14. Konstruktor kopiujący	201
48.6.5	Zadanie 15. Przekazywanie obiektu do funkcji	201
48.7	Poziom 4 - Dziedziczenie	201
48.7.1	Zadanie 16. Pracownik i menedżer	201
48.7.2	Zadanie 17. Samochód sportowy	201
48.7.3	Zadanie 18. Konto oszczędnościowe	201
48.7.4	Zadanie 19. Relacja „jest” czy „ma”	202
48.7.5	Zadanie 20. Opis hierarchii	202
48.8	Poziom 5 - Przeciążanie operatorów	202
48.8.1	Zadanie 21. Punkt i operator <code>+</code>	202
48.8.2	Zadanie 22. Punkt i operator <code>==</code>	202
48.8.3	Zadanie 23. Punkt i operator <code><<</code>	202
48.8.4	Zadanie 24. Kwota	202
48.8.5	Zadanie 25. Macierz	203
48.9	Zadania dodatkowe	203
48.9.1	Zadanie 26. Klasa do obsługi pliku	203
48.9.2	Zadanie 27. Prosty katalog produktów	203
48.9.3	Zadanie 28. Mini-projekt: konto bankowe	203
48.10	Wariant minimum	204
48.11	Mini-sprawdzian segmentu	204
48.12	Kryteria zaliczenia segmentu	204
48.13	Checklista oddania	204
48.14	Archiwum	205
49	08 - Ćwiczenie: figury, dziedziczenie i polimorfizm	206
49.1	Cel ćwiczenia	206
49.2	Wymagania wstępne	206
49.3	Opis zadania	206
49.4	Wymagania techniczne	206
49.5	Proponowany model klas	207
49.6	Wariant podstawowy	207
49.7	Proponowany podział pracy	208
49.8	Zadania dodatkowe	208

49.9 Rozszerzenie SFML	208
49.10 Pytania kontrolne	208
49.11 Scenariusze sprawdzenia	208
49.12 Kryteria zaliczenia	209
49.13 Źródło	209
50 07 - STL i struktury danych	210
50.1 Cele segmentu	210
50.2 Kolejność lekcji	210
50.3 Status porządkowania	210
50.4 Przykłady kodu	211
50.5 Testy	211
50.6 Format lekcji	211
50.7 Katalogi pomocnicze	211
51 01 - Wprowadzenie do STL	212
51.1 Cel lekcji	212
51.2 Wymagania wstępne	212
51.3 Krótka teoria	212
51.3.1 Czym jest STL	212
51.3.2 Kontener	212
51.3.3 Najważniejsze kontenery w tym segmencie	213
51.3.4 <code>std::vector</code>	213
51.3.5 <code>std::map</code>	213
51.3.6 <code>std::stack</code>	213
51.3.7 <code>std::queue</code>	213
51.3.8 <code>std::list</code>	213
51.4 Przykład kodu: kilka kontenerów	213
51.5 Przykład kodu: dobór kontenera	214
51.5.1 Pętla zakresowa	214
51.6 Typowe błędy	214
51.6.1 Ręczne pisanie struktury, gdy istnieje kontener	214
51.6.2 Zły kontener do problemu	214
51.6.3 Kopiowanie dużych elementów w pętli	214
51.7 Zadania do wykonania	215
51.8 Kryteria zaliczenia	215
51.9 Pytania kontrolne	215
52 02 - <code>std::vector</code>	216
52.1 Cel lekcji	216
52.2 Wymagania wstępne	216
52.3 Krótka teoria	216
52.3.1 Czym jest <code>std::vector</code>	216
52.4 Przykład kodu: tworzenie wektora	216
52.5 Przykład kodu: dodawanie elementów	217
52.5.1 Rozmiar wektora	217
52.5.2 Dostęp po indeksie	217
52.5.3 Pętla zakresowa	217
52.5.4 Usuwanie ostatniego elementu	217
52.5.5 <code>empty</code> , <code>front</code> , <code>back</code>	218
52.6 Przykład kodu: wektor struktur lub obiektów	218
52.6.1 <code>resize</code>	218
52.7 Typowe błędy	218
52.7.1 Wyjście poza zakres	218
52.7.2 <code>pop_back</code> na pustym wektorze	219
52.7.3 Kopiowanie dużych obiektów w pętli	219
52.8 Zadania do wykonania	219
52.9 Kryteria zaliczenia	219
52.10 Pytania kontrolne	219

53 03 - Algorytmy STL	220
53.1 Cel lekcji	220
53.2 Wymagania wstępne	220
53.3 Krótka teoria	220
53.3.1 Nagłówek <code><algorithm></code>	220
53.4 Przykład kodu: <code>std::sort</code>	220
53.4.1 Sortowanie malejące	220
53.4.2 Lambda	221
53.5 Przykład kodu: <code>std::find_if</code>	221
53.6 Przykład kodu: <code>std::copy_if</code>	221
53.6.1 <code>std::remove_if</code> i <code>erase</code>	221
53.6.2 Predykat	222
53.7 Typowe błędy	222
53.7.1 Brak sprawdzenia wyniku <code>find_if</code>	222
53.7.2 Samo <code>remove_if</code> bez <code>erase</code>	222
53.7.3 Sortowanie obiektów bez kryterium	222
53.8 Zadania do wykonania	222
53.9 Kryteria zaliczenia	222
53.10 Pytania kontrolne	223
54 04 - <code>std::map</code> i słownik	224
54.1 Cel lekcji	224
54.2 Wymagania wstępne	224
54.3 Krótka teoria	224
54.3.1 Czym jest <code>std::map</code>	224
54.4 Przykład kodu: tworzenie mapy	224
54.4.1 Klucz musi być unikalny	225
54.4.2 Odczyt przez operator <code>[]</code>	225
54.5 Przykład kodu: bezpieczne wyszukiwanie przez <code>find</code>	225
54.5.1 Przechodzenie po mapie	225
54.5.2 Usuwanie elementu	225
54.6 Przykład kodu: biblioteka filmów	225
54.7 Typowe błędy	226
54.7.1 Użycie <code>[]</code> do sprawdzania istnienia klucza	226
54.7.2 Brak sprawdzenia wyniku <code>find</code>	226
54.7.3 Zły wybór klucza	226
54.8 Zadania do wykonania	226
54.9 Kryteria zaliczenia	226
54.10 Pytania kontrolne	227
55 05 - <code>std::stack</code>, <code>std::queue</code>, <code>std::list</code>	228
55.1 Cel lekcji	228
55.2 Wymagania wstępne	228
55.3 Krótka teoria	228
55.3.1 <code>std::stack</code>	228
55.4 Przykład kodu: <code>std::queue</code>	229
55.4.1 LIFO kontra FIFO	229
55.5 Przykład kodu: <code>std::list</code>	229
55.5.1 Kiedy używać <code>std::list</code>	230
55.6 Typowe błędy	230
55.6.1 <code>top</code> , <code>front</code> albo <code>back</code> na pustym kontenerze	230
55.6.2 Oczekiwanie dostępu po indeksie w <code>std::list</code>	230
55.6.3 Mylenie <code>pop</code> ze zwracaniem wartości	230
55.7 Zadania do wykonania	230
55.8 Kryteria zaliczenia	231
55.9 Pytania kontrolne	231
56 Projekt HRMS	232
56.1 Cel lekcji	232

56.2	Wymagania wstępne	232
56.3	Krótką teoria	232
56.4	Przykład kodu: model pracownika	232
56.4.1	Dobór kontenerów	233
56.5	Przykład kodu: dodawanie pracownika	233
56.6	Przykład kodu: wyszukiwanie i zmiana wynagrodzenia	233
56.7	Przykład kodu: sortowanie wynagrodzeń	233
56.8	Przykład referencyjny	233
56.9	Zadania do wykonania	234
56.10	Kryteria zaliczenia	234
56.11	Pytania kontrolne	234
57	07 - Zadania	235
57.1	Cel zestawu zadań	235
57.2	Zasady wykonania	235
57.3	Wymagania jakościowe	235
57.4	Poziom 1 - <code>std::vector</code>	235
57.4.1	Zadanie 1. Liczby od 1 do 10	235
57.4.2	Zadanie 2. Suma i średnia	236
57.4.3	Zadanie 3. Dodawanie i usuwanie	236
57.4.4	Zadanie 4. Lista słów	236
57.4.5	Zadanie 5. Wektor obiektów	236
57.5	Poziom 2 - Algorytmy STL	236
57.5.1	Zadanie 6. Sortowanie liczb	236
57.5.2	Zadanie 7. Wyszukiwanie wartości	236
57.5.3	Zadanie 8. Pierwsza liczba parzysta	237
57.5.4	Zadanie 9. Zliczanie ocen	237
57.5.5	Zadanie 10. Usuwanie wartości	237
57.5.6	Zadanie 11. Sortowanie produktów	237
57.6	Poziom 3 - <code>std::map</code>	237
57.6.1	Zadanie 12. Numery telefonów	237
57.6.2	Zadanie 13. Bezpieczne wyszukiwanie	237
57.6.3	Zadanie 14. Licznik słów	237
57.6.4	Zadanie 15. Magazyn	238
57.6.5	Zadanie 16. Biblioteka filmów	238
57.7	Poziom 4 - Stos, kolejka i lista	238
57.7.1	Zadanie 17. Odwracanie kolejności	238
57.7.2	Zadanie 18. Kolejka klientów	238
57.7.3	Zadanie 19. Nawiasy	238
57.7.4	Zadanie 20. Lista uczestników	239
57.7.5	Zadanie 21. Dobór kontenera	239
57.8	Poziom 5 - Zadania projektowe	239
57.8.1	Zadanie 22. Ranking graczy	239
57.8.2	Zadanie 23. System zamówień	239
57.8.3	Zadanie 24. Rozbudowa HRMS	240
57.9	Wariant minimum	240
57.10	Mini-sprawdzian segmentu	240
57.11	Kryteria zaliczenia segmentu	240
57.12	Checklista oddania	241
57.13	Archiwum	241
58	08 - Projekt, build i testy	242
58.1	Cele segmentu	242
58.2	Kolejność lekcji	242
58.3	Status porządkowania	242
58.4	Przykłady kodu i konfiguracji	243
58.5	Testy	243
58.6	Format lekcji	243
58.7	Katalogi pomocnicze	243

59 01 - Preprocesor i #include	244
59.1 Cel lekcji	244
59.2 Wymagania wstępne	244
59.3 Krótka teoria	244
59.3.1 Czym jest preprocesor	244
59.3.2 #include	244
59.4 Przykład kodu: makra i stałe kompilacyjne	245
59.5 Przykład kodu: kompilacja warunkowa	245
59.5.1 Guardy w plikach nagłówkowych	245
59.5.2 Nazwy predefiniowane	245
59.6 Przykład referencyjny	245
59.7 Zadania do wykonania	246
59.8 Kryteria zaliczenia	246
 60 02 - Podział programu na pliki	 247
60.1 Cel lekcji	247
60.2 Wymagania wstępne	247
60.3 Krótka teoria	247
60.3.1 Problem jednego pliku	247
60.3.2 Deklaracja i definicja	247
60.3.3 Typowy podział	248
60.4 Przykład kodu: plik nagłówkowy	248
60.5 Przykład kodu: plik implementacji	248
60.6 Przykład kodu: plik <code>main.cpp</code>	248
60.7 Przykład komendy: kompilacja wielu plików	248
60.8 Przykład referencyjny	249
60.9 Typowe błędy	249
60.9.1 Brak pliku <code>.cpp</code> w kompilacji	249
60.9.2 Implementacja funkcji w nagłówku	249
60.9.3 Brak guarda	249
60.10 Zadania do wykonania	249
60.11 Kryteria zaliczenia	249
 61 03 - Struktura projektu	 250
61.1 Cel lekcji	250
61.2 Wymagania wstępne	250
61.3 Krótka teoria	250
61.3.1 Dlaczego struktura katalogów ma znaczenie	250
61.4 Przykład struktury projektu	250
61.4.1 Katalog <code>include</code>	251
61.4.2 Katalog <code>src</code>	251
61.4.3 Katalog <code>tests</code>	251
61.4.4 Katalog <code>build</code>	251
61.5 Przykład referencyjny	251
61.6 Przykład komendy: flaga <code>-I</code>	252
61.7 Zadania do wykonania	252
61.8 Kryteria zaliczenia	252
 62 04 - Kompilacja i prosty build	 253
62.1 Cel lekcji	253
62.2 Wymagania wstępne	253
62.3 Krótka teoria	253
62.4 Przykład komendy: kompilacja wielu plików jedną komendą	253
62.4.1 Ostrzeżenia kompilatora	253
62.5 Przykład komendy: kompilacja i linkowanie	254
62.6 Typowe błędy	254
62.6.1 Brak katalogu z nagłówkami	254
62.6.2 Brak pliku implementacji	254
62.6.3 Artefakty w repozytorium	254

62.7	Przykład komendy: prosty skrypt build	254
62.8	Przykład referencyjny	255
62.9	Zadania do wykonania	255
62.10	Kryteria zaliczenia	255
63	05 - Sprawdzanie składni	256
63.1	Cel lekcji	256
63.2	Wymagania wstępne	256
63.3	Krótką teorią	256
63.3.1	Po co sprawdzać składnię osobno	256
63.4	Przykład komendy: sprawdzanie składni	256
63.4.1	Ostrzeżenia	256
63.4.2	Błąd kompilacji a błąd linkowania	257
63.5	Przykład komendy: sprawdzanie projektu z katalogami	257
63.6	Przykład referencyjny	257
63.7	Zadania do wykonania	257
63.8	Kryteria zaliczenia	258
64	06 - Proste testy	259
64.1	Cel lekcji	259
64.2	Wymagania wstępne	259
64.3	Krótką teorią	259
64.3.1	Po co pisać testy	259
64.4	Przykład kodu: test jako osobny program	259
64.4.1	Funkcja pomocnicza	260
64.4.2	Przypadki testowe	260
64.4.3	Kod zakończenia programu	260
64.5	Przykład komendy: kompilacja testów	260
64.5.1	Testy w skrypcie build	260
64.6	Zadania do wykonania	261
64.7	Kryteria zaliczenia	261
65	07 - CI/CD	262
65.1	Cel lekcji	262
65.2	Wymagania wstępne	262
65.3	Krótką teorią	262
65.3.1	Co oznacza CI	262
65.3.2	Co oznacza CD	262
65.3.3	Dlaczego testy muszą zwracać kod błędu	262
65.4	Przykład komendy: lokalny odpowiednik CI	263
65.5	Przykład konfiguracji: GitHub Actions	263
65.5.1	Minimalny workflow	263
65.5.2	Co powinno trafić do CI	263
65.6	Typowe błędy	263
65.6.1	Workflow robi coś innego niż lokalny build	263
65.6.2	Testy wypisują błąd, ale zwracają 0	263
65.6.3	Build zapisuje artefakty w repozytorium	264
65.7	Zadania do wykonania	264
65.8	Kryteria zaliczenia	264
66	08 - Zadania	265
66.1	Cel zestawu zadań	265
66.2	Zasady wykonania	265
66.3	Wymagania jakościowe	265
66.4	Poziom 1 - Preprocesor i nagłówki	265
66.4.1	Zadanie 1. Własny nagłówek	265
66.4.2	Zadanie 2. Guard	266
66.4.3	Zadanie 3. Flaga debug	266
66.4.4	Zadanie 4. Informacja diagnostyczna	266

66.5	Poziom 2 - Podział na pliki	266
66.5.1	Zadanie 5. Kalkulator	266
66.5.2	Zadanie 6. Moduł tekstowy	266
66.5.3	Zadanie 7. Błąd linkera	267
66.5.4	Zadanie 8. Interfejs i implementacja	267
66.6	Poziom 3 - Struktura projektu i build	267
66.6.1	Zadanie 9. Projekt <code>notes</code>	267
66.6.2	Zadanie 10. Kompilacja z <code>-I</code>	267
66.6.3	Zadanie 11. Pliki obiektowe	267
66.6.4	Zadanie 12. Skrypt build	268
66.6.5	Zadanie 13. Ignorowanie artefaktów	268
66.7	Poziom 4 - Sprawdzanie składni i testy	268
66.7.1	Zadanie 14. <code>-fsyntax-only</code>	268
66.7.2	Zadanie 15. Ostrzeżenie jako błąd	268
66.7.3	Zadanie 16. Pierwszy test	268
66.7.4	Zadanie 17. Test dodawania	268
66.7.5	Zadanie 18. Funkcja <code>expectEqual</code>	268
66.7.6	Zadanie 19. Testy w buildzie	269
66.8	Poziom 5 - CI/CD i mini-projekt	269
66.8.1	Zadanie 20. Lokalny odpowiednik CI	269
66.8.2	Zadanie 21. Przykładowy workflow	269
66.8.3	Zadanie 22. Analiza awarii	269
66.8.4	Zadanie 23. Mini-projekt <code>gradebook</code>	269
66.8.5	Zadanie 24. Dokumentacja builda	270
66.9	Wariant minimum	270
66.10	Mini-sprawdzian segmentu	270
66.11	Kryteria zaliczenia segmentu	270
66.12	Checklista oddania	271
66.13	Archiwum	271
67 09	- Ćwiczenie: kalkulator RPN	272
67.1	Cel ćwiczenia	272
67.2	Wymagania wstępne	272
67.3	Opis zadania	272
67.4	Proponowana struktura plików	272
67.5	Wymagania techniczne	273
67.6	Wariant podstawowy	273
67.7	Proponowany podział pracy	273
67.8	Zadania dodatkowe	274
67.9	Pytania kontrolne	274
67.10	Scenariusze sprawdzenia	274
67.11	Kryteria zaliczenia	274
67.12	Źródło	274
68 09	- Modern C++	275
68.1	Cele segmentu	275
68.2	Kolejność lekcji	275
68.3	Status porządkowania	275
68.4	Przykłady kodu	276
68.5	Testy	276
68.6	Format lekcji	276
68.7	Katalogi pomocnicze	276
69 01	- Nowości modern C++	277
69.1	Cel lekcji	277
69.2	Wymagania wstępne	277
69.3	Krótką teoria	277
69.3.1	Co oznacza modern C++	277
69.4	Przykład kodu: wybrane zmiany	277

69.4.1	Inicjalizacja klamrowa	277
69.4.2	<code>auto</code>	277
69.4.3	<code>nullptr</code>	278
69.4.4	Pętla zakresowa	278
69.4.5	<code>enum class</code>	278
69.4.6	Lambda	278
69.4.7	Structured bindings	278
69.4.8	Co zostanie omówione dalej	278
69.5	Przykład referencyjny	278
69.6	Zadania do wykonania	279
69.7	Kryteria zaliczenia	279
70 02	- <code>auto</code>, <code>nullptr</code> i inicjalizacja	280
70.1	Cel lekcji	280
70.2	Wymagania wstępne	280
70.3	Krótką teorią	280
70.3.1	<code>auto</code>	280
70.3.2	<code>auto</code> i referencje	280
70.4	Przykład kodu: <code>nullptr</code>	281
70.4.1	Sprawdzanie wskaźnika	281
70.5	Przykład kodu: inicjalizacja klamrowa	281
70.6	Przykład referencyjny	281
70.7	Zadania do wykonania	281
70.8	Kryteria zaliczenia	282
71 03	- Lambdy i algorytmy	283
71.1	Cel lekcji	283
71.2	Wymagania wstępne	283
71.3	Krótką teorią	283
71.3.1	Czym jest lambda	283
71.4	Przykład kodu: lambda jako predykat	283
71.5	Przykład kodu: sortowanie obiektów	284
71.5.1	Przechwytywanie zmiennych	284
71.5.2	Czytelność lambd	284
71.6	Przykład referencyjny	284
71.7	Zadania do wykonania	285
71.8	Kryteria zaliczenia	285
72 04	- <code>enum class</code>, <code>override</code> i aliasy	286
72.1	Cel lekcji	286
72.2	Wymagania wstępne	286
72.3	Krótką teorią	286
72.3.1	<code>enum class</code>	286
72.3.2	Aliaszy typów	286
72.4	Przykład kodu: <code>override</code>	287
72.4.1	<code>= default</code>	287
72.4.2	<code>= delete</code>	287
72.5	Przykład referencyjny	287
72.6	Zadania do wykonania	287
72.7	Kryteria zaliczenia	287
73 05	- Move semantics	289
73.1	Cel lekcji	289
73.2	Wymagania wstępne	289
73.3	Krótką teorią	289
73.3.1	Problem kopiowania	289
73.3.2	Przenoszenie	289
73.4	Przykład kodu: <code>std::move</code>	289
73.4.1	Konstruktor przenoszący	290

73.4.2	Kiedy używać przenoszenia	290
73.5	Przykład referencyjny	290
73.6	Zadania do wykonania	290
73.7	Kryteria zaliczenia	290
74	06 - Przestrzenie nazw	291
74.1	Cel lekcji	291
74.2	Wymagania wstępne	291
74.3	Krótką teorią	291
74.3.1	Po co są przestrzenie nazw	291
74.4	Przykład kodu: operator ::	291
74.4.1	Zagnieżdżone przestrzenie nazw	292
74.4.2	using	292
74.4.3	using namespace std	292
74.5	Przykład referencyjny	292
74.6	Zadania do wykonania	292
74.7	Kryteria zaliczenia	292
75	07 - Moduły i podział kodu	294
75.1	Cel lekcji	294
75.2	Wymagania wstępne	294
75.3	Krótką teorią	294
75.3.1	Klasyczny podział na pliki	294
75.3.2	„Moduł” jako część programu	294
75.3.3	Moduły C++20	295
75.3.4	Co wybrać w tym kursie	295
75.4	Przykład referencyjny	295
75.5	Zadania do wykonania	295
75.6	Kryteria zaliczenia	295
76	08 - Wielowątkowość	296
76.1	Cel lekcji	296
76.2	Wymagania wstępne	296
76.3	Krótką teorią	296
76.3.1	Czym jest wątek	296
76.4	Przykład kodu: uruchomienie wątku	296
76.4.1	Dlaczego join jest ważny	297
76.5	Przykład kodu: wspólne dane	297
76.5.1	Przekazywanie danych do wątku	297
76.6	Przykład referencyjny	297
76.7	Zadania do wykonania	297
76.8	Kryteria zaliczenia	297
77	09 - Zadania	299
77.1	Cel zestawu zadań	299
77.2	Zasady wykonania	299
77.3	Wymagania jakościowe	299
77.4	Poziom 1 - Podstawy modern C++	299
77.4.1	Zadanie 1. Inicjalizacja klamrowa	299
77.4.2	Zadanie 2. auto w praktyce	300
77.4.3	Zadanie 3. nullptr	300
77.4.4	Zadanie 4. Pętla zakresowa	300
77.4.5	Zadanie 5. Structured bindings	300
77.5	Poziom 2 - Lambdy i algorytmy	300
77.5.1	Zadanie 6. Sortowanie przez lambdę	300
77.5.2	Zadanie 7. Filtrowanie z progiem	300
77.5.3	Zadanie 8. Liczenie elementów	300
77.5.4	Zadanie 9. find_if	301
77.5.5	Zadanie 10. Lambda czy funkcja	301

77.6	Poziom 3 - Silniejsze typy i API	301
77.6.1	Zadanie 11. <code>enum class</code>	301
77.6.2	Zadanie 12. Aliasy typów	301
77.6.3	Zadanie 13. <code>override</code>	301
77.6.4	Zadanie 14. <code>= delete</code>	301
77.6.5	Zadanie 15. <code>= default</code>	301
77.7	Poziom 4 - Przenoszenie i organizacja kodu	302
77.7.1	Zadanie 16. Kopiowanie i przenoszenie	302
77.7.2	Zadanie 17. <code>std::move</code>	302
77.7.3	Zadanie 18. Stan po przeniesieniu	302
77.7.4	Zadanie 19. Przestrzeń nazw	302
77.7.5	Zadanie 20. Podział na pliki	302
77.8	Poziom 5 - Wątki i mini-projekty	302
77.8.1	Zadanie 21. Pierwszy wątek	302
77.8.2	Zadanie 22. Dwa wątki	303
77.8.3	Zadanie 23. Bezpieczny licznik	303
77.8.4	Zadanie 24. Mini-projekt: monitor zadań	303
77.9	Wariant minimum	303
77.10	Mini-sprawdzian segmentu	303
77.11	Kryteria zaliczenia segmentu	304
77.12	Checklista oddania	304
77.13	Archiwum	304
78	10 - Projekty	305
78.1	Cele segmentu	305
78.2	Kategorie projektów	305
78.2.1	Narzędzia konsolowe	305
78.2.2	Systemy i aplikacje użytkowe	305
78.2.3	Gry	305
78.3	Status porządkowania	306
78.4	Dodatkowe zadania źródłowe	306
78.5	Docelowy format opisu projektu	306
78.6	Wymagania dla rozwiązania	306
78.7	Wariant minimum i rozszerzenie	307
78.8	Rubryka oceny projektu	307
78.9	Kryteria zaliczenia segmentu	307
78.10	Katalogi pomocnicze	307
79	Jak wybrać projekt	309
79.1	1. Wybierz poziom trudności	309
79.1.1	Poziom podstawowy	309
79.1.2	Poziom średni	309
79.1.3	Poziom rozszerzony	309
79.2	2. Dopasuj projekt do materiału	309
79.3	3. Sprawdź realny zakres	310
79.4	4. Przygotuj pierwszy plan pracy	310
79.5	5. Nie zaczynaj od rozszerzeń	310
79.6	6. Przed oddaniem	310
80	Plan pracy nad projektem	311
80.1	1. Etap startowy	311
80.2	2. Etap modelu danych	311
80.3	3. Etap pierwszej funkcji	311
80.4	4. Etap wariantu minimum	312
80.5	5. Etap testów i scenariuszy	312
80.6	6. Etap dokumentacji	312
80.7	7. Etap pokazu	312
80.8	8. Proponowany harmonogram	312
80.9	9. Sygnały ostrzegawcze	313

81 Checklista projektu zaliczeniowego	314
81.1 1. Wariant minimum	314
81.2 2. Kompilacja i uruchomienie	314
81.3 3. Organizacja kodu	314
81.4 4. Dane i błędy	314
81.5 5. Testy i scenariusze sprawdzenia	315
81.6 6. Dokumentacja projektu	315
81.7 7. Pokaz i omówienie	315
81.8 8. Rozszerzenia	315
82 Karta oceny projektu	316
82.1 Dane projektu	316
82.2 Warunek wstępny	316
82.3 Punktacja	316
82.4 Funkcjonalność minimum	317
82.5 Poprawność techniczna	317
82.6 Organizacja kodu	317
82.7 Testy lub scenariusze sprawdzenia	317
82.8 Dokumentacja	317
82.9 Rozmowa techniczna	317
82.10Decyzja	317
82.11Poprawki	318
82.12Notatki z odbioru	318
83 Gra kółko i krzyżyk	319
83.1 Cel projektu	319
83.2 Zakres funkcjonalny	319
83.3 Wymagania techniczne	319
83.4 Proponowana struktura plików	320
83.5 Model gry	320
83.6 Strategia komputera	321
83.7 Minimalny wariant zaliczeniowy	321
83.8 Rozszerzenia dla chętnych	321
83.9 Kryteria oceny	321
83.10Scenariusze sprawdzenia	322
84 Projekt 01 - Logger C++	323
84.1 Cel projektu	323
84.2 Zakres funkcjonalny	323
84.3 Wymagania techniczne	323
84.4 Proponowana struktura plików	324
84.5 Minimalny wariant zaliczeniowy	324
84.6 Rozszerzenia dla chętnych	324
84.7 Kryteria oceny	325
84.8 Scenariusze sprawdzenia	325
85 Projekt 02 - Kolejowanie zamówień w sklepie	326
85.1 Cel projektu	326
85.2 Zakres funkcjonalny	326
85.3 Wymagania techniczne	326
85.4 Proponowana struktura plików	327
85.5 Model danych	327
85.6 Minimalny wariant zaliczeniowy	327
85.7 Rozszerzenia dla chętnych	328
85.8 Kryteria oceny	328
85.9 Scenariusze sprawdzenia	328
86 Projekt 04 - Tetris	329
86.1 Cel projektu	329

86.2	Zakres funkcjonalny	329
86.3	Wymagania techniczne	329
86.4	Proponowana struktura plików	330
86.5	Model gry	330
86.6	Pętla gry	331
86.7	Minimalny wariant zaliczeniowy	331
86.8	Rozszerzenia dla chętnych	331
86.9	Kryteria oceny	332
86.10	Scenariusze sprawdzenia	332
87	Projekt 05 - Snake	333
87.1	Cel projektu	333
87.2	Zakres funkcjonalny	333
87.3	Wymagania techniczne	333
87.4	Proponowana struktura plików	334
87.5	Model gry	334
87.6	Pętla gry	335
87.7	Minimalny wariant zaliczeniowy	335
87.8	Rozszerzenia dla chętnych	335
87.9	Kryteria oceny	336
87.10	Scenariusze sprawdzenia	336
88	Projekt 06 - Pac-Man	337
88.1	Cel projektu	337
88.2	Zakres funkcjonalny	337
88.3	Wymagania techniczne	337
88.4	Proponowana struktura plików	338
88.5	Model gry	338
88.6	Ruch przeciwników	339
88.7	Pętla gry	339
88.8	Minimalny wariant zaliczeniowy	339
88.9	Rozszerzenia dla chętnych	340
88.10	Kryteria oceny	340
88.11	Scenariusze sprawdzenia	340
89	Projekt 07 - Baza danych pojazdów	341
89.1	Cel projektu	341
89.2	Zakres funkcjonalny	341
89.3	Wymagania techniczne	341
89.4	Proponowana struktura plików	341
89.5	Model danych	342
89.6	Minimalny wariant zaliczeniowy	342
89.7	Rozszerzenia dla chętnych	343
89.8	Kryteria oceny	343
89.9	Scenariusze sprawdzenia	343
90	Projekt 08 - Kalkulator	344
90.1	Cel projektu	344
90.2	Zakres funkcjonalny	344
90.3	Wymagania techniczne	344
90.4	Proponowana struktura plików	345
90.5	Obsługiwane operacje	345
90.6	Algorytm obliczania	345
90.7	Minimalny wariant zaliczeniowy	346
90.8	Rozszerzenia dla chętnych	346
90.9	Kryteria oceny	346
90.10	Scenariusze sprawdzenia	347
91	Projekt 09 - Kalkulator macierzy	348

91.1	Cel projektu	348
91.2	Zakres funkcjonalny	348
91.3	Wymagania techniczne	348
91.4	Proponowana struktura plików	349
91.5	Model danych	349
91.6	Obsługiwane operacje	350
91.7	Minimalny wariant zaliczeniowy	350
91.8	Rozszerzenia dla chętnych	350
91.9	Kryteria oceny	351
91.10	Scenariusze sprawdzenia	351
92	Projekt 10 - System zarządzania zadaniami	352
92.1	Cel projektu	352
92.2	Zakres funkcjonalny	352
92.3	Wymagania techniczne	353
92.4	Proponowana struktura plików	353
92.5	Model danych	353
92.6	Minimalny wariant zaliczeniowy	354
92.7	Rozszerzenia dla chętnych	354
92.8	Kryteria oceny	355
92.9	Scenariusze sprawdzenia	355
93	Projekt 11 - Parser plików CSV	356
93.1	Cel projektu	356
93.2	Zakres funkcjonalny	356
93.3	Wymagania techniczne	356
93.4	Proponowana struktura plików	357
93.5	Model danych	357
93.6	Zasady parsowania CSV	358
93.7	Minimalny wariant zaliczeniowy	358
93.8	Rozszerzenia dla chętnych	358
93.9	Kryteria oceny	359
93.10	Scenariusze sprawdzenia	359
94	Projekt 12 - Symulator bankomatu	360
94.1	Cel projektu	360
94.2	Zakres funkcjonalny	360
94.3	Wymagania techniczne	360
94.4	Proponowana struktura plików	361
94.5	Model danych	361
94.6	Minimalny wariant zaliczeniowy	362
94.7	Rozszerzenia dla chętnych	362
94.8	Kryteria oceny	362
94.9	Scenariusze sprawdzenia	363
94.10	Źródło	363

Rozdział 1

Wstęp

- Autor: Paweł Kisielewicz <pkisiel@gmail.com>
- Miejsce i data: Kraków, 20 czerwca 2026
- Wersja: 0.1.1

Ten ebook powstaje na podstawie repozytorium z materiałami do nauki C++. Materiał jest ułożony w segmenty: od przygotowania środowiska, przez podstawy języka, funkcje, tablice, pliki, obiekty, STL i organizację projektu, aż po tematy zaliczeniowe.

1.1 Jak czytać

Najlepiej pracować kolejno:

1. Przeczytaj cel segmentu.
2. Przejdź przez lekcje w podanej kolejności.
3. Uruchom przykłady kodu.
4. Wykonaj zadania.
5. Po wybranych segmentach wykonaj ćwiczenie pomostowe.
6. Na końcu wybierz projekt i przygotuj wariant minimum.

1.2 Wymagania techniczne

Do pracy z przykładami potrzebujesz kompilatora C++ obsługującego standard C++17. W repozytorium przykłady można sprawdzić poleceniem:

```
sh tools/check-examples.sh
```

1.3 Uwaga o wersji

To jest pierwsza uporządkowana edycja czytelnicza materiałów kursu. Kolejne kroki redakcyjne powinny skupiać się na wygładzeniu przejść między rozdziałami, usuwaniu powtórzeń i dopisywaniu krótkich podsumowań po większych segmentach.

Rozdział 2

00 - Start

Ten segment przygotowuje środowisko pracy i pokazuje, jak uruchamiać kod C++. Jest przeznaczony dla osób, które dopiero zaczynają pracę z kompilatorem, edytorem, repozytorium Git i prostą organizacją projektu.

2.1 Cele segmentu

Po zakończeniu segmentu student powinien umieć:

- wyjaśnić, czym jest kompilator,
- skompilować i uruchomić prosty program C++,
- skonfigurować podstawową pracę w Visual Studio Code,
- wykonać podstawowe operacje w Git,
- rozumieć rolę pliku `CMakeLists.txt`,
- odróżnić ręczną kompilację od budowania projektu,
- odnaleźć związek materiału z kwalifikacjami technika programisty.

2.2 Kolejność lekcji

1. kompilator.md - instalacja i podstawy kompilatora.
2. vs-code.md - edytor i konfiguracja pracy.
3. git.md - podstawy pracy z repozytorium.
4. cmake.md - wprowadzenie do CMake.
5. technik-programista.md - kontekst programu nauczania.

2.3 Status porządkowania

Lekcje w tym segmencie są uporządkowane w docelowym formacie:

1. Cel lekcji.
2. Wymagania wstępne.
3. Krótka teoria.
4. Przykład praktyczny.
5. Zadania do wykonania.
6. Kryteria zaliczenia.

2.4 Efekt końcowy

Student potrafi skompilować prosty program, uruchomić go lokalnie, pracować w edytorze i zapisać zmiany w repozytorium.

2.5 Checklist dla studenta

Po przejściu segmentu student powinien umieć:

- uruchomić edytor i terminal,
- sprawdzić dostępność kompilatora C++,
- skompilować prosty plik `.cpp`,
- wyjaśnić, do czego służą Git i CMake,
- przejść do segmentu 01-podstawy.

Rozdział 3

Kompilator C++

3.1 Cel lekcji

Celem lekcji jest zrozumienie, czym jest kompilator i jak użyć go do uruchomienia pierwszego programu C++. Po tej lekcji student powinien potrafić sprawdzić, czy kompilator jest zainstalowany, skompilować prosty plik `.cpp` i uruchomić powstały program.

3.2 Wymagania wstępne

Przed lekcją student powinien umieć:

- otworzyć terminal,
- przejść do katalogu z plikami,
- utworzyć prosty plik tekstowy,
- rozpoznać rozszerzenie pliku, np. `.cpp`.

3.3 Krótka teoria

Język programowania jest sposobem zapisu obliczeń w formie zrozumiałej dla ludzi i możliwej do przetworzenia przez maszynę. Kompilator tłumaczy kod źródłowy C++ na program wykonywalny.

Typowy przepływ pracy wygląda tak:

1. Programista pisze kod w pliku, np. `main.cpp`.
2. Kompilator sprawdza składnię i typy.
3. Jeśli kod jest poprawny, powstaje program wykonywalny.
4. Program można uruchomić w terminalu.

Najczęściej używane kompilatory C++:

- `g++` - część GCC, często używany na Linuxie i w MinGW,
- `clang++` - często dostępny na macOS i Linuxie,
- `cl` - kompilator Microsoft Visual C++ na Windows.

W tym repozytorium przykłady zakładają standard C++17.

3.4 Sprawdzenie kompilatora

W terminalu uruchom jedno z poleceń:

```
g++ --version
```

albo:

```
clang++ --version
```


Jeśli terminal wypisze wersję kompilatora, środowisko jest gotowe do prostych ćwiczeń. Jeśli pojawi się komunikat typu `command not found`, kompilator nie jest zainstalowany albo nie jest dostępny w zmiennej `PATH`.

3.5 Przykład praktyczny

Utwórz plik `main.cpp`:

```
#include <iostream>

int main()
{
    std::cout << "Hello, C++!\n";
    return 0;
}
```

Skompiluj program:

```
g++ -std=c++17 -Wall -Wextra -pedantic main.cpp -o hello
```

Uruchom program na Linuxie albo macOS:

```
./hello
```

Na Windows, jeśli używasz MinGW:

```
hello.exe
```

Oczekiwany wynik:

```
Hello, C++!
```

3.6 Znaczenie flag kompilacji

W poleceniu:

```
g++ -std=c++17 -Wall -Wextra -pedantic main.cpp -o hello
```

poszczególne elementy oznaczają:

- `g++` - uruchomienie kompilatora,
- `-std=c++17` - użycie standardu C++17,
- `-Wall` - włączenie podstawowych ostrzeżeń,
- `-Wextra` - włączenie dodatkowych ostrzeżeń,
- `-pedantic` - dokładniejsze sprawdzanie zgodności ze standardem,
- `main.cpp` - plik źródłowy,
- `-o hello` - nazwa programu wynikowego.

Ostrzeżenia nie zawsze zatrzymują kompilację, ale warto je traktować poważnie. Na kursie kod powinien kompilować się bez błędów i bez ostrzeżeń.

3.7 Typowe problemy

3.7.1 Brak kompilatora

Objaw:

```
g++: command not found
```

Co sprawdzić:

- czy kompilator jest zainstalowany,
- czy terminal został uruchomiony ponownie po instalacji,
- czy katalog z kompilatorem jest dodany do `PATH`.

3.7.2 Błąd składni

Przykład:

```
std::cout << "Hello"
```

Brakuje średnika. Kompilator wskaże linię, w której znalazł problem albo jego skutek.

3.7.3 Niepoprawna nazwa pliku

Jeśli kompilujesz `main.cpp`, a plik nazywa się inaczej, kompilator zgłosi, że nie może znaleźć pliku. Przed kompilacją warto sprawdzić zawartość katalogu:

```
ls
```

Na Windows w klasycznym `cmd`:

```
dir
```

3.8 Zadania do wykonania

1. Sprawdź wersję kompilatora poleceniem `g++ --version` albo `clang++ --version`.
2. Utwórz plik `main.cpp` z programem wypisującym `Hello, C++!`.
3. Skompiluj program z flagami `-std=c++17 -Wall -Wextra -pedantic`.
4. Uruchom program w terminalu.
5. Celowo usuń średnik z programu, skompiluj ponownie i przeczytaj komunikat błędu.
6. Przywróć poprawny kod i upewnij się, że program kompiluje się bez ostrzeżeń.

3.9 Checklist dla studenta

Przed zaliczeniem lekcji sprawdź, czy potrafisz:

- sprawdzić wersję kompilatora,
- skompilować pojedynczy plik `.cpp`,
- uruchomić program wynikowy z terminala,
- użyć flag `-Wall`, `-Wextra` i `-pedantic`,
- przeczytać podstawowy komunikat błędu kompilacji.

3.10 Kryteria zaliczenia

Lekcja jest zaliczona, jeśli student potrafi:

- sprawdzić wersję kompilatora,
- wyjaśnić różnicę między plikiem źródłowym a programem wykonywalnym,
- skompilować prosty program C++,
- uruchomić program z terminala,
- rozpoznać podstawowy błąd składni,
- skompilować kod z flagami `-Wall -Wextra -pedantic`.

3.11 Źródła

- Aho, A. V.; Lam, M. S.; Sethi, R.; Ullman, J., *Kompilatory*, s. 1.

Rozdział 4

Visual Studio Code dla C++

4.1 Cel lekcji

Celem lekcji jest przygotowanie edytora Visual Studio Code do pracy z kodem C++. Po tej lekcji student powinien umieć otworzyć katalog projektu, edytować plik `.cpp`, uruchomić terminal w edytorze i skompilować program.

4.2 Wymagania wstępne

Przed lekcją student powinien:

- mieć zainstalowany kompilator C++,
- umieć otworzyć terminal,
- znać podstawowe polecenie kompilacji z lekcji `kompilator.md`.

4.3 Krótka teoria

Visual Studio Code jest edytorem kodu. Sam edytor nie jest kompilatorem. Do pracy z C++ potrzebne są dwa elementy:

- edytor, w którym piszemy kod,
- kompilator, który tłumaczy kod na program wykonywalny.

VS Code może ułatwiać pracę przez:

- podświetlanie składni,
- podpowiedzi kodu,
- wbudowany terminal,
- szybkie wyszukiwanie w projekcie,
- integrację z Git,
- konfigurację zadań kompilacji.

4.4 Zalecane rozszerzenia

Na start wystarczy:

- C/C++ od Microsoft,
- opcjonalnie `CMake Tools`, jeśli projekt używa `CMake`.

Rozszerzenia pomagają w edycji kodu, ale nie zastępują kompilatora. Jeśli `g++ --version` albo `clang++ --version` nie działa w terminalu, VS Code również nie skompiluje programu.

4.5 Przykład praktyczny

1. Otwórz VS Code.
2. Wybierz File -> Open Folder.
3. Otwórz katalog z ćwiczeniem.
4. Utwórz plik `main.cpp`.
5. Wklej kod:

```
#include <iostream>

int main()
{
    std::cout << "Program uruchomiony z VS Code\n";
    return 0;
}
```

6. Otwórz terminal w VS Code:

Terminal -> New Terminal

7. Skompiluj program:

```
g++ -std=c++17 -Wall -Wextra -pedantic main.cpp -o app
```

8. Uruchom program:

```
./app
```

Na Windows z MinGW:

```
app.exe
```

4.6 Typowe problemy

4.6.1 Terminal otwiera się w złym katalogu

Sprawdź aktualny katalog:

```
pwd
```

Na Windows w cmd:

```
cd
```

Jeśli terminal nie jest w katalogu z plikiem `main.cpp`, przejdź do właściwego katalogu poleceniem `cd`.

4.6.2 VS Code nie widzi kompilatora

Sprawdź w terminalu VS Code:

```
g++ --version
```

Jeśli polecenie nie działa, problem dotyczy instalacji kompilatora albo zmiennej `PATH`, a nie samego kodu C++.

4.6.3 Podkreślenia w edytorze nie zawsze są błędem kompilacji

Najważniejszy jest wynik kompilacji w terminalu. Podpowiedzi edytora mogą być pomocne, ale ostatecznie kod musi przejść przez kompilator.

4.7 Zadania do wykonania

1. Otwórz katalog ćwiczenia w VS Code.
2. Utwórz plik `main.cpp`.
3. Wpisz prosty program wypisujący tekst.

4. Otwórz terminal w VS Code.
5. Sprawdź wersję kompilatora.
6. Skompiluj i uruchom program.
7. Zmień wypisywany tekst i uruchom program ponownie.

4.8 Checklist dla studenta

Przed zaliczeniem lekcji sprawdź, czy potrafisz:

- otworzyć folder projektu w VS Code,
- użyć wbudowanego terminala,
- odnaleźć i otworzyć plik `.cpp`,
- skompilować program bez wychodzenia z edytora,
- rozpoznać najczęstsze problemy z katalogiem roboczym.

4.9 Kryteria zaliczenia

Lekcja jest zaliczona, jeśli student potrafi:

- otworzyć katalog projektu w VS Code,
- utworzyć i zapisać plik `.cpp`,
- uruchomić terminal w VS Code,
- sprawdzić dostępność kompilatora,
- skompilować program z terminala,
- uruchomić program wynikowy.

4.10 Źródła

- Dokumentacja VS Code: `Using GCC with MinGW`.

Rozdział 5

CMake - pierwsze wprowadzenie

5.1 Cel lekcji

Celem lekcji jest zrozumienie, po co używa się CMake i jak wygląda minimalny plik `CMakeLists.txt` dla prostego programu C++. Po tej lekcji student powinien umieć odróżnić ręczną kompilację pojedynczego pliku od budowania projektu.

5.2 Wymagania wstępne

Przed lekcją student powinien:

- umieć skompilować pojedynczy plik `.cpp`,
- znać podstawowe pojęcie katalogu projektu,
- umieć uruchomić polecenie w terminalu.

5.3 Krótka teoria

CMake nie jest kompilatorem. CMake jest narzędziem, które generuje konfigurację budowania projektu dla właściwego kompilatora i systemu build.

Ręczna kompilacja jest wygodna dla jednego pliku:

```
g++ main.cpp -o app
```

Przy większym projekcie pojawiają się dodatkowe problemy:

- wiele plików `.cpp`,
- katalogi z nagłówkami,
- biblioteki,
- testy,
- różne systemy operacyjne,
- różne kompilatory.

CMake pomaga opisać projekt w jednym miejscu.

5.4 Minimalny projekt

Przykładowa struktura:

```
hello-cmake/  
  CMakeLists.txt  
  main.cpp
```

Plik `main.cpp`:

```
#include <iostream>

int main()
{
    std::cout << "Hello from CMake\n";
    return 0;
}
```

Plik CMakeLists.txt:

```
cmake_minimum_required(VERSION 3.16)

project(hello_cmake LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_executable(hello main.cpp)
```

5.5 Budowanie projektu

W katalogu projektu uruchom:

```
cmake -S . -B build
```

Następnie:

```
cmake --build build
```

Uruchomienie programu zależy od systemu i generatora. Na Linuxie albo macOS często będzie to:

```
./build/hello
```

Na Windows plik wykonywalny może znaleźć się głębiej w katalogu `build`, zależnie od użytego generatora.

5.6 Znaczenie poleceń

```
cmake -S . -B build
```

oznacza:

- `-S .` - katalog ze źródłami znajduje się tutaj,
- `-B build` - pliki budowania mają powstać w katalogu `build`.

```
cmake --build build
```

oznacza:

- zbuduj projekt używając konfiguracji zapisanej w katalogu `build`.

5.7 Typowe problemy

5.7.1 CMake nie jest kompilatorem

Jeśli kompilator nie jest zainstalowany, CMake nie rozwiąże tego problemu. Najpierw sprawdź:

```
g++ --version
```

albo:

```
clang++ --version
```

5.7.2 Literówka w nazwie pliku

Jeśli w CMakeLists.txt wpiszesz `main.cpp`, a plik nazywa się inaczej, budowanie zakończy się błędem.

5.7.3 Mieszanie plików źródłowych i build

Katalog `build/` powinien zawierać pliki generowane. Nie należy ręcznie edytować większości plików w tym katalogu.

5.8 Zadania do wykonania

1. Sprawdź wersję CMake:

```
cmake --version
```

2. Utwórz katalog `hello-cmake`.
3. Dodaj pliki `main.cpp` i `CMakeLists.txt`.
4. Wygeneruj katalog budowania poleceniem `cmake -S . -B build`.
5. Zbuduj projekt poleceniem `cmake --build build`.
6. Uruchom powstały program.
7. Zmień tekst wypisywany przez program i zbuduj projekt ponownie.

5.9 Checklist dla studenta

Przed zaliczeniem lekcji sprawdź, czy potrafisz:

- wskazać rolę pliku `CMakeLists.txt`,
- utworzyć katalog `build` poza kodem źródłowym,
- uruchomić `cmake -S . -B build`,
- zbudować projekt komendą `cmake --build build`,
- odróżnić pliki źródłowe od wygenerowanych artefaktów.

5.10 Kryteria zaliczenia

Lekcja jest zaliczona, jeśli student potrafi:

- wyjaśnić, że CMake nie jest kompilatorem,
- napisać minimalny `CMakeLists.txt`,
- wygenerować katalog `build`,
- zbudować projekt poleceniem `cmake --build`,
- uruchomić powstały program,
- wskazać, po co oddziela się katalog źródeł od katalogu `build`.

Rozdział 6

Git - podstawy pracy z repozytorium

6.1 Cel lekcji

Celem lekcji jest poznanie podstawowych poleceń Git potrzebnych do pracy z materiałami kursu. Po tej lekcji student powinien umieć sprawdzić stan repozytorium, dodać zmiany do commita i zapisać je lokalnie.

6.2 Wymagania wstępne

Przed lekcją student powinien:

- umieć otworzyć terminal,
- umieć przejść do katalogu projektu,
- mieć zainstalowany Git,
- rozumieć, że repozytorium przechowuje historię zmian.

6.3 Krótka teoria

Git zapisuje historię zmian w plikach projektu. Najważniejsze pojęcia:

- repozytorium - katalog śledzony przez Git,
- working tree - bieżące pliki na dysku,
- staging area - lista zmian przygotowanych do commita,
- commit - zapisany punkt historii,
- remote - zdalne repozytorium, np. na GitHubie.

Typowy cykl pracy:

1. Zmieniasz pliki.
2. Sprawdzasz stan repozytorium.
3. Dodajesz wybrane pliki do indeksu.
4. Tworzysz commit.
5. Wysyłasz commit do zdalnego repozytorium.

6.4 Podstawowe polecenia

Sprawdzenie stanu repozytorium:

```
git status
```

Krótki status:

```
git status --short
```

Podgląd zmian:

```
git diff
```

Dodanie pliku do indeksu:

```
git add main.cpp
```

Commit:

```
git commit -m "Add first C++ program"
```

Wysłanie zmian:

```
git push origin main
```

6.5 Praca z branchami

Pobranie informacji o zdalnych branchach:

```
git fetch origin
```

Przełączenie na istniejący branch:

```
git checkout nazwa-brancha
```

Utworzenie nowego brancha:

```
git checkout -b moja-zmiana
```

W nowszych wersjach Git można używać także:

```
git switch -c moja-zmiana
```

6.6 Typowe problemy

6.6.1 Plik nie trafił do commita

Sprawdź:

```
git status --short
```

Jeśli plik nadal jest oznaczony jako zmodyfikowany, prawdopodobnie nie został dodany poleceniem `git add`.

6.6.2 Przypadkowo dodany plik

Jeśli plik został dodany do indeksu, ale nie chcesz go commitować:

```
git restore --staged nazwa-pliku
```

6.6.3 Zmiana adresu zdalnego repozytorium

Przykład zmiany adresu na SSH:

```
git remote set-url origin git@github.com:USERNAME/REPOSITORY.git
```

6.7 Zadania do wykonania

1. Sprawdź wersję Git:

```
git --version
```

2. Sprawdź status repozytorium:

```
git status --short
```

3. Utwórz albo zmień mały plik ćwiczeniowy.
4. Sprawdź różnice poleceniem `git diff`.
5. Dodaj plik do indeksu poleceniem `git add`.

6. Sprawdź status ponownie.
7. Utwórz commit z krótkim opisem.

6.8 Checklist dla studenta

Przed zaliczeniem lekcji sprawdź, czy potrafisz:

- wyjaśnić różnicę między katalogiem roboczym, indeksem i commitem,
- użyć `git status` przed commitem,
- dodać wybrane pliki przez `git add`,
- utworzyć czytelny commit,
- utworzyć i przełączyć branch.

6.9 Kryteria zaliczenia

Lekcja jest zaliczona, jeśli student potrafi:

- sprawdzić status repozytorium,
- odróżnić pliki zmienione od dodanych do indeksu,
- podejrzeć diff,
- dodać plik poleceniem `git add`,
- utworzyć commit,
- wyjaśnić, po co używa się commita.

Rozdział 7

01 - Podstawy C++

Ten segment jest pierwszym właściwym blokiem kursu. Materiały są przygotowywane po polsku i mają prowadzić studenta od pierwszego programu do prostych zadań konsolowych.

7.1 Cele segmentu

Po zakończeniu segmentu student powinien umieć:

- opisać rolę algorytmu, programu i funkcji `main`,
- napisać, skompilować i uruchomić prosty program C++,
- używać `std::cin`, `std::cout` i `std::getline`,
- dobrać podstawowe typy danych do problemu,
- deklarować i inicjalizować zmienne,
- wykonywać proste obliczenia,
- formatować podstawowe wyjście tekstowe,
- rozwiązać proste zadania konsolowe.

7.2 Docelowa kolejność lekcji

1. 01-pierwszy-program.md - algorytm, program, `main`, komentarze, pierwszy kod.
2. 02-wejscie-wyjście.md - `cin`, `cout`, `getline`, argumenty programu.
3. 03-typy-zmienne-operatory.md - typy danych, zmienne, operatory, obliczenia.
4. 04-formatowanie-wyjścia.md - `iomanip`, precyzja, szerokość, wypełnienie.
5. 05-zadania.md - zadania podstawowe i dodatkowe.

7.3 Status porządkowania

Lekcje i zadania w tym segmencie są uporządkowane w docelowym formacie. Pierwotne notatki źródłowe zostały przeniesione do archive po wchłonięciu ich treści do lekcji i zadań.

7.4 Format lekcji

Każda docelowa lekcja powinna mieć taki układ:

1. Cel lekcji.
2. Wymagania wstępne.
3. Krótka teoria.
4. Przykład kodu.
5. Zadania do wykonania.
6. Kryteria zaliczenia.

7.5 Przykłady kodu

Katalog `examples` zawiera krótkie, kompilowalne programy powiązane z lekcjami. Przykłady służą do demonstracji składni i jako punkt startowy do zadań.

7.6 Testy

Katalog `tests` zawiera prosty test bez zewnętrznego frameworka. Test sprawdza podstawowe obliczenia, dobór typu wyniku, prostą klasyfikację tekstową oraz formatowanie liczby z ustaloną precyzją.

7.7 Katalogi pomocnicze

- `examples/` - kompilowalne przykłady `.cpp` dla tego segmentu.
- `tests/` - proste testy automatyczne dla podstaw C++.
- `archive/` - stare wersje plików po wchłonięciu ich treści do nowych lekcji.

Rozdział 8

01 - Pierwszy program

8.1 Cel lekcji

Celem lekcji jest zrozumienie, czym jest program w C++, gdzie zaczyna się jego wykonanie i jak uruchomić najprostszy kod w konsoli.

8.2 Wymagania wstępne

Student powinien mieć przygotowane środowisko z segmentu 00-start: edytor, kompilator i dostęp do terminala.

8.3 Krótka teoria

Algorytm to opis kolejnych kroków prowadzących do rozwiązania problemu.

Program to algorytm zapisany w języku programowania tak, aby komputer mógł go wykonać.

Kod źródłowy to tekst programu zapisany przez programistę, na przykład w pliku `.cpp`.

Kompilator tłumaczy kod źródłowy C++ na program wykonywalny.

8.4 Funkcja `main`

W typowym programie konsolowym C++ wykonanie zaczyna się od funkcji `main`.

```
int main()
{
    return 0;
}
```

Znaczenie elementów:

- `int` oznacza, że funkcja zwraca liczbę całkowitą,
- `main` to nazwa funkcji startowej programu,
- `{ ... }` wyznacza ciało funkcji,
- `return 0;` oznacza poprawne zakończenie programu.

8.5 Przykład kodu

```
#include <iostream>

int main()
{
```

```

    std::cout << "Witaj w C++!" << std::endl;
    return 0;
}

```

Program korzysta z biblioteki `iostream`, aby wypisać tekst na ekran.

- `#include <iostream>` dołącza obsługę wejścia i wyjścia.
- `std::cout` wypisuje dane na standardowe wyjście.
- `std::endl` kończy linię.
- Średnik `;` kończy instrukcję.

Ten sam przykład znajduje się w pliku `examples/hello.cpp`.

8.6 Kompilacja i uruchomienie

Dla kompilatora `g++`:

```

g++ examples/hello.cpp -o hello
./hello

```

Dla kompilatora `clang++`:

```

clang++ examples/hello.cpp -o hello
./hello

```

Oczekiwany wynik:

```
Witaj w C++!
```

8.7 Komentarze

Komentarze służą do opisywania kodu. Kompilator je pomija.

```

// Komentarz jednoliniowy

/*
    Komentarz
    wieloliniowy
*/

```

Komentarzy używamy do wyjaśnienia decyzji albo trudniejszego fragmentu kodu. Nie trzeba komentować każdej oczywistej instrukcji.

8.8 Zadania do wykonania

1. Napisz program, który wypisze Twoje imię.
2. Napisz program, który wypisze nazwę kierunku lub grupy.
3. Zmień przykład tak, aby wypisywał trzy linie tekstu.
4. Dodaj do programu komentarz z krótkim opisem działania.
5. Skompiluj program i uruchom go z terminala.

8.9 Checklist dla studenta

Przed zaliczeniem lekcji sprawdź, czy potrafisz:

- wskazać funkcję `main`,
- wypisać tekst przez `std::cout`,
- zakończyć instrukcję średnikiem,
- dodać komentarz do kodu,
- skompilować i uruchomić pierwszy program.

8.10 Kryteria zaliczenia

Program powinien:

- znajdować się w pliku `.cpp`,
- zawierać funkcję `main`,
- kompilować się bez błędów,
- wypisywać tekst w konsoli,
- kończyć się instrukcją `return 0;`.

8.11 Pytania kontrolne

1. Od której funkcji zaczyna się wykonanie typowego programu C++?
2. Do czego służy `#include <iostream>`?
3. Czym różni się kod źródłowy od programu wykonywalnego?
4. Co oznacza `return 0;` w funkcji `main`?

Rozdział 9

02 - Wejście i wyjście

9.1 Cel lekcji

Celem lekcji jest nauczanie podstawowej komunikacji programu z użytkownikiem: wypisywania tekstu na ekran, wczytywania danych z klawiatury i odczytywania argumentów programu.

9.2 Wymagania wstępne

Student powinien rozumieć materiał z lekcji 01-pierwszy-program.md: funkcję `main`, dołączanie biblioteki `iostream`, kompilację i uruchamianie programu.

9.3 Krótka teoria

9.3.1 Strumień wyjściowy `std::cout`

Do wypisywania tekstu na ekran używamy `std::cout`.

```
#include <iostream>

int main()
{
    std::cout << "Witaj!" << std::endl;
    return 0;
}
```

Operator `<<` przekazuje dane do strumienia wyjściowego.

Można wypisać kilka wartości w jednej instrukcji:

```
std::cout << "Wynik: " << 42 << std::endl;
```

9.3.2 `std::endl` i `\n`

`std::endl` kończy linię i opróżnia bufor wyjścia. W prostych programach jest wygodny, ale nie zawsze potrzebny.

Znak `\n` również przechodzi do nowej linii:

```
std::cout << "Pierwsza linia\n";
std::cout << "Druga linia\n";
```

W zadaniach podstawowych można używać obu form. W większych programach często wybiera się `\n`, gdy nie trzeba wymuszać opróżnienia bufora.

9.3.3 Strumień wejściowy `std::cin`

Do wczytywania danych z klawiatury używamy `std::cin`.

```
#include <iostream>
#include <string>

int main()
{
    std::string imie;

    std::cout << "Podaj imie: ";
    std::cin >> imie;

    std::cout << "Czesc, " << imie << "!" << std::endl;
    return 0;
}
```

Operator `>>` pobiera dane ze strumienia wejściowego i zapisuje je do zmiennej.

Ten przykład znajduje się w pliku `examples/input_name.cpp`.

9.3.4 Wczytywanie całej linii

`std::cin >> zmienna` czyta dane do pierwszej spacji. Jeżeli potrzebujemy wczytać cały tekst, na przykład imię i nazwisko, używamy `std::getline`.

```
#include <iostream>
#include <string>

int main()
{
    std::string imieINazwisko;

    std::cout << "Podaj imie i nazwisko: ";
    std::getline(std::cin, imieINazwisko);

    std::cout << "Uzytkownik: " << imieINazwisko << std::endl;
    return 0;
}
```

Ten przykład znajduje się w pliku `examples/input_line.cpp`.

9.3.5 Argumenty programu

Program może otrzymać dane już w chwili uruchomienia. Służą do tego argumenty funkcji `main`.

```
int main(int argc, char* argv[])
{
    // argc - liczba argumentow
    // argv - tablica argumentow jako tekst
}
```

Przykład:

```
#include <iostream>

int main(int argc, char* argv[])
{
    std::cout << "Liczba argumentow: " << argc << std::endl;

    for (int i = 0; i < argc; ++i)
    {
```

```

        std::cout << i << ": " << argv[i] << std::endl;
    }

    return 0;
}

```

Ten przykład znajduje się w pliku `examples/program_arguments.cpp`.

Uruchomienie:

```

c++ examples/program_arguments.cpp -o program_arguments
./program_arguments Ala ma kota

```

9.4 Zadania do wykonania

1. Napisz program, który pyta użytkownika o imię i wypisuje powitanie.
2. Napisz program, który pyta użytkownika o imię oraz wiek i wypisuje oba dane w jednym zdaniu.
3. Napisz program, który wczytuje dwie liczby całkowite i wypisuje ich sumę.
4. Napisz program, który wczytuje imię i nazwisko w jednej linii przy pomocy `std::getline`.
5. Napisz program, który wypisuje wszystkie argumenty przekazane przy uruchomieniu.

9.5 Checklist dla studenta

Przed zaliczeniem lekcji sprawdź, czy potrafisz:

- wczytać pojedynczą wartość przez `std::cin`,
- wypisać komunikat przez `std::cout`,
- wczytać linię tekstu przez `std::getline`,
- wyjaśnić problem połączenia `>>` i `getline`,
- przygotować czytelne komunikaty dla użytkownika.

9.6 Kryteria zaliczenia

Program powinien:

- używać `std::cout` do komunikatów dla użytkownika,
- używać `std::cin` albo `std::getline` do wczytywania danych,
- deklarować zmienne o odpowiednich typach,
- kompilować się bez błędów,
- wypisywać wynik w czytelnej formie.

9.7 Pytania kontrolne

1. Do czego służy `std::cout`?
2. Do czego służy `std::cin`?
3. Czym różni się `std::cin >> tekst` od `std::getline(std::cin, tekst)`?
4. Co oznacza parametr `argc`?
5. Co przechowuje `argv`?

Rozdział 10

03 - Typy, zmienne i operatory

10.1 Cel lekcji

Celem lekcji jest zrozumienie, czym są typy danych, jak deklarować zmienne i jak wykonywać podstawowe operacje arytmetyczne w C++.

10.2 Wymagania wstępne

Student powinien znać materiał z lekcji:

- 01-pierwszy-program.md,
- 02-wejscie-wyjscie.md.

10.3 Krótka teoria

10.3.1 Typ danych

Typ danych określa, jakiego rodzaju wartość może przechowywać zmienna.

Najczęściej używane typy na początku nauki:

Typ	Przykład wartości	Zastosowanie
int	10, -4, 0	liczby całkowite
double	3.14, -0.5	liczby zmiennoprzecinkowe
char	'A'	pojedynczy znak
bool	true, false	wartość logiczna
std::string	"Ala"	tekst

Typ `std::string` wymaga dołączenia biblioteki `string`:

```
#include <string>
```

10.3.2 Zmienna

Zmienna to nazwane miejsce w pamięci programu, w którym przechowujemy wartość.

```
int wiek = 20;  
double wzrost = 1.75;  
std::string imie = "Ala";
```

Deklaracja zmiennej składa się z typu i nazwy:

```
int liczba;
```

Inicjalizacja oznacza nadanie wartości początkowej:

```
int liczba = 5;
```

10.3.3 Nazwy zmiennych

Dobra nazwa zmiennej powinna opisywać jej znaczenie.

```
int liczbaStudentow = 24;
double temperatura = 21.5;
bool czyZalogowany = false;
```

Unikamy nazw, które nic nie mówią:

```
int x;
double a;
```

Krótkie nazwy są dopuszczalne w prostych wzorach matematycznych, ale w programach użytkowych lepiej stosować nazwy opisowe.

10.3.4 Operatory arytmetyczne

Podstawowe operatory:

Operator	Znaczenie	Przykład
+	dodawanie	a + b
-	odejmowanie	a - b
*	mnożenie	a * b
/	dzielenie	a / b
%	reszta z dzielenia	a % b

Operator % działa dla liczb całkowitych.

```
int reszta = 10 % 3; // wynik: 1
```

10.3.5 Dzielenie całkowite i zmiennoprzecinkowe

W C++ wynik dzielenia zależy od typów operandów.

```
int a = 5;
int b = 2;
std::cout << a / b << std::endl; // wynik: 2
```

Ponieważ a i b są typu int, wynik też jest całkowity.

```
double x = 5.0;
double y = 2.0;
std::cout << x / y << std::endl; // wynik: 2.5
```

Jeżeli potrzebujemy wyniku zmiennoprzecinkowego, co najmniej jedna wartość powinna być typu double.

10.3.6 Konwersja typu

Czasem trzeba jawnie potraktować wartość jako inny typ.

```
int a = 5;
int b = 2;

double wynik = static_cast<double>(a) / b;
std::cout << wynik << std::endl; // wynik: 2.5
```

static_cast<double>(a) tworzy wartość a potraktowaną jako double.

10.4 Przykład kodu: podstawowe typy

```
#include <iostream>
#include <string>

int main()
{
    int wiek = 20;
    double wzrost = 1.75;
    char ocena = 'A';
    bool aktywny = true;
    std::string imie = "Ala";

    std::cout << "Imie: " << imie << std::endl;
    std::cout << "Wiek: " << wiek << std::endl;
    std::cout << "Wzrost: " << wzrost << std::endl;
    std::cout << "Ocena: " << ocena << std::endl;
    std::cout << "Aktywny: " << aktywny << std::endl;

    return 0;
}
```

Ten przykład znajduje się w pliku `examples/basic_types.cpp`.

10.5 Przykład kodu: kalkulator sumy

```
#include <iostream>

int main()
{
    int a;
    int b;

    std::cout << "Podaj pierwsza liczbe: ";
    std::cin >> a;

    std::cout << "Podaj druga liczbe: ";
    std::cin >> b;

    std::cout << "Suma: " << a + b << std::endl;
    return 0;
}
```

Ten przykład znajduje się w pliku `examples/sum_two_numbers.cpp`.

10.6 Biblioteka `cmath`

Do wybranych obliczeń matematycznych używamy biblioteki `cmath`.

```
#include <cmath>
```

Przykładowe funkcje:

- `std::sqrt(x)` - pierwiastek kwadratowy,
- `std::pow(a, b)` - potęga `a` do `b`,
- `std::sin(x)` - sinus,
- `std::cos(x)` - cosinus.

Przykład:

```

#include <cmath>
#include <iostream>

int main()
{
    double wynik = std::sqrt(25.0);
    std::cout << wynik << std::endl;
    return 0;
}

```

Ten przykład znajduje się w pliku `examples/math_functions.cpp`.

10.7 Zadania do wykonania

1. Zadeklaruj zmienne: imię, wiek, wzrost i informację, czy student jest aktywny. Wypisz je na ekran.
2. Napisz program, który wczytuje dwie liczby całkowite i wypisuje ich sumę, różnicę oraz iloczyn.
3. Napisz program, który wczytuje dwie liczby typu `double` i wypisuje wynik dzielenia.
4. Napisz program, który wczytuje liczbę całkowitą i wypisuje resztę z dzielenia przez 2.
5. Napisz program, który oblicza pole prostokąta na podstawie boków podanych przez użytkownika.
6. Napisz program, który oblicza pierwiastek z liczby podanej przez użytkownika.

10.8 Checklist dla studenta

Przed zaliczeniem lekcji sprawdź, czy potrafisz:

- dobrać typ zmiennej do danych,
- wykonać proste obliczenie arytmetyczne,
- odróżnić dzielenie całkowite od zmiennoprzecinkowego,
- użyć funkcji z biblioteki `cmath`,
- nazwać zmienne w sposób czytelny dla osoby sprawdzającej.

10.9 Kryteria zaliczenia

Program powinien:

- używać poprawnych typów danych,
- mieć czytelne nazwy zmiennych,
- kompilować się bez błędów,
- poprawnie wczytywać dane od użytkownika,
- wypisywać wynik z opisem,
- stosować `double`, gdy wynik może mieć część ułamkową.

10.10 Pytania kontrolne

1. Czym różni się `int` od `double`?
2. Do czego służy typ `bool`?
3. Dlaczego `5 / 2` daje inny wynik niż `5.0 / 2.0`?
4. Kiedy można użyć operatora `%`?
5. Do czego służy `static_cast<double>`?

Rozdział 11

04 - Formatowanie wyjścia

11.1 Cel lekcji

Celem lekcji jest nauczenie podstawowego formatowania tekstu i liczb wypisywanych w konsoli.

11.2 Wymagania wstępne

Student powinien znać materiał z lekcji:

- 01-pierwszy-program.md,
- 02-wejscie-wyjście.md,
- 03-typy-zmienne-operatory.md.

11.3 Krótka teoria

11.3.1 Biblioteka `iomanip`

Do formatowania wyjścia używamy biblioteki `iomanip`.

```
#include <iomanip>
```

Pozwala ona między innymi ustawiać:

- liczbę cyfr wypisywanych dla wartości zmiennoprzecinkowych,
- stałą liczbę miejsc po przecinku,
- szerokość pola tekstowego,
- znak wypełnienia.

11.3.2 `std::setprecision`

`std::setprecision` ustawia precyzję wypisywania liczb zmiennoprzecinkowych.

```
#include <iomanip>
#include <iostream>
```

```
int main()
{
    double pi = 3.1415926535;

    std::cout << std::setprecision(4) << pi << std::endl;
    std::cout << std::setprecision(8) << pi << std::endl;

    return 0;
}
```


Bez `std::fixed` precyzja oznacza liczbę istotnych cyfr.

11.3.3 `std::fixed`

`std::fixed` zmienia sposób wypisywania liczb zmiennoprzecinkowych. Po jego użyciu `std::setprecision` oznacza liczbę miejsc po przecinku.

```
std::cout << std::fixed << std::setprecision(2) << 3.14159 << std::endl;
```

Wynik:

3.14

Przykład znajduje się w pliku `examples/format_precision.cpp`.

11.3.4 `std::setw`

`std::setw` ustawia szerokość pola dla kolejnej wypisywanej wartości.

```
std::cout << std::setw(10) << 77 << std::endl;
```

Jeżeli wartość jest krótsza niż podana szerokość, zostanie wyrównana spacjami.

Ważne: `std::setw` działa tylko na następną wartość wypisywaną do strumienia.

11.3.5 `std::setfill`

`std::setfill` ustawia znak wypełnienia używany razem z `std::setw`.

```
std::cout << std::setfill('*') << std::setw(10) << 15 << std::endl;
```

Przykładowy wynik:

*****15

11.4 Przykład kodu: prosta tabela

```
#include <iomanip>
#include <iostream>
#include <string>

int main()
{
    std::cout << std::left;
    std::cout << std::setw(12) << "Produkt"
              << std::right << std::setw(8) << "Cena" << std::endl;

    std::cout << std::left;
    std::cout << std::setw(12) << "Zeszyt"
              << std::right << std::setw(8) << 4.50 << std::endl;

    return 0;
}
```

Pełny przykład znajduje się w pliku `examples/format_table.cpp`.

11.5 Zadania do wykonania

1. Napisz program, który wypisuje liczbę 3.1415926535 z dokładnością do dwóch miejsc po przecinku.
2. Napisz program, który wypisuje wynik dzielenia dwóch liczb typu `double` z dokładnością do trzech miejsc po przecinku.
3. Napisz program, który wypisuje trzy liczby całkowite w polach o szerokości 8.
4. Napisz program, który wypisuje numer faktury w formacie podobnym do 00000123.

5. Napisz program, który wypisuje prostą tabelę: nazwa produktu, liczba sztuk, cena.

11.6 Checklist dla studenta

Przed zaliczeniem lekcji sprawdź, czy potrafisz:

- użyć `std::fixed`,
- ustawić precyzję przez `std::setprecision`,
- wyrównać kolumny przez `std::setw`,
- przygotować prostą tabelę tekstową,
- dobrać format wyniku do danych liczbowych.

11.7 Kryteria zaliczenia

Program powinien:

- dołączać bibliotekę `iomanip`,
- używać `std::setprecision` dla liczb zmiennoprzecinkowych,
- używać `std::fixed`, gdy potrzebna jest stała liczba miejsc po przecinku,
- używać `std::setw` do wyrównania kolumn,
- wypisywać wynik w czytelnej formie.

11.8 Pytania kontrolne

1. Do czego służy `std::setprecision`?
2. Co zmienia `std::fixed`?
3. Czy `std::setw` działa na wszystkie kolejne wartości, czy tylko na następną?
4. Do czego służy `std::setfill`?
5. Dlaczego formatowanie wyjścia jest ważne w programach konsolowych?

Rozdział 12

05 - Zadania

12.1 Cel zestawu zadań

Ten plik zbiera zadania do segmentu 01 - Podstawy C++. Zadania są ułożone od najprostszych do trudniejszych i korzystają z materiału z lekcji:

1. 01-pierwszy-program.md,
2. 02-wejscie-wyjscie.md,
3. 03-typy-zmienne-operatory.md,
4. 04-formatowanie-wyjscia.md.

12.2 Zasady wykonania

Każde zadanie powinno być zapisane w osobnym pliku `.cpp`. Program powinien kompilować się bez błędów i wypisywać wynik w czytelnej formie.

Przykład kompilacji:

```
c++ -std=c++17 -Wall -Wextra -pedantic nazwa_pliku.cpp -o program
./program
```

Przed oddaniem zadania sprawdź:

- czy plik ma rozszerzenie `.cpp`,
- czy program zawiera funkcję `main`,
- czy nazwy zmiennych opisują dane, które przechowują,
- czy program działa dla przykładowych danych z treści zadania,
- czy wynik jest wypisany w formie zrozumiałej dla użytkownika.

12.3 Poziom 1 - Pierwsze programy

12.3.1 Zadanie 1. Powitanie

Napisz program, który wypisuje Twoje imię oraz nazwę grupy.

Przykładowy wynik:

```
Imie: Jan
Grupa: 1A
```

Wymagania:

- użyj `#include <iostream>`,
- wypisz każdą informację w osobnej linii,
- zakończ program przez `return 0;`.

12.3.2 Zadanie 2. Trzy linie tekstu

Napisz program, który wypisuje trzy osobne linie:

- nazwę języka programowania,
- nazwę środowiska pracy,
- datę wykonania zadania.

Przykładowy wynik:

```
Jezyk: C++
Srodowisko: Visual Studio Code
Data: 2026-06-13
```

12.3.3 Zadanie 3. Komentarze

Napisz prosty program wypisujący dowolny tekst. Dodaj:

- jeden komentarz jednoliniowy,
- jeden komentarz wieloliniowy.

Komentarze powinny wyjaśniać cel programu albo znaczenie fragmentu kodu. Nie dodawaj komentarza typu `wypisuje tekst` nad oczywistą instrukcją `std::cout`.

12.3.4 Zadanie 4. Pierwsza wizytówka

Napisz program, który wypisuje prostą wizytówkę:

```
=====
Jan Kowalski
Technik programista
=====
```

Wymagania:

- użyj kilku instrukcji `std::cout`,
- zadbaj o czytelne łamanie linii,
- nie wczytuj danych od użytkownika.

12.4 Poziom 2 - Wejście i wyjście

12.4.1 Zadanie 5. Dane użytkownika

Napisz program, który pyta użytkownika o imię i wiek, a następnie wypisuje zdanie:

Czesc, Jan. Masz 20 lat.

Scenariusz sprawdzenia:

```
Dane wejsciowe:
Jan
20
```

Oczekiwany fragment wyniku:

Czesc, Jan. Masz 20 lat.

12.4.2 Zadanie 6. Imię i nazwisko

Napisz program, który wczytuje imię i nazwisko w jednej linii przy pomocy `std::getline`, a następnie wypisuje je na ekran.

Wskazówka: jeśli wcześniej używałeś `std::cin >>`, przed `std::getline` może być potrzebne oczyszczenie znaku końca linii. W tym zadaniu najlepiej użyć tylko `std::getline`.

12.4.3 Zadanie 7. Krótki formularz

Napisz program, który wczytuje:

- imię,
- miasto,
- nazwę szkoły.

Następnie wypisuje jedno podsumowanie:

Jan mieszka w Gdansk i uczy sie w Technikum nr 1.

12.4.4 Zadanie 8. Argumenty programu

Napisz program, który wypisuje wszystkie argumenty przekazane przy uruchomieniu programu.

Przykład uruchomienia:

```
./program Ala ma kota
```

Przykładowy wynik:

Liczba argumentow: 4

0: ./program

1: Ala

2: ma

3: kota

12.5 Poziom 3 - Typy, zmienne i operatory

12.5.1 Zadanie 9. Dwie liczby

Napisz program, który wczytuje dwie liczby całkowite i wypisuje:

- sumę,
- różnicę,
- iloczyn,
- wynik dzielenia całkowitego,
- resztę z dzielenia.

Scenariusz sprawdzenia:

Dane wejsciowe:

17

5

Oczekiwane wyniki:

Suma: 22

Roznica: 12

Iloczyn: 85

Dzielenie calkowite: 3

Reszta: 2

12.5.2 Zadanie 10. Pole prostokąta

Napisz program, który oblicza pole prostokąta. Długości boków *a* i *b* użytkownik podaje z klawiatury. Użyj typu `double`.

Wynik wypisz z dokładnością do dwóch miejsc po przecinku.

12.5.3 Zadanie 11. Pole i obwód koła

Napisz program, który wczytuje promień koła i oblicza:

- pole koła,

- obwód koła.

Przyjmij:

```
const double PI = 3.141592653589793;
```

Wyniki wypisz z dokładnością do dwóch miejsc po przecinku.

12.5.4 Zadanie 12. BMI

Napisz kalkulator BMI.

Użytkownik podaje:

- masę ciała w kilogramach,
- wzrost w metrach.

Wzór:

```
BMI = masa / (wzrost * wzrost)
```

Program powinien wypisać wartość BMI z dokładnością do dwóch miejsc po przecinku.

Scenariusz sprawdzenia:

Dane wejściowe:

80

2

Oczekiwany wynik:

BMI: 20.00

12.5.5 Zadanie 13. Pierwiastek i potęga

Napisz program, który wczytuje liczbę `x` typu `double`, a następnie wypisuje:

- pierwiastek kwadratowy z `x`,
- kwadrat liczby `x`,
- sześćcian liczby `x`.

Użyj funkcji `std::sqrt` i `std::pow` z biblioteki `cmath`.

12.5.6 Zadanie 14. Czas w sekundach

Napisz program, który wczytuje liczbę sekund jako liczbę całkowitą, a następnie przelicza ją na godziny, minuty i sekundy.

Przykład:

Dane wejściowe:

3671

Oczekiwany wynik:

1 h 1 min 11 s

Wymagania:

- użyj dzielenia całkowitego,
- użyj operatora reszty z dzielenia `%`,
- nie używaj instrukcji warunkowych.

12.6 Poziom 4 - Formatowanie wyjścia

12.6.1 Zadanie 15. Liczba PI

Napisz program, który wypisuje liczbę PI:

- z dokładnością do dwóch miejsc po przecinku,
- z dokładnością do pięciu miejsc po przecinku,
- z dokładnością do dziesięciu miejsc po przecinku.

12.6.2 Zadanie 16. Prosta tabela

Napisz program, który wypisuje tabelę produktów:

Produkt	Liczba sztuk	Cena
Zeszyt	3	4.50
Długoopis	1	6.99
Linijka	2	2.25

Użyj `std::setw`, aby wyrównać kolumny.

Wymagania:

- użyj `std::left` dla nazw produktów,
- użyj `std::right` dla liczb,
- ceny wypisz z dokładnością do dwóch miejsc po przecinku.

12.6.3 Zadanie 17. Numer z zerami

Napisz program, który przechowuje numer zamówienia jako liczbę całkowitą, na przykład 123, i wypisuje go w formacie:

```
00000123
```

Użyj `std::setw` i `std::setfill`.

12.6.4 Zadanie 18. Paragon

Napisz program, który wczytuje:

- nazwę produktu,
- cenę jednostkową,
- liczbę sztuk.

Następnie wypisuje prosty paragon:

Produkt	Cena	Ilosc	Wartosc
Zeszyt	4.50	3	13.50

Wymagania:

- użyj `std::setw`,
- wynik wartości oblicz jako `cena * liczbaSztuk`,
- ceny i wartość wypisz z dokładnością do dwóch miejsc po przecinku.

12.7 Poziom 5 - Zadania dodatkowe

12.7.1 Zadanie 19. Palindrom tekstowy

Napisz program, który sprawdza, czy tekst przekazany jako argument programu jest palindromem.

Na tym etapie możesz założyć, że tekst:

- nie zawiera spacji,
- składa się z małych liter,
- nie zawiera polskich znaków.

Przykład:

```
./program kajak
```

Wynik:

To jest palindrom.

12.7.2 Zadanie 20. Prosty kalkulator argumentów

Napisz program, który przyjmuje przez argumenty programu dwie liczby całkowite i wypisuje ich sumę.

Przykład:

```
./program 12 8
```

Wynik:

Suma: 20

Wskazówka: zamiana tekstu na liczbę wymaga funkcji takiej jak `std::stoi`.

12.7.3 Zadanie 21. Liczby skojarzone

Dwie różne liczby całkowite a i b większe od 1 nazwiemy skojarzonymi, jeśli:

- suma wszystkich dodatnich dzielników a mniejszych od a jest równa $b + 1$,
- suma wszystkich dodatnich dzielników b mniejszych od b jest równa $a + 1$.

Napisz program, który sprawdza, czy dwie liczby podane przez użytkownika są skojarzone.

Przykład: liczby 140 i 195 są skojarzone.

12.8 Wariant minimum

Do zaliczenia segmentu wykonaj co najmniej:

1. Zadanie 1 albo 2.
2. Zadanie 5 albo 6.
3. Zadanie 9.
4. Zadanie 10 albo 11.
5. Zadanie 15 albo 16.
6. Mini-sprawdzian segmentu z końca pliku.

Zadania z poziomu 5 są dodatkowe. Możesz je robić po wykonaniu wariantu minimum.

12.9 Zadania przeniesione na później

Niektóre stare zadania z materiałów źródłowych dotyczą tematów, które pojawiają się później:

- tablice,
- struktury,
- stos,
- ONP,
- sortowanie,
- macierze.

Ich oryginalne wersje są zachowane w katalogu archive. Wróć w segmentach o funkcjach, tablicach, STL i OOP.

12.10 Kryteria zaliczenia segmentu

Student zalicza segment, jeśli potrafi samodzielnie:

- napisać program z funkcją `main`,
- skompilować i uruchomić program,

- użyć `std::cin`, `std::cout` i `std::getline`,
- dobrać typy `int`, `double`, `bool`, `char`, `std::string`,
- wykonać podstawowe obliczenia,
- użyć `std::setprecision`, `std::fixed`, `std::setw` i `std::setfill`,
- przygotować czytelny wynik w konsoli.

12.11 Mini-sprawdzian segmentu

Przed przejściem dalej student powinien samodzielnie napisać mały program, który:

- pyta użytkownika o nazwę produktu, cenę i liczbę sztuk,
- oblicza wartość zamówienia,
- wypisuje wynik z dokładnością do dwóch miejsc po przecinku,
- używa czytelnych nazw zmiennych,
- kompiluje się z flagami `-Wall -Wextra -pedantic`.

Scenariusz sprawdzenia:

Dane wejściowe:

Zeszyt

4.5

3

Oczekiwany wynik:

Produkt: Zeszyt

Wartosc zamowienia: 13.50

12.12 Checklista oddania

Przed oddaniem rozwiązań sprawdź:

- ☐ Każde zadanie jest w osobnym pliku `.cpp`.
- ☐ Każdy program kompiluje się z `-Wall -Wextra -pedantic`.
- ☐ Programy nie wymagają plików wynikowych zapisanych w repozytorium.
- ☐ Wyniki są opisane, a nie wypisane jako same liczby.
- ☐ W zadaniach z `double` wynik ma wymaganą liczbę miejsc po przecinku.
- ☐ W zadaniach z wejściem przetestowano dane z treści zadania.

Rozdział 13

02 - Sterowanie i pętle

Ten segment uczy sterowania przepływem programu: podejmowania decyzji, powtarzania instrukcji i zapisu prostych algorytmów.

13.1 Cele segmentu

Po zakończeniu segmentu student powinien umieć:

- używać instrukcji `if`, `else if` i `else`,
- używać instrukcji `switch`,
- rozumieć operatory porównania i operatory logiczne,
- używać operatora warunkowego `?:`,
- pisać pętle `for`, `while` i `do while`,
- dobrać rodzaj pętli do problemu,
- używać `break` i `continue` w prostych przypadkach,
- rozwiązywać zadania wymagające warunków i powtórzeń.

13.2 Docelowa kolejność lekcji

1. 01-instrukcje-warunkowe.md - `if`, `else`, porównania, operatory logiczne.
2. 02-switch-i-operator-warunkowy.md - `switch` oraz `?:`.
3. 03-petla-for.md - klasyczna pętla `for` i typowe schematy iteracji.
4. 04-petle-while-do-while.md - pętle zależne od warunku.
5. 05-break-continue-i-zagniezdzenia.md - sterowanie wewnątrz pętli i pętle zagnieżdżone.
6. 06-zadania.md - zadania podstawowe i dodatkowe.
7. 07-cwiczenie-min-max-srednia.md - większe ćwiczenie z pętli, walidacji i wyniku częściowego.

13.3 Status porządkowania

Lekcje i zadania w tym segmencie są uporządkowane w docelowym formacie. Materiały źródłowe zostały przeniesione do archive po wchłonięciu ich treści do lekcji i zadań.

13.4 Format lekcji

Każda docelowa lekcja powinna mieć taki układ:

1. Cel lekcji.
2. Wymagania wstępne.
3. Krótka teoria.
4. Przykład kodu.
5. Zadania do wykonania.
6. Kryteria zaliczenia.

13.5 Przykłady kodu

Katalog `examples` zawiera krótkie programy pokazujące instrukcje warunkowe, `switch`, pętle oraz użycie `break` i `continue`. Przykłady są kompilowalne i mogą być punktem startowym do zadań.

Do większego ćwiczenia z minimum, maksimum i średnią przydatny jest przykład `examples/while__min__max__average.cpp`.

13.6 Testy

Katalog `tests` zawiera prosty test bez zewnętrznego frameworka. Test sprawdza warunki logiczne, `switch`, sumowanie w pętli `for`, pracę pętli `while`, przerwanie pętli przez `break` oraz pomijanie elementów przez `continue`.

13.7 Katalogi pomocnicze

- `examples/` - kompilowalne przykłady `.cpp` dla tego segmentu.
- `tests/` - proste testy automatyczne dla sterowania przepływem programu.
- `archive/` - stare wersje plików po wchłonięciu ich treści do nowych lekcji.

Rozdział 14

01 - Instrukcje warunkowe

14.1 Cel lekcji

Celem lekcji jest nauczenie zapisu decyzji w programie przy pomocy instrukcji `if`, `else if` i `else`.

14.2 Wymagania wstępne

Student powinien znać podstawy z segmentu 01-podstawy: zmienne, typy danych, operatory arytmetyczne oraz wejście i wyjście.

14.3 Krótka teoria

14.3.1 Po co są instrukcje warunkowe

Instrukcja warunkowa pozwala wykonać fragment kodu tylko wtedy, gdy spełniony jest określony warunek.

Przykład:

```
if (liczba > 0)
{
    std::cout << "Liczba jest dodatnia" << std::endl;
}
```

Kod w nawiasach klamrowych wykona się tylko wtedy, gdy `liczba > 0`.

14.3.2 Operatory porównania

Operator	Znaczenie
<code>==</code>	równe
<code>!=</code>	różne
<code><</code>	mniejsze
<code>></code>	większe
<code><=</code>	mniejsze lub równe
<code>>=</code>	większe lub równe

Uwaga: `==` porównuje wartości, `a =` przypisuje wartość.

```
if (a == 5) // porównanie
{
    std::cout << "a jest równe 5" << std::endl;
}
```

```
a = 5; // przypisanie
```

14.4 Przykład kodu: if i else

```
#include <iostream>

int main()
{
    int liczba;

    std::cout << "Podaj liczbę: ";
    std::cin >> liczba;

    if (liczba >= 0)
    {
        std::cout << "Liczba jest nieujemna" << std::endl;
    }
    else
    {
        std::cout << "Liczba jest ujemna" << std::endl;
    }

    return 0;
}
```

Ten przykład znajduje się w pliku examples/if_positive_negative.cpp.

14.4.1 else if

Jeżeli mamy więcej niż dwa przypadki, używamy else if.

```
if (liczba > 0)
{
    std::cout << "Liczba jest dodatnia" << std::endl;
}
else if (liczba < 0)
{
    std::cout << "Liczba jest ujemna" << std::endl;
}
else
{
    std::cout << "Liczba jest równa zero" << std::endl;
}
```

Warunki są sprawdzane od góry do dołu. Wykonuje się pierwszy pasujący blok.

14.4.2 Operatory logiczne

Warunki można łączyć operatorami logicznymi.

Operator	Znaczenie	Przykład
&&	i	wiek >= 18 && wiek < 65
	lub	liczba < -10 liczba > 10
!	negacja	!czyAktywny

Przykład:

```
if (liczba >= -10 && liczba <= 13)
{
    std::cout << "Liczba jest w przedziale <-10, 13>" << std::endl;
}
```

Pełny przykład znajduje się w pliku `examples/range_check.cpp`.

14.5 Typowy błąd: średnik po `if`

Nie stawiamy średnika bezpośrednio po warunku `if`.

Niepoprawny kod:

```
if (a == 1);
{
    std::cout << a << std::endl;
}
```

Ten kod się skompiluje, ale blok w nawiasach klamrowych wykona się zawsze. Średnik kończy pustą instrukcję warunkową.

Poprawny kod:

```
if (a == 1)
{
    std::cout << a << std::endl;
}
```

14.6 Zadania do wykonania

1. Napisz program, który sprawdza, czy liczba jest dodatnia, ujemna czy równa zero.
2. Napisz program, który sprawdza, czy liczba jest parzysta.
3. Napisz program, który sprawdza, czy trzy liczby są podane w kolejności rosnącej.
4. Napisz program, który sprawdza, czy liczba należy do przedziału `<-10, 13>`.
5. Napisz program, który sprawdza, czy liczba należy do jednego z przedziałów: `<-10, 13>` albo `<25, 35>`.

14.7 Ćwiczenia dodatkowe

1. Dodaj drugi warunek sprawdzający, czy liczba jest parzysta.
2. Zmień program tak, aby obsługiwał zakres od `-100` do `100`.
3. Dodaj komunikat dla wartości granicznych, np. `0`, `100` i `-100`.

14.8 Kryteria zaliczenia

Program powinien:

- używać `if`, `else if` albo `else` tam, gdzie jest to uzasadnione,
- stosować poprawne operatory porównania,
- stosować operatory logiczne dla warunków złożonych,
- kompilować się bez błędów,
- wypisywać czytelną informację dla użytkownika.

14.9 Pytania kontrolne

1. Czym różni się `=` od `==`?
2. Kiedy używamy `else if`?
3. Co oznacza operator `&&`?
4. Co oznacza operator `||`?
5. Dlaczego średnik po `if` (`warunek`); jest błędem logicznym?

Rozdział 15

02 - switch i operator warunkowy

15.1 Cel lekcji

Celem lekcji jest poznanie dwóch dodatkowych sposobów zapisu decyzji w programie: instrukcji `switch` oraz operatora warunkowego `?:`.

15.2 Wymagania wstępne

Student powinien znać materiał z lekcji 01-instrukcje-warunkowe.md: `if`, `else`, operatory porównania i operatory logiczne.

15.3 Krótka teoria

15.3.1 Instrukcja `switch`

Instrukcja `switch` pozwala wybrać jeden z wielu przypadków na podstawie wartości jednej zmiennej lub wyrażenia.

Najczęściej używamy jej, gdy porównujemy jedną wartość z kilkoma stałymi.

```
switch (wybor)
{
    case 1:
        std::cout << "Wybrano 1" << std::endl;
        break;
    case 2:
        std::cout << "Wybrano 2" << std::endl;
        break;
    default:
        std::cout << "Nieznana opcja" << std::endl;
        break;
}
```

Znaczenie elementów:

- `switch (wybor)` sprawdza wartość zmiennej `wybor`,
- `case` oznacza konkretny przypadek,
- `break` kończy wykonywanie instrukcji `switch`,
- `default` obsługuje sytuację, gdy żaden `case` nie pasuje.

15.4 Przykład kodu: proste menu

```
#include <iostream>

int main()
{
    int wybor;

    std::cout << "1. Nowa gra\n";
    std::cout << "2. Wczytaj gre\n";
    std::cout << "3. Wyjście\n";
    std::cout << "Wybor: ";
    std::cin >> wybor;

    switch (wybor)
    {
        case 1:
            std::cout << "Rozpoczynam nowa gre" << std::endl;
            break;
        case 2:
            std::cout << "Wczytuje gre" << std::endl;
            break;
        case 3:
            std::cout << "Koniec programu" << std::endl;
            break;
        default:
            std::cout << "Nieznana opcja" << std::endl;
            break;
    }

    return 0;
}
```

Ten przykład znajduje się w pliku `examples/switch_menu.cpp`.

15.4.1 Dlaczego `break` jest ważny

Jeżeli pominiemy `break`, program przejdzie do kolejnego przypadku. Takie zachowanie nazywa się przejściem dalej przez przypadki.

```
switch (wybor)
{
    case 1:
        std::cout << "Jeden" << std::endl;
    case 2:
        std::cout << "Dwa" << std::endl;
        break;
}
```

Dla `wybor == 1` program wypisze zarówno `Jeden`, jak i `Dwa`.

Czasem jest to celowe, ale na początku nauki zwykle oznacza błąd.

15.4.2 Kiedy użyć `switch`, a kiedy `if`

`switch` jest dobrym wyborem, gdy:

- sprawdzamy jedną zmienną,
- porównujemy ją z konkretnymi wartościami,
- przypadków jest kilka.

`if` jest lepszy, gdy:

- warunek dotyczy przedziału, np. `x >= 10 && x <= 20`,
- warunek łączy kilka zmiennych,
- potrzebne są operatory logiczne,
- sprawdzamy wartości zmiennoprzecinkowe.

15.4.3 Operator warunkowy ?:

Operator warunkowy pozwala zapisać prostą decyzję w jednym wyrażeniu.

Składnia:

`warunek ? wartosc_gdy_prawda : wartosc_gdy_falsz`

Przykład:

```
int a = 10;
int b = 20;
```

```
int wieksza = (a > b) ? a : b;
```

Jeżeli `a > b`, zmienna `wieksza` otrzyma wartość `a`. W przeciwnym razie otrzyma wartość `b`.

15.5 Przykład kodu: parzystość liczby

```
#include <iostream>
#include <string>

int main()
{
    int liczba;

    std::cout << "Podaj liczbę: ";
    std::cin >> liczba;

    std::string wynik = (liczba % 2 == 0) ? "parzysta" : "nieparzysta";
    std::cout << "Liczba jest " << wynik << std::endl;

    return 0;
}
```

Ten przykład znajduje się w pliku `examples/ternary_even_odd.cpp`.

15.6 Zadania do wykonania

1. Napisz program z menu kalkulatora:
 - 1 - dodawanie,
 - 2 - odejmowanie,
 - 3 - mnożenie,
 - 4 - dzielenie.
2. Napisz program, który dla numeru dnia tygodnia od 1 do 7 wypisuje nazwę dnia.
3. Napisz program, który dla oceny od 1 do 6 wypisuje komunikat tekstowy, np. *bardzo dobry*.
4. Napisz program, który przy pomocy operatora `?:` wybiera większą z dwóch liczb.
5. Napisz program, który przy pomocy operatora `?:` wypisuje, czy użytkownik jest pełnoletni.

15.7 Ćwiczenia dodatkowe

1. Dodaj do menu nową opcję i obsłuż ją w `switch`.
2. Zamień prosty `if else` na operator warunkowy tam, gdzie poprawia czytelność.
3. Dodaj przypadek `default` z czytelnym komunikatem o błędnym wyborze.

15.8 Kryteria zaliczenia

Program powinien:

- poprawnie używać `switch`, `case`, `break` i `default`,
- używać `switch` tylko wtedy, gdy pasuje do problemu,
- stosować operator `?:` tylko do krótkich, czytelnych decyzji,
- kompilować się bez błędów,
- obsługiwać niepoprawny wybór użytkownika.

15.9 Pytania kontrolne

1. Do czego służy `default` w instrukcji `switch`?
2. Co się stanie, gdy pominiemy `break`?
3. Kiedy `switch` jest czytelniejszy niż `if`?
4. Jak działa operator `?:`?
5. Dlaczego operator `?:` nie należy nadużywać?

Rozdział 16

03 - Pętla for

16.1 Cel lekcji

Celem lekcji jest nauczenie klasycznej pętli `for`, czyli konstrukcji używanej wtedy, gdy z góry wiemy, ile razy chcemy powtórzyć instrukcje.

16.2 Wymagania wstępne

Student powinien znać:

- 01-instrukcje-warunkowe.md,
- 02-switch-i-operator-warunkowy.md,
- podstawowe operatory arytmetyczne z segmentu 01-podstawy.

16.3 Krótka teoria

16.3.1 Składnia pętli for

```
for (inicjalizacja; warunek; zmiana)
{
    // instrukcje wykonywane w pętli
}
```

Najczęstszy przykład:

```
for (int i = 0; i < 10; i++)
{
    std::cout << i << std::endl;
}
```

Znaczenie elementów:

- `int i = 0` - utworzenie licznika pętli,
- `i < 10` - warunek kontynuowania pętli,
- `i++` - zwiększenie licznika po każdym obiegu.

16.3.2 Kolejność działania

Dla pętli:

```
for (int i = 1; i <= 3; i++)
{
    std::cout << i << std::endl;
}
```

Program działa tak:

1. Tworzy zmienną `i` i nadaje jej wartość 1.
2. Sprawdza warunek `i <= 3`.
3. Wykonuje ciało pętli.
4. Wykonuje `i++`.
5. Wraca do sprawdzenia warunku.

Wynik:

```
1
2
3
```

Ten przykład znajduje się w pliku `examples/for_count.cpp`.

16.3.3 Pętla z innym krokiem

Licznik nie musi zwiększać się o 1.

```
for (int i = 0; i <= 20; i += 2)
{
    std::cout << i << std::endl;
}
```

Ta pętla wypisuje liczby parzyste od 0 do 20.

16.3.4 Pętla malejąca

Pętla może też odliczać w dół.

```
for (int i = 10; i >= 1; i--)
{
    std::cout << i << std::endl;
}
```

16.4 Przykład kodu: suma liczb

Pętla `for` często służy do obliczania sumy.

```
int suma = 0;

for (int i = 1; i <= 100; i++)
{
    suma += i;
}

std::cout << "Suma: " << suma << std::endl;
```

Pełny przykład znajduje się w pliku `examples/for_sum.cpp`.

16.5 Przykład kodu: pętla i warunek

Wewnątrz pętli można używać instrukcji `if`.

```
for (int i = 1; i <= 20; i++)
{
    if (i % 2 == 0)
    {
        std::cout << i << std::endl;
    }
}
```

Ten kod wypisuje liczby parzyste od 1 do 20.

Pełny przykład znajduje się w pliku `examples/for_even_numbers.cpp`.

16.6 Typowe błędy

16.6.1 Zły warunek końca

```
for (int i = 0; i <= 10; i++)
```

Ta pętla wykona się 11 razy, bo obejmuje wartości od 0 do 10.

16.6.2 Brak zmiany licznika

```
for (int i = 0; i < 10;)
{
    std::cout << i << std::endl;
}
```

Jeżeli licznik się nie zmienia, pętla może nigdy się nie skończyć.

16.6.3 Użycie licznika poza zakresem

```
for (int i = 0; i < 10; i++)
{
    std::cout << i << std::endl;
}
```

// tutaj i nie jest już dostępne

Zmienna `i` zadeklarowana w pętli istnieje tylko w tej pętli.

16.7 Zadania do wykonania

1. Napisz program, który za pomocą pętli `for` wypisuje liczby od 1 do 110.
2. Napisz program, który za pomocą pętli `for` wypisuje liczby od 100 do 1.
3. Napisz program, który wypisuje liczby parzyste od 0 do 20.
4. Napisz program, który wypisuje liczby podzielne przez 3 z przedziału od 1 do 100.
5. Napisz program, który sumuje liczby całkowite od 1 do 110.
6. Napisz program, który sumuje liczby parzyste od 1 do 110.
7. Napisz program, który oblicza silnię liczby `n` podanej przez użytkownika.
8. Napisz program, który wypisuje litery alfabetu od `a` do `z`.

16.8 Ćwiczenia dodatkowe

1. Zmień zakres pętli tak, aby wypisywała liczby od 10 do 1.
2. Dodaj sumowanie tylko liczb parzystych.
3. Wypisz przy każdej iteracji aktualną wartość licznika i sumy.

16.9 Kryteria zaliczenia

Program powinien:

- poprawnie używać składni `for` (inicjalizacja; warunek; zmiana),
- kończyć pętlę po oczekiwanej liczbie obiegów,
- stosować czytelne nazwy zmiennych pomocniczych,
- kompilować się bez błędów,
- wypisywać wynik w czytelnej formie.

16.10 Pytania kontrolne

1. Z jakich trzech części składa się nagłówek pętli `for`?
2. Kiedy najczęściej wybieramy pętlę `for`?
3. Ile razy wykona się pętla `for (int i = 0; i < 10; i++)`?
4. Czym różni się `i++` od `i += 2` w nagłówku pętli?
5. Co może się stać, jeśli licznik pętli nigdy nie zmieni wartości?

Rozdział 17

04 - Pętle while i do while

17.1 Cel lekcji

Celem lekcji jest nauczenie pętli zależnych od warunku: `while` i `do while`.

17.2 Wymagania wstępne

Student powinien znać:

- 01-instrukcje-warunkowe.md,
- 03-pętla-for.md,
- podstawowe operatory porównania i logiczne.

17.3 Krótka teoria

17.3.1 Pętla while

Pętla `while` wykonuje instrukcje tak długo, jak długo warunek jest prawdziwy.

Składnia:

```
while (warunek)
{
    // instrukcje
}
```

Przykład:

```
int i = 1;

while (i <= 10)
{
    std::cout << i << std::endl;
    i++;
}
```

Pętla `while` najpierw sprawdza warunek, a dopiero potem wykonuje ciało pętli. Jeżeli warunek od początku jest fałszywy, pętla nie wykona się ani razu.

Pełny przykład znajduje się w pliku `examples/while_count.cpp`.

17.3.2 Kiedy używać while

Pętla `while` jest dobrym wyborem, gdy nie wiemy z góry, ile razy pętla ma się wykonać.

Przykłady:

- program czeka, aż użytkownik poda poprawną wartość,
- program wykonuje menu do momentu wyboru opcji wyjścia,
- program przetwarza dane, dopóki spełniony jest warunek.

17.4 Przykład kodu: suma do zera

```
int liczba;
int suma = 0;

std::cout << "Podaj liczbę, 0 kończy program: ";
std::cin >> liczba;

while (liczba != 0)
{
    suma += liczba;

    std::cout << "Podaj liczbę, 0 kończy program: ";
    std::cin >> liczba;
}

std::cout << "Suma: " << suma << std::endl;
```

Pełny przykład znajduje się w pliku `examples/while_sum_until_zero.cpp`.

17.4.1 Pętla do while

Pętla `do while` najpierw wykonuje ciało pętli, a dopiero potem sprawdza warunek.

Składnia:

```
do
{
    // instrukcje
}
while (warunek);
```

Uwaga: po `while (warunek)` na końcu pętli `do while` stawiamy średnik.

Przykład:

```
int wybor;

do
{
    std::cout << "1. Start\n";
    std::cout << "0. Wyjście\n";
    std::cout << "Wybor: ";
    std::cin >> wybor;
}
while (wybor != 0);
```

Pełny przykład znajduje się w pliku `examples/do_while_menu.cpp`.

17.4.2 Różnica między while i do while

`while` może nie wykonać się ani razu:

```
int i = 10;

while (i < 5)
{
```



```

    std::cout << i << std::endl;
}
do while wykona się co najmniej raz:
int i = 10;

do
{
    std::cout << i << std::endl;
}
while (i < 5);

```

17.4.3 Pętla nieskończona

Pętla nieskończona powstaje wtedy, gdy warunek nigdy nie staje się fałszywy.

```

int i = 1;

while (i <= 10)
{
    std::cout << i << std::endl;
    // brak i++
}

```

W tym przykładzie *i* nigdy się nie zmienia, więc warunek *i <= 10* pozostaje prawdziwy.

17.5 Zadania do wykonania

1. Napisz program, który za pomocą `while` wypisuje liczby od 1 do 110.
2. Napisz program, który za pomocą `while` wypisuje liczby od 100 do 1.
3. Napisz program, który za pomocą `while` sumuje liczby od 1 do 10.
4. Napisz program, który za pomocą `do while` wypisuje liczby parzyste od 0 do 20.
5. Napisz program, który za pomocą `do while` oblicza iloczyn liczb od 1 do 5.
6. Napisz program, który wczytuje liczby od użytkownika aż do podania 0, a następnie wypisuje ich sumę.
7. Napisz program z menu działającym w pętli `do while`, gdzie opcja 0 kończy program.

17.6 Ćwiczenia dodatkowe

1. Dodaj licznik prób w pętli `while`.
2. Zmień menu `do while`, aby kończyło pracę po wyborze 0.
3. Dodaj obsługę błędnej wartości bez przerywania programu.

17.7 Kryteria zaliczenia

Program powinien:

- poprawnie używać pętli `while` lub `do while`,
- mieć warunek kończący pętlę,
- aktualizować zmienne używane w warunku,
- kompilować się bez błędów,
- wypisywać wynik w czytelnej formie.

17.8 Pytania kontrolne

1. Czym różni się `while` od `do while`?
2. Kiedy pętla `while` może nie wykonać się ani razu?
3. Dlaczego `do while` dobrze pasuje do menu programu?

4. Co powoduje pętlę nieskończoną?
5. Dlaczego po `while` (warunek) w pętli do `while` stawiamy średnik?

Rozdział 18

05 - break, continue i pętle zagnieżdżone

18.1 Cel lekcji

Celem lekcji jest poznanie sterowania wykonaniem pętli za pomocą `break` i `continue` oraz używania pętli zagnieżdżonych.

18.2 Wymagania wstępne

Student powinien znać:

- 03-petla-for.md,
- 04-petle-while-do-while.md,
- instrukcje warunkowe z lekcji 01-instrukcje-warunkowe.md.

18.3 Krótka teoria

18.3.1 Instrukcja break

Instrukcja `break` natychmiast kończy najbliższą pętlę.

Przykład:

```
for (int i = 1; i <= 10; i++)
{
    if (i == 5)
    {
        break;
    }

    std::cout << i << std::endl;
}
```

Wynik:

```
1
2
3
4
```

Gdy `i` ma wartość 5, pętla kończy działanie.

18.4 Przykład kodu: szukanie liczby

```
int szukana = 7;
bool znaleziono = false;

for (int i = 1; i <= 10; i++)
{
    if (i == szukana)
    {
        znaleziono = true;
        break;
    }
}
```

Pełny przykład znajduje się w pliku `examples/break_search.cpp`.

18.4.1 Instrukcja `continue`

Instrukcja `continue` pomija resztę aktualnego obiegu pętli i przechodzi do następnego.

Przykład:

```
for (int i = 1; i <= 10; i++)
{
    if (i % 2 == 0)
    {
        continue;
    }

    std::cout << i << std::endl;
}
```

Ten kod wypisze tylko liczby nieparzyste. Gdy liczba jest parzysta, `continue` pomija `std::cout`.

Pełny przykład znajduje się w pliku `examples/continue_skip_even.cpp`.

18.4.2 Kiedy używać `break` i `continue`

`break` jest przydatny, gdy:

- znaleźliśmy szukany element,
- dalsze wykonywanie pętli nie ma sensu,
- użytkownik wybrał zakończenie działania.

`continue` jest przydatne, gdy:

- chcemy pominąć niektóre dane,
- dalsze instrukcje w aktualnym obiegu nie powinny się wykonać,
- chcemy uprościć warunki wewnątrz pętli.

Nie należy nadużywać `break` i `continue`. Jeśli pętla staje się trudna do czytania, warto uprościć warunki albo podzielić kod na funkcje.

18.4.3 Pętle zagnieżdżone

Pętla zagnieżdżona to pętla umieszczona wewnątrz innej pętli.

```
for (int i = 1; i <= 3; i++)
{
    for (int j = 1; j <= 3; j++)
    {
        std::cout << i << " " << j << std::endl;
    }
}
```

Dla każdej wartości *i* wewnętrzna pętla przechodzi przez wszystkie wartości *j*.

18.5 Przykład kodu: tabliczka mnożenia

```
for (int row = 1; row <= 10; row++)
{
    for (int col = 1; col <= 10; col++)
    {
        std::cout << row * col << '\t';
    }

    std::cout << std::endl;
}
```

Pełny przykład znajduje się w pliku `examples/nested_multiplication_table.cpp`.

18.5.1 break w pętli zagnieżdżonej

`break` kończy tylko najbliższą pętlę.

```
for (int i = 1; i <= 3; i++)
{
    for (int j = 1; j <= 3; j++)
    {
        if (j == 2)
        {
            break;
        }
    }
}
```

W tym przykładzie `break` kończy pętlę po *j*, ale pętla po *i* działa dalej.

18.6 Zadania do wykonania

1. Napisz program, który sprawdza liczby od 1 do 100 i kończy pętlę, gdy znajdzie pierwszą liczbę podzieloną przez 17.
2. Napisz program, który wypisuje liczby od 1 do 50, pomijając liczby podzielne przez 5.
3. Napisz program, który wczytuje liczby od użytkownika i kończy działanie po wpisaniu liczby ujemnej.
4. Napisz program, który wypisuje tabliczkę mnożenia od 1 do 10.
5. Napisz program, który wypisuje prostokąt z gwiazdek o wymiarach podanych przez użytkownika.
6. Napisz program, który wypisuje trójkąt z gwiazdek o wysokości podanej przez użytkownika.

18.7 Ćwiczenia dodatkowe

1. Dodaj warunek, który używa `continue` do pomijania liczb ujemnych.
2. Zmień przykład z `break`, aby kończył wyszukiwanie po znalezieniu dwóch wyników.
3. Rozbuduj pętlę zagnieżdżoną o nagłówki wierszy i kolumn.

18.8 Kryteria zaliczenia

Program powinien:

- poprawnie używać `break` do zakończenia pętli,
- poprawnie używać `continue` do pomijania obiegu pętli,
- stosować pętle zagnieżdżone tam, gdzie problem ma dwa wymiary,
- kompilować się bez błędów,

- wypisywać wynik w czytelnej formie.

18.9 Pytania kontrolne

1. Co robi `break` w pętli?
2. Co robi `continue` w pętli?
3. Czy `break` kończy wszystkie pętle zagnieżdżone?
4. Kiedy warto użyć pętli zagnieżdżonej?
5. Dlaczego nie należy nadużywać `break` i `continue`?

Rozdział 19

06 - Zadania

19.1 Cel zestawu zadań

Ten plik zbiera zadania do segmentu 02 - Sterowanie i pętle. Zadania sprawdzają instrukcje warunkowe, switch, operator ?:, pętle oraz sterowanie wykonaniem pętli.

19.2 Zasady wykonania

Każde zadanie powinno być zapisane w osobnym pliku .cpp. Program powinien kompilować się bez błędów i wypisywać wynik w czytelnej formie.

Przykład kompilacji:

```
c++ -std=c++17 -Wall -Wextra -pedantic nazwa_pliku.cpp -o program  
./program
```

Przed oddaniem zadania sprawdź:

- czy program kończy się dla wszystkich typowych danych wejściowych,
- czy warunki obejmują przypadki graniczne,
- czy pętle mają poprawny warunek zakończenia,
- czy wynik jest opisany, a nie wypisany jako sama liczba,
- czy program był uruchomiony dla danych poprawnych i błędnych.

19.3 Poziom 1 - Instrukcje warunkowe

19.3.1 Zadanie 1. Znak liczby

Napisz program, który wczytuje liczbę całkowitą i wypisuje, czy jest:

- dodatnia,
- ujemna,
- równa zero.

Scenariusze sprawdzenia:

Dane: 12
Wynik: dodatnia

Dane: -3
Wynik: ujemna

Dane: 0
Wynik: zero

19.3.2 Zadanie 2. Parzystość

Napisz program, który sprawdza, czy liczba podana przez użytkownika jest parzysta czy nieparzysta.

Wymagania:

- użyj operatora %,
- sprawdź program dla liczby dodatniej, ujemnej i zera.

19.3.3 Zadanie 3. Kolejność rosnąca

Napisz program, który wczytuje trzy liczby całkowite i sprawdza, czy są podane w kolejności rosnącej.

Przykłady:

- 1 2 3 - kolejność rosnąca,
- 1 1 2 - nie jest ściśle rosnąca,
- 3 2 1 - nie jest rosnąca.

19.3.4 Zadanie 4. Przedziały

Napisz program, który sprawdza, czy liczba należy do jednego z przedziałów:

- <-10, 13>,
- <25, 35>.

Sprawdź przypadki graniczne: -10, 13, 14, 25, 35, 36.

19.3.5 Zadanie 5. Klasyfikacja BMI

Rozbuduj kalkulator BMI z segmentu 01-podstawy. Program powinien obliczyć BMI i wypisać kategorię, np. niedowaga, wartość prawidłowa, nadwaga.

Przyjmij proste progi:

- poniżej 18.5 - niedowaga,
- od 18.5 do poniżej 25.0 - wartość prawidłowa,
- od 25.0 do poniżej 30.0 - nadwaga,
- od 30.0 - otyłość.

19.4 Poziom 2 - switch i operator ?:

19.4.1 Zadanie 6. Dzień tygodnia

Napisz program, który dla numeru od 1 do 7 wypisuje nazwę dnia tygodnia. Użyj switch.

Wymagania:

- dla wartości spoza zakresu wypisz komunikat Nieznany dzien,
- użyj gałęzi default.

19.4.2 Zadanie 7. Ocena opisowa

Napisz program, który dla oceny od 1 do 6 wypisuje opis, np. niedostateczny, dobry, bardzo dobry.

19.4.3 Zadanie 8. Prosty kalkulator

Napisz program z menu:

- 1 - dodawanie,
- 2 - odejmowanie,
- 3 - mnożenie,
- 4 - dzielenie.

Użytkownik wybiera operację i podaje dwie liczby.

Wymagania:

- użyj `switch`,
- obsłuż nieznaną opcję,
- przy dzieleniu sprawdź dzielenie przez zero.

Scenariusz sprawdzenia:

Dane:

4
10
0

Oczekiwany wynik:

Nie można dzielić przez zero.

19.4.4 Zadanie 9. Większa liczba

Napisz program, który wczytuje dwie liczby i przy pomocy operatora `?:` wybiera większą.

Jeśli liczby są równe, program może wypisać dowolną z nich jako większą albo osobny komunikat o równości. Opisz to zachowanie w wyniku programu.

19.4.5 Zadanie 10. Pełnoletność

Napisz program, który wczytuje wiek i przy pomocy operatora `?:` wypisuje `pełnoletni` albo `niepełnoletni`.

19.5 Poziom 3 - Pętla `for`

19.5.1 Zadanie 11. Liczby od 1 do 110

Napisz program, który za pomocą pętli `for` wypisuje liczby całkowite od 1 do 110.

Wymagania:

- każdą liczbę wypisz w osobnej linii albo oddziel spacją,
- nie wpisuj liczb ręcznie.

19.5.2 Zadanie 12. Liczby podzielne przez 3

Napisz program, który za pomocą pętli `for` wypisuje liczby podzielne przez 3 z przedziału od 1 do 100.

19.5.3 Zadanie 13. Suma liczb

Napisz program, który za pomocą pętli `for` sumuje liczby całkowite od 1 do 110.

Oczekiwany wynik:

Suma: 6105

19.5.4 Zadanie 14. Suma liczb parzystych

Napisz program, który za pomocą pętli `for` sumuje liczby parzyste od 1 do 110.

19.5.5 Zadanie 15. Silnia

Napisz program, który wczytuje liczbę `n` i oblicza `n!` przy pomocy pętli `for`.

Założenia:

- dla tego zadania przyjmij $0 \leq n \leq 12$,

- dla 0 wynik powinien wynosić 1,
- jeśli użytkownik poda liczbę spoza zakresu, wypisz komunikat o błędzie.

19.6 Poziom 4 - while i do while

19.6.1 Zadanie 16. Odliczanie w dół

Napisz program, który za pomocą pętli `while` wypisuje liczby od 100 do 1.

19.6.2 Zadanie 17. Suma do zera

Napisz program, który wczytuje liczby od użytkownika aż do podania 0, a następnie wypisuje ich sumę.

Scenariusz sprawdzenia:

Dane:

5
-2
10
0

Oczekiwany wynik:

Suma: 13

19.6.3 Zadanie 18. Iloczyn liczb

Napisz program, który za pomocą pętli `do while` oblicza iloczyn liczb od 1 do 5.

19.6.4 Zadanie 19. Menu w pętli

Napisz program z menu działającym w pętli `do while`. Program kończy działanie po wybraniu opcji 0.

Minimalne menu:

1. Wypisz powitanie
2. Wypisz aktualny licznik wyborow
0. Zakoncz

Program powinien obsługiwać nieznana opcję bez zakończenia działania.

19.6.5 Zadanie 20. Potęgi dwójki

Napisz program, który za pomocą pętli `while` wypisuje kolejne potęgi liczby 2, aż do osiągnięcia wartości `m` podanej przez użytkownika.

Przykład dla `m = 20`:

1
2
4
8
16

19.7 Poziom 5 - break, continue i zagnieżdżenia

19.7.1 Zadanie 21. Pierwsza liczba podzielna przez 17

Napisz program, który sprawdza liczby od 1 do 100 i kończy pętlę, gdy znajdzie pierwszą liczbę podzielną przez 17.

Wymagania:

- użyj `break`,

- wypisz znalezioną liczbę,
- nie kontynuuj sprawdzania po znalezieniu wyniku.

19.7.2 Zadanie 22. Pomijanie liczb

Napisz program, który wypisuje liczby od 1 do 50, pomijając liczby podzielne przez 5. Użyj `continue`.
Sprawdź, czy w wyniku nie pojawiają się liczby 5, 10, 15, 20, 25, 30, 35, 40, 45, 50.

19.7.3 Zadanie 23. Tabliczka mnożenia

Napisz program, który za pomocą pętli zagnieżdżonych wypisuje tabliczkę mnożenia od 1 do 10.

Wymagania:

- użyj dwóch pętli,
- wyrównaj kolumny przez `std::setw`,
- pierwszy wiersz powinien zawierać wyniki mnożenia przez 1.

19.7.4 Zadanie 24. Prostokąt z gwiazdek

Napisz program, który wczytuje szerokość i wysokość, a następnie wypisuje prostokąt z gwiazdek.

Przykład dla szerokości 5 i wysokości 3:

```
*****
*****
*****
```

19.7.5 Zadanie 25. Trójkąt z gwiazdek

Napisz program, który wczytuje wysokość, a następnie wypisuje trójkąt z gwiazdek.

Przykład dla wysokości 4:

```
*
**
***
****
```

19.8 Zadania dodatkowe

19.8.1 Zadanie 26. Liczba pierwsza

Napisz program, który sprawdza, czy liczba podana przez użytkownika jest liczbą pierwszą.

Scenariusze sprawdzenia:

- 1 - nie jest liczbą pierwszą,
- 2 - jest liczbą pierwszą,
- 9 - nie jest liczbą pierwszą,
- 17 - jest liczbą pierwszą.

19.8.2 Zadanie 27. Minimum, maksimum i średnia

Napisz program, który wczytuje `n` liczb, a następnie wypisuje:

- najmniejszą liczbę,
- największą liczbę,
- średnią arytmetyczną.

Wymagania:

- najpierw wczytaj `n`,
- jeśli `n <= 0`, wypisz komunikat o błędzie,

- nie używaj tablic.

Rozszerzony wariant tego zadania, z wczytywaniem do momentu wpisania komendy kończącej, znajduje się w osobnym ćwiczeniu 07-cwiczenie-min-max-srednia.md.

19.8.3 Zadanie 28. Liczby trójkątne

Napisz program, który wypisuje wszystkie liczby trójkątne z przedziału od 1 do 20.

Liczby trójkątne obliczaj jako kolejne sumy:

```
1
1 + 2 = 3
1 + 2 + 3 = 6
```

19.9 Wariant minimum

Do zaliczenia segmentu wykonaj co najmniej:

1. Zadanie 1 albo 2.
2. Zadanie 4 albo 5.
3. Zadanie 6 albo 8.
4. Zadanie 13 albo 15.
5. Zadanie 17 albo 19.
6. Zadanie 21 albo 22.
7. Mini-sprawdzian segmentu z końca pliku.

Zadania 26-28 są dodatkowe. Możesz je robić po wykonaniu wariantu minimum.

19.10 Zadania przeniesione na później

Oryginalne materiały zawierały także zadania, które lepiej pasują do późniejszych segmentów:

- tablice,
- struktury,
- większe zadania algorytmiczne,
- zadania z losowaniem danych.

Ich pierwotne wersje są zachowane w katalogu archive.

19.11 Kryteria zaliczenia segmentu

Student zalicza segment, jeśli potrafi samodzielnie:

- zapisać decyzję przy pomocy `if`, `else if`, `else`,
- użyć `switch` dla menu lub wyboru jednej z wielu stałych wartości,
- zastosować operator `?:` w prostym przypadku,
- napisać pętlę `for`, `while` i `do while`,
- uniknąć pętli nieskończonej,
- użyć `break` i `continue` w uzasadnionych sytuacjach,
- zastosować pętlę zagnieżdżoną do problemu dwuwymiarowego.

19.12 Mini-sprawdzian segmentu

Napisz program z menu, który pozwala:

- wczytywać liczby do momentu wyboru opcji zakończenia,
- zliczać liczby dodatnie, ujemne i zera,
- pominąć liczby spoza zakresu od -100 do 100,
- wypisać podsumowanie po zakończeniu,
- skompilować program z `-Wall -Wextra -pedantic`.

Wymagania szczegółowe:

- użyj menu w pętli `do while`,
- użyj `continue` do pomijania liczb spoza zakresu,
- użyj warunków do klasyfikacji liczby,
- zakończ program dopiero po wybraniu opcji zakończenia.

Scenariusz sprawdzenia:

Dane:

1
10
1
-5
1
0
1
150
0

Oczekiwane podsumowanie:

Dodatnie: 1

Ujemne: 1

Zera: 1

Pominiete: 1

19.13 Checklista oddania

Przed oddaniem rozwiązań sprawdź:

- ☐ Każde zadanie jest w osobnym pliku `.cpp`.
- ☐ Programy kompilują się z `-Wall -Wextra -pedantic`.
- ☐ Każda pętla ma jasny warunek zakończenia.
- ☐ Programy obsługują wartości graniczne z treści zadania.
- ☐ Menu obsługuje nieznaną opcję.
- ☐ Dzielenie przez zero jest obsługiwane tam, gdzie może wystąpić.
- ☐ Wyniki są opisane tekstem zrozumiałym dla użytkownika.

Rozdział 20

07 - Ćwiczenie: minimum, maksimum i średnia

20.1 Cel ćwiczenia

Celem ćwiczenia jest napisanie programu konsolowego, który wczytuje liczby rzeczywiste od użytkownika, a następnie wyznacza wartość minimalną, wartość maksymalną oraz średnią arytmetyczną.

Ćwiczenie utrwała pętle zależne od warunku, walidację danych wejściowych, aktualizowanie wyniku częściowego i obsługę przypadku braku danych.

20.2 Wymagania wstępne

Przed wykonaniem ćwiczenia student powinien znać:

- instrukcję `if`, `else if`, `else`,
- pętle `while` albo `do while`,
- typ `double`,
- podstawowe operacje wejścia i wyjścia,
- sprawdzanie poprawności danych wejściowych.

20.3 Opis zadania

Napisz program, który:

- wczytuje kolejne liczby rzeczywiste,
- pozwala zakończyć wprowadzanie danych komendą tekstową,
- oblicza minimum,
- oblicza maksimum,
- oblicza średnią arytmetyczną,
- wyświetla liczbę poprawnie wczytanych wartości,
- obsługuje sytuację, gdy użytkownik nie poda żadnej liczby.

Zamiast obsługi klawisza `ESC` użyj rozwiązania przenośnego: użytkownik wpisuje `q`, `koniec` albo pustą linię, aby zakończyć wprowadzanie danych.

20.4 Przykład działania

```
Podaj liczbe albo q, aby zakonczyc: 10
Podaj liczbe albo q, aby zakonczyc: -2.5
Podaj liczbe albo q, aby zakonczyc: 4.5
Podaj liczbe albo q, aby zakonczyc: q
```

Liczba wartosci: 3
Minimum: -2.5
Maksimum: 10
Srednia: 4

20.5 Wymagania techniczne

Program powinien:

- być napisany w C++17,
- używać pętli `while` albo `do while`,
- przechowywać sumę wartości w zmiennej typu `double`,
- przechowywać licznik poprawnych wartości,
- aktualizować minimum i maksimum po każdym poprawnym odczycie,
- wykrywać błędne dane wejściowe,
- nie dzielić przez zero, gdy nie podano żadnej liczby,
- kompilować się z flagami `-Wall -Wextra -pedantic`.

Kompilowalny punkt startowy znajduje się w `examples/while_min_max_average.cpp`.

20.6 Wariant podstawowy

Minimalna wersja ćwiczenia powinna zawierać:

1. Wczytywanie wielu liczb od użytkownika.
2. Komendę końzącą wprowadzanie danych.
3. Obliczenie minimum.
4. Obliczenie maksimum.
5. Obliczenie średniej arytmetycznej.
6. Obsługę przypadku pustej listy danych.
7. Komunikat dla błędnej wartości wejściowej.
8. Krótką instrukcję kompilacji i uruchomienia.

20.7 Proponowany podział pracy

1. Napisz pętlę, która czyta tekst do momentu wpisania `q`.
2. Dodaj licznik poprawnych wartości.
3. Dodaj sumę i obliczanie średniej.
4. Dodaj minimum i maksimum.
5. Dodaj komunikat dla pustej listy danych.
6. Dodaj obsługę błędnego tekstu, np. `abc`.

20.8 Zadania dodatkowe

Po wykonaniu wariantu podstawowego możesz dodać:

- przechowywanie wszystkich liczb w `std::vector<double>`,
- sortowanie wprowadzonych liczb,
- obliczenie mediany,
- obliczenie sumy,
- obliczenie odchylenia standardowego,
- wczytanie liczb z pliku,
- zapis raportu do pliku,
- testy automatyczne funkcji liczącej statystyki.

20.9 Pytania kontrolne

1. Dlaczego program nie może obliczyć średniej, gdy licznik wynosi 0?
2. Kiedy należy ustawić pierwszą wartość minimum i maksimum?
3. Czym różni się błędna wartość wejściowa od komendy kończącej program?
4. Dlaczego w tym zadaniu pętla `while` jest wygodniejsza niż pętla `for`?

20.10 Scenariusze sprawdzenia

1. Wpisz liczby 10, -2.5, 4.5 i sprawdź wynik:
 - minimum: -2.5,
 - maksimum: 10,
 - średnia: 4.
2. Wpisz tylko jedną liczbę i sprawdź, czy minimum, maksimum i średnia są takie same.
3. Zakończ wprowadzanie bez podania liczby i sprawdź komunikat programu.
4. Wpisz błędną wartość, np. `abc`, i sprawdź, czy program prosi o kolejną wartość zamiast kończyć działanie.
5. Wpisz kilka liczb ujemnych i sprawdź poprawność minimum oraz maksimum.

20.11 Kryteria zaliczenia

Ćwiczenie jest zaliczone, jeśli:

- program kompiluje się bez błędów i ostrzeżeń,
- poprawnie liczy minimum,
- poprawnie liczy maksimum,
- poprawnie liczy średnią,
- poprawnie działa dla liczb ujemnych,
- poprawnie działa dla jednej liczby,
- nie wykonuje dzielenia przez zero,
- informuje użytkownika o błędnym wejściu,
- kod jest czytelnie sformatowany.

20.12 Źródło

Ćwiczenie powstało na podstawie starszego zadania z zadań pomocniczych TNT.

Rozdział 21

03 - Funkcje, tablice, napisy

Ten segment uczy dzielenia programu na funkcje oraz pracy z prostymi kolekcjami danych: tablicami i napisami.

21.1 Cele segmentu

Po zakończeniu segmentu student powinien umieć:

- deklarować i definiować własne funkcje,
- przekazywać argumenty do funkcji,
- zwracać wynik z funkcji,
- rozumieć różnicę między deklaracją a definicją funkcji,
- używać tablic jednowymiarowych i dwuwymiarowych,
- iterować po elementach tablicy,
- wykonywać podstawowe operacje na `std::string`,
- dzielić większe zadanie na mniejsze części.

21.2 Docelowa kolejność lekcji

1. 01-funkcje-podstawy.md - deklaracja, definicja, wywołanie, wartość zwracana.
2. 02-parametry-i-zakres.md - parametry funkcji, zmienne lokalne, proste przypadki przekazywania danych.
3. 03-tablice.md - tablice jednowymiarowe, indeksy, pętle.
4. 04-tablice-dwuwymiarowe.md - macierze i proste operacje na siatkach danych.
5. 05-napisy.md - `std::string`, długość, znaki, proste przetwarzanie tekstu.
6. 06-zadania.md - zadania podstawowe i dodatkowe.
7. 07-cwiczenie-szyfr-gaderypoluki.md - większe ćwiczenie z funkcji i przetwarzania napisów.

21.3 Status porządkowania

Lekcje i zadania w tym segmencie są uporządkowane w docelowym formacie. Materiały źródłowe zostały przeniesione do archive po wchłonięciu ich treści do lekcji i zadań.

21.4 Format lekcji

Każda docelowa lekcja powinna mieć taki układ:

1. Cel lekcji.
2. Wymagania wstępne.
3. Krótka teoria.
4. Przykład kodu.
5. Zadania do wykonania.

6. Kryteria zaliczenia.

21.5 Przykłady kodu

Katalog `examples` zawiera krótkie programy pokazujące funkcje, parametry, tablice jedno- i dwuwymiarowe oraz podstawowe operacje na napisach. Przykłady są kompilowalne i mogą być punktem startowym do zadań.

Do większego ćwiczenia z szyfrem GADERYPOLUKI przydatny jest przykład `examples/string_gaderypoluki.cpp`.

21.6 Testy

Katalog `tests` zawiera prosty przykład testu automatycznego bez zewnętrznego frameworka. Test pokazuje, jak sprawdzać funkcje zwracające wynik i jak zwracać kod błędu, gdy przypadek testowy nie przechodzi.

21.7 Katalogi pomocnicze

- `examples/` - kompilowalne przykłady `.cpp` dla tego segmentu.
- `tests/` - proste testy automatyczne dla funkcji.
- `archive/` - stare wersje plików po wchłonięciu ich treści do nowych lekcji.

Rozdział 22

01 - Funkcje: podstawy

22.1 Cel lekcji

Celem lekcji jest zrozumienie, czym jest funkcja, jak ją zadeklarować, zdefiniować i wywołać w programie C++.

22.2 Wymagania wstępne

Student powinien znać:

- podstawową strukturę programu z funkcją `main`,
- typy danych i zmienne,
- instrukcje warunkowe oraz pętle z wcześniejszych segmentów.

22.3 Krótka teoria

22.3.1 Po co są funkcje

Funkcja to nazwany blok kodu, który wykonuje określone zadanie.

Funkcje pomagają:

- dzielić program na mniejsze części,
- unikać powtarzania kodu,
- nadawać nazwę konkretnemu działaniu,
- łatwiej testować i poprawiać program.

Przykład myślenia funkcjami:

program BMI:

1. wczytaj dane
2. oblicz BMI
3. wypisz wynik

Krok oblicz BMI może być osobną funkcją.

22.3.2 Definicja funkcji

Definicja funkcji zawiera nagłówek oraz ciało funkcji.

```
typ_zwracany nazwa_funkcji(lista_parametrow)
{
    // instrukcje
}
```

Przykład:

```
int dodaj(int a, int b)
{
    return a + b;
}
```

Znaczenie elementów:

- `int` przed nazwą funkcji oznacza typ zwracanej wartości,
- `dodaj` to nazwa funkcji,
- `int a, int b` to parametry,
- `return a + b;` zwraca wynik działania funkcji.

22.4 Przykład kodu: wywołanie funkcji

Funkcję wywołujemy przez podanie jej nazwy i argumentów.

```
int wynik = dodaj(2, 3);
```

Wartość 2 trafia do parametru `a`, a wartość 3 trafia do parametru `b`.

Pełny przykład znajduje się w pliku `examples/function_add.cpp`.

22.5 Przykład kodu: funkcja bez wartości zwracanej

Jeżeli funkcja nie zwraca wyniku, używamy typu `void`.

```
void wypiszPowitanie()
{
    std::cout << "Witaj!" << std::endl;
}
```

Wywołanie:

```
wypiszPowitanie();
```

Pełny przykład znajduje się w pliku `examples/function_void.cpp`.

22.5.1 Deklaracja funkcji

Deklaracja informuje kompilator, że funkcja istnieje. Deklaracja kończy się średnikiem.

```
int dodaj(int a, int b);
```

Można też pominąć nazwy parametrów:

```
int dodaj(int, int);
```

Deklaracja jest potrzebna, gdy funkcja jest zdefiniowana po funkcji `main`.

```
#include <iostream>

int dodaj(int a, int b); // deklaracja

int main()
{
    std::cout << dodaj(2, 3) << std::endl;
    return 0;
}

int dodaj(int a, int b) // definicja
{
    return a + b;
}
```

22.5.2 Deklaracja a definicja

Deklaracja mówi, jak funkcja wygląda z zewnątrz:

```
int dodaj(int a, int b);
```

Definicja zawiera kod funkcji:

```
int dodaj(int a, int b)
{
    return a + b;
}
```

Funkcja może mieć wiele deklaracji, ale powinna mieć jedną definicję w programie.

22.5.3 Nazwy funkcji

Dobra nazwa funkcji powinna mówić, co funkcja robi.

Dobre przykłady:

- dodaj,
- obliczBmi,
- czyParzysta,
- wypiszMenu.

Słabe przykłady:

- f,
- x,
- test,
- licz.

22.6 Zadania do wykonania

1. Napisz funkcję `dodaj`, która przyjmuje dwie liczby całkowite i zwraca ich sumę.
2. Napisz funkcję `kwadrat`, która przyjmuje liczbę całkowitą i zwraca jej kwadrat.
3. Napisz funkcję `wypiszPowitanie`, która nic nie zwraca i wypisuje tekst powitania.
4. Napisz funkcję `czyParzysta`, która przyjmuje liczbę całkowitą i zwraca `true`, jeśli liczba jest parzysta.
5. Napisz program, który korzysta z co najmniej trzech własnych funkcji.

22.7 Ćwiczenia dodatkowe

1. Dodaj funkcję obliczającą obwód prostokąta.
2. Zmień przykład tak, aby funkcja przyjmowała dane od użytkownika.
3. Dodaj funkcję `wypiszWynik`, która oddziela obliczenia od prezentacji.

22.8 Kryteria zaliczenia

Program powinien:

- zawierać co najmniej jedną własną funkcję poza `main`,
- poprawnie deklarować i definiować funkcje,
- używać `return` w funkcjach zwracających wartość,
- używać `void` dla funkcji, które nie zwracają wyniku,
- kompilować się bez błędów.

22.9 Pytania kontrolne

1. Czym jest funkcja?

2. Czym różni się deklaracja funkcji od definicji?
3. Do czego służy `return`?
4. Co oznacza typ zwracany `void`?
5. Dlaczego warto dzielić program na funkcje?

Rozdział 23

02 - Parametry i zakres zmiennych

23.1 Cel lekcji

Celem lekcji jest zrozumienie, jak funkcje otrzymują dane przez parametry, czym różnią się parametry od argumentów oraz gdzie są dostępne zmienne lokalne.

23.2 Wymagania wstępne

Student powinien znać materiał z lekcji 01-funkcje-podstawy.md: definicję funkcji, deklarację funkcji, wywołanie funkcji i `return`.

23.3 Krótka teoria

23.3.1 Parametr i argument

Parametr to zmienna zapisana w definicji funkcji.

```
int dodaj(int a, int b)
{
    return a + b;
}
```

W tym przykładzie `a` i `b` są parametrami.

Argument to konkretna wartość przekazana podczas wywołania funkcji.

```
int wynik = dodaj(2, 3);
```

Wartości 2 i 3 są argumentami.

23.4 Przykład kodu: przekazywanie przez wartość

Na tym etapie używamy przekazywania przez wartość. Oznacza to, że funkcja dostaje kopię przekazanej wartości.

```
void zmien(int liczba)
{
    liczba = 100;
}

int main()
{
    int x = 5;
    zmien(x);
}
```

```
    std::cout << x << std::endl; // nadal 5
}
```

Zmiana parametru `liczba` wewnątrz funkcji nie zmienia zmiennej `x` w `main`.

Pełny przykład znajduje się w pliku `examples/pass_by_value.cpp`.

23.4.1 Zmienne lokalne

Zmienna zadeklarowana wewnątrz funkcji jest zmienną lokalną. Jest dostępna tylko w tej funkcji.

```
int obliczKwadrat(int liczba)
{
    int wynik = liczba * liczba;
    return wynik;
}
```

Zmienna `wynik` istnieje tylko w funkcji `obliczKwadrat`.

23.4.2 Zakres zmiennej

Zakres zmiennej określa, w której części programu można użyć tej zmiennej.

```
int main()
{
    int x = 10;

    if (x > 0)
    {
        int y = 5;
        std::cout << y << std::endl;
    }

    // tutaj y nie jest dostępne
    return 0;
}
```

Zmienna `y` istnieje tylko wewnątrz bloku `if`.

23.4.3 Nazwy parametrów w deklaracji

W deklaracji funkcji nazwy parametrów można podać, ale nie trzeba.

```
double poleProstokata(double szerokosc, double wysokosc);
```

To samo można zapisać tak:

```
double poleProstokata(double, double);
```

Typy i kolejność parametrów są ważne. Nazwy w deklaracji służą głównie czytelności.

W definicji funkcji nazwy parametrów są potrzebne, ponieważ używamy ich w kodzie.

```
double poleProstokata(double szerokosc, double wysokosc)
{
    return szerokosc * wysokosc;
}
```

23.5 Przykład kodu: pole prostokąta

```
#include <iostream>

double poleProstokata(double szerokosc, double wysokosc)
{
```



```

    return szerokosc * wysokosc;
}

int main()
{
    std::cout << poleProstokata(4.0, 2.5) << std::endl;
    return 0;
}

```

Pełny przykład znajduje się w pliku `examples/function_rectangle_area.cpp`.

23.6 Przykład kodu: BMI

```

double obliczBmi(double masa, double wzrost)
{
    return masa / (wzrost * wzrost);
}

```

Taka funkcja ukrywa wzór w jednym miejscu. Program główny może skupić się na wczytywaniu danych i wypisywaniu wyniku.

Pełny przykład znajduje się w pliku `examples/function_bmi.cpp`.

23.7 Zadania do wykonania

1. Napisz funkcję `poleProstokata`, która przyjmuje dwa argumenty typu `double` i zwraca pole prostokąta.
2. Napisz funkcję `obwodProstokata`, która przyjmuje dwa argumenty typu `double` i zwraca obwód prostokąta.
3. Napisz funkcję `czyParzysta`, która przyjmuje liczbę całkowitą i zwraca `true` albo `false`.
4. Napisz funkcję `maksimum`, która przyjmuje dwie liczby całkowite i zwraca większą z nich.
5. Napisz funkcję `obliczBmi`, która przyjmuje masę i wzrost, a następnie zwraca wartość BMI.
6. Napisz program pokazujący, że zmiana parametru przekazanego przez wartość nie zmienia zmiennej w funkcji `main`.

23.8 Ćwiczenia dodatkowe

1. Dodaj przykład funkcji z parametrem przekazywanym przez `const` referencję.
2. Zmień nazwę zmiennej lokalnej tak, aby nie myliła się z parametrem.
3. Dodaj funkcję, która zwraca wynik zamiast wypisywać go w środku funkcji.

23.9 Kryteria zaliczenia

Program powinien:

- stosować parametry funkcji o odpowiednich typach,
- rozróżniać argumenty od parametrów,
- używać zmiennych lokalnych w ograniczonym zakresie,
- nie korzystać z niepotrzebnych zmiennych globalnych,
- kompilować się bez błędów.

23.10 Pytania kontrolne

1. Czym różni się parametr od argumentu?
2. Co oznacza przekazywanie przez wartość?
3. Czy zmienna lokalna z jednej funkcji jest dostępna w innej funkcji?
4. Czy nazwy parametrów w deklaracji funkcji są obowiązkowe?
5. Dlaczego warto ograniczać zakres zmiennych?

Rozdział 24

03 - Tablice jednowymiarowe

24.1 Cel lekcji

Celem lekcji jest poznanie tablic jednowymiarowych: deklaracji, inicjalizacji, indeksowania i iteracji po elementach.

24.2 Wymagania wstępne

Student powinien znać:

- pętlę `for` z segmentu 02-sterowanie-i-petle,
- funkcje z lekcji 01-funkcje-podstawy.md,
- parametry i zmienne lokalne z lekcji 02-parametry-i-zakres.md.

24.3 Krótka teoria

24.3.1 Czym jest tablica

Tablica przechowuje wiele wartości tego samego typu pod jedną nazwą.

Przykład:

```
int liczby[5] = {10, 20, 30, 40, 50};
```

Ta tablica:

- ma nazwę `liczby`,
- przechowuje wartości typu `int`,
- ma 5 elementów.

24.3.2 Indeksy

Elementy tablicy numerujemy od 0.

```
int liczby[5] = {10, 20, 30, 40, 50};
```

```
std::cout << liczby[0] << std::endl; // 10  
std::cout << liczby[4] << std::endl; // 50
```

Dla tablicy o rozmiarze 5 poprawne indeksy to:

0, 1, 2, 3, 4

Indeks 5 jest już poza zakresem.

24.3.3 Deklaracja i inicjalizacja

Tablicę można zadeklarować bez wartości początkowych:

```
int wyniki[3];

wyniki[0] = 12;
wyniki[1] = 15;
wyniki[2] = 18;
```

Można też podać wartości od razu:

```
int wyniki[3] = {12, 15, 18};
```

Jeżeli podajemy wartości przy inicjalizacji, kompilator może sam określić rozmiar:

```
int wyniki[] = {12, 15, 18};
```

24.4 Przykład kodu: iteracja po tablicy

Do przechodzenia po elementach tablicy najczęściej używamy pętli for.

```
int liczby[] = {1, 3, 6, 3, 4, 7, 8, 4, 3};
int rozmiar = sizeof(liczby) / sizeof(liczby[0]);

for (int i = 0; i < rozmiar; i++)
{
    std::cout << liczby[i] << std::endl;
}
```

Pełny przykład znajduje się w pliku examples/array_print.cpp.

24.4.1 Rozmiar tablicy

Dla tablic statycznych w tym samym zakresie można obliczyć liczbę elementów:

```
int rozmiar = sizeof(liczby) / sizeof(liczby[0]);
```

`sizeof(liczby)` zwraca rozmiar całej tablicy w bajtach, a `sizeof(liczby[0])` rozmiar jednego elementu.

24.5 Przykład kodu: suma i średnia

Tablice często przetwarzamy w pętli.

```
int suma = 0;

for (int i = 0; i < rozmiar; i++)
{
    suma += liczby[i];
}
```

```
double srednia = static_cast<double>(suma) / rozmiar;
```

Pełny przykład znajduje się w pliku examples/array_average.cpp.

24.6 Przykład kodu: minimum i maksimum

Typowy algorytm:

1. Przyjmij pierwszy element jako aktualne minimum i maksimum.
2. Przejdź po pozostałych elementach.
3. Aktualizuj minimum i maksimum, gdy znajdziesz mniejszą lub większą wartość.

```

int minimum = liczby[0];
int maksimum = liczby[0];

for (int i = 1; i < rozmiar; i++)
{
    if (liczby[i] < minimum)
    {
        minimum = liczby[i];
    }

    if (liczby[i] > maksimum)
    {
        maksimum = liczby[i];
    }
}

```

Pełny przykład znajduje się w pliku `examples/array_min_max.cpp`.

24.7 Typowy błąd: wyjście poza zakres tablicy

Dla tablicy:

```
int liczby[3] = {1, 2, 3};
```

poprawne indeksy to 0, 1, 2.

Kod:

```
liczby[3] = 10;
```

jest błędny, bo zapisuje poza tablicą. Taki błąd może prowadzić do nieprzewidywalnego działania programu.

24.8 Zadania do wykonania

1. Utwórz tablicę pięciu liczb całkowitych i wypisz wszystkie elementy.
2. Utwórz tablicę liczb i oblicz sumę jej elementów.
3. Utwórz tablicę liczb i oblicz średnią arytmetyczną.
4. Utwórz tablicę liczb i znajdź najmniejszą oraz największą wartość.
5. Wczytaj od użytkownika pięć liczb do tablicy, a następnie wypisz je w odwrotnej kolejności.
6. Policz, ile elementów tablicy jest parzystych.

24.9 Ćwiczenia dodatkowe

1. Dodaj funkcję liczącą sumę elementów tablicy.
2. Dodaj funkcję zliczającą elementy większe od podanej wartości.
3. Zmień przykład tak, aby rozmiar tablicy był stałą nazwaną.

24.10 Kryteria zaliczenia

Program powinien:

- poprawnie deklarować tablicę,
- używać indeksów od 0 do `rozmiar - 1`,
- iterować po tablicy pętlą,
- nie wychodzić poza zakres tablicy,
- kompilować się bez błędów.

24.11 Pytania kontrolne

1. Od jakiego indeksu zaczyna się tablica w C++?
2. Jakie są poprawne indeksy dla tablicy `int x[4]`?
3. Do czego służy `sizeof(tablica) / sizeof(tablica[0])`?
4. Dlaczego wyjście poza zakres tablicy jest błędem?
5. Kiedy warto użyć tablicy zamiast kilku osobnych zmiennych?

Rozdział 25

04 - Tablice dwuwymiarowe

25.1 Cel lekcji

Celem lekcji jest poznanie tablic dwuwymiarowych, czyli struktur przydatnych do reprezentowania tabel, macierzy i plansz.

25.2 Wymagania wstępne

Student powinien znać:

- tablice jednowymiarowe z lekcji 03-tablice.md,
- pętle zagnieżdżone z segmentu 02-sterowanie-i-petle.

25.3 Krótka teoria

25.3.1 Czym jest tablica dwuwymiarowa

Tablica dwuwymiarowa przechowuje dane w układzie wierszy i kolumn.

Przykład:

```
int macierz[2][3] = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

Ta tablica ma:

- 2 wiersze,
- 3 kolumny,
- 6 elementów.

25.3.2 Indeksy

Do elementu tablicy dwuwymiarowej odwołujemy się przez dwa indeksy:

```
macierz[wiersz][kolumna]
```

Przykład:

```
std::cout << macierz[0][0] << std::endl; // 1  
std::cout << macierz[1][2] << std::endl; // 6
```

Indeksy, tak jak w tablicach jednowymiarowych, zaczynają się od 0.

25.3.3 Deklaracja

Składnia:

```
typ nazwa[liczba_wierszy][liczba_kolumn];
```

Przykład:

```
int plansza[3][3];
```

Taka tablica może reprezentować planszę 3 x 3, np. do gry w kółko i krzyżyk.

25.4 Przykład kodu: iteracja po tablicy dwuwymiarowej

Do przechodzenia po wszystkich elementach używamy pętli zagnieżdżonych.

```
for (int row = 0; row < rows; row++)
{
    for (int col = 0; col < cols; col++)
    {
        std::cout << macierz[row][col] << " ";
    }

    std::cout << std::endl;
}
```

Zewnętrzna pętla przechodzi po wierszach, a wewnętrzna po kolumnach.

Pełny przykład znajduje się w pliku examples/matrix_print.cpp.

25.5 Przykład kodu: suma wszystkich elementów

```
int suma = 0;

for (int row = 0; row < rows; row++)
{
    for (int col = 0; col < cols; col++)
    {
        suma += macierz[row][col];
    }
}
```

Pełny przykład znajduje się w pliku examples/matrix_sum.cpp.

25.6 Przykład kodu: suma wierszy

Możemy też przetwarzać każdy wiersz osobno.

```
for (int row = 0; row < rows; row++)
{
    int sumaWiersza = 0;

    for (int col = 0; col < cols; col++)
    {
        sumaWiersza += macierz[row][col];
    }

    std::cout << "Suma wiersza " << row << ": " << sumaWiersza << std::endl;
}
```

Pełny przykład znajduje się w pliku examples/matrix_row_sums.cpp.

25.6.1 Tablica dwuwymiarowa jako plansza

Tablica dwuwymiarowa dobrze nadaje się do reprezentowania plansz.

```
char plansza[3][3] = {
    {' ', ' ', ' '},
    {' ', ' ', ' '},
    {' ', ' ', ' '}
};
```

Przykładowe użycie:

```
plansza[0][0] = 'X';
plansza[1][1] = 'O';
```

To dobry punkt startowy do projektu gry w kółko i krzyżyk.

25.7 Typowe błędy

25.7.1 Zamiana wiersza z kolumną

Jeżeli przyjmujemy zapis `macierz[row][col]`, pierwszy indeks oznacza wiersz, a drugi kolumnę.

25.7.2 Wyjście poza zakres

Dla tablicy:

```
int macierz[2][3];
```

poprawne indeksy to:

- wiersze: 0, 1,
- kolumny: 0, 1, 2.

`macierz[2][0]` i `macierz[0][3]` są poza zakresem.

25.8 Zadania do wykonania

1. Utwórz tablicę 3 x 3 liczb całkowitych i wypisz ją w formie tabeli.
2. Oblicz sumę wszystkich elementów tablicy 3 x 3.
3. Oblicz sumę każdego wiersza osobno.
4. Oblicz sumę każdej kolumny osobno.
5. Znajdź największy element w tablicy dwuwymiarowej.
6. Utwórz pustą planszę 3 x 3 dla gry w kółko i krzyżyk i wypisz ją na ekran.

25.9 Ćwiczenia dodatkowe

1. Dodaj obliczanie sumy każdej kolumny.
2. Wypisz największą wartość w całej macierzy.
3. Zmień rozmiar macierzy i sprawdź, które fragmenty kodu wymagają aktualizacji.

25.10 Kryteria zaliczenia

Program powinien:

- poprawnie deklarować tablicę dwuwymiarową,
- używać pierwszego indeksu jako wiersza,
- używać drugiego indeksu jako kolumny,
- iterować po tablicy pętlami zagnieżdżonymi,
- nie wychodzić poza zakres tablicy,
- kompilować się bez błędów.

25.11 Pytania kontrolne

1. Co oznaczają dwa indeksy w zapisie `tablica[row][col]`?
2. Jakie są poprawne indeksy dla tablicy `int x[2][3]`?
3. Dlaczego do tablicy dwuwymiarowej zwykle używamy dwóch pętli?
4. Jak można reprezentować planszę gry za pomocą tablicy dwuwymiarowej?
5. Czym różni się suma wszystkich elementów od sumy pojedynczego wiersza?

Rozdział 26

05 - Napisy `std::string`

26.1 Cel lekcji

Celem lekcji jest poznanie podstaw pracy z napisami w C++ przy pomocy typu `std::string`.

26.2 Wymagania wstępne

Student powinien znać:

- wejście i wyjście z segmentu 01-podstawy,
- pętle z segmentu 02-sterowanie-i-petle,
- funkcje z lekcji 01-funkcje-podstawy.md.

26.3 Krótka teoria

26.3.1 Typ `std::string`

`std::string` służy do przechowywania tekstu.

```
#include <string>
```

```
std::string imie = "Ala";
```

Do wypisywania i wczytywania `std::string` używamy `std::cout`, `std::cin` oraz `std::getline`.

26.3.2 `std::cin` i `std::getline`

`std::cin >> tekst` wczytuje tekst do pierwszej spacji.

```
std::string imie;  
std::cin >> imie;
```

`std::getline` wczytuje całą linię.

```
std::string imieINazwisko;  
std::getline(std::cin, imieINazwisko);
```

Pełny przykład znajduje się w pliku `examples/string_input.cpp`.

26.3.3 Długość napisu

Długość napisu można sprawdzić metodą `.length()` albo `.size()`.

```
std::string tekst = "program";
```

```
std::cout << tekst.length() << std::endl; // 7
std::cout << tekst.size() << std::endl;    // 7
```

W prostych programach można używać obu form.

26.3.4 Indeksowanie znaków

Znaki w napisie są indeksowane od 0, podobnie jak elementy tablicy.

```
std::string tekst = "kot";

std::cout << tekst[0] << std::endl; // k
std::cout << tekst[1] << std::endl; // o
std::cout << tekst[2] << std::endl; // t
```

Dla napisu o długości 3 poprawne indeksy to 0, 1, 2.

Pełny przykład znajduje się w pliku `examples/string_chars.cpp`.

26.4 Przykład kodu: iteracja po znakach

Możemy przechodzić po znakach napisu pętlą `for`.

```
std::string tekst = "program";

for (int i = 0; i < tekst.length(); i++)
{
    std::cout << tekst[i] << std::endl;
}
```

Możemy też użyć pętli zakresowej:

```
for (char znak : tekst)
{
    std::cout << znak << std::endl;
}
```

26.4.1 Łączenie napisów

Napisy można łączyć operatorem `+`.

```
std::string imie = "Ala";
std::string komunikat = "Czesc, " + imie + "!";
```

26.5 Przykład kodu: proste sprawdzanie palindromu

Palindrom to tekst, który czytany od lewej i od prawej strony jest taki sam.

Przykłady:

- kajak,
- oko,
- level.

Funkcja sprawdzająca palindrom:

```
bool czyPalindrom(std::string tekst)
{
    int left = 0;
    int right = tekst.length() - 1;

    while (left < right)
    {
```

```

        if (tekst[left] != tekst[right])
        {
            return false;
        }

        left++;
        right--;
    }

    return true;
}

```

Pełny przykład znajduje się w pliku `examples/string_palindrome.cpp`.

26.6 Uwaga o polskich znakach

Na tym etapie zakładamy, że tekst składa się z prostych znaków ASCII, np. `a-z`, `A-Z`, `0-9`.

Polskie znaki, takie jak `ą`, `ę`, `ł`, wymagają dodatkowej wiedzy o kodowaniu tekstu i nie są częścią tej lekcji.

26.7 Zadania do wykonania

1. Napisz program, który wczytuje imię i wypisuje liczbę znaków.
2. Napisz program, który wczytuje całe imię i nazwisko przy pomocy `std::getline`.
3. Napisz program, który wypisuje każdy znak napisu w osobnej linii.
4. Napisz funkcję `pierwszyZnak`, która zwraca pierwszy znak napisu.
5. Napisz funkcję `czyPalindrom`, która sprawdza, czy napis jest palindromem.
6. Napisz program, który zlicza, ile razy litera `a` występuje w napisie.

26.8 Ćwiczenia dodatkowe

1. Dodaj zliczanie spacji w napisie.
2. Zmień program tak, aby ignorował wielkość liter przy porównaniu.
3. Dodaj funkcję sprawdzającą, czy napis zaczyna się od podanego znaku.

26.9 Kryteria zaliczenia

Program powinien:

- dołączać bibliotekę `string`,
- poprawnie używać `std::string`,
- rozróżniać `std::cin >> tekst` i `std::getline`,
- nie wychodzić poza zakres napisu,
- kompilować się bez błędów.

26.10 Pytania kontrolne

1. Do czego służy `std::string`?
2. Czym różni się `std::cin >> tekst` od `std::getline`?
3. Od jakiego indeksu zaczynają się znaki w napisie?
4. Co zwraca metoda `.length()`?
5. Czym jest palindrom?

Rozdział 27

06 - Zadania

27.1 Cel zestawu zadań

Ten plik zbiera zadania do segmentu 03 - Funkcje, tablice, napisy. Zadania sprawdzają umiejętność dzielenia programu na funkcje oraz pracy z tablicami i napisami.

27.2 Zasady wykonania

Każde zadanie powinno być zapisane w osobnym pliku `.cpp`. Program powinien kompilować się bez błędów i wypisywać wynik w czytelnej formie.

Przykład kompilacji:

```
c++ -std=c++17 -Wall -Wextra -pedantic nazwa_pliku.cpp -o program
./program
```

Przed oddaniem zadania sprawdź:

- czy główna logika jest wydzielona do funkcji,
- czy funkcje mają czytelne nazwy i pojedynczą odpowiedzialność,
- czy tablice są przekazywane razem z rozmiarem,
- czy indeksy nie wychodzą poza zakres tablicy,
- czy program był sprawdzony dla danych typowych i granicznych.

27.3 Poziom 1 - Funkcje

27.3.1 Zadanie 1. Podstawowe działania

Napisz funkcje:

- `dodaj(int a, int b)`,
- `odejmij(int a, int b)`,
- `pomnoz(int a, int b)`,
- `podziel(double a, double b)`.

W funkcji `main` pokaż działanie każdej funkcji.

Wymagania:

- funkcje obliczające powinny zwracać wynik przez `return`,
- `main` powinien tylko przygotować dane, wywołać funkcje i wypisać wyniki,
- przy dzieleniu przez zero wypisz komunikat zamiast wykonywać działanie.

Scenariusz sprawdzenia:

Dane: 10 i 5
Suma: 15

Roznica: 5
Iloczyn: 50
Iloraz: 2

27.3.2 Zadanie 2. Kwadrat i sześćian

Napisz funkcje `kwadrat` i `sześcián`, które przyjmują liczbę i zwracają odpowiednio drugą oraz trzecią potęgę tej liczby.

Sprawdź funkcje dla wartości 0, 2 i -3.

27.3.3 Zadanie 3. Parzystość

Napisz funkcję `czyParzysta`, która przyjmuje liczbę całkowitą i zwraca `true`, jeśli liczba jest parzysta.

W `main` wypisz wynik jako tekst `parzysta` albo `nieparzysta`, nie jako 0 lub 1.

27.3.4 Zadanie 4. BMI

Napisz funkcję `obliczBmi`, która przyjmuje masę i wzrost, a następnie zwraca wartość BMI.

Rozbuduj program o funkcję `wypiszKategorieBmi`, która wypisuje opis wyniku.

Wymagania:

- `obliczBmi` nie powinna nic wypisywać,
- `wypiszKategorieBmi` powinna przyjmować gotową wartość BMI,
- wynik BMI wypisz z dokładnością do dwóch miejsc po przecinku.

27.3.5 Zadanie 5. Maksimum

Napisz funkcję `maksimum`, która przyjmuje dwie liczby całkowite i zwraca większą z nich.

Sprawdź przypadki:

- pierwsza liczba większa,
- druga liczba większa,
- obie liczby równe.

27.4 Poziom 2 - Tablice jednowymiarowe

27.4.1 Zadanie 6. Wypisywanie tablicy

Utwórz tablicę pięciu liczb całkowitych i wypisz wszystkie elementy.

Wymagania:

- użyj pętli,
- nie wypisuj elementów ręcznie przez pięć osobnych instrukcji,
- wydziel funkcję `wypiszTablice`.

27.4.2 Zadanie 7. Suma i średnia

Utwórz tablicę liczb całkowitych. Napisz funkcje:

- `sumaTablicy`,
- `sredniaTablicy`.

Funkcje powinny przyjmować tablicę i jej rozmiar.

Scenariusz sprawdzenia:

Tablica: 2 4 6 8
Suma: 20
Srednia: 5

27.4.3 Zadanie 8. Minimum i maksimum

Napisz funkcje `minimumTablicy` i `maksimumTablicy`, które znajdują najmniejszą oraz największą wartość w tablicy.

Sprawdź tablicę zawierającą liczby dodatnie, ujemne i zero.

27.4.4 Zadanie 9. Odwrotna kolejność

Wczytaj od użytkownika pięć liczb do tablicy, a następnie wypisz je w odwrotnej kolejności.

Przykład:

Dane:

1 2 3 4 5

Wynik:

5 4 3 2 1

27.4.5 Zadanie 10. Liczby parzyste

Napisz funkcję, która zlicza, ile elementów tablicy jest parzystych.

Wymagania:

- funkcja powinna zwracać liczbę elementów parzystych,
- sprawdź przypadek, w którym tablica nie ma żadnej liczby parzystej.

27.5 Poziom 3 - Tablice dwuwymiarowe

27.5.1 Zadanie 11. Wypisywanie macierzy

Utwórz tablicę 3 x 3 liczb całkowitych i wypisz ją w formie tabeli.

Wymagania:

- użyj dwóch pętli,
- każdy wiersz macierzy wypisz w osobnej linii.

27.5.2 Zadanie 12. Suma macierzy

Napisz program, który oblicza sumę wszystkich elementów tablicy 3 x 3.

Scenariusz sprawdzenia:

Macierz:

1 2 3

4 5 6

7 8 9

Suma: 45

27.5.3 Zadanie 13. Sumy wierszy i kolumn

Napisz program, który wypisuje:

- sumę każdego wiersza,
- sumę każdej kolumny.

Wymagania:

- przygotuj osobną funkcję do sumy wiersza,
- przygotuj osobną funkcję do sumy kolumny,
- użyj stałych dla liczby wierszy i kolumn.

27.5.4 Zadanie 14. Największy element

Napisz program, który znajduje największy element w tablicy dwuwymiarowej.

Sprawdź przypadek, w którym największa wartość znajduje się w ostatnim wierszu i ostatniej kolumnie.

27.5.5 Zadanie 15. Plansza 3 x 3

Utwórz pustą planszę 3 x 3 dla gry w kółko i krzyżyk. Wypełnij ją spacjami i wypisz na ekran.

Wymagania:

- wydziel funkcję `wypełnijPlansze`,
- wydziel funkcję `wypiszPlansze`,
- do przechowywania pól użyj tablicy znaków.

27.6 Poziom 4 - Napisy

27.6.1 Zadanie 16. Długość napisu

Napisz program, który wczytuje imię i wypisuje liczbę znaków.

Wymagania:

- użyj `std::string`,
- użyj metody `.size()` albo `.length()`,
- wczytaj tekst przez `std::getline`.

27.6.2 Zadanie 17. Znaki napisu

Napisz program, który wypisuje każdy znak napisu w osobnej linii.

Przykład:

Dane:

kot

Wynik:

k

o

t

27.6.3 Zadanie 18. Pierwszy i ostatni znak

Napisz funkcje:

- `pierwszyZnak`,
- `ostatniZnak`.

Funkcje powinny przyjmować `std::string` i zwracać znak.

Wymagania:

- jeśli napis jest pusty, wypisz komunikat o błędzie w `main`,
- funkcji nie wywołuj dla pustego napisu.

27.6.4 Zadanie 19. Liczenie litery

Napisz funkcję, która zlicza, ile razy litera `a` występuje w napisie.

Sprawdź napisy:

- `ala` - wynik 2,
- `kot` - wynik 0,
- pusty napis - wynik 0.

27.6.5 Zadanie 20. Palindrom

Napisz funkcję `czyPalindrom`, która sprawdza, czy napis jest palindromem.

Na tym etapie załóż, że tekst:

- nie zawiera spacji,
- nie zawiera polskich znaków,
- ma tylko małe litery.

Scenariusze sprawdzenia:

- `kajak` - palindrom,
- `anna` - palindrom,
- `program` - nie jest palindromem.

Większe ćwiczenie z funkcji i przetwarzania znaków znajduje się w pliku 07-cwiczenie-szyfr-gaderypolu.ki.md.

27.7 Poziom 5 - Zadania dodatkowe

27.7.1 Zadanie 21. Kółko i krzyżyk - plansza

Przygotuj program, który przechowuje planszę gry w kółko i krzyżyk jako tablicę 3 x 3 znaków.

Program powinien:

- wypisać pustą planszę,
- pozwolić wpisać znak `X` lub `O` w wybrane pole,
- wypisać planszę po zmianie.

Wymagania:

- sprawdź, czy wybrane pole mieści się w zakresie planszy,
- nie pozwól nadpisać zajętego pola,
- wydziel funkcję do sprawdzania poprawności ruchu.

27.7.2 Zadanie 22. Najdłuższy wspólny podciąg

Napisz funkcję, która przyjmuje dwa napisy i zwraca długość ich najdłuższego wspólnego podciągu.

Przykład:

```
input: "ABCD", "ACDF"  
output: 2
```

To zadanie jest dodatkowe, ponieważ wymaga dynamicznego podejścia do problemu.

27.7.3 Zadanie 23. Liczby skojarzone z funkcjami

Przepisz zadanie o liczbach skojarzonych tak, aby program był podzielony na funkcje:

- `sumaDzielnikow`,
- `czySkojarzone`,
- funkcja odpowiedzialna za wypisanie wyniku.

27.8 Warian minimum

Do zaliczenia segmentu wykonaj co najmniej:

1. Zadanie 1 albo 2.
2. Zadanie 4 albo 5.
3. Zadanie 7 albo 8.
4. Zadanie 11 albo 12.
5. Zadanie 16 albo 17.

6. Zadanie 20.

7. Mini-sprawdzian segmentu z końca pliku.

Zadania 21-23 są dodatkowe. Możesz je robić po wykonaniu wariantu minimum.

27.9 Kryteria zaliczenia segmentu

Student zalicza segment, jeśli potrafi samodzielnie:

- napisać własną funkcję i wywołać ją z `main`,
- przekazać argumenty do funkcji,
- zwrócić wynik przez `return`,
- użyć tablicy jednowymiarowej,
- użyć tablicy dwuwymiarowej,
- przejść po tablicy pętlą,
- wykonać podstawowe operacje na `std::string`,
- podzielić większy program na mniejsze funkcje.

27.10 Mini-sprawdzian segmentu

Napisz program analizujący listę ocen, który:

- przechowuje oceny w tablicy,
- ma osobne funkcje do obliczania średniej, minimum i maksimum,
- wypisuje wyniki w czytelnej formie,
- sprawdza, czy każda ocena mieści się w zakresie od 1 do 6,
- kompiluje się z `-Wall -Wextra -pedantic`.

Wymagania szczegółowe:

- przygotuj funkcję `czyPoprawnaOcena`,
- przygotuj funkcje `sredniaOcen`, `minimumOcen`, `maksimumOcen`,
- nie licz średniej, jeśli w danych jest niepoprawna ocena,
- wypisz jasny komunikat dla błędnych danych.

Scenariusz sprawdzenia:

Oceny:

5 4 3 6 2

Srednia: 4

Minimum: 2

Maksimum: 6

27.11 Checklista oddania

Przed oddaniem rozwiązań sprawdź:

- ☐ Każde zadanie jest w osobnym pliku `.cpp`.
- ☐ Programy kompilują się z `-Wall -Wextra -pedantic`.
- ☐ Program jest podzielony na funkcje, a `main` nie zawiera całej logiki.
- ☐ Funkcje obliczeniowe zwracają wynik zamiast wypisywać go bezpośrednio.
- ☐ Tablice są przekazywane razem z rozmiarem.
- ☐ Program nie wychodzi poza zakres tablicy ani napisu.
- ☐ Przypadki pustego napisu i błędnych danych są obsługiwane tam, gdzie mogą wystąpić.

27.12 Archiwum

Oryginalne materiały źródłowe tego segmentu są zachowane w katalogu `archive`.

Rozdział 28

07 - Ćwiczenie: szyfr GADERYPOLUKI

28.1 Cel ćwiczenia

Celem ćwiczenia jest napisanie programu konsolowego, który szyfruje tekst podany przez użytkownika za pomocą prostego szyfru podstawieniowego GADERYPOLUKI.

Ćwiczenie utrwała pracę z `std::string`, iterowanie po znakach napisu, tworzenie funkcji zwracającej wynik i rozdzielenie logiki od wejścia oraz wyjścia programu.

28.2 Wymagania wstępne

Przed wykonaniem ćwiczenia student powinien znać:

- deklarowanie i definiowanie funkcji,
- przekazywanie napisu do funkcji,
- zwracanie wartości przez `return`,
- pętlę przechodzącą po znakach napisu,
- instrukcje warunkowe.

28.3 Opis szyfru

Szyfr GADERYPOLUKI używa par zamienników:

GA DE RY PO LU KI

Zasady:

- G zamienia się na A,
- A zamienia się na G,
- D zamienia się na E,
- E zamienia się na D,
- i tak dalej dla pozostałych par,
- znaki, których nie ma w kluczu, pozostają bez zmian.

Przykład:

PROGRAM -> OYPAYGM

28.4 Opis zadania

Napisz program, który:

- prosi użytkownika o wpisanie tekstu,

- szyfruje tekst szyfrem GADERYPOLUKI,
- wyświetla wynik,
- zachowuje bez zmian znaki spoza klucza,
- działa dla tekstu składającego się z wielu znaków.

28.5 Wymagania techniczne

Program powinien:

- być napisany w C++17,
- zawierać funkcję `encryptGaderypoluki`,
- przyjmować tekst jako `std::string`,
- zwracać zaszyfrowany tekst jako `std::string`,
- nie modyfikować bezpośrednio tekstu wejściowego,
- używać czytelnych nazw zmiennych i funkcji,
- kompilować się z flagami `-Wall -Wextra -pedantic`.

Przykładowy nagłówek funkcji:

```
std::string encryptGaderypoluki(const std::string& text);
```

Kompilowalny punkt startowy znajduje się w `examples/string_gaderypoluki.cpp`.

28.6 Wariant podstawowy

Minimalna wersja ćwiczenia powinna zawierać:

1. Wczytanie tekstu od użytkownika.
2. Funkcję szyfrującą.
3. Obsługę wszystkich par z klucza GA-DE-RY-PO-LU-KI.
4. Pozostawianie bez zmian znaków spoza klucza.
5. Wyświetlenie tekstu zaszyfrowanego.
6. Krótką instrukcję kompilacji i uruchomienia.

28.7 Proponowany podział pracy

1. Napisz funkcję, która zamienia pojedynczy znak.
2. Sprawdź zamianę dla pary G i A.
3. Dodaj pozostałe pary klucza.
4. Napisz funkcję, która przechodzi po całym napisie.
5. Dodaj wczytywanie tekstu przez `std::getline`.
6. Sprawdź tekst ze znakami spoza klucza.

28.8 Zadania dodatkowe

Po wykonaniu wariantu podstawowego możesz dodać:

- obsługę małych liter,
- zachowanie wielkości liter,
- odszyfrowywanie tekstu tą samą funkcją,
- testy automatyczne dla funkcji szyfrującej,
- wczytanie tekstu z pliku,
- zapis wyniku do pliku,
- wybór własnego klucza podstawieniowego.

28.9 Pytania kontrolne

1. Dlaczego funkcja szyfrująca powinna zwracać nowy napis zamiast wypisywać wynik?

2. Jak sprawdzić, czy znak nie występuje w kluczu?
3. Dlaczego `std::getline` jest lepsze niż `std::cin >> text`, jeśli tekst może zawierać spacje?
4. Czy ten sam algorytm może odszyfrować tekst? Uzasadnij odpowiedź.

28.10 Scenariusze sprawdzenia

1. Wpisz `PROGRAM` i sprawdź, czy wynik to `OYPAYGM`.
2. Wpisz `GADERYPOLUKI` i sprawdź, czy każda litera została zamieniona na swoją parę.
3. Wpisz tekst zawierający spacje i cyfry, np. `ALA 123`, i sprawdź, czy znaki spoza klucza pozostały bez zmian.
4. Wpisz pusty tekst i sprawdź, czy program nie kończy się błędem.

28.11 Kryteria zaliczenia

Ćwiczenie jest zaliczone, jeśli:

- program kompiluje się bez błędów i ostrzeżeń,
- funkcja szyfrująca jest wydzielona z `main`,
- program poprawnie szyfruje słowo `PROGRAM` jako `OYPAYGM`,
- program poprawnie zamienia wszystkie pary z klucza,
- program zostawia bez zmian znaki spoza klucza,
- kod jest czytelnie sformatowany.

28.12 Źródło

Ćwiczenie powstało na podstawie starszego zadania z zadań pomocniczych TNT.

Rozdział 29

04 - Wskaźniki, referencje, pamięć

Ten segment wprowadza pojęcia adresu, wskaźnika i referencji. Materiał powinien być prowadzony ostrożnie, ponieważ błędy w tej części często prowadzą do trudnych do diagnozowania problemów.

29.1 Cele segmentu

Po zakończeniu segmentu student powinien umieć:

- odróżnić wartość zmiennej od jej adresu,
- używać operatora pobrania adresu `&`,
- deklarować wskaźniki,
- rozumieć znaczenie `nullptr`,
- używać operatora dereferencji `*`,
- rozumieć podstawy referencji,
- przekazywać dane do funkcji przez wskaźnik i przez referencję,
- wskazać podstawowe ryzyka pracy z pamięcią.

29.2 Kolejność lekcji

1. 01-adresy-i-wskazniki.md - adres zmiennej, wskaźnik, `nullptr`, dereferencja.
2. 02-referencje.md - referencje i różnice względem wskaźników.
3. 03-przekazywanie-do-funkcji.md - przekazywanie przez wartość, wskaźnik i referencję.
4. 04-bezpieczenstwo-pamieci.md - typowe błędy i dobre praktyki.
5. 05-zadania.md - zadania podstawowe i dodatkowe.

29.3 Status porządkowania

Lekcje i zadania w tym segmencie są uporządkowane w docelowym formacie. Materiały źródłowe zostały wchłonięte do lekcji, a ich pierwotne wersje są zachowane w katalogu archive:

- `archive/wskazniki-i-referencje.md`
- `archive/lab-02-petle-wskazniki.md`
- `archive/memory-safe.md`

29.4 Format lekcji

Każda docelowa lekcja powinna mieć taki układ:

1. Cel lekcji.
2. Wymagania wstępne.
3. Krótka teoria.
4. Przykład kodu.

5. Zadania do wykonania.
6. Kryteria zaliczenia.

29.5 Przykłady kodu

Katalog `examples` zawiera krótkie programy pokazujące adresy, wskaźniki, referencje, przekazywanie argumentów do funkcji i podstawowe zasady bezpiecznej pracy z pamięcią.

29.6 Testy

Katalog `tests` zawiera prosty test bez zewnętrznego frameworka. Test sprawdza przekazywanie przez referencję, pracę ze wskaźnikiem, obsługę `nullptr`, przechodzenie po tablicy przez wskaźnik oraz zwracanie kilku wyników przez parametry referencyjne.

29.7 Katalogi pomocnicze

- `examples/` - kompilowalne przykłady `.cpp` dla tego segmentu.
- `tests/` - proste testy automatyczne dla wskaźników i referencji.
- `archive/` - pierwotne wersje materiałów zachowane do wglądu.

Rozdział 30

01 - Adresy i wskaźniki

30.1 Cel lekcji

Celem lekcji jest zrozumienie, czym jest adres zmiennej, czym jest wskaźnik i jak bezpiecznie odczytywać wartość wskazywaną przez wskaźnik.

30.2 Wymagania wstępne

Student powinien znać:

- zmienne i typy danych z segmentu 01-podstawy,
- funkcje z segmentu 03-funkcje-tablice-napisy.

30.3 Krótka teoria

30.3.1 Wartość i adres

Zmienna ma wartość oraz miejsce w pamięci.

```
int liczba = 42;
```

Wartością zmiennej `liczba` jest 42.

Adres zmiennej można pobrać operatorem `&`.

```
std::cout << &liczba << std::endl;
```

Adres zwykle jest wypisywany jako liczba szesnastkowa. Konkretna wartość adresu może być inna przy każdym uruchomieniu programu.

Pełny przykład znajduje się w pliku `examples/address_of_variable.cpp`.

30.3.2 Czym jest wskaźnik

Wskaźnik to zmienna, która przechowuje adres innej zmiennej.

```
int liczba = 42;  
int* wskaznik = &liczba;
```

`wskaznik` przechowuje adres zmiennej `liczba`.

Zapis:

```
int* wskaznik;
```

oznacza, że `wskaznik` może przechowywać adres zmiennej typu `int`.

30.3.3 Deklarowanie wskaźników

Możliwe są różne style zapisu:

```
int* p;  
int *p;
```

W tym kursie stosujemy zapis:

```
int* p;
```

Warto uważać przy deklarowaniu kilku zmiennych w jednej linii:

```
int* p, liczba;
```

W tym przykładzie tylko `p` jest wskaźnikiem. `liczba` jest zwykłą zmienną typu `int`.

Dla czytelności lepiej pisać:

```
int* p;  
int liczba;
```

30.4 Przykład kodu: dereferencja

Operator `*` przed wskaźnikiem pozwala dostać się do wartości, na którą wskaźnik wskazuje.

```
int liczba = 42;  
int* wskaznik = &liczba;  
  
std::cout << *wskaznik << std::endl; // 42
```

Można też zmienić wartość zmiennej przez wskaźnik:

```
*wskaznik = 100;  
std::cout << liczba << std::endl; // 100
```

Pełny przykład znajduje się w pliku `examples/pointer_dereference.cpp`.

30.4.1 nullptr

Wskaźnik może nie wskazywać na żadną poprawną zmienną. W C++ używamy wtedy wartości `nullptr`.

```
int* wskaznik = nullptr;
```

Przed dereferencją wskaźnika warto sprawdzić, czy nie jest pusty.

```
if (wskaznik != nullptr)  
{  
    std::cout << *wskaznik << std::endl;  
}
```

Nie wolno dereferencjonować `nullptr`.

```
int* wskaznik = nullptr;  
// std::cout << *wskaznik << std::endl; // błąd w czasie działania
```

Pełny przykład znajduje się w pliku `examples/nullptr_check.cpp`.

30.5 Przykład kodu: wskaźnik i adres wskaźnika

Wskaźnik też jest zmienną, więc ma własny adres.

```
int liczba = 42;  
int* wskaznik = &liczba;  
  
std::cout << "Wartosc liczby: " << liczba << std::endl;  
std::cout << "Adres liczby: " << &liczba << std::endl;
```

```
std::cout << "Wartosc wskaznika: " << wskaznik << std::endl;
std::cout << "Adres wskaznika: " << &wskaznik << std::endl;
```

wskaznik przechowuje adres liczby, ale `&wskaznik` to adres samego wskaźnika.

30.6 Typowe błędy

30.6.1 Błędny typ

Niepoprawnie:

```
int liczba = 42;
int p = &liczba;
```

Poprawnie:

```
int liczba = 42;
int* p = &liczba;
```

30.6.2 Dereferencja pustego wskaźnika

Niepoprawnie:

```
int* p = nullptr;
std::cout << *p << std::endl;
```

Poprawnie:

```
int* p = nullptr;

if (p != nullptr)
{
    std::cout << *p << std::endl;
}
```

30.7 Zadania do wykonania

1. Utwórz zmienną typu `int`, wypisz jej wartość i adres.
2. Utwórz wskaźnik na zmienną typu `int` i wypisz wartość zmiennej przez wskaźnik.
3. Zmień wartość zmiennej przez wskaźnik.
4. Utwórz wskaźnik ustawiony na `nullptr` i sprawdź go instrukcją `if`.
5. Wypisz adres zmiennej, wartość wskaźnika oraz adres samego wskaźnika.

30.8 Ćwiczenia dodatkowe

1. Wypisz adresy dwóch różnych zmiennych i porównaj wynik.
2. Dodaj wskaźnik do zmiennej typu `double`.
3. Zmień wartość zmiennej przez wskaźnik i wypisz wynik przed oraz po zmianie.

30.9 Kryteria zaliczenia

Program powinien:

- poprawnie używać operatora `&`,
- poprawnie deklarować wskaźniki,
- używać operatora `*` tylko dla poprawnych wskaźników,
- sprawdzać `nullptr` przed dereferencją,
- kompilować się bez błędów.

30.10 Pytania kontrolne

1. Co zwraca operator `&`?
2. Czym jest wskaźnik?
3. Co robi operator `*` przed wskaźnikiem?
4. Do czego służy `nullptr`?
5. Dlaczego nie wolno dereferencjonować pustego wskaźnika?

Rozdział 31

02 - Referencje

31.1 Cel lekcji

Celem lekcji jest zrozumienie, czym jest referencja, jak różni się od wskaźnika i kiedy warto jej używać.

31.2 Wymagania wstępne

Student powinien znać:

- zmienne i typy danych,
- funkcje,
- podstawy wskaźników z lekcji 01-adresy-i-wskazniki.md.

31.3 Krótka teoria

31.3.1 Czym jest referencja

Referencja to inna nazwa dla istniejącej zmiennej.

```
int liczba = 10;  
int& ref = liczba;
```

ref nie jest osobną kopią wartości. ref odnosi się do tej samej zmiennej co liczba.

```
ref = 20;  
std::cout << liczba << std::endl; // 20
```

Pełny przykład znajduje się w pliku examples/reference_alias.cpp.

31.3.2 Referencję trzeba zainicjalizować

Referencja musi od razu odnosić się do istniejącej zmiennej.

Poprawnie:

```
int liczba = 10;  
int& ref = liczba;
```

Niepoprawnie:

```
int& ref; // błąd: referencja musi być zainicjalizowana
```

31.3.3 Referencji nie przepinamy

Po utworzeniu referencja pozostaje aliasem tej samej zmiennej.

```
int a = 1;
int b = 2;

int& ref = a;
ref = b;
```

Ostatnia linia nie sprawia, że `ref` zaczyna odnosić się do `b`. Ona przypisuje wartość `b` do `a`.

Po wykonaniu tego kodu:

```
a == 2
b == 2
```

31.3.4 Referencja a wskaźnik

Cecha	Referencja	Wskaźnik
Wymaga inicjalizacji	tak	nie
Może być <code>nullptr</code>	nie	tak
Dostęp do wartości	przez nazwę	przez <code>*</code>
Można przepiąć na inny obiekt	nie	tak

Referencja jest wygodna, gdy funkcja ma pracować na istniejącej zmiennej i nie chcemy obsługiwać `nullptr`.

Wskaźnik jest potrzebny, gdy wartość może być pusta albo gdy chcemy jawnie pokazać pracę na adresach.

31.4 Przykład kodu: referencje w funkcjach

Referencje często służą do modyfikowania argumentów przekazanych do funkcji.

```
void zwieksz(int& liczba)
{
    liczba++;
}
```

Wywołanie:

```
int x = 5;
zwieksz(x);
std::cout << x << std::endl; // 6
```

Pełny przykład znajduje się w pliku `examples/reference_increment.cpp`.

31.5 Przykład kodu: zamiana wartości przez referencje

```
void zamien(int& a, int& b)
{
    int temp = a;
    a = b;
    b = temp;
}
```

Ta funkcja modyfikuje oryginalne zmienne przekazane przy wywołaniu.

Pełny przykład znajduje się w pliku `examples/reference_swap.cpp`.

31.5.1 `const` referencja

Jeżeli funkcja ma tylko odczytać argument i nie powinna go zmieniać, można użyć `const`.

```
void wypisz(const std::string& tekst)
{
    std::cout << tekst << std::endl;
}
```

`const std::string&` oznacza: przyjmij istniejący napis bez kopiowania i nie pozwalaj funkcji go zmienić. To jest szczególnie przydatne dla większych obiektów, takich jak `std::string`.

31.6 Zadania do wykonania

1. Utwórz zmienną `int` i referencję do niej. Zmień wartość przez referencję i wypisz oryginalną zmienną.
2. Napisz funkcję `zwiększ`, która przyjmuje `int&` i zwiększa wartość o 1.
3. Napisz funkcję `wyzeruj`, która przyjmuje `int&` i ustawia wartość na 0.
4. Napisz funkcję `zamien`, która przyjmuje dwie referencje do `int` i zamienia wartości miejscami.
5. Napisz funkcję `wypiszTekst`, która przyjmuje `const std::string&` i wypisuje tekst.

31.7 Ćwiczenia dodatkowe

1. Dodaj funkcję zwiększającą wartość przez referencję.
2. Porównaj wynik przekazania przez wartość i przez referencję.
3. Dodaj przykład referencji `const` dla wartości tylko do odczytu.

31.8 Kryteria zaliczenia

Program powinien:

- poprawnie deklarować referencje,
- inicjalizować referencje przy deklaracji,
- używać referencji do modyfikowania argumentów funkcji,
- używać `const` referencji tam, gdzie funkcja nie powinna zmieniać argumentu,
- kompilować się bez błędów.

31.9 Pytania kontrolne

1. Czym jest referencja?
2. Czy referencja może być niezainicjalizowana?
3. Czy referencję można przepiąć na inną zmienną?
4. Czym różni się referencja od wskaźnika?
5. Po co stosuje się `const std::string&`?

Rozdział 32

03 - Przekazywanie danych do funkcji

32.1 Cel lekcji

Celem lekcji jest porównanie trzech sposobów przekazywania danych do funkcji: przez wartość, przez wskaźnik i przez referencję.

32.2 Wymagania wstępne

Student powinien znać:

- funkcje z segmentu 03-funkcje-tablice-napisy,
- wskaźniki z lekcji 01-adresy-i-wskazniki.md,
- referencje z lekcji 02-referencje.md.

32.3 Krótka teoria

32.3.1 Przekazywanie przez wartość

Przy przekazywaniu przez wartość funkcja dostaje kopię argumentu.

```
void ustawNaZero(int liczba)
{
    liczba = 0;
}
```

Wywołanie:

```
int x = 5;
ustawNaZero(x);
std::cout << x << std::endl; // 5
```

Oryginalna zmienna `x` nie zmienia się, bo funkcja modyfikuje tylko kopię.

Pełny przykład znajduje się w pliku `examples/pass_by_value_no_change.cpp`.

32.3.2 Przekazywanie przez wskaźnik

Funkcja może otrzymać adres zmiennej. Wtedy może zmienić oryginalną wartość.

```
void ustawNaZero(int* liczba)
{
    if (liczba != nullptr)
    {
```

```

        *liczba = 0;
    }
}

```

Wywołanie:

```

int x = 5;
ustawNaZero(&x);
std::cout << x << std::endl; // 0

```

W tej wersji funkcja przyjmuje wskaźnik, więc trzeba przekazać adres zmiennej operatorem `&`.

32.3.3 Przekazywanie przez referencję

Funkcja może otrzymać referencję do zmiennej. Wtedy pracuje bezpośrednio na oryginalnej zmiennej.

```

void ustawNaZero(int& liczba)
{
    liczba = 0;
}

```

Wywołanie:

```

int x = 5;
ustawNaZero(x);
std::cout << x << std::endl; // 0

```

Dla użytkownika funkcji wywołanie wygląda prościej niż przy wskaźniku, bo nie trzeba pisać `&x`.

32.4 Przykład kodu: zamiana wartości przez wskaźniki

```

void zamien(int* a, int* b)
{
    if (a == nullptr || b == nullptr)
    {
        return;
    }

    int temp = *a;
    *a = *b;
    *b = temp;
}

```

Pełny przykład znajduje się w pliku `examples/swap_by_pointer.cpp`.

32.5 Przykład kodu: zamiana wartości przez referencje

```

void zamien(int& a, int& b)
{
    int temp = a;
    a = b;
    b = temp;
}

```

Pełny przykład znajduje się w pliku `examples/swap_by_reference.cpp`.

32.6 Przykład kodu: zwracanie więcej niż jednego wyniku

Funkcja może zwrócić tylko jedną wartość przez `return`, ale dodatkowe wyniki można przekazać przez referencje lub wskaźniki.

Przykład przez referencje:


```
void sumaIloczyn(int a, int b, int& suma, int& iloczyn)
{
    suma = a + b;
    iloczyn = a * b;
}
```

Wywołanie:

```
int suma;
int iloczyn;
```

```
sumaIloczyn(3, 4, suma, iloczyn);
```

Pełny przykład znajduje się w pliku `examples/multiple_results_reference.cpp`.

32.6.1 Kiedy używać którego sposobu

Sposób	Kiedy używać
przez wartość	gdy funkcja nie ma zmieniać argumentu
przez wskaźnik	gdy argument może być pusty albo chcemy jawnie pracować na adresie
przez referencję	gdy funkcja ma zmienić argument i argument musi istnieć
przez <code>const</code> referencję	gdy chcemy uniknąć kopii, ale nie zmieniać argumentu

32.7 Zadania do wykonania

1. Napisz funkcję `ustawNaZero` w trzech wersjach: przez wartość, przez wskaźnik i przez referencję. Porównaj wyniki.
2. Napisz funkcję, która zamienia miejscami dwie liczby przekazane przez wskaźniki.
3. Napisz funkcję, która zamienia miejscami dwie liczby przekazane przez referencje.
4. Napisz funkcję, która przyjmuje dwie liczby i zwraca przez referencje ich sumę oraz iloczyn.
5. Napisz funkcję, która przyjmuje wskaźnik na `int` i zwiększa wartość tylko wtedy, gdy wskaźnik nie jest `nullptr`.

32.8 Ćwiczenia dodatkowe

1. Dodaj funkcję zwracającą dwa wyniki przez referencje.
2. Zmień funkcję ze wskaźnikiem tak, aby sprawdzała `nullptr`.
3. Porównaj czytelność wersji ze wskaźnikiem i wersji z referencją.

32.9 Kryteria zaliczenia

Program powinien:

- rozróżniać przekazywanie przez wartość, wskaźnik i referencję,
- sprawdzać wskaźnik przed dereferencją,
- używać referencji do argumentów, które muszą istnieć,
- kompilować się bez błędów,
- jasno pokazywać, które funkcje zmieniają oryginalne zmienne.

32.10 Pytania kontrolne

1. Dlaczego funkcja przyjmująca argument przez wartość nie zmienia oryginalnej zmiennej?
2. Po co przy wywołaniu funkcji wskaźnikowej używamy `&x`?

3. Dlaczego wskaźnik trzeba sprawdzić przed dereferencją?
4. Kiedy referencja jest wygodniejsza od wskaźnika?
5. Jak można zwrócić z funkcji więcej niż jeden wynik?

Rozdział 33

04 - Bezpieczeństwo pamięci

33.1 Cel lekcji

Celem lekcji jest poznanie podstawowych zasad bezpiecznej pracy ze wskaźnikami i referencjami.

33.2 Wymagania wstępne

Student powinien znać:

- wskaźniki z lekcji 01-adresy-i-wskazniki.md,
- referencje z lekcji 02-referencje.md,
- przekazywanie argumentów z lekcji 03-przekazywanie-do-funkcji.md.

33.3 Krótka teoria

33.3.1 Zasada 1: inicjalizuj wskaźniki

Nie zostawiaj wskaźnika bez wartości początkowej.

Niepoprawnie:

```
int* p;
```

Poprawnie:

```
int* p = nullptr;
```

Jeżeli wskaźnik nie wskazuje jeszcze na poprawną zmienną, ustaw go na `nullptr`.

33.3.2 Zasada 2: sprawdzaj `nullptr`

Przed dereferencją wskaźnika sprawdź, czy nie jest pusty.

```
if (p != nullptr)
{
    std::cout << *p << std::endl;
}
```

Pełny przykład znajduje się w pliku `examples/safe__pointer__read.cpp`.

33.3.3 Zasada 3: nie zwracaj adresu zmiennej lokalnej

Zmienna lokalna przestaje istnieć po zakończeniu funkcji.

Niepoprawny pomysł:

```
int* niepoprawnie()
{
    int lokalna = 10;
    return &lokalna;
}
```

Po wyjściu z funkcji `lokalna` już nie istnieje. Wskaźnik zwrócony z takiej funkcji wskazywałby na niepoprawne miejsce.

Bezpieczniej zwrócić wartość:

```
int poprawnie()
{
    int lokalna = 10;
    return lokalna;
}
```

33.3.4 Zasada 4: nie wychodź poza zakres tablicy

Dla tablicy:

```
int tablica[3] = {1, 2, 3};
```

poprawne indeksy to 0, 1, 2.

Ten kod jest błędny:

```
tablica[3] = 10;
```

Podobny problem może wystąpić przy pracy ze wskaźnikiem na pierwszy element tablicy.

```
int* p = tablica;
```

`p[0]`, `p[1]`, `p[2]` są poprawne. `p[3]` jest poza zakresem.

Pełny przykład bezpiecznego przejścia po tablicy znajduje się w pliku `examples/safe_array_pointer.cpp`.

33.3.5 Zasada 5: używaj referencji, gdy argument musi istnieć

Jeżeli funkcja wymaga poprawnej zmiennej, referencja jest często bezpieczniejsza i czytelniejsza niż wskaźnik.

```
void zwieksz(int& liczba)
{
    liczba++;
}
```

Referencja nie może być `nullptr`, więc nie trzeba sprawdzać pustego argumentu.

Wskaźnik wybierz wtedy, gdy brak wartości jest dopuszczalny:

```
void zwiekszJesliIstnieje(int* liczba)
{
    if (liczba != nullptr)
    {
        (*liczba)++;
    }
}
```

Pełny przykład znajduje się w pliku `examples/safe_optional_pointer.cpp`.

33.3.6 Zasada 6: ograniczaj zakres zmiennych

Twórz zmienne możliwie blisko miejsca użycia.

```

if (warunek)
{
    int wynik = 10;
    std::cout << wynik << std::endl;
}

```

Im mniejszy zakres zmiennej, tym mniejsze ryzyko przypadkowego użycia jej w złym miejscu.

33.4 Dobre praktyki

- Ustawiaj pusty wskaźnik na `nullptr`.
- Sprawdzaj wskaźnik przed dereferencją.
- Nie zwracaj adresów zmiennych lokalnych.
- Nie wychodź poza zakres tablicy.
- Używaj referencji, gdy argument musi istnieć.
- Używaj `const` referencji, gdy funkcja ma tylko czytać większy obiekt.
- Pisz małe funkcje, które jasno pokazują, kto modyfikuje dane.

33.5 Zadania do wykonania

1. Napisz funkcję `wypiszJesliIstnieje`, która przyjmuje `int*` i wypisuje wartość tylko wtedy, gdy wskaźnik nie jest `nullptr`.
2. Napisz funkcję `zwiększJesliIstnieje`, która przyjmuje `int*` i zwiększa wartość tylko wtedy, gdy wskaźnik nie jest `nullptr`.
3. Napisz program, który bezpiecznie przechodzi po tablicy przez wskaźnik i jej rozmiar.
4. Napisz funkcję `zwiększ`, która przyjmuje `int&`. Porównaj ją z wersją wskaźnikową.
5. Znajdź w swoich wcześniejszych przykładach miejsca, w których można użyć `const` referencji.

33.6 Ćwiczenia dodatkowe

1. Dodaj sprawdzanie wskaźnika przed każdym odczytem.
2. Zmień przykład tak, aby funkcja zwracała informację o błędzie.
3. Wypisz komunikat diagnostyczny, gdy dane wejściowe są niepoprawne.

33.7 Kryteria zaliczenia

Program powinien:

- inicjalizować wskaźniki,
- sprawdzać `nullptr` przed dereferencją,
- nie zwracać adresów zmiennych lokalnych,
- nie wychodzić poza zakres tablic,
- kompilować się bez błędów.

33.8 Pytania kontrolne

1. Dlaczego warto inicjalizować wskaźnik wartością `nullptr`?
2. Co może się stać przy dereferencji pustego wskaźnika?
3. Dlaczego nie wolno zwracać adresu zmiennej lokalnej?
4. Co oznacza wyjście poza zakres tablicy?
5. Kiedy referencja jest bezpieczniejsza niż wskaźnik?

Rozdział 34

05 - Zadania

34.1 Cel zestawu zadań

Ten plik zbiera zadania do segmentu 04 - Wskaźniki, referencje, pamięć. Zadania sprawdzają rozumienie adresów, wskaźników, referencji, przekazywania argumentów do funkcji oraz podstawowych zasad bezpieczeństwa pamięci.

34.2 Zasady wykonania

Każde zadanie powinno być zapisane w osobnym pliku `.cpp`. Program powinien kompilować się bez błędów i wypisywać wynik w czytelnej formie.

Przykład kompilacji:

```
c++ -std=c++17 -Wall -Wextra -pedantic nazwa_pliku.cpp -o program
./program
```

W zadaniach ze wskaźnikami pamiętaj o sprawdzaniu `nullptr`, jeśli funkcja dopuszcza brak wartości.

Przed oddaniem zadania sprawdź:

- czy każdy wskaźnik jest sprawdzony przed dereferencją, jeśli może być pusty,
- czy funkcje nie zwracają adresów zmiennych lokalnych,
- czy tablice są przekazywane razem z rozmiarem,
- czy indeksy tablic są sprawdzane przed użyciem,
- czy w komentarzach nie ma nieprawdziwych założeń o czasie życia zmiennych.

34.3 Poziom 1 - Adresy i wskaźniki

34.3.1 Zadanie 1. Adres zmiennej

Utwórz zmienną typu `int`, przypisz jej wartość i wypisz:

- wartość zmiennej,
- adres zmiennej,
- wartość odczytaną przez wskaźnik.

Wymagania:

- użyj operatora `&` do pobrania adresu,
- użyj operatora `*` do dereferencji,
- w komentarzu opisz różnicę między wartością zmiennej a jej adresem.

34.3.2 Zadanie 2. Zmiana przez wskaźnik

Utwórz zmienną `liczba` i wskaźnik wskazujący na tę zmienną. Zmień wartość zmiennej przez wskaźnik i wypisz wynik przed oraz po zmianie.

Scenariusz sprawdzenia:

Przed: 10

Po: 25

34.3.3 Zadanie 3. `nullptr`

Napisz funkcję `wypiszWartosc`, która przyjmuje wskaźnik do `int`.

Funkcja powinna:

- wypisać wartość, jeśli wskaźnik nie jest pusty,
- wypisać komunikat `Brak wartosci`, jeśli wskaźnik ma wartość `nullptr`.

W `main` wywołaj funkcję dwa razy: raz z adresem poprawnej zmiennej i raz z `nullptr`.

34.3.4 Zadanie 4. Wskaźnik do elementu tablicy

Utwórz tablicę pięciu liczb całkowitych. Ustaw wskaźnik na pierwszy element tablicy i wypisz wszystkie elementy, używając wskaźnika oraz indeksu.

Wymagania:

- użyj zapisu `*(wskaznik + i)`,
- nie wychodź poza zakres pięciu elementów,
- wypisz indeks i wartość elementu.

34.3.5 Zadanie 5. Największa wartość przez wskaźnik

Napisz funkcję `wskaznikNaWieksza`, która przyjmuje dwa wskaźniki do `int` i zwraca wskaźnik do większej wartości.

Założ, że oba wskaźniki muszą wskazywać poprawne zmienne. W funkcji `main` pokaż użycie funkcji.

Rozszerzenie: obsłuż sytuację, w której jeden ze wskaźników jest `nullptr`.

34.4 Poziom 2 - Referencje

34.4.1 Zadanie 6. Zmiana przez referencję

Napisz funkcję `zwieksz`, która przyjmuje referencję do `int` i zwiększa wartość zmiennej o 1.

W `main` pokaż wartość przed i po wywołaniu funkcji.

34.4.2 Zadanie 7. Zamiana wartości

Napisz funkcję `zamien`, która przyjmuje dwie referencje do `int` i zamienia wartości zmiennych.

Nie używaj `std::swap`.

Scenariusz sprawdzenia:

Przed: `a = 7, b = 12`

Po: `a = 12, b = 7`

34.4.3 Zadanie 8. Minimum i maksimum

Napisz funkcję `minimumMaksimum`, która przyjmuje tablicę, jej rozmiar oraz dwie referencje wyjściowe:

- `minimum`,
- `maksimum`.

Funkcja powinna zapisać w referencjach najmniejszą i największą wartość z tablicy.

Wymagania:

- jeśli rozmiar tablicy jest mniejszy lub równy 0, wypisz komunikat o błędzie,
- nie próbuj odczytywać pierwszego elementu pustej tablicy,
- sprawdź tablicę z liczbami dodatnimi i ujemnymi.

34.4.4 Zadanie 9. Poprawianie wyniku

Napisz funkcję `ograniczDoZakresu`, która przyjmuje referencję do liczby oraz dwie granice: `min` i `max`.

Jeśli liczba jest mniejsza niż `min`, funkcja powinna ustawić ją na `min`. Jeśli jest większa niż `max`, powinna ustawić ją na `max`.

Scenariusze sprawdzenia:

- -5 przy zakresie 0..100 daje 0,
- 120 przy zakresie 0..100 daje 100,
- 60 przy zakresie 0..100 zostaje 60.

34.4.5 Zadanie 10. Referencja czy wskaźnik

Napisz dwie wersje funkcji zwiększającej liczbę o podaną wartość:

- wersję przyjmującą wskaźnik,
- wersję przyjmującą referencję.

W komentarzu w kodzie napisz, kiedy wybrałbyś wskaźnik, a kiedy referencję.

Wymagania:

- wersja wskaźnikowa ma obsługiwać `nullptr`,
- wersja referencyjna zakłada, że obiekt zawsze istnieje,
- w `main` pokaż oba wywołania.

34.5 Poziom 3 - Przekazywanie argumentów do funkcji

34.5.1 Zadanie 11. Przekazanie przez wartość

Napisz funkcję, która próbuje zmienić wartość argumentu przekazanego przez wartość. Pokaż w `main`, że oryginalna zmienna się nie zmieniła.

Wynik programu powinien jasno pokazać dwie wartości: wewnątrz funkcji i po powrocie do `main`.

34.5.2 Zadanie 12. Przekazanie przez wskaźnik

Napisz funkcję `ustawZero`, która przyjmuje wskaźnik do `int` i ustawia wartość zmiennej na 0, jeśli wskaźnik nie jest pusty.

W `main` wywołaj funkcję dla poprawnego wskaźnika i dla `nullptr`.

34.5.3 Zadanie 13. Przekazanie przez referencję

Napisz funkcję `podwoj`, która przyjmuje referencję do `int` i podwaja wartość zmiennej.

34.5.4 Zadanie 14. Dwa wyniki z funkcji

Napisz funkcję `sumaIloczyn`, która przyjmuje dwie liczby i zwraca dwa wyniki:

- sumę,
- iloczyn.

Jeden wynik zwróć przez `return`, a drugi przez parametr wskaźnikowy albo referencję.

Przykład:

Dane: 4 i 5
Suma: 9
Iloczyn: 20

34.5.5 Zadanie 15. Warunkowa zamiana

Napisz funkcję przyjmującą wskaźniki do dwóch zmiennych typu `int`. Funkcja powinna zamienić wartości tylko wtedy, gdy druga wartość jest mniejsza od pierwszej.

Przykład:

przed: a = 10, b = 3
po: a = 3, b = 10

Wymagania:

- sprawdź oba wskaźniki przed użyciem,
- jeśli któryś wskaźnik jest pusty, nie zmieniaj danych,
- jeśli wartości są już w kolejności rosnącej, nie zamieniaj ich.

34.6 Poziom 4 - Bezpieczeństwo pamięci

34.6.1 Zadanie 16. Bezpieczne wypisywanie tablicy

Napisz funkcję `wypiszTablice`, która przyjmuje wskaźnik do pierwszego elementu tablicy oraz jej rozmiar.

Funkcja powinna:

- sprawdzić, czy wskaźnik nie jest pusty,
- sprawdzić, czy rozmiar jest większy od 0,
- wypisać elementy tylko wtedy, gdy dane wejściowe są poprawne.

Scenariusze sprawdzenia:

- poprawna tablica i rozmiar 5 - wypisuje elementy,
- `nullptr` i rozmiar 5 - wypisuje komunikat o błędzie,
- poprawna tablica i rozmiar 0 - wypisuje komunikat o błędzie.

34.6.2 Zadanie 17. Bezpieczna suma tablicy

Napisz funkcję `sumaTablicy`, która przyjmuje wskaźnik do tablicy oraz rozmiar.

Jeśli wskaźnik jest pusty albo rozmiar jest niepoprawny, funkcja powinna zwrócić 0.

Scenariusz sprawdzenia:

Tablica: 2 4 6
Suma: 12
`nullptr`: 0
rozmiar 0: 0

34.6.3 Zadanie 18. Nie wychodź poza tablicę

Napisz program, który wczytuje indeks elementu tablicy pięcioelementowej.

Program powinien:

- sprawdzić, czy indeks jest w zakresie,
- wypisać element tylko dla poprawnego indeksu,
- wypisać komunikat o błędzie dla indeksu spoza zakresu.

Sprawdź indeksy 0, 4, -1 i 5.

34.6.4 Zadanie 19. Brak wskaźnika do zmiennej lokalnej

Napisz przykład funkcji, która zwraca wynik przez wartość zamiast zwracać wskaźnik do zmiennej lokalnej.

W komentarzu wyjaśnij, dlaczego zwracanie adresu zmiennej lokalnej byłoby błędem.

Wymagania:

- nie pisz funkcji, która faktycznie zwraca błędny wskaźnik,
- opisz błąd w komentarzu,
- pokaż poprawną funkcję zwracającą wynik przez wartość.

34.6.5 Zadanie 20. Wskaźnik opcjonalny

Napisz funkcję `ustawJesliPodano`, która przyjmuje wskaźnik do `int` oraz nową wartość.

Funkcja powinna zmienić zmienną tylko wtedy, gdy wskaźnik nie jest pusty.

W `main` pokaż wywołanie z adresem zmiennej i z `nullptr`.

34.7 Zadania dodatkowe

34.7.1 Zadanie 21. Statystyki tablicy

Napisz funkcję `statystyki`, która przyjmuje tablicę i jej rozmiar, a następnie zwraca przez parametry wyjściowe:

- minimum,
- maksimum,
- sumę,
- średnią.

Dobierz wskaźniki albo referencje tak, aby kod był czytelny i bezpieczny.

Wymagania:

- funkcja powinna zwracać informację, czy obliczenia się udały,
- dla pustej tablicy albo `nullptr` nie zapisuj nieprawdziwych wyników,
- średnią zwróć jako `double`.

34.7.2 Zadanie 22. Sortowanie dwóch liczb

Napisz funkcję, która przyjmuje dwie referencje i ustawia wartości w kolejności rosnącej.

Przykład:

```
przed: a = 8, b = 2
po:    a = 2, b = 8
```

34.7.3 Zadanie 23. Proste menu z funkcjami

Napisz program z menu:

- 1 - pokaż wartość,
- 2 - zwiększ przez referencję,
- 3 - ustaw przez wskaźnik,
- 0 - zakończ.

Program powinien przechowywać jedną zmienną typu `int` i przekazywać ją do funkcji różnymi sposobami.

34.8 Wariant minimum

Do zaliczenia segmentu wykonaj co najmniej:

1. Zadanie 1 albo 2.
2. Zadanie 3.
3. Zadanie 6 albo 7.
4. Zadanie 8 albo 9.
5. Zadanie 12 albo 14.
6. Zadanie 16 albo 17.
7. Mini-sprawdzian segmentu z końca pliku.

Zadania 21-23 są dodatkowe. Możesz je robić po wykonaniu wariantu minimum.

34.9 Kryteria zaliczenia segmentu

Student zalicza segment, jeśli potrafi samodzielnie:

- wypisać adres zmiennej,
- utworzyć wskaźnik i odczytać wartość przez dereferencję,
- sprawdzić wskaźnik przed dereferencją,
- użyć `nullptr`,
- zmienić wartość zmiennej przez wskaźnik,
- zmienić wartość zmiennej przez referencję,
- dobrać wskaźnik albo referencję do prostego przypadku,
- przekazać wynik z funkcji przez parametr wyjściowy,
- nie wychodzić poza zakres tablicy,
- wyjaśnić, dlaczego nie wolno zwracać adresu zmiennej lokalnej.

34.10 Mini-sprawdzian segmentu

Napisz program z trzema funkcjami, który:

- zamienia dwie wartości przez referencje,
- próbuje ustawić wartość przez wskaźnik po sprawdzeniu `nullptr`,
- zwraca dwa wyniki przez parametry wyjściowe,
- wypisuje adresy i wartości przed oraz po zmianie,
- kompiluje się z `-Wall -Wextra -pedantic`.

Wymagania szczegółowe:

- funkcja zamiany ma używać referencji,
- funkcja ustawiania przez wskaźnik ma obsługiwać `nullptr`,
- funkcja z dwoma wynikami ma zwracać jeden wynik przez referencję, a drugi przez wskaźnik albo drugą referencję,
- program ma pokazać przypadek poprawnego wskaźnika i `nullptr`.

Scenariusz sprawdzenia:

```
Przed zamiana: a = 4, b = 9
Po zamianie:   a = 9, b = 4
Ustawiono przez wskaznik: 100
Brak wskaznika - bez zmiany
Suma: 13
Iloczyn: 36
```

34.11 Checklista oddania

Przed oddaniem rozwiązań sprawdź:

- ☐ Każde zadanie jest w osobnym pliku `.cpp`.
- ☐ Programy kompilują się z `-Wall -Wextra -pedantic`.
- ☐ Wskaźniki są sprawdzane przed dereferencją, jeśli mogą być `nullptr`.
- ☐ Funkcje nie zwracają adresów zmiennych lokalnych.
- ☐ Tablice są przekazywane razem z rozmiarem.

- ☐ Program nie odczytuje tablicy poza zakresem.
- ☐ W kodzie jest jasne, kiedy użyto wskaźnika, a kiedy referencji.

34.12 Archiwum

Oryginalne materiały źródłowe tego segmentu są zachowane w katalogu archive.

Rozdział 35

05 - Pliki i wyjątki

Ten segment pokazuje, jak program może zapisywać dane poza pamięcią operacyjną oraz jak reagować na sytuacje błędne. Materiał łączy obsługę plików tekstowych z podstawami walidacji, kontroli błędów i wyjątków.

35.1 Cele segmentu

Po zakończeniu segmentu student powinien umieć:

- otworzyć plik tekstowy do odczytu,
- otworzyć plik tekstowy do zapisu,
- dopisać dane na końcu pliku,
- odczytać plik linia po linii,
- sprawdzić, czy operacja na pliku się udała,
- rozumieć różnicę między błędem możliwym do obsłużenia a błędem programistycznym,
- użyć podstawowego bloku `try`, `throw`, `catch`,
- dobrać prostą strategię obsługi błędów do zadania.

35.2 Kolejność lekcji

1. 01-pliki-tekstowe.md - zapis, odczyt i dopisywanie danych.
2. 02-odczyt-i-walidacja.md - sprawdzanie błędów wejścia i stanu pliku.
3. 03-wyjątki.md - podstawy `try`, `throw`, `catch`.
4. 04-zadanie-menu-plikowe.md - mały program z menu do zarządzania plikiem.
5. 05-zadania.md - zadania podstawowe i dodatkowe.

35.3 Status porządkowania

Lekcje i zadania zostały uporządkowane do wspólnego formatu segmentów. Pierwotne wersje materiałów są zachowane w katalogu `archive`:

- `archive/pliki-tekstowe.md`
- `archive/pliki-tekstowe-zadanie.md`
- `archive/zadanie-io-plik.md`
- `archive/wyjątki.md`

35.4 Przykłady kodu

Przykłady w katalogu `examples` można kompilować osobno:

- `examples/write_text_file.cpp` - zapis do pliku.
- `examples/append_text_file.cpp` - dopisywanie danych.

- `examples/read_text_file.cpp` - odczyt liniami.
- `examples/missing_file_check.cpp` - sprawdzanie braku pliku.
- `examples/read_numbers_with_validation.cpp` - odczyt liczb z walidacją.
- `examples/validate_lines.cpp` - walidacja danych w liniach.
- `examples/config_key_value.cpp` - parser konfiguracji `klucz=wartosc`.
- `examples/basic_exception.cpp` - podstawowy wyjątek.
- `examples/function_exception.cpp` - rzucanie wyjątku z funkcji.
- `examples/standard_exception.cpp` - obsługa wyjątku standardowego.
- `examples/file_menu.cpp` - program z menu plikowym.

35.5 Testy

Katalog `tests` zawiera prosty test walidacji danych wejściowych bez zewnętrznego frameworka. Test pokazuje, jak wydzielić parsowanie i sprawdzanie zakresu do funkcji, którą można uruchomić bez otwierania pliku.

35.6 Format lekcji

Każda docelowa lekcja powinna mieć taki układ:

1. Cel lekcji.
2. Wymagania wstępne.
3. Krótka teoria.
4. Przykład kodu.
5. Zadania do wykonania.
6. Kryteria zaliczenia.

35.7 Katalogi pomocnicze

- `examples/` - kompilowalne przykłady `.cpp` dla tego segmentu.
- `tests/` - proste testy automatyczne walidacji danych.
- `archive/` - pierwotne wersje materiałów zachowane do wglądu.

Rozdział 36

01 - Pliki tekstowe

36.1 Cel lekcji

Celem lekcji jest poznanie podstaw zapisu, odczytu i dopisywania danych do pliku tekstowego w C++.

36.2 Wymagania wstępne

Student powinien znać:

- zmienne i typy danych z segmentu 01-podstawy,
- instrukcje warunkowe i pętle z segmentu 02-sterowanie-i-petle,
- funkcje i napisy z segmentu 03-funkcje-tablice-napisy.

36.3 Krótka teoria

36.3.1 Biblioteka `<fstream>`

Do pracy z plikami tekstowymi używamy nagłówka:

```
#include <fstream>
```

Najczęściej używane klasy to:

- `std::ofstream` - zapis do pliku,
- `std::ifstream` - odczyt z pliku,
- `std::fstream` - odczyt i zapis w jednym strumieniu.

Na początku warto rozdzielać zapis i odczyt, bo kod jest wtedy prostszy do zrozumienia.

36.4 Przykład kodu: zapis do pliku

Plik można otworzyć do zapisu przy pomocy `std::ofstream`.

```
std::ofstream plik("dane.txt");  
plik << "Ala ma kota" << std::endl;
```

Jeśli plik `dane.txt` nie istnieje, zostanie utworzony. Jeśli istnieje, jego poprzednia zawartość zostanie nadpisana.

Pełny przykład znajduje się w pliku `examples/write_text_file.cpp`.

36.4.1 Sprawdzanie otwarcia pliku

Operacja otwarcia pliku może się nie udać. Program powinien to sprawdzić przed zapisem albo odczytem.

```
std::ofstream plik("dane.txt");

if (!plik)
{
    std::cout << "Nie mozna otworzyc pliku" << std::endl;
    return 1;
}
```

Warunek `!plik` oznacza, że strumień jest w stanie błędu.

36.5 Przykład kodu: dopisywanie do pliku

Domyślny zapis przez `std::ofstream` nadpisuje plik. Jeśli chcemy dopisać dane na końcu pliku, używamy trybu `std::ios::app`.

```
std::ofstream plik("dane.txt", std::ios::app);
plik << "Nowy wpis" << std::endl;
```

Pełny przykład znajduje się w pliku `examples/append_text_file.cpp`.

36.6 Przykład kodu: odczyt z pliku

Plik można otworzyć do odczytu przy pomocy `std::ifstream`.

```
std::ifstream plik("dane.txt");
std::string linia;

while (std::getline(plik, linia))
{
    std::cout << linia << std::endl;
}
```

`std::getline` czyta całą linię tekstu, razem ze spacjami wewnątrz linii.

Pełny przykład znajduje się w pliku `examples/read_text_file.cpp`.

36.6.1 Odczyt liniami i odczyt słowami

Dla plików tekstowych są dwa częste sposoby odczytu.

Odczyt liniami:

```
std::getline(plik, linia);
```

Ten sposób zachowuje spacje w linii.

Odczyt słowami:

```
plik >> slowo;
```

Ten sposób pomija białe znaki i czyta kolejne słowa osobno.

W zadaniach z tekstem opisowym zwykle lepszy jest `std::getline`.

36.6.2 Zamykanie pliku

Strumień plikowy zamyka plik automatycznie, gdy obiekt wychodzi poza zakres.

```
{
    std::ofstream plik("dane.txt");
    plik << "tekst" << std::endl;
} // tutaj plik zostaje zamknięty
```

Można też wywołać `close()`, ale w prostych programach często nie jest to potrzebne.

36.7 Typowe błędy

36.7.1 Brak sprawdzenia otwarcia pliku

Niepoprawnie:

```
std::ifstream plik("brak.txt");
std::string linia;
std::getline(plik, linia);
```

Poprawnie:

```
std::ifstream plik("brak.txt");

if (!plik)
{
    std::cout << "Nie mozna otworzyc pliku" << std::endl;
    return 1;
}
```

36.7.2 Nadpisanie pliku zamiast dopisania

Niepoprawnie, jeśli chcemy zachować stare dane:

```
std::ofstream plik("dane.txt");
```

Poprawnie:

```
std::ofstream plik("dane.txt", std::ios::app);
```

36.7.3 Użycie >> tam, gdzie potrzebna jest cała linia

Dla tekstu:

Ala ma kota

operator >> wczyta najpierw tylko Ala. Funkcja `std::getline` wczyta całą linię.

36.8 Zadania do wykonania

1. Napisz program, który tworzy plik `dane.txt` i zapisuje do niego trzy linie tekstu.
2. Napisz program, który odczytuje plik `dane.txt` linia po linii i wypisuje jego zawartość.
3. Napisz program, który dopisuje nową linię na końcu pliku `dane.txt`.
4. Rozbuduj program odczytujący plik tak, aby wypisywał numer każdej linii.
5. Napisz program, który liczy, ile linii znajduje się w pliku tekstowym.

36.9 Ćwiczenia dodatkowe

1. Zmień nazwę pliku wynikowego na `notatki.txt`.
2. Dopisz do pliku trzy linie bez usuwania poprzedniej zawartości.
3. Odczytaj plik i wypisz numery linii.

36.10 Kryteria zaliczenia

Program powinien:

- używać `std::ofstream` do zapisu,
- używać `std::ifstream` do odczytu,
- sprawdzać, czy plik został poprawnie otwarty,
- używać `std::ios::app`, gdy dane mają być dopisane,
- czytać tekst liniami przy pomocy `std::getline`,
- kompilować się bez błędów.

36.11 Pytania kontrolne

1. Czym różni się `std::ofstream` od `std::ifstream`?
2. Co stanie się z istniejącym plikiem po otwarciu go zwykłym `std::ofstream`?
3. Do czego służy `std::ios::app`?
4. Dlaczego warto sprawdzić `if (!plik)`?
5. Czym różni się `std::getline` od operatora `>>`?

Rozdział 37

02 - Odczyt i walidacja

37.1 Cel lekcji

Celem lekcji jest nauczenie się sprawdzania, czy odczyt z pliku lub wejścia użytkownika rzeczywiście się udał oraz czy wczytane dane mają oczekiwaną postać.

37.2 Wymagania wstępne

Student powinien znać:

- instrukcje warunkowe z segmentu 02-sterowanie-i-petle,
- pętle z segmentu 02-sterowanie-i-petle,
- funkcje z segmentu 03-funkcje-tablice-napisy,
- podstawowy zapis i odczyt plików z lekcji 01-pliki-tekstowe.md.

37.3 Krótka teoria

37.3.1 Czym jest walidacja

Walidacja to sprawdzenie, czy dane spełniają wymagania programu.

Przykłady wymagań:

- liczba musi być większa od 0,
- wiek musi być w sensownym zakresie,
- linia w pliku nie może być pusta,
- plik musi istnieć,
- w pliku powinny znajdować się liczby, a nie dowolny tekst.

Program nie powinien zakładać, że dane zawsze są poprawne.

37.4 Przykład kodu: sprawdzanie otwarcia pliku

Przed odczytem trzeba sprawdzić, czy plik został poprawnie otwarty.

```
std::ifstream plik("liczby.txt");

if (!plik)
{
    std::cout << "Nie mozna otworzyc pliku" << std::endl;
    return 1;
}
```

Pełny przykład znajduje się w pliku `examples/missing_file_check.cpp`.

37.5 Przykład kodu: sprawdzanie odczytu liczby

Operator >> zwraca strumień. Strumień można sprawdzić w warunku.

```
int liczba = 0;

if (std::cin >> liczba)
{
    std::cout << "Wczytano: " << liczba << std::endl;
}
else
{
    std::cout << "To nie jest poprawna liczba" << std::endl;
}
```

Ten sam mechanizm działa dla plików.

```
std::ifstream plik("liczby.txt");
int liczba = 0;

while (plik >> liczba)
{
    std::cout << liczba << std::endl;
}
```

Pełny przykład znajduje się w pliku examples/read_numbers_with_validation.cpp.

37.5.1 Walidacja zakresu

Poprawny typ danych nie zawsze oznacza poprawną wartość.

```
if (wiek < 0 || wiek > 130)
{
    std::cout << "Niepoprawny wiek" << std::endl;
}
```

Najpierw sprawdzamy, czy odczyt się udał. Dopiero potem sprawdzamy sens wartości.

```
int wiek = 0;

if (!(std::cin >> wiek))
{
    std::cout << "Wiek musi być liczbą" << std::endl;
}
else if (wiek < 0 || wiek > 130)
{
    std::cout << "Wiek poza zakresem" << std::endl;
}
else
{
    std::cout << "Wiek poprawny" << std::endl;
}
```

37.6 Przykład kodu: odczyt linii i parsowanie

Jeśli plik może zawierać błędne linie, wygodnie jest czytać go liniami, a potem analizować każdą linię osobno.

```
std::string linia;

while (std::getline(plik, linia))
{

```

```

std::istringstream strumienLinii(linia);
int liczba = 0;

if (strumienLinii >> liczba)
{
    std::cout << "Liczba: " << liczba << std::endl;
}
else
{
    std::cout << "Pominięto błędna linie" << std::endl;
}
}

```

Do `std::istringstream` potrzebny jest nagłówek:

```
#include <sstream>
```

Pełny przykład znajduje się w pliku `examples/validate_lines.cpp`.

37.6.1 Stan strumienia

Strumień może być w różnych stanach:

- poprawny odczyt,
- koniec pliku,
- błąd formatu danych,
- błąd otwarcia lub odczytu.

Na tym etapie najważniejsze są dwa wzorce:

```

if (!plik)
{
    // plik nie został otwarty albo strumień jest w stanie błęd
}

```

oraz:

```

while (plik >> liczba)
{
    // wykonuje się tylko dla poprawnie odczytanych liczb
}

```

37.7 Typowe błędy

37.7.1 Użycie wartości po nieudanym odczycie

Niepoprawnie:

```

int liczba;
std::cin >> liczba;
std::cout << liczba << std::endl;

```

Jeśli użytkownik wpisze tekst zamiast liczby, program będzie pracował na niepoprawnych danych.

Poprawnie:

```

int liczba = 0;

if (std::cin >> liczba)
{
    std::cout << liczba << std::endl;
}
else
{

```

```
std::cout << "Niepoprawne dane" << std::endl;  
}
```

37.7.2 Sprawdzenie zakresu przed sprawdzeniem typu

Najpierw sprawdzamy, czy udało się odczytać liczbę. Dopiero później sprawdzamy jej zakres.

37.7.3 Przerywanie programu przy pierwszej błędnej linii

W wielu zadaniach lepiej pominąć błędną linię, wypisać ostrzeżenie i kontynuować odczyt kolejnych danych.

37.8 Zadania do wykonania

1. Napisz program, który wczytuje liczbę z klawiatury i sprawdza, czy odczyt się udał.
2. Rozbuduj program tak, aby sprawdzał, czy liczba należy do zakresu od 1 do 100.
3. Napisz program, który odczytuje liczby z pliku `liczby.txt` i wypisuje ich sumę.
4. Rozbuduj program z zadania 3 tak, aby obsługiwał brak pliku.
5. Napisz program, który czyta plik linia po linii i wypisuje tylko te linie, które zawierają poprawne liczby całkowite.

37.9 Ćwiczenia dodatkowe

1. Dodaj komunikat z numerem błędnej linii.
2. Zmień zakres poprawnych wartości i sprawdź przypadki graniczne.
3. Policz, ile poprawnych i błędnych danych było w pliku.

37.10 Kryteria zaliczenia

Program powinien:

- sprawdzać, czy plik został otwarty,
- sprawdzać wynik operacji odczytu,
- nie używać danych po nieudanym odczycie,
- rozdzielać sprawdzenie typu danych od sprawdzenia zakresu,
- jasno informować użytkownika o błędnych danych,
- kompilować się bez błędów.

37.11 Pytania kontrolne

1. Co oznacza warunek `if (!plik)`?
2. Dlaczego nie wolno zakładać, że odczyt liczby zawsze się uda?
3. Czym różni się błąd formatu danych od wartości spoza zakresu?
4. Po co czytać plik liniami przed parsowaniem danych?
5. Kiedy lepiej pominąć błędną linię, a kiedy zakończyć program?

Rozdział 38

03 - Wyjątki

38.1 Cel lekcji

Celem lekcji jest zrozumienie podstaw mechanizmu wyjątków w C++: `try`, `throw` i `catch`, oraz rozpoznanie sytuacji, w których wyjątek jest sensownym sposobem zgłoszenia błędu.

38.2 Wymagania wstępne

Student powinien znać:

- funkcje z segmentu 03-funkcje-tablice-napisy,
- podstawy odczytu i walidacji z lekcji 02-odczyt-i-walidacja.md,
- podstawowe komunikaty o błędach zwracane przez program.

38.3 Krótka teoria

38.3.1 Po co są wyjątki

Wyjątek służy do zgłoszenia sytuacji błędnej, której funkcja nie chce albo nie potrafi obsłużyć lokalnie.

Przykład:

- funkcja ma obliczyć wynik,
- wykrywa niepoprawne dane,
- przerywa normalne wykonanie przez `throw`,
- kod wyżej obsługuje problem w bloku `catch`.

Wyjątki nie zastępują zwykłych instrukcji `if`. Dla prostych walidacji, które można obsłużyć od razu, zwykły warunek często jest czytelniejszy.

38.4 Przykład kodu: `try`, `throw`, `catch`

Podstawowy schemat wygląda tak:

```
try
{
    throw std::runtime_error("Opis błędu");
}
catch (const std::runtime_error& blad)
{
    std::cout << blad.what() << std::endl;
}
```

Do `std::runtime_error` potrzebny jest nagłówek:

```
#include <stdexcept>
```

Pełny przykład znajduje się w pliku `examples/basic_exception.cpp`.

38.5 Przykład kodu: rzucanie wyjątku z funkcji

Najczęściej wyjątek jest zgłaszany w funkcji, która wykrywa błąd.

```
double podziel(double a, double b)
{
    if (b == 0)
    {
        throw std::invalid_argument("Dzielenie przez zero");
    }

    return a / b;
}
```

Kod wywołujący funkcję decyduje, jak zareagować.

```
try
{
    std::cout << podziel(10, 0) << std::endl;
}
catch (const std::invalid_argument& blad)
{
    std::cout << blad.what() << std::endl;
}
```

Pełny przykład znajduje się w pliku `examples/function_exception.cpp`.

38.6 Przykład kodu: łapanie wyjątków standardowych

Wiele elementów biblioteki standardowej zgłasza wyjątki. Przykładem jest metoda `at()` w `std::vector`, która sprawdza zakres indeksu.

```
std::vector<int> liczby = {10, 20, 30};
std::cout << liczby.at(10) << std::endl;
```

Ten kod zgłosi wyjątek `std::out_of_range`.

Można go obsługiwać jako konkretny typ:

```
catch (const std::out_of_range& blad)
{
    std::cout << "Indeks poza zakresem" << std::endl;
}
```

albo szerzej jako `std::exception`:

```
catch (const std::exception& blad)
{
    std::cout << blad.what() << std::endl;
}
```

Pełny przykład znajduje się w pliku `examples/standard_exception.cpp`.

38.6.1 Kolejność bloków `catch`

Najpierw zapisujemy bardziej szczegółowe typy wyjątków, a później ogólniejsze.

```
catch (const std::out_of_range& blad)
{
    // konkretny blad zakresu
}
```



```

}
catch (const std::exception& blad)
{
    // inne standardowe wyjatki
}

```

Jeśli `std::exception` byłoby pierwsze, złapałoby też wyjątki bardziej szczegółowe.

38.6.2 catch (...)

Zapis:

```

catch (...)
{
    std::cout << "Nieznany blad" << std::endl;
}

```

łapie każdy wyjątek. Używamy go ostrożnie, bo nie daje informacji o typie błędu. W prostych programach lepiej łapać konkretne typy albo `std::exception`.

38.6.3 Kiedy używać wyjątków

Wyjątek ma sens, gdy:

- funkcja wykrywa błąd, ale nie powinna sama pytać użytkownika o poprawkę,
- błąd uniemożliwia normalne wykonanie operacji,
- chcemy oddzielić logikę obliczeń od obsługi błędów,
- błąd powinien zostać obsłużony wyżej w programie.

Zwykły `if` jest często lepszy, gdy:

- błąd można obsłużyć od razu,
- sprawdzamy prosty warunek wejściowy,
- niepoprawna wartość jest normalnym elementem działania programu.

38.7 Typowe błędy

38.7.1 Łapanie wyjątku przez wartość

Mniej zalecane:

```

catch (std::exception blad)
{
    std::cout << blad.what() << std::endl;
}

```

Lepiej:

```

catch (const std::exception& blad)
{
    std::cout << blad.what() << std::endl;
}

```

Łapanie przez referencję unika kopiowania i zachowuje właściwy typ wyjątku.

38.7.2 Puste catch

Niepoprawnie:

```

catch (const std::exception&)
{
}

```

Jeśli błąd jest ignorowany bez komunikatu, program może zachowywać się niezrozumiale.

38.7.3 Używanie wyjątków do zwykłego sterowania programem

Wyjątki nie powinny zastępować pętli, `if` ani menu. Służą do obsługi sytuacji błędnych.

38.8 Zadania do wykonania

1. Napisz funkcję `podziel`, która rzuca `std::invalid_argument`, gdy drugi argument jest równy 0.
2. Obsłuż wyjątek z zadania 1 w funkcji `main` i wypisz czytelny komunikat.
3. Napisz funkcję `sprawdzWiek`, która rzuca wyjątek, jeśli wiek jest mniejszy od 0 albo większy od 130.
4. Przygotuj przykład z `std::vector` i metodą `at()`, który obsługuje indeks spoza zakresu.
5. Napisz program, który próbuje otworzyć plik i rzuca wyjątek, jeśli plik nie istnieje.

38.9 Ćwiczenia dodatkowe

1. Dodaj własny komunikat do wyjątku `std::runtime_error`.
2. Obsłuż osobno `std::invalid_argument` i `std::exception`.
3. Przenieś walidację danych do osobnej funkcji rzucającej wyjątek.

38.10 Kryteria zaliczenia

Program powinien:

- używać bloku `try`,
- zgłaszać błąd przez `throw`,
- obsługiwać błąd przez `catch`,
- łapać wyjątki standardowe przez `const` referencję,
- wypisywać czytelny komunikat błędu,
- używać wyjątków tylko dla sytuacji błędnych,
- kompilować się bez błędów.

38.11 Pytania kontrolne

1. Co robi instrukcja `throw`?
2. Po co stosujemy blok `try`?
3. Co robi blok `catch`?
4. Dlaczego zwykle łapiemy wyjątki jako `const std::exception&`?
5. Kiedy zwykły `if` jest lepszy niż wyjątek?

Rozdział 39

04 - Zadanie: menu plikowe

39.1 Cel lekcji

Celem lekcji jest połączenie obsługi plików, walidacji wejścia i funkcji w jednym małym programie użytkowym.

39.2 Wymagania wstępne

Student powinien znać:

- podstawy plików tekstowych z lekcji 01-pliki-tekstowe.md,
- walidację odczytu z lekcji 02-odczyt-i-walidacja.md,
- funkcje z segmentu 03-funkcje-tablice-napisy.

39.3 Krótka teoria

Napisz program, który zarządza prostym plikiem tekstowym `dane.txt`.

Program powinien wyświetlać menu:

1. Dodaj wpis
2. Wyświetl wpisy
3. Usun wszystkie wpisy
0. Wyjdź

Użytkownik wybiera opcję, a program wykonuje odpowiednią operację na pliku.

39.3.1 Wymagane funkcje programu

Program powinien być podzielony na funkcje:

- `pokazMenu()` - wypisuje dostępne opcje,
- `dodajWpis()` - dopisuje linię tekstu do pliku,
- `wyswietlWpisy()` - odczytuje i wypisuje cały plik,
- `usunWpisy()` - czyści zawartość pliku,
- `wczytajWybor()` - pobiera i sprawdza wybór użytkownika.

Podział na funkcje jest ważny, bo dzięki temu `main` pozostaje czytelny.

39.4 Przykład kodu: dopisywanie wpisu

Dopisywanie wpisu wymaga otwarcia pliku w trybie `std::ios::app`.

```
std::ofstream plik("dane.txt", std::ios::app);  
plik << wpis << std::endl;
```

Jeśli plik nie istnieje, zostanie utworzony.

39.5 Przykład kodu: wyświetlanie wpisów

Odczyt wykonujemy linia po linii.

```
std::ifstream plik("dane.txt");
std::string linia;

while (std::getline(plik, linia))
{
    std::cout << linia << std::endl;
}
```

Warto numerować wpisy, bo wtedy wynik jest czytelniejszy.

39.6 Przykład kodu: czyszczenie pliku

Najprostszy sposób wyczyszczenia pliku to otwarcie go zwykłym `std::ofstream`.

```
std::ofstream plik("dane.txt");
```

Taki zapis nadpisuje poprzednią zawartość pustą treścią.

39.6.1 Walidacja wyboru

Nie można zakładać, że użytkownik zawsze wpisze liczbę.

```
int wybor = -1;

if (!(std::cin >> wybor))
{
    std::cout << "Niepoprawny wybor" << std::endl;
}
```

Po nieudanym odczycie trzeba wyczyścić stan `std::cin` i usunąć błędne dane z bufora. W przykładzie referencyjnym użyta jest do tego funkcja `ignorujReszteLinii()`.

39.7 Przykład referencyjny

Pełny przykład znajduje się w pliku `examples/file_menu.cpp`.

Przykład pokazuje:

- pętlę menu,
- dopisywanie wpisów,
- odczyt pliku,
- czyszczenie pliku,
- prostą walidację wyboru.

39.8 Zadania do wykonania

1. Uruchom przykład referencyjny i sprawdź wszystkie opcje menu.
2. Zmień nazwę pliku z `dane.txt` na `notatki.txt`.
3. Dodaj opcję 4. **Policz wpisy**, która wypisuje liczbę linii w pliku.
4. Dodaj komunikat **Brak wpisow**, jeśli plik jest pusty.
5. Rozbuduj wpis tak, aby przed tekstem zapisywany był numer wpisu.

39.9 Ćwiczenia dodatkowe

1. Dodaj opcję liczenia wpisów w pliku.
2. Dodaj walidację pustego wpisu.
3. Zmień program tak, aby nazwa pliku była stałą na początku programu.

39.10 Kryteria zaliczenia

Program powinien:

- wyświetlać menu w pętli,
- obsługiwać wybór 0 jako zakończenie programu,
- dopisywać wpisy do pliku,
- wyświetlać wszystkie wpisy z pliku,
- czyścić zawartość pliku,
- sprawdzać błędny wybór użytkownika,
- być podzielony na funkcje,
- kompilować się bez błędów.

39.11 Pytania kontrolne

1. Dlaczego dopisywanie wymaga `std::ios::app`?
2. Dlaczego `main` nie powinien zawierać całej logiki programu?
3. Co robi zwykle otwarcie pliku przez `std::ofstream`?
4. Dlaczego trzeba obsługiwać niepoprawny wybór z menu?
5. Jak sprawdzić, czy plik jest pusty?

Rozdział 40

05 - Zadania

40.1 Cel zestawu zadań

Ten plik zbiera zadania do segmentu 05 - Pliki i wyjątki. Zadania sprawdzają zapis i odczyt plików tekstowych, walidację danych, obsługę błędów oraz podstawy wyjątków.

40.2 Zasady wykonania

Każde zadanie powinno być zapisane w osobnym pliku `.cpp`. Program powinien kompilować się bez błędów i wypisywać wynik w czytelnej formie.

Przykład kompilacji:

```
c++ -std=c++17 -Wall -Wextra -pedantic nazwa_pliku.cpp -o program
./program
```

Jeśli program tworzy pliki pomocnicze, używaj prostych nazw, np. `dane.txt`, `liczby.txt`, `raport.txt`. Nie dodawaj plików wynikowych ani lokalnych danych testowych do commita, jeśli prowadzący nie poprosił o to wprost.

Przed oddaniem zadania sprawdź:

- czy program reaguje czytelnie na brak pliku,
- czy błędne dane nie kończą programu awaryjnie,
- czy pliki są zamykane automatycznie przez obiekty strumieni,
- czy funkcje odczytu i walidacji są wydzielone z `main`,
- czy przykładowe pliki wejściowe są małe i opisane.

40.3 Poziom 1 - Zapis i odczyt pliku

40.3.1 Zadanie 1. Pierwszy zapis

Napisz program, który tworzy plik `dane.txt` i zapisuje do niego trzy linie tekstu.

Wymagania:

- sprawdź, czy plik udało się otworzyć do zapisu,
- wypisz komunikat po poprawnym zapisie,
- po uruchomieniu sprawdź zawartość pliku w edytorze albo przez `cat dane.txt`.

40.3.2 Zadanie 2. Odczyt liniami

Napisz program, który odczytuje plik `dane.txt` linia po linii i wypisuje jego zawartość.

Scenariusz sprawdzenia:

Zawartosc dane.txt:

Ala
ma
kota

Oczekiwany wynik:

Ala
ma
kota

40.3.3 Zadanie 3. Numerowanie linii

Rozbuduj program z zadania 2 tak, aby przed każdą linią wypisywał jej numer.

Przykład:

1. Pierwsza linia
2. Druga linia

Wymagania:

- numerowanie zacznij od 1,
- pustą linię też licz jako linię.

40.3.4 Zadanie 4. Dopisywanie do pliku

Napisz program, który dopisuje nową linię na końcu pliku `dane.txt`, nie usuwając wcześniejszej zawartości.

Wymagania:

- użyj trybu dopisywania,
- po dopisaniu odczytaj plik i pokaż całą zawartość.

40.3.5 Zadanie 5. Liczba linii

Napisz program, który zlicza, ile linii znajduje się w pliku tekstowym.

Sprawdź program dla pliku pustego, pliku z jedną linią i pliku z kilkoma liniami.

40.4 Poziom 2 - Walidacja danych

40.4.1 Zadanie 6. Brak pliku

Napisz program, który próbuje otworzyć plik `liczby.txt`.

Jeśli plik nie istnieje, program powinien wypisać czytelny komunikat i zakończyć działanie.

Oczekiwany komunikat powinien zawierać nazwę pliku, którego nie udało się otworzyć.

40.4.2 Zadanie 7. Suma liczb z pliku

Napisz program, który odczytuje liczby całkowite z pliku `liczby.txt` i wypisuje ich sumę.

Przykładowy plik:

2
4
-1
10

Oczekiwany wynik:

Suma: 15

40.4.3 Zadanie 8. Błędne dane w pliku

Rozbuduj program z zadania 7 tak, aby wykrywał sytuację, w której plik zawiera tekst zamiast liczby.

Program powinien wypisać komunikat o błędnych danych.

Przykładowy plik:

```
10
abc
5
```

Program powinien zgłosić błąd dla `abc`. Może przerwać działanie albo pominąć błędną linię, ale to zachowanie powinno być jasno opisane w komunikacie.

40.4.4 Zadanie 9. Odczyt i zakres

Napisz program, który odczytuje z pliku wiek użytkownika.

Program powinien sprawdzić:

- czy odczyt się udał,
- czy wiek jest w zakresie od 0 do 130.

Sprawdź dane: 20, -1, 131, `abc`.

40.4.5 Zadanie 10. Pomijanie błędnych linii

Napisz program, który czyta plik `linie.txt` linia po linii.

Program powinien wypisywać tylko te linie, które zawierają poprawną liczbę całkowitą. Dla błędnych linii powinien wypisać komunikat `Pominięto błędna linie`.

Przykładowy plik:

```
10
kot
-3
4.5
0
```

Oczekiwane poprawne wartości: 10, -3, 0.

40.5 Poziom 3 - Wyjątki

40.5.1 Zadanie 11. Dzielenie przez zero

Napisz funkcję `podziel`, która przyjmuje dwie liczby typu `double`.

Jeśli drugi argument jest równy 0, funkcja powinna rzucić `std::invalid_argument`.

Wymagania:

- dołącz `<stdexcept>`,
- w treści wyjątku podaj informację o dzieleniu przez zero,
- dla poprawnych danych funkcja powinna zwracać wynik dzielenia.

40.5.2 Zadanie 12. Obsługa wyjątku

Rozbuduj program z zadania 11 tak, aby funkcja `main` obsługiwała wyjątek i wypisywała czytelny komunikat.

Wymagania:

- łap wyjątek przez `const std::exception&`,
- po obsłudze wyjątku program powinien zakończyć się kontrolowanie.

40.5.3 Zadanie 13. Sprawdzanie wieku

Napisz funkcję `sprawdzWiek`, która rzuca wyjątek, jeśli wiek jest mniejszy od 0 albo większy od 130.

Scenariusze sprawdzenia:

- 0 - poprawny,
- 130 - poprawny,
- -1 - wyjątek,
- 131 - wyjątek.

40.5.4 Zadanie 14. Indeks poza zakresem

Utwórz `std::vector<int>` z kilkoma wartościami. Użyj metody `at()` i obsłuż wyjątek `std::out_of_range` dla indeksu spoza zakresu.

Sprawdź indeks poprawny oraz indeks równy rozmiarowi wektora.

40.5.5 Zadanie 15. Plik jako wyjątek

Napisz funkcję `otworzPlik`, która próbuje otworzyć plik do odczytu.

Jeśli pliku nie da się otworzyć, funkcja powinna rzucić `std::runtime_error`.

W `main` obsłuż wyjątek i wypisz komunikat z `what()`.

40.6 Poziom 4 - Menu plikowe

40.6.1 Zadanie 16. Proste menu

Napisz program z menu:

- 1 - dodaj wpis,
- 2 - wyświetl wpisy,
- 0 - zakończ.

Wpisy zapisuj w pliku `dane.txt`.

Wymagania:

- menu ma działać w pętli,
- dodawanie wpisu ma dopisywać do pliku,
- wyświetlanie ma obsługiwać brak pliku albo pusty plik.

40.6.2 Zadanie 17. Czyszczenie pliku

Rozbuduj program z zadania 16 o opcję:

- 3 - usuń wszystkie wpisy.

Czyszczenie powinno otworzyć plik w trybie zapisu, tak aby poprzednia zawartość została zastąpiona pustą zawartością.

40.6.3 Zadanie 18. Liczenie wpisów

Dodaj opcję:

- 4 - policz wpisy.

Program powinien wypisać liczbę linii zapisanych w pliku.

Sprawdź wynik po dodaniu dwóch wpisów i po wyczyszczeniu pliku.

40.6.4 Zadanie 19. Walidacja wyboru menu

Rozbuduj menu tak, aby program poprawnie reagował na wpisanie tekstu zamiast liczby.

Program nie powinien wpadać w pętlę nieskończoną.

Wskazówka: po błędnym odczycie z `std::cin` trzeba wyczyścić stan strumienia i usunąć błędne dane z bufora.

40.6.5 Zadanie 20. Pusty wpis

Rozbuduj dodawanie wpisu tak, aby pusty wpis nie był zapisywany do pliku.

Program powinien wypisać komunikat, że pusty wpis został odrzucony.

40.7 Zadania dodatkowe

40.7.1 Zadanie 21. Lista zadań w pliku

Napisz program, który zapisuje listę zadań do pliku `todo.txt`.

Program powinien umożliwiać:

- dodanie zadania,
- wypisanie wszystkich zadań,
- oznaczenie zadania jako wykonane przez dopisanie `[x]`.

Wymagania:

- zadania numeruj od 1,
- obsłuż próbę oznaczenia nieistniejącego zadania,
- po zmianie zapisz aktualną listę zadań do pliku.

40.7.2 Zadanie 22. Prosty dziennik

Napisz program, który dopisuje wpis do pliku `dziennik.txt`.

Każdy wpis powinien zawierać:

- numer wpisu,
- temat,
- treść.

Wymagania:

- nowy wpis dopisuj na końcu pliku,
- numer wpisu wylicz na podstawie istniejącej liczby wpisów,
- oddziel wpisy pustą linią.

40.7.3 Zadanie 23. Raport z liczb

Napisz program, który odczytuje liczby z pliku `liczby.txt` i zapisuje do pliku `raport.txt`:

- liczbę poprawnych wartości,
- sumę,
- średnią,
- najmniejszą wartość,
- największą wartość.

Program powinien obsługiwać błędne linie.

40.8 Wariant minimum

Do zaliczenia segmentu wykonaj co najmniej:

1. Zadanie 1 albo 2.
2. Zadanie 4 albo 5.
3. Zadanie 6.
4. Zadanie 7 albo 10.
5. Zadanie 11 albo 12.
6. Zadanie 16 albo 18.
7. Mini-sprawdzian segmentu z końca pliku.

Zadania 21-23 są dodatkowe. Możesz je robić po wykonaniu wariantu minimum.

40.9 Kryteria zaliczenia segmentu

Student zalicza segment, jeśli potrafi samodzielnie:

- zapisać dane do pliku tekstowego,
- odczytać plik linia po linii,
- dopisać dane na końcu pliku,
- sprawdzić, czy plik został otwarty,
- sprawdzić wynik operacji odczytu,
- odróżnić błąd typu danych od wartości spoza zakresu,
- użyć `try`, `throw` i `catch`,
- złapać wyjątek standardowy przez `const` referencję,
- podzielić program plikowy na funkcje,
- zbudować proste menu działające w pętli.

40.10 Mini-sprawdzian segmentu

Napisz program tworzący raport z pliku tekstowego, który:

- odczytuje dane linia po linii,
- pomija błędne linie z komunikatem,
- liczy poprawne wartości i ich sumę,
- zapisuje wynik do osobnego pliku,
- obsługuje brak pliku przez wyjątek albo czytelny komunikat błędu.

Wymagania szczegółowe:

- plik wejściowy powinien mieć nazwę `liczby.txt`,
- plik wynikowy powinien mieć nazwę `raport.txt`,
- błędne linie powinny być policzone osobno,
- raport powinien zawierać liczbę poprawnych wartości, liczbę błędnych linii i sumę,
- brak pliku wejściowego nie może zakończyć programu awaryjnie.

Scenariusz sprawdzenia:

Zawartosc `liczby.txt`:

```
10
abc
-2
5
```

Oczekiwany raport:

Poprawne wartosci: 3

Bledne linie: 1

Suma: 13

40.11 Checklista oddania

Przed oddaniem rozwiązań sprawdź:

- ☐ Każde zadanie jest w osobnym pliku `.cpp`.

- ☐ Programy kompilują się z `-Wall -Wextra -pedantic`.
- ☐ Program sprawdza, czy plik udało się otworzyć.
- ☐ Brak pliku ma czytelny komunikat.
- ☐ Błędne dane wejściowe nie powodują awaryjnego zakończenia.
- ☐ Program nie zostawia w repozytorium lokalnych plików wynikowych.
- ☐ Wyjątki są łapane przez `const` referencję.

40.12 Archiwum

Oryginalne materiały źródłowe tego segmentu są zachowane w katalogu archive.

Rozdział 41

06 - Programowanie obiektowe

Ten segment wprowadza programowanie obiektowe jako sposób modelowania większych programów. Materiał przechodzi od prostych klas i obiektów do konstruktorów, enkapsulacji, kopiowania, dziedziczenia oraz przeciążania operatorów.

41.1 Cele segmentu

Po zakończeniu segmentu student powinien umieć:

- zdefiniować prostą klasę,
- utworzyć obiekt klasy,
- odróżnić pola od metod,
- stosować `public` i `private`,
- napisać konstruktor,
- rozumieć podstawy enkapsulacji,
- użyć metod `const`,
- rozumieć, kiedy powstaje kopia obiektu,
- wyjaśnić podstawy dziedziczenia,
- przeciążyć prosty operator,
- zaprojektować małą klasę do zadania praktycznego.

41.2 Kolejność lekcji

1. 01-klasy-i-obiekty.md - klasa, obiekt, pola i metody.
2. 02-konstruktory-i-enkapsulacja.md - konstruktory, `private`, gettery i settery.
3. 03-metody-const-i-this.md - metody `const`, wskaźnik `this`, dobre nazwy.
4. 04-kopiowanie-obiektow.md - konstruktor kopiujący i przekazywanie obiektów.
5. 05-dziedziczenie.md - klasy bazowe i pochodne.
6. 06-przeciazanie-operatorow.md - podstawy przeciążania operatorów.
7. 07-zadania.md - zadania podstawowe i projektowe.
8. 08-cwiczenie-figury-polimorfizm.md - większe ćwiczenie z dziedziczenia, klasy abstrakcyjnej i polimorfizmu.

41.3 Status porządkowania

Lekcje i zadania zostały uporządkowane do wspólnego formatu segmentów. Pierwotne wersje materiałów są zachowane w katalogu archive:

- archive/klasa-obiekt.md
- archive/nazwy-klas.md
- archive/klasy-zadania.md
- archive/zadania-oop-1.md

- [archive/zadania-oop-t1.md](#)
- [archive/lab-03-klasa-macierz.md](#)
- [archive/lab-03-zadania.md](#)
- [archive/konstruktor-kopiujacy.md](#)
- [archive/friends.md](#)
- [archive/dziedziczenie-zadania.md](#)
- [archive/przeciazanie-operatorow.md](#)
- [archive/drzewo-binarne.md](#)
- [archive/composite.md](#)
- [archive/uml-composite.md](#)
- [archive/linki-oop.md](#)

41.4 Przykłady kodu

Przykłady w katalogu examples można kompilować osobno:

- `examples/simple_point.cpp` - prosta klasa i obiekt.
- `examples/rectangle_methods.cpp` - pola i metody klasy.
- `examples/car_encapsulation.cpp` - enkapsulacja pól.
- `examples/account_constructor.cpp` - konstruktor i walidacja.
- `examples/const_methods.cpp` - metody `const`.
- `examples/this_pointer.cpp` - wskaźnik `this`.
- `examples/copy_constructor.cpp` - konstruktor kopiujący.
- `examples/pass_object.cpp` - przekazywanie obiektów.
- `examples/basic_inheritance.cpp` - podstawowe dziedziczenie.
- `examples/car_inheritance.cpp` - rozszerzanie klasy.
- `examples/figures_polymorphism.cpp` - klasa abstrakcyjna, `override` i polimorfizm.
- `examples/point_operators.cpp` - przeciążanie operatorów.
- `examples/money_output_operator.cpp` - operator wyjścia `<<`.

Większe ćwiczenie integrujące dziedziczenie i polimorfizm znajduje się w pliku `08-cwiczenie-figury-polimorfizm.md`.

41.5 Testy

Katalog `tests` zawiera prosty test klasy bez zewnętrznego frameworka. Test pokazuje, jak sprawdzać konstruktor, metody zmieniające stan, metody `const` oraz operator porównania.

41.6 Format lekcji

Każda docelowa lekcja powinna mieć taki układ:

1. Cel lekcji.
2. Wymagania wstępne.
3. Krótka teoria.
4. Przykład kodu.
5. Zadania do wykonania.
6. Kryteria zaliczenia.

41.7 Katalogi pomocnicze

- `examples/` - kompilowalne przykłady `.cpp` dla tego segmentu.
- `tests/` - proste testy automatyczne dla klas.
- `archive/` - pierwotne wersje materiałów zachowane do wglądu.

Rozdział 42

01 - Klasy i obiekty

42.1 Cel lekcji

Celem lekcji jest zrozumienie, czym jest klasa, czym jest obiekt oraz jak w C++ zapisać pola i metody klasy.

42.2 Wymagania wstępne

Student powinien znać:

- zmienne i typy danych z segmentu 01-podstawy,
- funkcje z segmentu 03-funkcje-tablice-napisy,
- podstawy pracy z plikami .cpp.

42.3 Krótka teoria

42.3.1 Klasa

Klasa to opis typu obiektu. Określa, jakie dane obiekt przechowuje i jakie operacje można na nim wykonać.

Przykład:

```
class Punkt
{
public:
    int x;
    int y;
};
```

Klasa Punkt opisuje obiekty, które mają dwa pola: x i y.

42.3.2 Obiekt

Obiekt to konkretna zmienna utworzona na podstawie klasy.

```
Punkt p;
p.x = 10;
p.y = 20;
```

Punkt jest typem, a p jest obiektem tego typu.

Pełny przykład znajduje się w pliku examples/simple_point.cpp.

42.4 Przykład kodu: pola klasy

Pola to dane przechowywane w obiekcie.

```
class Osoba
{
public:
    std::string imie;
    int wiek;
};
```

Każdy obiekt klasy `Osoba` ma własne pole `imie` i własne pole `wiek`.

```
Osoba adam;
adam.imie = "Adam";
adam.wiek = 20;
```

```
Osoba ewa;
ewa.imie = "Ewa";
ewa.wiek = 21;
```

Zmiana pola w obiekcie `adam` nie zmienia pola w obiekcie `ewa`.

42.5 Przykład kodu: metody klasy

Metoda to funkcja należąca do klasy. Metoda może korzystać z pól obiektu.

```
class Prostokat
{
public:
    double szerokosc;
    double wysokosc;

    double pole()
    {
        return szerokosc * wysokosc;
    }
};
```

Wywołanie metody:

```
Prostokat p;
p.szerokosc = 5;
p.wysokosc = 3;
```

```
std::cout << p.pole() << std::endl;
```

Pełny przykład znajduje się w pliku `examples/rectangle_methods.cpp`.

42.5.1 public

Słowo `public` oznacza, że pola i metody są dostępne z zewnątrz klasy.

```
class Punkt
{
public:
    int x;
    int y;
};
```

W pierwszej lekcji używamy `public`, żeby skupić się na samej idei klasy i obiektu. W kolejnej lekcji pojawi się `private`, czyli ukrywanie danych przed bezpośrednią zmianą z zewnątrz.

42.5.2 Nazwy klas

Nazwy klas powinny być rzeczownikami albo krótkimi frazami rzeczownikowymi.

Dobre przykłady:

- Punkt,
- Osoba,
- KontoBankowe,
- Rezerwacja,
- Produkt.

W tym kursie nazwy klas zapisujemy stylem `PascalCase`, czyli każde słowo zaczyna się wielką literą.

```
class KontoBankowe
{
public:
    double saldo;
};
```

42.6 Typowe błędy

42.6.1 Brak średnika po klasie

Niepoprawnie:

```
class Punkt
{
public:
    int x;
    int y;
}
```

Poprawnie:

```
class Punkt
{
public:
    int x;
    int y;
};
```

Definicja klasy kończy się średnikiem.

42.6.2 Mylenie klasy z obiektem

Niepoprawnie:

```
Punkt.x = 10;
```

Poprawnie:

```
Punkt p;
p.x = 10;
```

Najpierw trzeba utworzyć obiekt, potem można korzystać z jego pól.

42.6.3 Zbyt techniczna nazwa klasy

Mniej czytelnie:

```
class DaneTablica
{
};
```

Czytelniej:

```
class ListaZakupow
{
};
```

Nazwa klasy powinna opisywać pojęcie z problemu, a nie szczegół implementacji.

42.7 Zadania do wykonania

1. Utwórz klasę `Punkt` z polami `x` i `y`. Utwórz dwa obiekty tej klasy i wypisz ich wartości.
2. Utwórz klasę `Osoba` z polami `imie` i `wiek`. Wypisz dane dwóch osób.
3. Utwórz klasę `Prostokat` z polami `szerokosc` i `wysokosc` oraz metodą `pole`.
4. Rozbuduj klasę `Prostokat` o metodę `obwod`.
5. Utwórz klasę `Produkt` z polami `nazwa`, `cena` i `ilosc`. Dodaj metodę `wartosc`, która zwraca `cena * ilosc`.

42.8 Kryteria zaliczenia

Program powinien:

- poprawnie definiować klasę,
- kończyć definicję klasy średnikiem,
- tworzyć obiekty klasy,
- odczytywać i zmieniać pola obiektu,
- wywoływać metody obiektu,
- używać czytelnych nazw klas,
- kompilować się bez błędów.

42.9 Pytania kontrolne

1. Czym różni się klasa od obiektu?
2. Czym są pola klasy?
3. Czym jest metoda?
4. Do czego służy `public`?
5. Dlaczego definicja klasy kończy się średnikiem?

Rozdział 43

02 - Konstruktory i enkapsulacja

43.1 Cel lekcji

Celem lekcji jest zrozumienie, po co stosujemy konstruktory, pola `private` oraz metody dostępne, czyli gettery i settery.

43.2 Wymagania wstępne

Student powinien znać:

- klasy i obiekty z lekcji 01-klasy-i-obiekty.md,
- funkcje z segmentu 03-funkcje-tablice-napisy,
- instrukcje warunkowe z segmentu 02-sterowanie-i-petle.

43.3 Krótka teoria

43.3.1 Problem z polami publicznymi

W pierwszej lekcji pola były publiczne:

```
class Konto
{
public:
    double saldo;
};
```

Taki zapis pozwala zmienić pole z dowolnego miejsca programu.

```
Konto konto;
konto.saldo = -1000;
```

Dla konta bankowego ujemne saldo może być niedozwolone. Klasa powinna chronić swoje dane przed niepoprawną zmianą.

43.3.2 `private`

Słowo `private` oznacza, że pola są dostępne tylko wewnątrz klasy.

```
class Konto
{
private:
    double saldo;
};
```

Teraz kod poza klasą nie może bezpośrednio zmienić pola `saldo`.

```
Konto konto;  
// konto.saldo = 100; // blad kompilacji
```

To jest początek enkapsulacji: dane obiektu są ukryte, a dostęp do nich odbywa się przez metody.

43.4 Przykład kodu: konstruktor

Konstruktor to specjalna metoda wywoływana podczas tworzenia obiektu. Ma taką samą nazwę jak klasa i nie ma typu zwracanego.

```
class Konto  
{  
public:  
    Konto(double saldoPoczkowe)  
    {  
        saldo = saldoPoczkowe;  
    }  
  
private:  
    double saldo;  
};
```

Użycie:

```
Konto konto(100);
```

Konstruktor pozwala utworzyć obiekt od razu w poprawnym stanie.

Pełny przykład znajduje się w pliku `examples/account_constructor.cpp`.

43.4.1 Getter

Getter to metoda, która zwraca wartość pola.

```
double pobierzSaldo()  
{  
    return saldo;  
}
```

Getter pozwala odczytać wartość bez bezpośredniego dostępu do pola.

```
std::cout << konto.pobierzSaldo() << std::endl;
```

43.4.2 Setter

Setter to metoda, która zmienia wartość pola. Setter może sprawdzić, czy nowa wartość jest poprawna.

```
void ustawSaldo(double noweSaldo)  
{  
    if (noweSaldo >= 0)  
    {  
        saldo = noweSaldo;  
    }  
}
```

Dzięki temu klasa może odrzucić niepoprawne dane.

43.5 Przykład kodu: enkapsulacja

Enkapsulacja oznacza ukrywanie szczegółów działania klasy i udostępnianie tylko potrzebnych operacji.

Dobra klasa:

- ukrywa pola jako `private`,

- udostępnia metody, które mają sens dla użytkownika klasy,
- pilnuje poprawności swoich danych,
- nie pozwala ominąć reguł obiektu.

Przykład:

```
class Samochod
{
public:
    Samochod(std::string marka, int rokProdukcji)
    {
        this->marka = marka;
        this->rokProdukcji = rokProdukcji;
        przebieg = 0;
    }

    void jedz(int kilometry)
    {
        if (kilometry > 0)
        {
            przebieg += kilometry;
        }
    }

private:
    std::string marka;
    int rokProdukcji;
    int przebieg;
};
```

Pełny przykład znajduje się w pliku examples/car_encapsulation.cpp.

43.5.1 Konstruktor domyślny i konstruktor z parametrami

Konstruktor domyślny nie przyjmuje argumentów.

```
Konto()
{
    saldo = 0;
}
```

Konstruktor z parametrami przyjmuje dane potrzebne do utworzenia obiektu.

```
Konto(double saldoPoczkowe)
{
    saldo = saldoPoczkowe;
}
```

Klasa może mieć więcej niż jeden konstruktor, jeśli różnią się listą parametrów.

43.6 Typowe błędy

43.6.1 Publiczne pola bez kontroli

Niepoprawnie:

```
class Samochod
{
public:
    int przebieg;
};
```

Taki kod pozwala ustawić **przebieg** na wartość ujemną.

Poprawniej:

```
class Samochod
{
public:
    void jedz(int kilometry)
    {
        if (kilometry > 0)
        {
            przebieg += kilometry;
        }
    }

private:
    int przebieg = 0;
};
```

43.6.2 Getter, który zmienia stan

Getter powinien tylko odczytywać dane. Zmiana stanu w getterze jest myląca.

43.6.3 Setter bez walidacji

Jeśli setter tylko przypisuje wartość bez żadnych zasad, to często nie daje przewagi nad polem publicznym.

43.7 Zadania do wykonania

1. Utwórz klasę **Konto** z prywatnym polem **saldo** i konstruktorem ustawiającym saldo początkowe.
2. Dodaj getter **pobierzSaldo**.
3. Dodaj metodę **wplac**, która zwiększa saldo tylko dla kwoty większej od 0.
4. Dodaj metodę **wypłac**, która zmniejsza saldo tylko wtedy, gdy kwota jest większa od 0 i nie przekracza salda.
5. Utwórz klasę **Samochod** z prywatnymi polami **marka**, **model**, **rokProdukcji** i **przebieg**. Dodaj konstruktor oraz metodę **jedz**.

43.8 Kryteria zaliczenia

Program powinien:

- używać pól **private**,
- tworzyć obiekty przez konstruktor,
- udostępniać dane przez gettery,
- zmieniać dane przez metody z walidacją,
- nie pozwalać ustawić niepoprawnego stanu obiektu,
- kompilować się bez błędów.

43.9 Pytania kontrolne

1. Po co stosujemy **private**?
2. Czym jest konstruktor?
3. Czym getter różni się od settera?
4. Co oznacza enkapsulacja?
5. Dlaczego obiekt powinien pilnować poprawności własnych danych?

Rozdział 44

03 - Metody const i this

44.1 Cel lekcji

Celem lekcji jest zrozumienie, kiedy metoda klasy powinna być oznaczona jako `const`, czym jest wskaźnik `this` oraz jak nazywać klasy, pola i metody w czytelny sposób.

44.2 Wymagania wstępne

Student powinien znać:

- klasy i obiekty z lekcji 01-klasy-i-obiekty.md,
- konstruktory i enkapsulację z lekcji 02-konstruktory-i-enkapsulacja.md.

44.3 Krótka teoria

44.3.1 Metoda const

Metoda oznaczona jako `const` obiecuje, że nie zmieni stanu obiektu.

```
class Konto
{
public:
    double pobierzSaldo() const
    {
        return saldo;
    }

private:
    double saldo = 0;
};
```

Słowo `const` zapisujemy po liście parametrów metody.

```
double pobierzSaldo() const
```

Getter zwykle powinien być metodą `const`, ponieważ tylko odczytuje dane.

Pełny przykład znajduje się w pliku `examples/const_methods.cpp`.

44.3.2 Dlaczego const jest przydatne

Jeśli obiekt jest stały, można na nim wywołać tylko metody `const`.

```
const Konto konto(100);
std::cout << konto.pobierzSaldo() << std::endl;
```

To chroni obiekt przed przypadkową zmianą.
Metoda `const` nie może zmienić pola obiektu.

```
double pobierzSaldo() const
{
    // saldo = 0; // bład kompilacji
    return saldo;
}
```

44.4 Przykład kodu: metody odczytujące i modyfikujące

Warto rozdzielać metody na dwie grupy.

Metody odczytujące:

- zwracają stan obiektu,
- nie zmieniają pól,
- zwykle powinny być `const`.

Przykłady:

```
double pobierzSaldo() const;
std::string pobierzNazwe() const;
int pobierzPrzebieg() const;
```

Metody modyfikujące:

- zmieniają stan obiektu,
- nie są oznaczane jako `const`.

Przykłady:

```
void wplac(double kwota);
void jedz(int kilometry);
void ustawNazwe(std::string nowaNazwa);
```

44.5 Przykład kodu: wskaźnik `this`

Każda metoda obiektu ma dostęp do specjalnego wskaźnika `this`.

`this` wskazuje na obiekt, na którym została wywołana metoda.

```
class Produkt
{
public:
    Produkt(std::string nazwa)
    {
        this->nazwa = nazwa;
    }

private:
    std::string nazwa;
};
```

W przykładzie parametr konstruktora i pole klasy mają tę samą nazwę. Zapis `this->nazwa` oznacza pole obiektu.

Pełny przykład znajduje się w pliku `examples/this_pointer.cpp`.

44.5.1 Kiedy używać `this`

`this` jest przydatne, gdy:

- parametr ma taką samą nazwę jak pole,

- chcemy jasno pokazać, że chodzi o pole obiektu,
- metoda zwraca bieżący obiekt albo wskaźnik do niego.

Na tym etapie najczęstsze użycie to rozróżnienie pola i parametru.

```
void ustawNazwe(std::string nazwa)
{
    this->nazwa = nazwa;
}
```

Można też unikać konfliktu nazw przez inne nazwy parametrów:

```
void ustawNazwe(std::string nowaNazwa)
{
    nazwa = nowaNazwa;
}
```

Oba style są poprawne. Najważniejsza jest spójność.

44.5.2 Dobre nazwy w klasach

Nazwy klas powinny być rzeczownikami.

Dobrze:

- Samochod,
- KontoBankowe,
- Produkt,
- Rezerwacja.

Nazwy metod powinny mówić, co metoda robi.

Dobrze:

- pobierzSaldo,
- ustawNazwe,
- wplac,
- wypłac,
- wypiszInformacje.

Mniej czytelnie:

- doIt,
- data,
- manager,
- process.

44.5.3 Konwencja w tym kursie

W materiałach stosujemy:

- klasy: `PascalCase`, np. `KontoBankowe`,
- zmienne i metody: `camelCase` albo polskie nazwy bez znaków diakrytycznych, np. `pobierzSaldo`,
- stałe: wielkie litery tylko wtedy, gdy są naprawdę stałymi globalnymi,
- nazwy opisujące pojęcie z zadania, a nie szczegół techniczny.

W istniejących przykładach mogą pojawiać się starsze style. Nowe materiały powinny być spójne i czytelne.

44.6 Typowe błędy

44.6.1 Brak `const` przy getterze

Mniej precyzyjnie:

```
double pobierzSaldo()
{
    return saldo;
}
```

Lepiej:

```
double pobierzSaldo() const
{
    return saldo;
}
```

44.6.2 Próba zmiany pola w metodzie const

Niepoprawnie:

```
double pobierzSaldo() const
{
    saldo = 0;
    return saldo;
}
```

Metoda `const` nie może zmieniać zwykłych pól obiektu.

44.6.3 Niejasne nazwy

Niepoprawnie:

```
class Data
{
};
```

`Data` może oznaczać datę albo dane. Lepiej użyć nazwy bardziej jednoznacznej, np. `DaneKlienta` albo `DataUrodzenia`.

44.7 Zadania do wykonania

1. Dodaj `const` do getterów w klasie `Konto`.
2. Utwórz klasę `Produkt` z polami `nazwa`, `cena` i `ilosc`. Gettery powinny być metodami `const`.
3. Dodaj metodę `wartosc()` `const`, która zwraca `cena * ilosc`.
4. Napisz konstruktor klasy `Produkt`, w którym użyjesz `this`, aby odróżnić pola od parametrów.
5. Przejrzyj własny kod z poprzednich zadań i popraw nazwy klas oraz metod tak, aby były bardziej opisowe.

44.8 Kryteria zaliczenia

Program powinien:

- oznaczać metody odczytujące jako `const`,
- nie zmieniać pól w metodach `const`,
- poprawnie używać `this` do odróżniania pól od parametrów,
- stosować czytelne nazwy klas i metod,
- kompilować się bez błędów.

44.9 Pytania kontrolne

1. Co oznacza `const` po liście parametrów metody?
2. Dlaczego getter zwykle powinien być metodą `const`?
3. Czym jest `this`?
4. Kiedy `this->nazwa` jest bardziej czytelne niż samo `nazwa`?
5. Jakie nazwy są dobre dla klas, a jakie dla metod?

Rozdział 45

04 - Kopiowanie obiektów

45.1 Cel lekcji

Celem lekcji jest zrozumienie, kiedy w C++ powstaje kopia obiektu, czym jest konstruktor kopiujący oraz dlaczego obiekty często przekazujemy do funkcji przez referencję.

45.2 Wymagania wstępne

Student powinien znać:

- klasy, obiekty i metody z lekcji 01-klasy-i-obiekty.md,
- konstruktory i enkapsulację z lekcji 02-konstruktory-i-enkapsulacja.md,
- referencje z segmentu 04-wskazniki-referencje-pamiec.

45.3 Krótka teoria

45.3.1 Kiedy powstaje kopia

Kopia obiektu może powstać między innymi wtedy, gdy:

- tworzymy nowy obiekt na podstawie istniejącego,
- przekazujemy obiekt do funkcji przez wartość,
- funkcja zwraca obiekt przez wartość.

Przykład:

```
Konto konto1("Anna", 100);  
Konto konto2 = konto1;
```

konto2 jest osobnym obiektem. Ma takie same wartości pól jak konto1, ale jest niezależną kopią.

45.4 Przykład kodu: konstruktor kopiujący

Konstruktor kopiujący tworzy nowy obiekt na podstawie istniejącego obiektu tej samej klasy.

```
class Konto  
{  
public:  
    Konto(const Konto& inne)  
    {  
        wlasciciel = inne.wlasciciel;  
        saldo = inne.saldo;  
    }  
}
```

```
private:
    std::string wlasciciel;
    double saldo;
};
```

Parametr konstruktora kopiującego zwykle ma typ:

```
const Konto& inne
```

Używamy referencji, żeby nie tworzyć kolejnej kopii. Używamy `const`, bo konstruktor kopiujący nie powinien zmieniać obiektu źródłowego.

Pełny przykład znajduje się w pliku `examples/copy_constructor.cpp`.

45.4.1 Domyślne kopiowanie

Jeśli klasa ma proste pola, C++ potrafi wygenerować konstruktor kopiujący automatycznie.

```
class Punkt
{
public:
    int x;
    int y;
};
```

Dla takiej klasy kopiowanie działa bez pisania własnego konstruktora kopiującego.

```
Punkt a;
a.x = 1;
a.y = 2;
```

```
Punkt b = a;
```

Własny konstruktor kopiujący piszemy wtedy, gdy chcemy mieć specjalne zachowanie podczas kopiowania albo gdy klasa zarządza zasobem.

Na tym etapie najważniejsze jest rozumienie, kiedy kopia powstaje.

45.5 Przykład kodu: przekazywanie przez wartość

Gdy funkcja przyjmuje obiekt przez wartość, powstaje kopia.

```
void wypisz(Konto konto)
{
    std::cout << konto.pobierzSaldo() << std::endl;
}
```

Dla małych klas to może nie mieć znaczenia. Dla większych obiektów kopiowanie może być kosztowne.

Pełny przykład znajduje się w pliku `examples/pass_object.cpp`.

45.5.1 Przekazywanie przez referencję `const`

Jeśli funkcja ma tylko odczytać obiekt, zwykle lepiej przekazać go przez referencję `const`.

```
void wypisz(const Konto& konto)
{
    std::cout << konto.pobierzSaldo() << std::endl;
}
```

Taki zapis:

- nie tworzy kopii,
- nie pozwala funkcji zmienić obiektu,
- jest czytelny dla osoby czytającej kod.

45.5.2 Przekazywanie przez referencję bez const

Jeśli funkcja ma zmienić obiekt, można przekazać go przez zwykłą referencję.

```
void dopiszOdsetki(Konto& konto)
{
    konto.wplac(10);
}
```

Taki zapis oznacza, że funkcja może zmienić oryginalny obiekt.

45.5.3 Zwracanie obiektów z funkcji

Funkcja może zwrócić obiekt przez wartość.

```
Konto utworzKonto()
{
    Konto konto("Anna", 100);
    return konto;
}
```

W nowoczesnym C++ kompilator często optymalizuje takie przypadki. Na tym etapie warto jednak pamiętać, że zwracanie obiektu oznacza przekazanie gotowego wyniku do miejsca wywołania.

45.6 Typowe błędy

45.6.1 Niepotrzebne kopiowanie dużych obiektów

Mniej korzystnie:

```
void wypisz(Raport raport)
{
}
```

Lepiej:

```
void wypisz(const Rapport& raport)
{
}
```

45.6.2 Brak const przy referencji tylko do odczytu

Mniej precyzyjnie:

```
void wypisz(Konto& konto)
{
}
```

Lepiej:

```
void wypisz(const Konto& konto)
{
}
```

Jeśli funkcja nie zmienia obiektu, powinna to pokazać w typie parametru.

45.6.3 Mylenie kopii z tym samym obiektem

```
Konto a("Anna", 100);
Konto b = a;
```

a i b to dwa osobne obiekty. Zmiana b nie zmienia a.

45.7 Zadania do wykonania

1. Utwórz klasę `Konto` z polami prywatnymi `wlasciciel` i `saldo`.
2. Dodaj konstruktor z parametrami oraz konstruktor kopiujący.
3. W konstruktorze kopiującym wypisz komunikat `Kopiowanie konta`.
4. Napisz funkcję, która przyjmuje `Konto` przez wartość i zaobserwuj, że powstaje kopia.
5. Napisz drugą funkcję, która przyjmuje `const Konto&` i porównaj działanie obu funkcji.

45.8 Kryteria zaliczenia

Program powinien:

- poprawnie definiować konstruktor kopiujący,
- przekazywać obiekt źródłowy jako `const` referencję,
- pokazać sytuację, w której powstaje kopia,
- używać `const` referencji dla parametrów tylko do odczytu,
- odróżniać kopię obiektu od oryginału,
- kompilować się bez błędów.

45.9 Pytania kontrolne

1. Kiedy powstaje kopia obiektu?
2. Jak wygląda typowy parametr konstruktora kopiującego?
3. Dlaczego konstruktor kopiujący przyjmuje `const Konto&`, a nie `Konto`?
4. Kiedy funkcja powinna przyjmować obiekt przez `const` referencję?
5. Czy zmiana kopii obiektu zmienia oryginał?

Rozdział 46

05 - Dziedziczenie

46.1 Cel lekcji

Celem lekcji jest zrozumienie podstaw dziedziczenia: czym jest klasa bazowa, czym jest klasa pochodna i jak klasa pochodna może ponownie wykorzystać kod klasy bazowej.

46.2 Wymagania wstępne

Student powinien znać:

- klasy i obiekty z lekcji 01-klasy-i-obiekty.md,
- konstruktory i enkapsulację z lekcji 02-konstruktory-i-enkapsulacja.md,
- metody `const` z lekcji 03-metody-const-i-this.md.

46.3 Krótka teoria

46.3.1 Klasa bazowa i klasa pochodna

Dziedziczenie pozwala opisać relację typu „jest rodzajem”.

Przykłady:

- Menedzer jest rodzajem Pracownika,
- SportowySamochod jest rodzajem Samochodu,
- KontoOszczednosciowe jest rodzajem Konta.

Klasa bazowa zawiera wspólne pola i metody.

```
class Pracownik
{
public:
    Pracownik(std::string imie, int pensja)
    {
        this->imie = imie;
        this->pensja = pensja;
    }

private:
    std::string imie;
    int pensja;
};
```

Klasa pochodna rozszerza klasę bazową.

```

class Menedzer : public Pracownik
{
public:
    Menedzer(std::string imie, int pensja, int liczbaOsob)
        : Pracownik(imie, pensja)
    {
        this->liczbaOsob = liczbaOsob;
    }

private:
    int liczbaOsob;
};

```

Pełny przykład znajduje się w pliku `examples/basic_inheritance.cpp`.

46.3.2 Dziedziczenie public

Najczęściej na początku używamy dziedziczenia publicznego:

```

class Menedzer : public Pracownik
{
};

```

Oznacza to, że publiczne metody klasy bazowej pozostają publiczne w klasie pochodnej.

Jeśli `Pracownik` ma publiczną metodę `opis()`, to obiekt klasy `Menedzer` też może jej użyć.

46.4 Przykład kodu: konstruktor klasy bazowej

Klasa pochodna musi zbudować część bazową obiektu.

Robimy to na liście inicjalizacyjnej konstruktora:

```

Menedzer(std::string imie, int pensja, int liczbaOsob)
    : Pracownik(imie, pensja)
{
    this->liczbaOsob = liczbaOsob;
}

```

Fragment:

```

: Pracownik(imie, pensja)

```

wywołuje konstruktor klasy bazowej.

46.4.1 protected

Pola `private` klasy bazowej nie są dostępne bezpośrednio w klasie pochodnej.

```

class Pracownik
{
private:
    int pensja;
};

```

Klasa `Menedzer` nie może bezpośrednio użyć pola `pensja`, jeśli jest ono `private`.

Można użyć `protected`, aby pole było dostępne w klasie bazowej i pochodnych.

```

class Pracownik
{
protected:
    int pensja;
};

```


Na początku warto jednak nadal preferować `private` i publiczne metody dostępne. `protected` jest przydatne, ale łatwo przez nie osłabić enkapsulację.

46.5 Przykład kodu: rozszerzanie zachowania

Klasa pochodna może dodać własne pola i metody.

```
class SportowySamochod : public Samochod
{
public:
    void turbo()
    {
        maksymalnaPredkosc += 20;
    }

private:
    int maksymalnaPredkosc;
};
```

Pełny przykład znajduje się w pliku `examples/car_inheritance.cpp`.

46.5.1 Kiedy dziedziczenie ma sens

Dziedziczenie ma sens, gdy relacja naprawdę oznacza „jest rodzajem”.

Dobrze:

- Menedżer jest rodzajem Pracownika,
- SportowySamochod jest rodzajem Samochodu.

Podważane:

- Samochod dziedziczy po Silnik,
- Zamowienie dziedziczy po Produkt.

W takich przypadkach lepiej użyć pola w klasie.

```
class Samochod
{
private:
    Silnik silnik;
};
```

To oznacza relację „ma”.

46.6 Typowe błędy

46.6.1 Dziedziczenie zamiast pola

Jeśli obiekt coś posiada, nie powinien po tym dziedziczyć.

Samochód ma silnik, ale nie jest silnikiem.

46.6.2 Bezpośredni dostęp do zbyt wielu pól

Nadużywanie `protected` może sprawić, że klasy pochodne zbyt mocno zależą od szczegółów klasy bazowej.

46.6.3 Brak wywołania konstruktora bazowego

Jeśli klasa bazowa nie ma konstruktora domyślnego, klasa pochodna musi jawnie wywołać konstruktor bazowy na liście inicjalizacyjnej.

46.7 Zadania do wykonania

1. Utwórz klasę `Pracownik` z polami `imie` i `pensja` oraz metodą `opis`.
2. Utwórz klasę `Menedzer`, która dziedziczy publicznie po `Pracownik` i dodaje pole `liczbaOsob`.
3. Dodaj do klasy `Menedzer` metodę `opisZespolu`.
4. Utwórz klasę `Samochod` oraz klasę `SportowySamochod`, która po niej dziedziczy.
5. W klasie `SportowySamochod` dodaj metodę `turbo`, która zwiększa maksymalną prędkość tylko dla dodatniej wartości.

46.8 Kryteria zaliczenia

Program powinien:

- poprawnie definiować klasę bazową,
- poprawnie definiować klasę pochodną,
- używać dziedziczenia `public`,
- wywoływać konstruktor klasy bazowej,
- odróżniać relację „jest” od relacji „ma”,
- kompilować się bez błędów.

46.9 Pytania kontrolne

1. Czym jest klasa bazowa?
2. Czym jest klasa pochodna?
3. Co oznacza zapis `class Menedzer : public Pracownik`?
4. Po co klasa pochodna wywołuje konstruktor klasy bazowej?
5. Kiedy lepiej użyć pola zamiast dziedziczenia?

Rozdział 47

06 - Przeciążanie operatorów

47.1 Cel lekcji

Celem lekcji jest zrozumienie, czym jest przeciążanie operatorów i jak użyć go w prostych klasach, aby kod był czytelniejszy.

47.2 Wymagania wstępne

Student powinien znać:

- klasy i obiekty z lekcji 01-klasy-i-obiekty.md,
- enkapsulację z lekcji 02-konstruktory-i-enkapsulacja.md,
- metody `const` z lekcji 03-metody-const-i-this.md,
- kopiowanie obiektów z lekcji 04-kopiowanie-obiektow.md.

47.3 Krótka teoria

47.3.1 Czym jest przeciążanie operatorów

Przeciążanie operatorów pozwala zdefiniować, co oznacza operator dla obiektów własnej klasy.

Przykład:

```
Punkt a(1, 2);
Punkt b(3, 4);
Punkt c = a + b;
```

Dla klasy `Punkt` możemy ustalić, że `+` oznacza dodanie współrzędnych.

```
Punkt operator+(const Punkt& inny) const
{
    return Punkt(x + inny.x, y + inny.y);
}
```

Pełny przykład znajduje się w pliku `examples/point_operators.cpp`.

47.4 Przykład kodu: operator jako metoda

Operator można zapisać jako metodę klasy.

```
class Punkt
{
public:
    Punkt operator+(const Punkt& inny) const
    {
```

```

        return Punkt(x + inny.x, y + inny.y);
    }

private:
    int x;
    int y;
};

```

Słowo `operator+` oznacza przeciążenie operatora `+`.

Parametr `const Punkt& inny` oznacza drugi obiekt po prawej stronie operatora.

Metoda jest `const`, bo dodawanie nie powinno zmieniać istniejących punktów.

47.5 Przykład kodu: operator ==

Operator `==` powinien zwracać `bool`.

```

bool operator==(const Punkt& inny) const
{
    return x == inny.x && y == inny.y;
}

```

Dzięki temu można pisać:

```

if (a == b)
{
    std::cout << "Punkty sa takie same" << std::endl;
}

```

47.6 Przykład kodu: operator <<

Operator `<<` pozwala wypisywać obiekt do strumienia.

```
std::cout << punkt << std::endl;
```

Często zapisuje się go jako funkcję poza klasą.

```

std::ostream& operator<<(std::ostream& out, const Punkt& punkt)
{
    out << "(" << punkt.pobierzX() << ", " << punkt.pobierzY() << ")";
    return out;
}

```

Funkcja zwraca `std::ostream&`, żeby można było łączyć wypisywanie.

```
std::cout << punkt << std::endl;
```

47.6.1 Operator jako funkcja zaprzyjaźniona

Jeśli operator zewnętrzny potrzebuje dostępu do prywatnych pól, można użyć `friend`.

```
friend std::ostream& operator<<(std::ostream& out, const Punkt& punkt);
```

W tym kursie na początku preferujemy gettery, bo są prostsze do zrozumienia i nie otwierają prywatnych pól dla dodatkowej funkcji.

Pełny przykład operatora `<<` znajduje się w pliku `examples/money__output__operator.cpp`.

47.6.2 Kiedy przeciążanie operatorów ma sens

Przeciążanie operatorów ma sens, gdy operator jest naturalny dla danej klasy.

Dobre przykłady:

- `Punkt + Punkt`,
- `Kwota + Kwota`,
- `Macierz + Macierz`,
- `Macierz == Macierz`.

Podejrzane przykłady:

- `Student + Student`,
- `Plik * Plik`,
- `Samochod == Kierowca`.

Operator powinien zachowywać się intuicyjnie.

47.7 Typowe błędy

47.7.1 Operator robi coś zaskakującego

Jeśli `+` usuwa dane albo zapisuje plik, kod będzie trudny do zrozumienia. Operator powinien robić coś zgodnego z oczekiwaniem.

47.7.2 Brak `const`

Mniej precyzyjnie:

```
Punkt operator+(Punkt& inny)
{
}
```

Lepiej:

```
Punkt operator+(const Punkt& inny) const
{
}
```

Dodawanie nie powinno zmieniać ani lewego, ani prawego argumentu.

47.7.3 Zwracanie referencji do obiektu lokalnego

Nie wolno zwracać referencji do zmiennej utworzonej wewnątrz operatora.

```
// Niepoprawny pomysł
Punkt& operator+(const Punkt& inny)
{
    Punkt wynik(0, 0);
    return wynik;
}
```

Wynik dodawania zwracamy przez wartość.

47.8 Zadania do wykonania

1. Utwórz klasę `Punkt` z polami `x` i `y`. Dodaj operator `+`.
2. Dodaj operator `==` dla klasy `Punkt`.
3. Dodaj operator `<<`, który wypisuje punkt w formacie `(x, y)`.
4. Utwórz klasę `Kwota` z polami `złote` i `grosze`. Dodaj operator `+`.
5. Rozbuduj klasę `macierz` z wcześniejszych zadań o operator `+` i operator `==`.

47.9 Kryteria zaliczenia

Program powinien:

- poprawnie przeciążać operator jako metodę klasy,

- używać `const` dla operatorów, które nie zmieniają obiektu,
- przekazywać drugi operand przez `const` referencję,
- zwracać wynik operatora przez wartość, jeśli powstaje nowy obiekt,
- implementować operator `<<` jako funkcję zwracającą `std::ostream&`,
- kompilować się bez błędów.

47.10 Pytania kontrolne

1. Co oznacza `operator+`?
2. Dlaczego operator `==` zwraca `bool`?
3. Dlaczego operator `+` zwykle zwraca nowy obiekt?
4. Po co operator `<<` zwraca `std::ostream&`?
5. Kiedy przeciążanie operatora jest dobrym pomysłem?

Rozdział 48

07 - Zadania

48.1 Cel zestawu zadań

Ten plik zbiera zadania do segmentu 06 - Programowanie obiektowe. Zadania sprawdzają klasy, obiekty, konstruktory, enkapsulację, metody `const`, kopiowanie, dziedziczenie oraz przeciążanie operatorów.

48.2 Zasady wykonania

Każde zadanie powinno być zapisane w osobnym pliku `.cpp`. Program powinien kompilować się bez błędów i wypisywać wynik w czytelnej formie.

Przykład kompilacji:

```
c++ -std=c++17 -Wall -Wextra -pedantic nazwa_pliku.cpp -o program
./program
```

Klasy powinny mieć czytelne nazwy, a pola powinny być prywatne, jeśli zadanie nie mówi inaczej.

48.3 Wymagania jakościowe

W zadaniach z tego segmentu zwracaj uwagę na:

- kompilację z flagami `-std=c++17 -Wall -Wextra -pedantic`,
- prywatne pola i publiczne metody tylko tam, gdzie są potrzebne,
- metody `const` dla operacji, które nie zmieniają obiektu,
- konstruktory ustawiające obiekt od razu w poprawnym stanie,
- krótkie funkcje i czytelne nazwy klas, metod oraz pól.
- rozdzielenie logiki klasy od demonstracji w `main`.

48.4 Poziom 1 - Klasy i obiekty

48.4.1 Zadanie 1. Punkt

Utwórz klasę `Punkt` z polami `x` i `y`. Utwórz dwa obiekty tej klasy i wypisz ich współrzędne.

Wymagania:

- użyj nazwy klasy zapisanej wielką literą,
- przygotuj metodę `wypisz`,
- w `main` utwórz dwa punkty o różnych współrzędnych.

48.4.2 Zadanie 2. Prostokąt

Utwórz klasę `Prostokat` z polami `szerokosc` i `wysokosc`.

Dodaj metody:

- `pole`,
- `obwod`,
- `wypiszInformacje`.

Scenariusz sprawdzenia:

`Szerokosc: 4`

`Wysokosc: 3`

`Pole: 12`

`Obwod: 14`

48.4.3 Zadanie 3. Osoba

Utwórz klasę `Osoba` z polami `imie` i `wiek`. Utwórz kilka obiektów i wypisz ich dane.

Wymagania:

- `wiek` nie powinien być ujemny,
- przygotuj metodę wypisującą dane osoby.

48.4.4 Zadanie 4. Produkt

Utwórz klasę `Produkt` z polami:

- `nazwa`,
- `cena`,
- `ilosc`.

Dodaj metodę `wartosc`, która zwraca `cena * ilosc`.

Wymagania:

- `wartosc` powinna zwracać `double`,
- nie wypisuj wyniku bezpośrednio w metodzie `wartosc`.

48.4.5 Zadanie 5. Konto

Utwórz klasę `Konto` z polami:

- `wlasciciel`,
- `numer`,
- `saldo`.

Dodaj metodę wypisującą dane konta.

Nie dodawaj jeszcze wpłat i wypłat. To pojawi się w kolejnych zadaniach.

48.5 Poziom 2 - Konstruktory i enkapsulacja

48.5.1 Zadanie 6. Konto z konstruktorem

Przepisz klasę `Konto` tak, aby pola były prywatne. Dodaj konstruktor ustawiający właściciela, numer i saldo początkowe.

Wymagania:

- saldo początkowe nie może być ujemne,
- jeśli konstruktor dostaje ujemne saldo, ustaw saldo na 0,
- dodaj getter do salda.

48.5.2 Zadanie 7. Wpłata i wypłata

Dodaj do klasy `Konto` metody:

- `wplac`,
- `wyplac`,
- `pobierzSaldo`.

Program powinien odrzucać wpłaty ujemne oraz wypłatę większą niż saldo.

Scenariusz sprawdzenia:

```
Saldo początkowe: 100
Po wpłacie 50: 150
Po wypłacie 30: 120
Wypłata 200 odrzucona
Wpłata -10 odrzucona
Saldo końcowe: 120
```

48.5.3 Zadanie 8. Samochód

Utwórz klasę `Samochod` z prywatnymi polami:

- `marka`,
- `model`,
- `rokProdukcji`,
- `przebieg`.

Dodaj konstruktor, gettery i metodę `jedz`, która zwiększa przebieg tylko dla dodatniej liczby kilometrów.

Wymagania:

- gettery oznacz jako `const`,
- `jedz(0)` i `jedz(-10)` nie powinny zmieniać przebiegu.

48.5.4 Zadanie 9. Walidacja settera

Dodaj setter `ustawPrzebieg`, który nie pozwala ustawić przebiegu na wartość ujemną.

Setter powinien zwracać `true`, jeśli zmiana się udała, i `false`, jeśli została odrzucona.

48.5.5 Zadanie 10. Klasa Uczeń

Utwórz klasę `Uczen` z prywatnymi polami `imie`, `nazwisko` i `ocena`.

Ocena powinna być w zakresie od 1 do 6.

Wymagania:

- pole `ocena` ma być prywatne,
- konstruktor albo setter ma odrzucać ocenę spoza zakresu,
- dodaj metodę `czyZaliczyl`, która zwraca `true` dla ocen od 2 do 6.

48.6 Poziom 3 - `const`, `this` i kopiowanie

48.6.1 Zadanie 11. Gettery `const`

W klasie `Produkt` dodaj gettery oznaczone jako `const`.

Sprawdź, czy możesz wywołać gettery na obiekcie zadeklarowanym jako `const Produkt`.

48.6.2 Zadanie 12. Wartość produktu

Dodaj metodę `wartosc()` `const`, która nie zmienia obiektu.

Scenariusz sprawdzenia:

Produkt: Zeszyt
Cena: 4.50
Ilosc: 3
Wartosc: 13.50

48.6.3 Zadanie 13. Konstruktor z `this`

Napisz konstruktor klasy `Produkt`, w którym parametry mają takie same nazwy jak pola. Użyj `this`, aby odróżnić pola od parametrów.

48.6.4 Zadanie 14. Konstruktor kopiujący

Dodaj konstruktor kopiujący do klasy `Konto`. W konstruktorze wypisz komunikat `Kopiowanie konta`. W `main` utwórz kopię konta i pokaż, że oba obiekty mają te same dane, ale są osobnymi obiektami.

48.6.5 Zadanie 15. Przekazywanie obiektu do funkcji

Napisz dwie funkcje wypisujące konto:

- pierwsza przyjmuje konto przez wartość,
- druga przyjmuje konto przez `const` referencję.

Zaobserwuj, kiedy powstaje kopia.

Wymagania:

- wykorzystaj komunikat z konstruktora kopiującego,
- w komentarzu zapisz, która wersja funkcji jest lepsza do samego odczytu i dlaczego.

48.7 Poziom 4 - Dziedziczenie

48.7.1 Zadanie 16. Pracownik i menedżer

Utwórz klasę `Pracownik` z polami `imie` i `pensja`.

Utwórz klasę `Menedzer`, która dziedziczy po `Pracownik` i dodaje pole `liczbaOsob`.

Wymagania:

- pola klasy bazowej powinny być prywatne albo chronione zgodnie z wybranym projektem,
- dodaj metodę wypisującą dane pracownika i menedżera,
- pokaż utworzenie obu typów obiektów.

48.7.2 Zadanie 17. Samochód sportowy

Utwórz klasę `SportowySamochod`, która dziedziczy po `Samochod` i dodaje pole `maksymalnaPredkosc`.

Dodaj metodę `turbo`, która zwiększa maksymalną prędkość tylko dla dodatniej wartości.

Scenariusz sprawdzenia:

Predkosc poczatkowa: 220
Po turbo 30: 250
Po turbo -10: 250

48.7.3 Zadanie 18. Konto oszczędnościowe

Utwórz klasę `KontoOszczednosciowe`, która dziedziczy po `Konto`.

Dodaj pole `oprocentowanie` oraz metodę `doliczOdsetki`.

Wymagania:

- `oprocentowanie` nie może być ujemne,
- `odsetki` powinny zwiększać saldo,

- wykorzystaj publiczne metody klasy bazowej zamiast bezpośredniego dostępu do pól prywatnych.

48.7.4 Zadanie 19. Relacja „jest” czy „ma”

Dla poniższych par zdecyduj, czy lepsze jest dziedziczenie, czy pole w klasie:

- samochód i silnik,
- menedżer i pracownik,
- zamówienie i produkt,
- sportowy samochód i samochód.

Zapisz krótkie uzasadnienie w komentarzu.

Wymagania:

- dla każdej pary napisz jednozdaniowe uzasadnienie,
- nie implementuj pełnych klas, jeśli zadanie ma być tylko analizą.

48.7.5 Zadanie 20. Opis hierarchii

Napisz program z klasami `Pracownik`, `Menedzer` i `Inzynier`.

Każda klasa powinna mieć metodę wypisującą opis obiektu.

Wymagania:

- pokaż wspólne elementy w klasie bazowej,
- pokaż różnice w klasach pochodnych,
- nie powielaj tych samych pól bez potrzeby.

Większe ćwiczenie z klasą abstrakcyjną i polimorfizmem znajduje się w pliku `08-cwiczenie-figury-polimorfizm.md`.

48.8 Poziom 5 - Przeciążanie operatorów

48.8.1 Zadanie 21. Punkt i operator +

Dodaj do klasy `Punkt` operator `+`, który dodaje współrzędne dwóch punktów.

Scenariusz sprawdzenia:

$(2, 3) + (4, -1) = (6, 2)$

48.8.2 Zadanie 22. Punkt i operator ==

Dodaj operator `==`, który zwraca `true`, jeśli dwa punkty mają takie same współrzędne.

W `main` sprawdź parę równych i parę różnych punktów.

48.8.3 Zadanie 23. Punkt i operator <<

Dodaj operator `<<`, który wypisuje punkt w formacie:

`(x, y)`

Operator powinien zwracać referencję do strumienia, aby można było łączyć wypisywanie kilku wartości.

48.8.4 Zadanie 24. Kwota

Utwórz klasę `Kwota` z polami `zlote` i `grosze`. Dodaj operator `+`, który poprawnie sumuje grosze.

Scenariusz sprawdzenia:

`12 zl 80 gr + 3 zl 50 gr = 16 zl 30 gr`

Wymagania:

- po dodawaniu **grosze** powinny być w zakresie 0..99,
- pola powinny być prywatne.

48.8.5 Zadanie 25. Macierz

Utwórz klasę **Macierz** reprezentującą macierz liczb `double`.

Dodaj:

- konstruktor z liczbą wierszy i kolumn,
- metody `set` i `get`,
- operator `+`,
- operator `==`,
- operator `<<`.

Wymagania:

- sprawdzaj zgodność wymiarów przy dodawaniu,
- `get` i `set` powinny obsługiwać błędny indeks,
- dla pierwszej wersji możesz ograniczyć macierz do stałego rozmiaru 2 x 2, jeśli pełna wersja jest zbyt trudna.

48.9 Zadania dodatkowe

48.9.1 Zadanie 26. Klasa do obsługi pliku

Utwórz klasę **PlikTekstowy**, która przechowuje nazwę pliku i ma metody:

- `zapisz`,
- `dopisz`,
- `odczytaj`.

Wykorzystaj wiedzę z segmentu 05-pliki-wyjatki.

Wymagania:

- metody powinny zgłaszać błąd, jeśli operacja na pliku się nie uda,
- nazwa pliku powinna być ustawiana w konstruktorze.

48.9.2 Zadanie 27. Prosty katalog produktów

Utwórz klasy **Produkt** i **KatalogProduktow**.

Katalog powinien pozwalać:

- dodać produkt,
- wypisać produkty,
- policzyć łączną wartość magazynu.

Wymagania:

- **KatalogProduktow** powinien przechowywać kolekcję produktów,
- nie pozwól dodać produktu z ujemną ceną albo ilością,
- metoda licząca wartość magazynu powinna być `const`.

48.9.3 Zadanie 28. Mini-projekt: konto bankowe

Przygotuj program z klasami:

- **Konto**,
- **KontoOszczednosciowe**,
- **HistoriaOperacji**.

Program powinien pozwalać wykonać wpłatę, wypłatę i wypisać historię operacji.

48.10 Wariant minimum

Do zaliczenia segmentu wykonaj co najmniej:

1. Zadanie 1 albo 2.
2. Zadanie 6 albo 7.
3. Zadanie 8 albo 10.
4. Zadanie 11 albo 12.
5. Zadanie 14 albo 15.
6. Zadanie 16 albo 18.
7. Zadanie 21 albo 22.

Zadania 25-28 są dodatkowe albo projektowe. Możesz je robić po wykonaniu wariantu minimum.

48.11 Mini-sprawdzian segmentu

Napisz program z klasą `Konto`, który:

- ma prywatne pola `wlasciciel`, `numer`, `saldo`,
- ustawia poprawny stan w konstruktorze,
- odrzuca ujemne saldo początkowe,
- pozwala wykonać wpłatę i wypłatę z walidacją,
- ma metodę `pobierzSaldo()` `const`,
- ma operator `==` porównujący numer konta,
- kompiluje się z `-Wall -Wextra -pedantic`.

Scenariusz sprawdzenia:

Saldo początkowe: 100

Wpłata 50: zaakceptowana

Wypłata 30: zaakceptowana

Wypłata 500: odrzucona

Saldo końcowe: 120

Konta o tym samym numerze: równe

48.12 Kryteria zaliczenia segmentu

Student zalicza segment, jeśli potrafi samodzielnie:

- zdefiniować klasę i utworzyć obiekty,
- dobrać pola i metody do prostego problemu,
- ukryć pola jako `private`,
- napisać konstruktor,
- dodać gettery i metody modyfikujące z walidacją,
- oznaczyć metody odczytujące jako `const`,
- użyć `this` w konstruktorze lub setterze,
- rozpoznać moment kopiowania obiektu,
- przekazać obiekt przez `const` referencję,
- zbudować prostą hierarchię dziedziczenia,
- przeciążyć podstawowy operator w czytelny sposób.

48.13 Checklista oddania

Przed oddaniem rozwiązań sprawdź:

- ☐ Każde zadanie jest w osobnym pliku `.cpp`.
- ☐ Programy kompilują się z `-Wall -Wextra -pedantic`.
- ☐ Pola klas są prywatne, jeśli nie ma dobrego powodu, aby były publiczne.
- ☐ Konstruktory ustawiają obiekty w poprawnym stanie.
- ☐ Metody odczytujące są oznaczone jako `const`.

- ☐ Settery i metody modyfikujące walidują dane.
- ☐ `main` pokazuje użycie klasy, ale nie zawiera logiki należącej do klasy.
- ☐ Przeciążone operatory mają intuicyjne znaczenie.

48.14 Archiwum

Oryginalne materiały źródłowe tego segmentu są zachowane w katalogu archive.

Rozdział 49

08 - Ćwiczenie: figury, dziedziczenie i polimorfizm

49.1 Cel ćwiczenia

Celem ćwiczenia jest napisanie programu, który pokazuje działanie klasy abstrakcyjnej, dziedziczenia i polimorfizmu na przykładzie figur geometrycznych.

Ćwiczenie utrwała klasy abstrakcyjne, metody wirtualne, enkapsulację, dziedziczenie oraz wywoływanie metod przez interfejs klasy bazowej.

49.2 Wymagania wstępne

Przed wykonaniem ćwiczenia student powinien znać:

- klasy i obiekty,
- pola prywatne,
- konstruktory,
- metody `const`,
- podstawy dziedziczenia,
- podstawy pracy z `std::vector`.

49.3 Opis zadania

Napisz program, który:

- definiuje abstrakcyjną klasę bazową **Figura**,
- definiuje klasy pochodne **Kwadrat** i **Kolo**,
- oblicza pole każdej figury,
- zwraca opis każdej figury,
- przechowuje różne figury przez wskaźniki do klasy bazowej,
- iteruje po kolekcji figur i wywołuje metody polimorficznie.

Minimalna wersja programu ma działać w konsoli i nie wymaga żadnych bibliotek zewnętrznych.

49.4 Wymagania techniczne

Program powinien:

- być napisany w C++17,
- używać metod czysto wirtualnych,
- używać destruktora wirtualnego w klasie bazowej,
- ukrywać dane figur w sekcji `private`,

- oznaczać metody niezmiennające obiektu jako `const`,
- używać `override` w klasach pochodnych,
- używać `std::vector` do przechowywania figur,
- kompilować się z flagami `-Wall -Wextra -pedantic`.

49.5 Proponowany model klas

Klasa bazowa:

```
class Figura {
public:
    virtual ~Figura() = default;

    virtual double pole() const = 0;
    virtual std::string opis() const = 0;
};
```

Klasy pochodne:

```
class Kwadrat : public Figura {
public:
    explicit Kwadrat(double bok);

    double pole() const override;
    std::string opis() const override;

private:
    double bok_;
};
```

```
class Kolo : public Figura {
public:
    explicit Kolo(double promien);

    double pole() const override;
    std::string opis() const override;

private:
    double promien_;
};
```

W funkcji `main` możesz użyć `std::unique_ptr<Figura>`, aby bezpiecznie przechowywać obiekty różnych klas w jednej kolekcji:

```
std::vector<std::unique_ptr<Figura>> figury;
```

Kompilowalny punkt startowy znajduje się w `examples/figures__polymorphism.cpp`.

49.6 Wariant podstawowy

Minimalna wersja ćwiczenia powinna zawierać:

1. Klasę abstrakcyjną `Figura`.
2. Metodę czysto wirtualną `pole`.
3. Metodę czysto wirtualną `opis`.
4. Wirtualny destruktor w klasie bazowej.
5. Klasę `Kwadrat` z prywatnym polem `bok`.
6. Klasę `Kolo` z prywatnym polem `promien`.
7. Implementację metod `pole` i `opis` w obu klasach pochodnych.
8. Kolekcję przechowującą co najmniej jeden kwadrat i jedno koło.
9. Pętlę, która wyświetla opis i pole każdej figury.

10. Krótką instrukcję kompilacji i uruchomienia.

49.7 Proponowany podział pracy

1. Napisz klasę abstrakcyjną `Figura`.
2. Dodaj klasę `Kwadrat` i sprawdź obliczanie pola.
3. Dodaj klasę `Kolo` i stałą dla liczby π .
4. Dodaj metodę `opis` w każdej klasie.
5. Utwórz kolekcję figur przez wskaźniki do klasy bazowej.
6. Przejdź po kolekcji i wywołaj `opis()` oraz `pole()`.
7. Dodaj walidację wymiarów figur.

49.8 Zadania dodatkowe

Po wykonaniu wariantu podstawowego możesz dodać:

- klasę `Prostokat`,
- klasę `Trojkat`,
- metodę `obwod`,
- sortowanie figur po polu,
- sumę pól wszystkich figur,
- testy automatyczne metod `pole`,
- zapis listy figur do pliku.

49.9 Rozszerzenie SFML

Wizualizacja w SFML może być dodatkiem, ale nie jest wymagana w minimalnej wersji ćwiczenia.

Jeśli student chce użyć SFML, powinien potraktować to jako osobny etap po działającej wersji konsolowej.

Wariant z SFML może zawierać:

- okno aplikacji,
- narysowanie kwadratu i koła,
- podpisy figur,
- skalowanie figur do rozmiaru okna,
- zamknięcie programu po naciśnięciu przycisku zamknięcia okna.

Kod OOP powinien pozostać oddzielony od kodu odpowiedzialnego za rysowanie. Klasy `Figura`, `Kwadrat` i `Kolo` nie powinny zależeć bezpośrednio od SFML w minimalnej wersji.

49.10 Pytania kontrolne

1. Dlaczego klasa `Figura` powinna mieć wirtualny destruktor?
2. Po czym poznać, że klasa jest abstrakcyjna?
3. Dlaczego pola `bok_` i `promien_` powinny być prywatne?
4. Co daje wywołanie `pole()` przez wskaźnik do klasy bazowej?
5. Dlaczego kod rysowania w SFML nie powinien być wymagany w wariantcie minimum?

49.11 Scenariusze sprawdzenia

1. Utwórz kwadrat o boku 4 i sprawdź, czy pole wynosi 16.
2. Utwórz koło o promieniu 2 i sprawdź, czy pole jest bliskie 12.566.
3. Umieść kwadrat i koło w jednej kolekcji typu bazowego.
4. Przejdź pętlą po kolekcji i wyświetl wynik `opis()` oraz `pole()`.
5. Sprawdź, czy usunięcie obiektów przez wskaźnik do klasy bazowej jest bezpieczne dzięki wirtualnemu destruktorowi.

49.12 Kryteria zaliczenia

Ćwiczenie jest zaliczone, jeśli:

- program kompiluje się bez błędów i ostrzeżeń,
- klasa `Figura` jest abstrakcyjna,
- `pole` i `opis` są metodami wirtualnymi,
- klasy `Kwadrat` i `Kolo` dziedziczą po `Figura`,
- metody w klasach pochodnych używają `override`,
- dane figur są prywatne,
- program przechowuje figury przez wskaźnik albo referencję do klasy bazowej,
- wywołanie metod działa polimorficznie,
- wyniki pól są poprawne,
- kod jest czytelnie sformatowany.

49.13 Źródło

Ćwiczenie powstało na podstawie starszego zadania z zadań pomocniczych TNT.

Rozdział 50

07 - STL i struktury danych

Ten segment pokazuje standardowe kontenery C++ i ich praktyczne zastosowania. Materiał przechodzi od `std::vector` do kontenerów asocjacyjnych, adapterów takich jak `std::stack` i `std::queue`, podstaw algorytmów STL oraz większego zadania projektowego.

50.1 Cele segmentu

Po zakończeniu segmentu student powinien umieć:

- dobrać kontener do prostego problemu,
- używać `std::vector` do przechowywania listy danych,
- przechodzić po kontenerach pętlą zakresową,
- dodawać, usuwać i wyszukiwać elementy w kontenerze,
- używać `std::map` do par klucz-wartość,
- używać `std::stack` i `std::queue`,
- zastosować podstawowe algorytmy STL, np. `std::sort` i `std::find_if`,
- połączyć kontenery z klasami w małym programie.

50.2 Kolejność lekcji

1. 01-wprowadzenie-do-stl.md - po co są kontenery i kiedy ich używać.
2. 02-vector.md - `std::vector`, dodawanie, usuwanie, iteracja.
3. 03-algorytmy-stl.md - `std::sort`, `std::find_if`, predykaty i lambdy.
4. 04-map-i-słownik.md - `std::map`, klucze, wartości i wyszukiwanie.
5. 05-stack-queue-list.md - stos, kolejka i lista.
6. 06-projekt-hrms.md - mini-projekt z kilkoma kontenerami.
7. 07-zadania.md - zadania podstawowe i projektowe.

50.3 Status porządkowania

Lekcje i zadania zostały uporządkowane do wspólnego formatu segmentów. Pierwotne materiały są zachowane w katalogu archive:

- archive/kontenery-stl.md
- archive/vector.md
- archive/vector-zadania.md
- archive/vector-part2.md
- archive/map-zadania.md
- archive/kolejka-linki.md
- archive/console.md
- archive/converter_v1.cpp
- archive/egzamin-tablice.md

- [archive/powtorzenie-zadania.md](#)
- [archive/powtorzenie-part2.md](#)

50.4 Przykłady kodu

Przykłady w katalogu `examples` można kompilować osobno:

- `examples/stl_containers_overview.cpp` - przegląd kontenerów STL.
- `examples/choose_container.cpp` - dobór kontenera do problemu.
- `examples/vector_basics.cpp` - podstawy `std::vector`.
- `examples/vector_records.cpp` - wektor rekordów.
- `examples/sort_numbers.cpp` - sortowanie liczb.
- `examples/filter_people.cpp` - filtrowanie danych.
- `examples/map_basics.cpp` - podstawy `std::map`.
- `examples/map_vector_join.cpp` - łączenie `std::map` z wektorem rekordów.
- `examples/movie_library.cpp` - słownik filmów.
- `examples/stack_queue.cpp` - stos i kolejka.
- `examples/list_operations.cpp` - operacje na liście.
- `examples/hrms.cpp` - przykład projektu HRMS.

50.5 Testy

Katalog `tests` zawiera prosty test kontenerów i algorytmów STL bez zewnętrznego frameworka. Test pokazuje sortowanie, filtrowanie, wyszukiwanie w `std::map` oraz kolejność obsługi `std::queue`.

50.6 Format lekcji

Każda docelowa lekcja powinna mieć taki układ:

1. Cel lekcji.
2. Wymagania wstępne.
3. Krótka teoria.
4. Przykład kodu.
5. Zadania do wykonania.
6. Kryteria zaliczenia.

50.7 Katalogi pomocnicze

- `examples/` - kompilowalne przykłady `.cpp` dla tego segmentu.
- `tests/` - proste testy automatyczne dla kontenerów i algorytmów STL.
- `archive/` - pierwotne wersje materiałów zachowane do wglądu.

Rozdział 51

01 - Wprowadzenie do STL

51.1 Cel lekcji

Celem lekcji jest zrozumienie, czym jest STL, po co używamy kontenerów standardowych i jak wstępnie dobrać kontener do problemu.

51.2 Wymagania wstępne

Student powinien znać:

- tablice i pętle z segmentu 03-funkcje-tablice-napisy,
- klasy i obiekty z segmentu 06-oop,
- podstawowe operacje wejścia i wyjścia.

51.3 Krótka teoria

51.3.1 Czym jest STL

STL to część biblioteki standardowej C++, która dostarcza gotowe narzędzia do pracy z danymi.

W tym segmencie najważniejsze będą:

- kontenery, czyli gotowe struktury do przechowywania danych,
- iteracja po kontenerach,
- algorytmy, np. sortowanie i wyszukiwanie,
- dobór kontenera do problemu.

Zamiast samodzielnie pisać tablicę dynamiczną, kolejkę albo mapowanie klucz-wartość, zwykle lepiej użyć gotowego kontenera z biblioteki standardowej.

51.3.2 Kontener

Kontener to obiekt przechowujący wiele elementów.

Przykład:

```
std::vector<int> liczby = {10, 20, 30};
```

`std::vector<int>` przechowuje wiele liczb typu `int`.

Do użycia `std::vector` potrzebny jest nagłówek:

```
#include <vector>
```

51.3.3 Najważniejsze kontenery w tym segmencie

51.3.4 `std::vector`

`std::vector` jest listą elementów w ciągłej pamięci.

Dobry wybór, gdy:

- potrzebujesz listy elementów,
- często dodajesz elementy na końcu,
- chcesz szybko dostać się do elementu po indeksie,
- nie znasz z góry liczby elementów.

51.3.5 `std::map`

`std::map` przechowuje pary klucz-wartość.

Dobry wybór, gdy:

- wyszukujesz dane po unikalnym kluczu,
- chcesz powiązać nazwę z wartością,
- np. tytuł filmu z oceną albo identyfikator pracownika z wynagrodzeniem.

51.3.6 `std::stack`

`std::stack` to stos. Działa według zasady: ostatni dodany element wychodzi pierwszy.

Dobry wybór, gdy:

- obsługujesz cofanie operacji,
- analizujesz nawiasy,
- potrzebujesz struktury LIFO.

51.3.7 `std::queue`

`std::queue` to kolejka. Działa według zasady: pierwszy dodany element wychodzi pierwszy.

Dobry wybór, gdy:

- obsługujesz kolejkę zadań,
- modelujesz klientów w kolejce,
- przetwarzasz zgłoszenia w kolejności przyjścia.

51.3.8 `std::list`

`std::list` to lista dwukierunkowa.

Dobry wybór, gdy:

- często wstawiasz lub usuwasz elementy ze środka,
- nie potrzebujesz szybkiego dostępu po indeksie.

W praktyce na początku częściej używa się `std::vector`, a `std::list` wybiera dopiero wtedy, gdy problem naprawdę tego wymaga.

51.4 Przykład kodu: kilka kontenerów

Pełny przykład znajduje się w pliku `examples/stl_containers_overview.cpp`.

Pokazuje on:

- `std::vector` jako listę liczb,
- `std::map` jako oceny filmów,
- `std::queue` jako kolejkę zadań,
- `std::stack` jako historię cofania.

51.5 Przykład kodu: dobór kontenera

Dobór kontenera zaczynamy od pytania: jak będę używać danych?

Jeśli chcesz przechowywać listę:

```
std::vector<std::string> imiona;
```

Jeśli chcesz szukać po kluczu:

```
std::map<std::string, double> ocenyFilmow;
```

Jeśli chcesz obsługiwać kolejkę:

```
std::queue<std::string> zadania;
```

Jeśli chcesz zdejmować ostatnio dodany element:

```
std::stack<std::string> historia;
```

Pełny przykład znajduje się w pliku `examples/choose_container.cpp`.

51.5.1 Pętla zakresowa

Po kontenerach często przechodzimy pętlą zakresową.

```
for (const auto& liczba : liczby)
{
    std::cout << liczba << std::endl;
}
```

`const auto&` oznacza, że:

- typ elementu zostanie rozpoznany automatycznie,
- element nie będzie kopiowany,
- element nie zostanie zmieniony.

Na początku można też pisać typ jawnie.

```
for (const int& liczba : liczby)
{
    std::cout << liczba << std::endl;
}
```

51.6 Typowe błędy

51.6.1 Ręczne pisanie struktury, gdy istnieje kontener

Jeśli potrzebujesz listy elementów, zacznij od `std::vector`, a nie od ręcznej tablicy dynamicznej.

51.6.2 Zły kontener do problemu

Jeśli często szukasz danych po nazwie, `std::vector` może wymagać przeglądania wszystkich elementów. `std::map` będzie bardziej naturalny.

51.6.3 Kopiowanie dużych elementów w pętli

Mniej korzystnie:

```
for (auto element : elementy)
{
}
```

Lepiej dla większych obiektów:

```
for (const auto& element : elementy)
{
}
```

51.7 Zadania do wykonania

1. Wypisz trzy przykłady problemów, dla których dobrym wyborem będzie `std::vector`.
2. Wypisz trzy przykłady problemów, dla których dobrym wyborem będzie `std::map`.
3. Napisz program, który przechowuje imiona w `std::vector` i wypisuje je pętlą zakresową.
4. Napisz program, który przechowuje oceny filmów w `std::map`.
5. Napisz krótki komentarz w kodzie: czym różni się `std::stack` od `std::queue`.

51.8 Kryteria zaliczenia

Program powinien:

- używać właściwych nagłówków,
- tworzyć co najmniej jeden kontener STL,
- dodawać elementy do kontenera,
- przechodzić po kontenerze pętlą,
- umieć uzasadnić wybór kontenera,
- kompilować się bez błędów.

51.9 Pytania kontrolne

1. Czym jest kontener?
2. Kiedy warto użyć `std::vector`?
3. Kiedy warto użyć `std::map`?
4. Czym różni się `std::stack` od `std::queue`?
5. Po co używamy `const auto&` w pętli zakresowej?

Rozdział 52

02 - `std::vector`

52.1 Cel lekcji

Celem lekcji jest nauczenie się podstaw pracy z `std::vector`: tworzenia wektora, dodawania elementów, usuwania ostatniego elementu, odczytu po indeksie i iteracji.

52.2 Wymagania wstępne

Student powinien znać:

- tablice z segmentu 03-funkcje-tablice-napisy,
- pętle z segmentu 02-sterowanie-i-petle,
- podstawy STL z lekcji 01-wprowadzenie-do-stl.md.

52.3 Krótka teoria

52.3.1 Czym jest `std::vector`

`std::vector` to kontener przechowujący elementy w kolejności.

Można myśleć o nim jak o tablicy, która może zmieniać rozmiar.

```
std::vector<int> liczby;
```

Do użycia `std::vector` potrzebny jest nagłówek:

```
#include <vector>
```

52.4 Przykład kodu: tworzenie wektora

Pusty wektor:

```
std::vector<int> liczby;
```

Wektor z wartościami początkowymi:

```
std::vector<int> liczby = {10, 20, 30};
```

Wektor o podanym rozmiarze:

```
std::vector<int> liczby(5);
```

Taki wektor ma pięć elementów ustawionych na 0.

52.5 Przykład kodu: dodawanie elementów

Do dodania elementu na końcu używamy `push_back`.

```
liczby.push_back(10);  
liczby.push_back(20);  
liczby.push_back(30);
```

Pełny przykład znajduje się w pliku `examples/vector_basics.cpp`.

52.5.1 Rozmiar wektora

Liczbę elementów sprawdzamy metodą `size`.

```
std::cout << liczby.size() << std::endl;
```

`size()` zwraca typ bez znaku. W prostych pętlach indeksowych można użyć:

```
for (std::size_t i = 0; i < liczby.size(); i++)  
{  
    std::cout << liczby[i] << std::endl;  
}
```

Do `std::size_t` zwykle wystarczy dołączony już nagłówek standardowy, ale w większych programach można też użyć:

```
#include <cstddef>
```

52.5.2 Dostęp po indeksie

Dostęp przez `[]`:

```
std::cout << liczby[0] << std::endl;
```

Jest szybki, ale nie sprawdza zakresu.

Dostęp przez `at()`:

```
std::cout << liczby.at(0) << std::endl;
```

`at()` sprawdza zakres i w razie błędu zgłasza wyjątek `std::out_of_range`.

Na początku warto używać `at()`, gdy indeks pochodzi od użytkownika albo z obliczeń.

52.5.3 Pętla zakresowa

Najczytelniejszy sposób wypisania wszystkich elementów:

```
for (const auto& liczba : liczby)  
{  
    std::cout << liczba << std::endl;  
}
```

Jeśli elementy są małe, np. `int`, można też pisać:

```
for (int liczba : liczby)  
{  
    std::cout << liczba << std::endl;  
}
```

52.5.4 Usuwanie ostatniego elementu

Ostatni element usuwamy metodą `pop_back`.

```
liczby.pop_back();
```

Nie wolno wywoływać `pop_back()` na pustym wektorze.

```
if (!liczby.empty())
{
    liczby.pop_back();
}
```

52.5.5 empty, front, back

Przydatne metody:

- `empty()` - czy wektor jest pusty,
- `front()` - pierwszy element,
- `back()` - ostatni element.

```
if (!liczby.empty())
{
    std::cout << liczby.front() << std::endl;
    std::cout << liczby.back() << std::endl;
}
```

52.6 Przykład kodu: wektor struktur lub obiektów

Wektor może przechowywać nie tylko liczby, ale też struktury i obiekty.

```
struct Kandydat
{
    std::string imie;
    int wiek;
};

std::vector<Kandydat> kandydaci;
```

Pełny przykład znajduje się w pliku `examples/vector_records.cpp`.

52.6.1 resize

Metoda `resize` zmienia rozmiar wektora.

```
std::vector<int> liczby = {1, 2, 3};
liczby.resize(5);
```

Po powiększeniu nowe elementy będą miały wartość domyślną, np. 0 dla `int`.

```
liczby.resize(6, 4);
```

Ten zapis powiększa wektor i wstawia 4 jako wartość nowych elementów.

52.7 Typowe błędy

52.7.1 Wyjście poza zakres

Niepoprawnie:

```
std::vector<int> liczby = {1, 2, 3};
std::cout << liczby[10] << std::endl;
```

Poprawniej:

```
if (indeks >= 0 && indeks < static_cast<int>(liczby.size()))
{
    std::cout << liczby[indeks] << std::endl;
}
```

52.7.2 pop_back na pustym wektorze

Niepoprawnie:

```
liczby.pop_back();
```

Poprawnie:

```
if (!liczby.empty())
{
    liczby.pop_back();
}
```

52.7.3 Kopiowanie dużych obiektów w pętli

Mniej korzystnie:

```
for (auto kandydat : kandydaci)
{
}
```

Lepiej:

```
for (const auto& kandydat : kandydaci)
{
}
```

52.8 Zadania do wykonania

1. Utwórz wektor liczb całkowitych i dodaj do niego liczby od 1 do 10.
2. Wypisz zawartość wektora pętlą zakresową.
3. Usuń ostatni element wektora, jeśli wektor nie jest pusty.
4. Wypisz pierwszy i ostatni element wektora.
5. Utwórz wektor struktur `Osoba` z polami `imie` i `wiek`. Wypisz wszystkie osoby.

52.9 Kryteria zaliczenia

Program powinien:

- dołączać nagłówek `<vector>`,
- poprawnie tworzyć `std::vector`,
- dodawać elementy przez `push_back`,
- sprawdzać `empty()` przed `pop_back`, `front` i `back`,
- przechodzić po wektorze pętlą zakresową,
- kompilować się bez błędów.

52.10 Pytania kontrolne

1. Czym `std::vector` różni się od zwykłej tablicy?
2. Do czego służy `push_back`?
3. Do czego służy `size`?
4. Czym różni się `liczby[0]` od `liczby.at(0)`?
5. Dlaczego warto sprawdzić `empty()` przed `pop_back()`?

Rozdział 53

03 - Algorytmy STL

53.1 Cel lekcji

Celem lekcji jest poznanie podstawowych algorytmów STL: `std::sort`, `std::find_if`, `std::copy_if` i `std::remove_if`, oraz zrozumienie prostych predykatów i `lambda`.

53.2 Wymagania wstępne

Student powinien znać:

- podstawy STL z lekcji 01-wprowadzenie-do-stl.md,
- `std::vector` z lekcji 02-vector.md,
- funkcje z segmentu 03-funkcje-tablice-napisy.

53.3 Krótka teoria

53.3.1 Nagłówek `<algorithm>`

Do wielu algorytmów STL potrzebny jest nagłówek:

```
#include <algorithm>
```

Algorytmy działają na zakresie elementów. Zakres zwykle zapisujemy przez `begin()` i `end()`.

```
std::sort(liczby.begin(), liczby.end());
```

53.4 Przykład kodu: `std::sort`

`std::sort` sortuje elementy w kontenerze.

```
std::vector<int> liczby = {5, 1, 4, 2, 3};  
std::sort(liczby.begin(), liczby.end());
```

Po sortowaniu wektor będzie miał wartości:

```
1 2 3 4 5
```

Pełny przykład znajduje się w pliku `examples/sort_numbers.cpp`.

53.4.1 Sortowanie malejące

Do sortowania malejącego można przekazać predykat.

```
std::sort(liczby.begin(), liczby.end(), [](int a, int b)  
{
```

```

    return a > b;
});

```

Predykat mówi, który element powinien być wcześniej.

53.4.2 Lambda

Lambda to krótka funkcja zapisana w miejscu użycia.

```

[](int a, int b)
{
    return a > b;
}

```

W tym przykładzie lambda przyjmuje dwie liczby i zwraca `true`, jeśli pierwsza ma być przed drugą.

53.5 Przykład kodu: `std::find_if`

`std::find_if` szuka pierwszego elementu spełniającego warunek.

```

auto wynik = std::find_if(liczby.begin(), liczby.end(), [](int liczba)
{
    return liczba > 10;
});

```

Po wywołaniu trzeba sprawdzić, czy coś znaleziono.

```

if (wynik != liczby.end())
{
    std::cout << "Znaleziono: " << *wynik << std::endl;
}

```

53.6 Przykład kodu: `std::copy_if`

`std::copy_if` kopiuje do nowego kontenera tylko elementy spełniające warunek.

```

std::vector<int> dodatnie;

std::copy_if(liczby.begin(), liczby.end(), std::back_inserter(dodatnie), [](int liczba)
{
    return liczba > 0;
});

```

Do `std::back_inserter` potrzebny jest nagłówek:

```

#include <iterator>

```

53.6.1 `std::remove_if` i `erase`

`std::remove_if` samo nie zmniejsza rozmiaru wektora. Przesuwa elementy, które mają zostać zachowane.

Dla `std::vector` używamy wzorca `erase-remove`:

```

liczby.erase(
    std::remove_if(liczby.begin(), liczby.end(), [](int liczba)
    {
        return liczba < 0;
    }),
    liczby.end()
);

```

Ten kod usuwa liczby ujemne.

Pełny przykład znajduje się w pliku `examples/filter_people.cpp`.

53.6.2 Predykat

Predykat to funkcja albo lambda zwracająca `bool`.

Przykłady predykatów:

```
[] (int liczba) { return liczba > 0; }
[] (const Osoba& osoba) { return osoba.wiek >= 18; }
[] (const Produkt& produkt) { return produkt.cena < 100; }
```

Predykaty są używane w algorytmach takich jak `find_if`, `copy_if` i `remove_if`.

53.7 Typowe błędy

53.7.1 Brak sprawdzenia wyniku `find_if`

Niepoprawnie:

```
auto wynik = std::find_if(liczby.begin(), liczby.end(), warunek);
std::cout << *wynik << std::endl;
```

Poprawnie:

```
if (wynik != liczby.end())
{
    std::cout << *wynik << std::endl;
}
```

53.7.2 Samo `remove_if` bez `erase`

`remove_if` nie usuwa fizycznie elementów z wektora. Do usunięcia potrzebne jest `erase`.

53.7.3 Sortowanie obiektów bez kryterium

Jeśli sortujesz obiekty własnej klasy lub struktury, musisz powiedzieć, według czego sortować.

```
std::sort(osoby.begin(), osoby.end(), [](const Osoba& a, const Osoba& b)
{
    return a.wiek < b.wiek;
});
```

53.8 Zadania do wykonania

1. Utwórz wektor liczb i posortuj go rosnąco.
2. Posortuj ten sam wektor malejąco przy pomocy lambdy.
3. Znajdź pierwszą liczbę większą od 50 przy pomocy `std::find_if`.
4. Skopiuj do nowego wektora tylko liczby parzyste przy pomocy `std::copy_if`.
5. Usuń z wektora wszystkie liczby ujemne przy pomocy wzorca `erase-remove`.

53.9 Kryteria zaliczenia

Program powinien:

- dołączać nagłówki `<algorithm>`,
- używać `begin()` i `end()`,
- poprawnie używać `std::sort`,
- sprawdzać wynik `std::find_if`,
- używać lambdy jako predykatu,
- poprawnie stosować `remove_if` razem z `erase`,
- kompilować się bez błędów.

53.10 Pytania kontrolne

1. Do czego służy `std::sort`?
2. Czym jest predykat?
3. Co zwraca `std::find_if`, gdy nic nie znajdzie?
4. Dlaczego `remove_if` trzeba połączyć z `erase`?
5. Po co używamy `lambda` w algorytmach STL?

Rozdział 54

04 - `std::map` i słownik

54.1 Cel lekcji

Celem lekcji jest nauczenie się pracy z `std::map`, czyli kontenerem przechowującym pary klucz-wartość.

54.2 Wymagania wstępne

Student powinien znać:

- podstawy STL z lekcji 01-wprowadzenie-do-stl.md,
- `std::vector` z lekcji 02-vector.md,
- klasy z segmentu 06-oop.

54.3 Krótka teoria

54.3.1 Czym jest `std::map`

`std::map` przechowuje dane jako pary:

klucz -> wartość

Przykłady:

- tytuł filmu -> ocena,
- identyfikator pracownika -> pensja,
- nazwa produktu -> liczba sztuk,
- słowo -> liczba wystąpień.

Do użycia `std::map` potrzebny jest nagłówek:

```
#include <map>
```

54.4 Przykład kodu: tworzenie mapy

Mapa z kluczem typu `std::string` i wartością typu `double`:

```
std::map<std::string, double> ocenyFilmow;
```

Dodawanie wartości:

```
ocenyFilmow["Matrix"] = 5.0;  
ocenyFilmow["Incepcja"] = 4.8;
```

Pełny przykład znajduje się w pliku `examples/map_basics.cpp`.

54.4.1 Klucz musi być unikalny

W `std::map` każdy klucz występuje tylko raz.

```
ocenyFilmow["Matrix"] = 5.0;
ocenyFilmow["Matrix"] = 4.5;
```

Po drugim przypisaniu wartość dla klucza "Matrix" zostanie zmieniona na 4.5.

54.4.2 Odczyt przez operator[]

Można odczytać wartość tak:

```
std::cout << ocenyFilmow["Matrix"] << std::endl;
```

Trzeba jednak uważać: jeśli klucza nie ma, `operator[]` utworzy nowy wpis z wartością domyślną.

```
std::cout << ocenyFilmow["Nieznany film"] << std::endl;
```

Ten kod doda do mapy nowy film z oceną 0.0.

54.5 Przykład kodu: bezpieczne wyszukiwanie przez find

Do sprawdzania, czy klucz istnieje, używamy `find`.

```
auto wynik = ocenyFilmow.find("Matrix");

if (wynik != ocenyFilmow.end())
{
    std::cout << wynik->second << std::endl;
}
```

`wynik->first` to klucz, a `wynik->second` to wartość.

54.5.1 Przechodzenie po mapie

Po mapie można przechodzić pętlą zakresową.

```
for (const auto& para : ocenyFilmow)
{
    std::cout << para.first << ": " << para.second << std::endl;
}
```

Elementy w `std::map` są uporządkowane według klucza.

54.5.2 Usuwanie elementu

Element można usunąć metodą `erase`.

```
ocenyFilmow.erase("Matrix");
```

Jeśli klucza nie ma, metoda nie usuwa niczego.

Przed usunięciem można sprawdzić, czy element istnieje.

```
if (ocenyFilmow.find("Matrix") != ocenyFilmow.end())
{
    ocenyFilmow.erase("Matrix");
}
```

54.6 Przykład kodu: biblioteka filmów

`std::map` dobrze pasuje do biblioteki filmów, w której tytuł jest kluczem, a ocena jest wartością.

```
std::map<std::string, double> filmy;
```

Pełny przykład klasy `MovieLibrary` znajduje się w pliku `examples/movie_library.cpp`.

Przykład pokazuje:

- dodawanie filmu,
- sprawdzanie duplikatu,
- usuwanie filmu,
- pobieranie oceny,
- obliczanie średniej,
- szukanie najwyższej ocenionego filmu.

54.7 Typowe błędy

54.7.1 Użycie `[]` do sprawdzania istnienia klucza

Niepoprawnie:

```
if (ocenyFilmow["Matrix"] > 0)
{
}
```

Jeśli filmu nie ma, taki kod doda go do mapy.

Poprawnie:

```
auto wynik = ocenyFilmow.find("Matrix");
if (wynik != ocenyFilmow.end())
{
}
```

54.7.2 Brak sprawdzenia wyniku `find`

Niepoprawnie:

```
auto wynik = ocenyFilmow.find("Matrix");
std::cout << wynik->second << std::endl;
```

Poprawnie:

```
if (wynik != ocenyFilmow.end())
{
    std::cout << wynik->second << std::endl;
}
```

54.7.3 Zły wybór klucza

Klucz powinien jednoznacznie identyfikować wpis. Jeśli tytuły filmów mogą się powtarzać, sam tytuł może nie wystarczyć.

54.8 Zadania do wykonania

1. Utwórz `std::map<std::string, int>` przechowującą liczbę sztuk produktów w magazynie.
2. Dodaj kilka produktów i wypisz całą mapę.
3. Sprawdź przez `find`, czy produkt istnieje.
4. Usuń produkt z mapy.
5. Utwórz klasę `MovieLibrary` z mapą `std::map<std::string, double>` i metodami `addMovie`, `removeMovie`, `getRating`.

54.9 Kryteria zaliczenia

Program powinien:

- dołączać nagłówek `<map>`,
- tworzyć mapę z odpowiednim typem klucza i wartości,
- dodawać i aktualizować wartości,
- sprawdzać istnienie klucza przez `find`,
- przechodzić po mapie pętlą zakresową,
- usuwać elementy przez `erase`,
- kompilować się bez błędów.

54.10 Pytania kontrolne

1. Co przechowuje `std::map`?
2. Dlaczego klucz w mapie musi być unikalny?
3. Jaka jest różnica między `mapa["klucz"]` a `mapa.find("klucz")`?
4. Co oznaczają `first` i `second` w elemencie mapy?
5. Kiedy `std::map` jest lepszym wyborem niż `std::vector`?

Rozdział 55

05 - `std::stack`, `std::queue`, `std::list`

55.1 Cel lekcji

Celem lekcji jest poznanie trzech struktur danych z biblioteki standardowej: stosu, kolejki i listy dwukierunkowej. Student powinien rozumieć różnicę między LIFO i FIFO oraz wiedzieć, kiedy `std::list` jest lepszym wyborem niż `std::vector`.

55.2 Wymagania wstępne

Student powinien znać:

- podstawy STL z lekcji 01-wprowadzenie-do-stl.md,
- `std::vector` z lekcji 02-vector.md,
- algorytmy STL z lekcji 03-algorytmy-stl.md.

55.3 Krótka teoria

55.3.1 `std::stack`

`std::stack` to stos. Działa według zasady LIFO:

last in, first out

Ostatni dodany element jest zdejmowany jako pierwszy.

Do użycia stosu potrzebny jest nagłówek:

```
#include <stack>
```

Podstawowe operacje:

- `push` - dodaje element,
- `top` - zwraca element na szczycie,
- `pop` - usuwa element ze szczytu,
- `empty` - sprawdza, czy stos jest pusty,
- `size` - zwraca liczbę elementów.

Przykład:

```
std::stack<std::string> historia;  
historia.push("Dodano rekord");  
historia.push("Zmieniono rekord");
```

```
std::cout << historia.top() << std::endl;
historia.pop();
```

Pełny przykład znajduje się w pliku `examples/stack_queue.cpp`.

55.4 Przykład kodu: `std::queue`

`std::queue` to kolejka. Działa według zasady FIFO:

`first in, first out`

Pierwszy dodany element jest obsługiwany jako pierwszy.

Do użycia kolejki potrzebny jest nagłówek:

```
#include <queue>
```

Podstawowe operacje:

- `push` - dodaje element na koniec,
- `front` - zwraca pierwszy element,
- `back` - zwraca ostatni element,
- `pop` - usuwa pierwszy element,
- `empty` - sprawdza, czy kolejka jest pusta,
- `size` - zwraca liczbę elementów.

Przykład:

```
std::queue<std::string> zgloszenia;
zgloszenia.push("Zgłoszenie 1");
zgloszenia.push("Zgłoszenie 2");

std::cout << zgloszenia.front() << std::endl;
zgloszenia.pop();
```

55.4.1 LIFO kontra FIFO

Stos:

dodaj: A, B, C
zdejmij: C, B, A

Kolejka:

dodaj: A, B, C
obsłuż: A, B, C

Stos pasuje do historii cofania, a kolejka do obsługi zgłoszeń.

55.5 Przykład kodu: `std::list`

`std::list` to lista dwukierunkowa.

Do użycia listy potrzebny jest nagłówek:

```
#include <list>
```

Przydatne operacje:

- `push_front` - dodaje element na początek,
- `push_back` - dodaje element na koniec,
- `insert` - wstawia element w wybrane miejsce,
- `erase` - usuwa element,
- `remove` - usuwa elementy o podanej wartości.

Przykład:

```
std::list<int> liczby = {7, 5, 16, 8};
liczby.push_front(25);
liczby.push_back(13);
```

Pełny przykład znajduje się w pliku `examples/list_operations.cpp`.

55.5.1 Kiedy używać `std::list`

`std::list` ma sens, gdy:

- często wstawiasz elementy w środku kolekcji,
- często usuwasz elementy ze środka,
- nie potrzebujesz dostępu po indeksie.

`std::vector` jest zwykle lepszy, gdy:

- często przechodzisz po wszystkich elementach,
- potrzebujesz dostępu po indeksie,
- dodajesz głównie na końcu.

Na początku wybieraj `std::vector`, dopóki problem wyraźnie nie pasuje do listy.

55.6 Typowe błędy

55.6.1 `top`, `front` albo `back` na pustym kontenerze

Niepoprawnie:

```
std::stack<int> stos;
std::cout << stos.top() << std::endl;
```

Poprawnie:

```
if (!stos.empty())
{
    std::cout << stos.top() << std::endl;
}
```

55.6.2 Oczekiwanie dostępu po indeksie w `std::list`

`std::list` nie obsługuje wygodnego dostępu:

```
// lista[0] // tak nie działa
```

Jeśli potrzebujesz indeksów, zwykle wybierz `std::vector`.

55.6.3 Mylenie `pop` ze zwracaniem wartości

`pop()` usuwa element, ale go nie zwraca.

Najpierw odczytaj element przez `top()` albo `front()`, potem wywołaj `pop()`.

55.7 Zadania do wykonania

1. Utwórz `std::stack<std::string>` jako historię operacji. Dodaj trzy operacje i zdejmij je w kolejności cofania.
2. Utwórz `std::queue<std::string>` jako kolejkę zgłoszeń. Dodaj trzy zgłoszenia i obsłuż je w kolejności przyjścia.
3. Utwórz `std::list<int>`, dodaj elementy na początek i koniec, a następnie wypisz listę.
4. Wstaw wartość 42 przed pierwszą wartością 16 w liście.
5. Napisz komentarz w kodzie, kiedy wybrałbyś `std::list`, a kiedy `std::vector`.

55.8 Kryteria zaliczenia

Program powinien:

- używać nagłówków `<stack>`, `<queue>` albo `<list>`,
- sprawdzać `empty()` przed odczytem ze stosu lub kolejki,
- poprawnie odróżniać LIFO od FIFO,
- przechodzić po `std::list` pętlą zakresową,
- umieć uzasadnić wybór struktury danych,
- kompilować się bez błędów.

55.9 Pytania kontrolne

1. Co oznacza LIFO?
2. Co oznacza FIFO?
3. Czym różni się `top()` od `front()`?
4. Dlaczego `pop()` nie zwraca usuwanego elementu?
5. Kiedy `std::list` może być lepsza niż `std::vector`?

Rozdział 56

Projekt HRMS

56.1 Cel lekcji

Celem projektu jest połączenie klas, kontenerów STL i algorytmów w jednym programie. Zbudujemy prosty system do zarządzania pracownikami, działami oraz wynagrodzeniami.

56.2 Wymagania wstępne

Przed rozpoczęciem projektu warto znać:

- klasy i enkapsulację,
- `std::vector`,
- algorytmy STL,
- `std::map`.

56.3 Krótka teoria

System HRMS przechowuje pracowników i pozwala:

- dodać pracownika wraz z wynagrodzeniem,
- wyświetlić pracowników wskazanego działu,
- zmienić wynagrodzenie pracownika,
- wyświetlić wszystkie wynagrodzenia,
- wyświetlić wynagrodzenia posortowane malejąco.

56.4 Przykład kodu: model pracownika

Klasa `Employee` opisuje jednego pracownika:

```
class Employee {  
private:  
    std::string id;  
    std::string name;  
    std::string surname;  
    std::string departmentId;  
    std::string position;  
};
```

Pola są prywatne. Pozostała część programu korzysta z publicznych metod dostępowych, dzięki czemu klasa kontroluje sposób udostępniania danych.

56.4.1 Dobór kontenerów

System używa trzech kontenerów:

```
std::vector<Employee> employees;
std::map<std::string, std::vector<std::string>> departmentEmployees;
std::map<std::string, double> salaries;
```

- `employees` przechowuje pełne dane wszystkich pracowników,
- `departmentEmployees` przypisuje identyfikator działu do listy identyfikatorów pracowników,
- `salaries` przypisuje identyfikator pracownika do jego wynagrodzenia.

Taki podział pozwala ćwiczyć łączenie kilku struktur danych. W większym systemie należałoby dodatkowo przeanalizować koszt wyszukiwania pracowników w wektorze.

56.5 Przykład kodu: dodawanie pracownika

Przed dodaniem pracownika system sprawdza, czy:

- identyfikator nie jest już używany,
- wynagrodzenie nie jest ujemne.

Po poprawnej walidacji dane trafiają do wszystkich właściwych kontenerów. Ważne jest zachowanie ich spójności.

56.6 Przykład kodu: wyszukiwanie i zmiana wynagrodzenia

Do wyszukiwania wpisów w `std::map` używamy metody `find`:

```
auto salaryIt = salaries.find(employeeId);

if (salaryIt != salaries.end()) {
    salaryIt->second = newSalary;
}
```

Metoda `find` pozwala odróżnić istniejący wpis od brakującego. Samo użycie operatora `[]` utworzyłoby nowy wpis, gdyby klucza nie było w mapie.

56.7 Przykład kodu: sortowanie wynagrodzeń

Mapa porządkuje elementy według klucza, a nie według wynagrodzenia. Aby wyświetlić pensje malejąco:

1. kopiujemy wpisy mapy do pomocniczego wektora,
2. sortujemy wektor funkcją `std::sort`,
3. wyświetlamy posortowane elementy.

```
std::sort(sortedSalaries.begin(), sortedSalaries.end(),
    [](const auto& left, const auto& right) {
        return left.second > right.second;
    });
```

56.8 Przykład referencyjny

Kod kompletnego programu znajduje się w pliku `examples/hrms.cpp`.

Kompilacja i uruchomienie:

```
c++ -std=c++17 examples/hrms.cpp -o hrms
./hrms
```

56.9 Zadania do wykonania

1. Dodaj metodę usuwającą pracownika ze wszystkich kontenerów.
2. Dodaj możliwość przeniesienia pracownika do innego działu.
3. Oblicz średnie wynagrodzenie w wybranym dziale.
4. Wyświetl pracowników posortowanych alfabetycznie.
5. Zapisz dane pracowników do pliku i odczytaj je po ponownym uruchomieniu.

56.10 Kryteria zaliczenia

Projekt jest ukończony, gdy:

- nie pozwala dodać dwóch pracowników z tym samym identyfikatorem,
- poprawnie wyświetla pracowników działu,
- pozwala zmienić istniejące wynagrodzenie,
- sortuje wynagrodzenia malejąco,
- kompiluje się bez błędów.

56.11 Pytania kontrolne

1. Dlaczego dane jednego pracownika przechowujemy w klasie?
2. Dlaczego mapa działów przechowuje identyfikatory zamiast kopii pracowników?
3. Co się stanie, jeśli użyjemy `salaries[employeeId]` dla nieznanego klucza?
4. Dlaczego przed sortowaniem wynagrodzeń tworzymy wektor?

Rozdział 57

07 - Zadania

57.1 Cel zestawu zadań

Ten plik zbiera zadania do segmentu 07 - STL i struktury danych. Zadania sprawdzają użycie kontenerów, algorytmów STL, iteratorów, predykatów oraz łączenie struktur danych z klasami.

57.2 Zasady wykonania

Każde zadanie powinno być zapisane w osobnym pliku `.cpp`. Program powinien kompilować się bez błędów i wypisywać wynik w czytelnej formie.

Przykład kompilacji:

```
c++ -std=c++17 -Wall -Wextra -pedantic nazwa_pliku.cpp -o program
./program
```

Jeśli zadanie wymaga wyszukiwania lub sortowania, użyj odpowiedniego algorytmu STL zamiast pisać własną pętlę realizującą ten sam algorytm.

57.3 Wymagania jakościowe

W zadaniach z tego segmentu zwracaj uwagę na:

- kompilację z flagami `-std=c++17 -Wall -Wextra -pedantic`,
- dobór kontenera do sposobu użycia danych,
- pętle zakresowe i `const auto&` tam, gdzie poprawiają czytelność,
- algorytmy STL zamiast ręcznego sortowania lub wyszukiwania,
- sprawdzanie wyniku `find` przed użyciem znalezionego elementu.
- unikanie operator `[]` w `std::map`, jeśli samo wyszukiwanie nie powinno tworzyć wpisu.

57.4 Poziom 1 - `std::vector`

57.4.1 Zadanie 1. Liczby od 1 do 10

Utwórz `std::vector<int>` i wypełnij go liczbami od 1 do 10. Wyświetl wszystkie elementy za pomocą pętli zakresowej.

Wymagania:

- użyj `push_back` albo inicjalizacji listą,
- do wypisywania użyj pętli zakresowej,
- nie wypisuj elementów ręcznie po indeksach.

57.4.2 Zadanie 2. Suma i średnia

Wczytaj od użytkownika pięć liczb rzeczywistych i zapisz je w wektorze. Wyświetl ich sumę oraz średnią.

Scenariusz sprawdzenia:

Dane:

2 4 6 8 10

Suma: 30

Srednia: 6

57.4.3 Zadanie 3. Dodawanie i usuwanie

Utwórz wektor liczb od 1 do 10. Usuń z niego liczbę 5, a następnie dodaj liczbę 0 na początku wektora.

Wymagania:

- usuń wartość 5 bez założenia, że znasz jej indeks,
- sprawdź, czy wartość została znaleziona przed usunięciem,
- po operacjach wypisz cały wektor.

57.4.4 Zadanie 4. Lista słów

Utwórz wektor zawierający co najmniej pięć słów. Wyświetl:

- liczbę zapisanych słów,
- pierwsze słowo,
- ostatnie słowo,
- wszystkie słowa dłuższe niż pięć znaków.

Wymagania:

- przed odczytem pierwszego i ostatniego elementu upewnij się, że wektor nie jest pusty,
- do filtrowania dłuższych słów możesz użyć pętli zakresowej.

57.4.5 Zadanie 5. Wektor obiektów

Utwórz klasę `Produkt` z polami `nazwa` i `cena`. Zapisz kilka produktów w wektorze i wyświetl ich dane.

Wymagania:

- pola powinny być prywatne,
- dodaj konstruktor i gettery,
- cena nie może być ujemna.

57.5 Poziom 2 - Algorytmy STL

57.5.1 Zadanie 6. Sortowanie liczb

Posortuj wektor liczb całkowitych rosnąco, a następnie malejąco. Użyj `std::sort`.

Wymagania:

- sortowanie rosnące wykonaj domyślnym `std::sort`,
- sortowanie malejące wykonaj z lambdą albo `std::greater<int>`,
- wypisz wynik po każdym sortowaniu.

57.5.2 Zadanie 7. Wyszukiwanie wartości

Wyszukaj wskazaną przez użytkownika liczbę w wektorze za pomocą `std::find`. Wyświetl informację, czy liczba została znaleziona.

Sprawdź przypadek, w którym liczba istnieje, oraz przypadek, w którym jej nie ma.

57.5.3 Zadanie 8. Pierwsza liczba parzysta

Znajdź pierwszą liczbę parzystą w wektorze za pomocą `std::find_if` i wyrażenia lambda.

Jeśli liczby parzystej nie ma, wypisz komunikat `Brak liczby parzystej`.

57.5.4 Zadanie 9. Zliczanie ocen

W wektorze ocen zlicz wartości większe lub równe 3 za pomocą `std::count_if`.

Scenariusz sprawdzenia:

Oceny: 2 3 5 1 4

Zaliczone: 3

57.5.5 Zadanie 10. Usuwanie wartości

Usuń z wektora wszystkie liczby ujemne. Użyj `std::remove_if` oraz metody `erase`.

Wymagania:

- użyj idiomu `erase-remove`,
- po usunięciu wypisz pozostałe liczby,
- sprawdź przypadek, w którym wektor nie zawiera liczb ujemnych.

57.5.6 Zadanie 11. Sortowanie produktów

Rozbuduj klasę `Produkt` z zadania 5. Posortuj produkty:

- rosnąco według ceny,
- alfabetycznie według nazwy.

Do każdego sortowania użyj osobnego wyrażenia lambda.

Wymagania:

- sortowanie po cenie ma zachować ceny rosnąco,
- sortowanie po nazwie ma zachować kolejność alfabetyczną,
- nie duplikuj kodu wypisującego produkty.

57.6 Poziom 3 - `std::map`

57.6.1 Zadanie 12. Numery telefonów

Utwórz mapę, w której kluczem jest imię, a wartością numer telefonu. Dodaj kilka wpisów i wyświetl całą książkę telefoniczną.

Wymagania:

- użyj `std::map<std::string, std::string>`,
- pokaż, że klucze są wypisywane w kolejności uporządkowanej przez mapę.

57.6.2 Zadanie 13. Bezpieczne wyszukiwanie

Rozbuduj książkę telefoniczną o wyszukiwanie osoby. Użyj metody `find`, aby program nie tworzył nowego wpisu dla nieznanego imienia.

Sprawdź wyszukiwanie istniejącego i nieistniejącego imienia.

57.6.3 Zadanie 14. Licznik słów

Wczytaj zdanie i policz wystąpienia każdego słowa. Wyniki zapisz w `std::map<std::string, int>`.

Przykład:

Dane:
ala ma kota ala

Wynik:
ala: 2
ma: 1
kota: 1

57.6.4 Zadanie 15. Magazyn

Utwórz mapę przechowującą nazwę produktu i liczbę sztuk w magazynie. Program powinien pozwalać:

- dodać nowy produkt,
- zwiększyć stan produktu,
- zmniejszyć stan produktu,
- odrzucić operację prowadzącą do ujemnego stanu.

Wymagania:

- wyszukuj produkty przez `find`,
- nie pozwól na ujemny stan magazynu,
- wypisz czytelny komunikat dla nieznanego produktu.

57.6.5 Zadanie 16. Biblioteka filmów

Utwórz klasę `MovieLibrary`, która przechowuje oceny filmów w `std::map<std::string, double>`.

Klasa powinna pozwalać:

- dodać i usunąć film,
- odczytać ocenę filmu,
- obliczyć średnią ocenę,
- znaleźć najlepiej oceniony film.

Wymagania:

- odczyt oceny nie powinien tworzyć nowego wpisu,
- średnia pustej biblioteki powinna być obsługana czytelnym komunikatem,
- usunięcie nieznanego filmu powinno zwrócić informację o niepowodzeniu.

57.7 Poziom 4 - Stos, kolejka i lista

57.7.1 Zadanie 17. Odwracanie kolejności

Umieść kilka słów w `std::stack<std::string>`, a następnie zdejmuj je ze stosu i wyświetlaj. Zaobserwuj kolejność elementów.

Wynik powinien pokazać zasadę LIFO: ostatni dodany element wychodzi pierwszy.

57.7.2 Zadanie 18. Kolejka klientów

Utwórz kolejkę klientów oczekujących na obsługę. Program powinien pozwalać dodać klienta oraz obsłużyć pierwszą osobę z kolejki.

Wymagania:

- jeśli kolejka jest pusta, obsługa klienta ma wypisać komunikat,
- pokaż zasadę FIFO: pierwszy dodany klient jest obsługiwany jako pierwszy.

57.7.3 Zadanie 19. Nawiasy

Napisz funkcję, która za pomocą stosu sprawdza, czy w podanym napisie nawiasy okrągłe są poprawnie zamknięte.

Przykłady:

```
(a + b)          -> poprawne
((a + b) * c)    -> poprawne
(a + b))         -> niepoprawne
```

Dodaj też test dla napisu ((a) oraz dla pustego napisu.

57.7.4 Zadanie 20. Lista uczestników

Utwórz `std::list<std::string>` z nazwiskami uczestników. Dodaj i usuń osoby z początku oraz końca listy. Następnie posortuj listę alfabetycznie.

Wymagania:

- użyj metod `push_front`, `push_back`, `pop_front`, `pop_back`,
- przed usuwaniem sprawdź, czy lista nie jest pusta,
- do sortowania użyj metody `sort` listy.

57.7.5 Zadanie 21. Dobór kontenera

Dla każdego przypadku wybierz właściwy kontener: `std::vector`, `std::map`, `std::stack`, `std::queue` albo `std::list`. Zapisz krótkie uzasadnienie w komentarzu.

- historia cofania operacji w edytorze,
- klienci oczekujący na połączenie z konsultantem,
- oceny przypisane do numeru indeksu studenta,
- lista wyników przeznaczona do częstego sortowania i odczytu,
- elementy często dodawane i usuwane w środku kolekcji.

Wymagania:

- dla każdego przypadku wskaż jeden kontener,
- uzasadnienie ma odnosić się do sposobu użycia danych, nie tylko do nazwy kontenera.

57.8 Poziom 5 - Zadania projektowe

57.8.1 Zadanie 22. Ranking graczy

Utwórz klasę `Player` z nazwą i liczbą punktów. Przechowuj graczy w wektorze. Program powinien:

- odrzucać powtarzające się nazwy,
- pozwalać zmienić wynik gracza,
- wyświetlać ranking malejąco według punktów,
- znaleźć gracza o podanej nazwie.

Wymagania:

- do sprawdzania powtarzających się nazw użyj `std::find_if` albo dodatkowej mapy,
- ranking sortuj przez `std::sort`,
- aktualizacja nieznanego gracza powinna zwrócić komunikat o błędzie.

57.8.2 Zadanie 23. System zamówień

Utwórz klasy `Order` i `OrderSystem`. System powinien przechowywać zamówienia, umożliwiać wyszukiwanie po identyfikatorze oraz obsługiwać je w kolejności dodania.

Dobierz kontenery do każdej odpowiedzialności i uzasadnij wybór w komentarzu.

Wymagania:

- wyszukiwanie po identyfikatorze powinno być szybkie i bezpieczne,
- obsługa zamówień ma zachować kolejność dodania,
- system nie powinien pozwolić dodać dwóch zamówień o tym samym identyfikatorze.

57.8.3 Zadanie 24. Rozbudowa HRMS

Rozbuduj projekt z lekcji 06-projekt-hrms.md.

Dodaj:

- usuwanie pracownika ze wszystkich kontenerów,
- przenoszenie pracownika do innego działu,
- średnie wynagrodzenie w dziale,
- listę pracowników posortowaną alfabetycznie,
- informację o błędnej operacji bez przerywania programu.

57.9 Wariant minimum

Do zaliczenia segmentu wykonaj co najmniej:

1. Zadanie 1 albo 2.
2. Zadanie 6 albo 7.
3. Zadanie 8 albo 10.
4. Zadanie 12 albo 13.
5. Zadanie 17 albo 18.
6. Zadanie 19 albo 21.
7. Mini-sprawdzian segmentu z końca pliku.

Zadania 22-24 są projektowe. Możesz je robić po wykonaniu wariantu minimum.

57.10 Mini-sprawdzian segmentu

Napisz program Biblioteka ocen filmów, który:

- przechowuje filmy i oceny w `std::map<std::string, double>`,
- pozwala dodać film, zmienić ocenę i usunąć film,
- wyszukuje film bez tworzenia nowego wpisu,
- wypisuje filmy posortowane alfabetycznie po tytule,
- tworzy ranking filmów malejąco po ocenie z użyciem `std::vector` i `std::sort`,
- obsługuje pustą bibliotekę bez błędów.

Scenariusz sprawdzenia:

```
Dodaj: Matrix 9.0
Dodaj: Alien 8.5
Zmien: Matrix 9.5
Szukaj: Matrix -> 9.5
Szukaj: Brak -> nie znaleziono
Ranking:
Matrix 9.5
Alien 8.5
```

57.11 Kryteria zaliczenia segmentu

Student zalicza segment, jeśli potrafi samodzielnie:

- utworzyć i modyfikować `std::vector`,
- bezpiecznie przechodzić po elementach kontenera,
- użyć `std::sort`, `std::find`, `std::find_if` i `std::count_if`,
- zastosować idiom `erase-remove`,
- napisać prosty predykat jako wyrażenie lambda,
- użyć `std::map` i bezpiecznie wyszukać klucz metodą `find`,
- wyjaśnić różnicę między stosem a kolejką,
- dobrać kontener do prostego problemu,
- połączyć klasę z kilkoma kontenerami STL,

- zachować spójność danych przechowywanych w kilku kontenerach.

57.12 Checklista oddania

Przed oddaniem rozwiązań sprawdź:

- ☐ Każde zadanie jest w osobnym pliku `.cpp`.
- ☐ Programy kompilują się z `-Wall -Wextra -pedantic`.
- ☐ Kontener jest dobrany do sposobu użycia danych.
- ☐ Sortowanie, wyszukiwanie i filtrowanie używa algorytmów STL tam, gdzie ma to sens.
- ☐ Wynik `find` albo `find_if` jest sprawdzony przed dereferencją.
- ☐ `std::map::find` jest używany tam, gdzie wyszukiwanie nie powinno tworzyć wpisu.
- ☐ Operacje na pustym kontenerze są obsługiwane czytelnym komunikatem.

57.13 Archiwum

Oryginalne materiały źródłowe tego segmentu są zachowane w katalogu `archive`.

Rozdział 58

08 - Projekt, build i testy

Ten segment pokazuje, jak przejść od pojedynczego pliku `.cpp` do małego projektu C++ z podziałem na pliki, prostym buildem, sprawdzaniem składni, testami i podstawową automatyzacją.

58.1 Cele segmentu

Po zakończeniu segmentu student powinien umieć:

- rozdzielić program na pliki nagłówkowe i źródłowe,
- wyjaśnić rolę preprocesora i dyrektywy `#include`,
- stosować proste zabezpieczenia przed wielokrotnym dołączeniem nagłówka,
- przygotować minimalną strukturę projektu,
- skompilować projekt złożony z kilku plików,
- sprawdzić składnię bez budowania programu,
- uruchomić proste testy funkcji,
- opisać podstawową ideę CI/CD.

58.2 Kolejność lekcji

1. 01-preprocesor-i-include.md - preprocesor, `#include`, `#define`, guardy.
2. 02-podzial-na-pliki.md - pliki `.h` i `.cpp`, interfejs i implementacja.
3. 03-struktura-projektu.md - katalogi `include`, `src`, `tests`, `build`.
4. 04-kompilacja-i-build.md - kompilacja wielu plików i prosty build.
5. 05-sprawdzanie-skladni.md - `-fsyntax-only`, ostrzeżenia i podstawowa analiza.
6. 06-testy.md - proste testy bez frameworka i organizacja przypadków testowych.
7. 07-ci-cd.md - czym jest automatyzacja testów i budowania.
8. 08-zadania.md - zadania podstawowe i projektowe.
9. 09-cwiczenie-kalkulator-rpn.md - ćwiczenie pomostowe: mały projekt wieloplikowy z tokenizacją i testami.

58.3 Status porządkowania

Lekcje i zadania zostały uporządkowane do wspólnego formatu segmentów. Pierwotne materiały są zachowane w katalogu archive:

- archive/preprocesor.md
- archive/organizacja-projektu.md
- archive/structure.md
- archive/check-cpp-syntax.md
- archive/testy-ci-cd.md
- archive/sqlite-macos.md

58.4 Przykłady kodu i konfiguracji

Przykłady w katalogu examples można uruchamiać osobno:

- `examples/preprocessor_info.cpp` - preprocesor i nazwy predefiniowane.
- `examples/split-project` - podział programu na `.h` i `.cpp`.
- `examples/project-layout` - struktura projektu, build i testy.
- `examples/syntax-check/student_report.cpp` - plik do sprawdzania składni.
- `examples/ci-cd/github-actions-cpp.yml` - przykładowy workflow CI.

Ćwiczenie 09-cwiczenie-kalkulator-rpn.md łączy podział na pliki, build i testy na mniejszym wariancie kalkulatora.

58.5 Testy

Katalog tests zawiera prosty test bez zewnętrznego frameworka. Test sprawdza funkcje z przykładu `examples/split-project` i pokazuje, jak kompilować osobny program testujący razem z plikiem implementacji `.cpp`.

58.6 Format lekcji

Każda docelowa lekcja powinna mieć taki układ:

1. Cel lekcji.
2. Wymagania wstępne.
3. Krótka teoria.
4. Przykład kodu lub komend.
5. Zadania do wykonania.
6. Kryteria zaliczenia.

58.7 Katalogi pomocnicze

- `examples/` - kompilowalne przykłady i małe projekty dla tego segmentu.
- `tests/` - proste testy automatyczne dla kodu dzielonego na pliki.
- `archive/` - pierwotne wersje materiałów zachowane do wglądu.

Rozdział 59

01 - Preprocesor i #include

59.1 Cel lekcji

Celem lekcji jest zrozumienie, co dzieje się z plikiem C++ przed właściwą kompilacją. Student powinien umieć używać `#include`, prostych makr, kompilacji warunkowej oraz zabezpieczeń przed wielokrotnym dołączeniem pliku nagłówkowego.

59.2 Wymagania wstępne

Przed rozpoczęciem lekcji warto znać:

- podstawową strukturę programu C++,
- funkcje,
- pliki nagłówkowe z biblioteki standardowej, np. `<iostream>`,
- kompilację pojedynczego pliku `.cpp`.

59.3 Krótka teoria

59.3.1 Czym jest preprocesor

Preprocesor działa przed kompilatorem. Przetwarza dyrektywy zaczynające się od znaku `#`, a dopiero wynik tego przetwarzania trafia do właściwej kompilacji.

Najczęściej spotykane dyrektywy to:

- `#include` - dołącza treść innego pliku,
- `#define` - definiuje makro,
- `#ifdef`, `#ifndef`, `#if` - włącza lub wyłącza fragmenty kodu,
- `#endif` - kończy blok kompilacji warunkowej.

59.3.2 #include

Dyrektywa `#include` dołącza zawartość pliku do aktualnego pliku źródłowego.

```
#include <iostream>
```

Nagłówki standardowe zapisujemy w nawiasach ostrych:

```
#include <vector>
```

```
#include <string>
```

Własne nagłówki zapisujemy w cudzysłowie:

```
#include "calculator.h"
```

59.4 Przykład kodu: makra i stałe kompilacyjne

Makro zdefiniowane przez `#define` jest podstawiane tekstowo przed kompilacją:

```
#define MAX_USERS 100
```

W nowym kodzie C++ dla zwykłych stałych częściej używa się `const` albo `constexpr`:

```
constexpr int maxUsers = 100;
```

Makra nadal są przydatne do kompilacji warunkowej.

59.5 Przykład kodu: kompilacja warunkowa

Kompilacja warunkowa pozwala włączyć fragment kodu tylko wtedy, gdy dana flaga jest zdefiniowana:

```
#ifdef DEBUG_MODE
std::cout << "Debug information\n";
#endif
```

Flagę można zdefiniować w kodzie:

```
#define DEBUG_MODE
```

albo podczas kompilacji:

```
c++ -std=c++17 -DDEBUG_MODE examples/preprocessor_info.cpp -o preprocessor_info
```

59.5.1 Guardy w plikach nagłówkowych

Ten sam nagłówek może zostać dołączony kilka razy przez różne pliki. Guard chroni przed wielokrotną definicją tych samych elementów.

Przykład:

```
#ifndef CALCULATOR_H
#define CALCULATOR_H

int add(int a, int b);

#endif
```

Alternatywą jest:

```
#pragma once
```

`#pragma once` jest proste i powszechnie obsługiwane, ale klasyczny guard `#ifndef` / `#define` / `#endif` dobrze pokazuje mechanizm preprocesora.

59.5.2 Nazwy predefiniowane

Preprocesor udostępnia specjalne nazwy, które pomagają w diagnostyce:

```
__FILE__
__LINE__
__DATE__
__TIME__
```

Nie należy budować logiki programu na dacie kompilacji, ale takie wartości są przydatne w prostych komunikatach diagnostycznych.

59.6 Przykład referencyjny

Przykład znajduje się w pliku `examples/preprocessor_info.cpp`.

Kompilacja bez trybu debug:

```
c++ -std=c++17 examples/preprocessor_info.cpp -o preprocessor_info
./preprocessor_info
```

Kompilacja z trybem debug:

```
c++ -std=c++17 -DDEBUG_MODE examples/preprocessor_info.cpp -o preprocessor_info
./preprocessor_info
```

59.7 Zadania do wykonania

1. Skompiluj przykład bez flagi `DEBUG_MODE` i z flagą `DEBUG_MODE`. Porównaj wynik działania programu.
2. Dodaj w przykładzie własną flagę `SHOW_BUILD_INFO`, która wyświetli `__DATE__` oraz `__TIME__`.
3. Utwórz plik `math_utils.h` z guardem i deklaracją funkcji `square`.
4. Utwórz plik `math_utils.cpp` z implementacją funkcji `square`.
5. Dołącz `math_utils.h` w `main.cpp` i użyj funkcji `square`.

59.8 Kryteria zaliczenia

Student zalicza lekcję, jeśli potrafi:

- wyjaśnić, że preprocesor działa przed kompilatorem,
- odróżnić nagłówki standardowe od własnych,
- użyć `#include`,
- włączyć fragment kodu za pomocą `#ifdef`,
- zdefiniować flagę kompilacji przez `-D`,
- napisać prosty guard dla pliku nagłówkowego.

Rozdział 60

02 - Podział programu na pliki

60.1 Cel lekcji

Celem lekcji jest rozdzielenie programu C++ na kilka plików. Student powinien rozumieć różnicę między deklaracją a definicją oraz wiedzieć, dlaczego pliki nagłówkowe nie powinny zawierać przypadkowych implementacji.

60.2 Wymagania wstępne

Przed rozpoczęciem lekcji warto znać:

- funkcje,
- podstawy kompilacji programu C++,
- dyrektywę `#include`,
- guardy z lekcji 01-preprocesor-i-include.md.

60.3 Krótka teoria

60.3.1 Problem jednego pliku

Małe programy można pisać w jednym pliku `main.cpp`. Gdy kod rośnie, taki plik staje się trudny do czytania i testowania.

Podział na pliki pozwala:

- oddzielić interfejs od implementacji,
- łatwiej znaleźć odpowiedzialność danego fragmentu kodu,
- ponownie używać funkcji w różnych miejscach programu,
- kompilować większe projekty w bardziej uporządkowany sposób.

60.3.2 Deklaracja i definicja

Deklaracja mówi kompilatorowi, że dana funkcja istnieje:

```
int add(int a, int b);
```

Definicja zawiera właściwe ciało funkcji:

```
int add(int a, int b) {  
    return a + b;  
}
```

Deklaracja najczęściej trafia do pliku `.h`, a definicja do pliku `.cpp`.

60.3.3 Typowy podział

Prosty program może mieć trzy pliki:

```
calculator.h  
calculator.cpp  
main.cpp
```

Rola plików:

- calculator.h - deklaracje funkcji widoczne dla innych plików,
- calculator.cpp - implementacje funkcji,
- main.cpp - punkt startowy programu.

60.4 Przykład kodu: plik nagłówkowy

Plik calculator.h opisuje interfejs:

```
#ifndef CALCULATOR_H  
#define CALCULATOR_H  
  
int add(int a, int b);  
int subtract(int a, int b);  
int multiply(int a, int b);  
  
#endif
```

Nagłówek powinien być możliwie mały i czytelny. Osoba korzystająca z modułu powinna wiedzieć, co może wywołać, bez czytania szczegółów implementacji.

60.5 Przykład kodu: plik implementacji

Plik calculator.cpp zawiera definicje funkcji:

```
#include "calculator.h"  
  
int add(int a, int b) {  
    return a + b;  
}
```

Dołączenie własnego nagłówka w pliku implementacji pomaga sprawdzić, czy deklaracje i definicje są ze sobą zgodne.

60.6 Przykład kodu: plik main.cpp

Plik main.cpp korzysta z interfejsu:

```
#include <iostream>  
  
#include "calculator.h"  
  
int main() {  
    std::cout << add(2, 3) << '\n';  
    return 0;  
}
```

main.cpp nie musi znać szczegółów implementacji funkcji add. Wystarczy mu deklaracja z nagłówka.

60.7 Przykład komendy: kompilacja wielu plików

Kompilator musi dostać wszystkie pliki .cpp, które zawierają potrzebne implementacje:

```
c++ -std=c++17 main.cpp calculator.cpp -o calculator_app
```

Jeśli podamy tylko `main.cpp`, kompilator zobaczy deklarację funkcji, ale linker nie znajdzie jej definicji.

60.8 Przykład referencyjny

Przykład znajduje się w katalogu `examples/split-project`.

Kompilacja:

```
c++ -std=c++17 \  
  examples/split-project/main.cpp \  
  examples/split-project/calculator.cpp \  
  -o split_project \  
  ./split_project
```

60.9 Typowe błędy

60.9.1 Brak pliku `.cpp` w kompilacji

Objawem jest błąd linkera podobny do:

```
undefined reference to `add(int, int)'
```

Rozwiązanie: dopisz do komendy kompilacji plik `.cpp` z implementacją.

60.9.2 Implementacja funkcji w nagłówku

W prostych przykładach może działać, ale w większym projekcie łatwo prowadzi do wielokrotnych definicji. Na tym etapie trzymaj implementacje zwykłych funkcji w plikach `.cpp`.

60.9.3 Brak guarda

Nagłówek bez guarda może zostać dołączony więcej niż raz i spowodować błędy kompilacji.

60.10 Zadania do wykonania

1. Skompiluj i uruchom przykład `examples/split-project`.
2. Dodaj do `calculator.h` deklarację funkcji `divide`.
3. Dodaj implementację `divide` w `calculator.cpp`. Dla dzielenia przez zero zwróć 0 i wypisz komunikat w `main.cpp`.
4. Dodaj drugi moduł `text_utils.h` oraz `text_utils.cpp` z funkcją `repeat(std::string text, int count)`.
5. Skompiluj program z trzema plikami `.cpp`.

60.11 Kryteria zaliczenia

Student zalicza lekcję, jeśli potrafi:

- wskazać różnicę między deklaracją i definicją,
- utworzyć plik `.h` z guardem,
- utworzyć plik `.cpp` z implementacją,
- dołączyć własny nagłówek w `main.cpp`,
- skompilować program złożony z kilku plików źródłowych,
- rozpoznać błąd linkera wynikający z pominięcia pliku `.cpp`.

Rozdział 61

03 - Struktura projektu

61.1 Cel lekcji

Celem lekcji jest uporządkowanie małego projektu C++ w katalogach. Student powinien rozumieć, gdzie trzymać publiczne nagłówki, implementacje, testy oraz pliki wynikowe budowania.

61.2 Wymagania wstępne

Przed rozpoczęciem lekcji warto znać:

- podział programu na pliki `.h` i `.cpp`,
- kompilację programu złożonego z kilku plików,
- podstawową rolę plików nagłówkowych.

61.3 Krótka teoria

61.3.1 Dlaczego struktura katalogów ma znaczenie

Gdy projekt ma kilka plików, płaski katalog szybko staje się nieczytelny. Ustalona struktura pomaga:

- znaleźć interfejs modułu,
- oddzielić kod aplikacji od testów,
- nie mieszać kodu źródłowego z plikami wynikowymi,
- łatwiej przejść później do CMake albo innego narzędzia build.

61.4 Przykład struktury projektu

Na tym etapie wystarczy taki układ:

```
project-name/  
  include/  
    project_name/  
      public_header.h  
  src/  
    main.cpp  
    implementation.cpp  
  tests/  
    test_something.cpp  
  build/
```

Znaczenie katalogów:

- `include/` - nagłówki używane przez inne części projektu,
- `src/` - implementacja programu,

- `tests/` - programy testujące,
- `build/` - pliki wygenerowane podczas kompilacji.

61.4.1 Katalog `include`

W `include` trzymamy nagłówki publiczne. Dobrą praktyką jest dodanie katalogu z nazwą projektu:

```
include/task_counter/counter.h
```

Dzięki temu w kodzie można pisać:

```
#include "task_counter/counter.h"
```

Taki zapis zmniejsza ryzyko konfliktu nazw z innymi projektami.

61.4.2 Katalog `src`

W `src` trzymamy pliki `.cpp` z implementacją:

```
src/counter.cpp
src/main.cpp
```

Plik `main.cpp` jest punktem startowym aplikacji. Pozostałe pliki `.cpp` zawierają funkcje i klasy używane przez aplikację.

61.4.3 Katalog `tests`

W `tests` można umieścić osobne programy testujące. Na tym etapie nie trzeba jeszcze używać frameworka testowego.

Przykład:

```
tests/counter_tests.cpp
```

Taki plik może kompilować się jako osobny program i zwracać 0, jeśli testy przechodzą.

61.4.4 Katalog `build`

`build` służy na pliki wynikowe:

- programy wykonywalne,
- pliki obiektowe,
- pliki wygenerowane przez narzędzia `build`.

Zwykle nie commitujemy zawartości `build/`. W repozytorium można zostawić pusty plik `.gitkeep`, jeśli chcemy pokazać, że katalog ma istnieć.

61.5 Przykład referencyjny

Przykład znajduje się w katalogu `examples/project-layout`.

Struktura:

```
examples/project-layout/
  include/task_counter/counter.h
  src/counter.cpp
  src/main.cpp
  tests/counter_tests.cpp
  build/.gitkeep
```

Kompilacja aplikacji:

```
c++ -std=c++17 \
-I examples/project-layout/include \
examples/project-layout/src/main.cpp \
```

```
examples/project-layout/src/counter.cpp \  
-o examples/project-layout/build/task_counter_app
```

Uruchomienie aplikacji:

```
./examples/project-layout/build/task_counter_app
```

Kompilacja testów:

```
c++ -std=c++17 \  
-I examples/project-layout/include \  
examples/project-layout/tests/counter_tests.cpp \  
examples/project-layout/src/counter.cpp \  
-o examples/project-layout/build/counter_tests
```

Uruchomienie testów:

```
./examples/project-layout/build/counter_tests
```

61.6 Przykład komendy: flaga -I

Flaga -I mówi kompilatorowi, gdzie szukać własnych nagłówków:

```
-I examples/project-layout/include
```

Bez tej flagi kompilator może nie znaleźć pliku:

```
#include "task_counter/counter.h"
```

61.7 Zadania do wykonania

1. Skompiluj i uruchom aplikację z `examples/project-layout`.
2. Skompiluj i uruchom testy.
3. Dodaj do klasy `Counter` metodę `reset`.
4. Dopisz test sprawdzający metodę `reset`.
5. Dodaj drugi moduł `report`, który tworzy tekstowy opis stanu licznika.

61.8 Kryteria zaliczenia

Student zalicza lekcję, jeśli potrafi:

- wyjaśnić rolę katalogów `include`, `src`, `tests` i `build`,
- użyć flagi `-I`,
- skompilować aplikację z kodem w kilku katalogach,
- skompilować osobny program testujący,
- nie umieszczać plików wynikowych w kodzie źródłowym.

Rozdział 62

04 - Kompilacja i prosty build

62.1 Cel lekcji

Celem lekcji jest zrozumienie, jak kompilować projekt złożony z kilku plików oraz czym różni się kompilacja od linkowania. Student powinien umieć zbudować aplikację jedną komendą, przygotować pliki obiektowe i użyć prostego skryptu build.

62.2 Wymagania wstępne

Przed rozpoczęciem lekcji warto znać:

- podział programu na pliki `.h` i `.cpp`,
- strukturę projektu z katalogami `include`, `src`, `tests` i `build`,
- podstawową komendę kompilacji `c++`.

62.3 Krótka teoria

Kompilacja projektu składa się z przetworzenia plików źródłowych i połączenia wyniku w program wykonywalny. W małym projekcie można zrobić to jedną komendą, a w większym rozdzielić pracę na kompilację plików obiektowych i linkowanie.

62.4 Przykład komendy: kompilacja wielu plików jedną komendą

Najprostszy sposób budowania małego projektu to podanie kompilatorowi wszystkich plików `.cpp`:

```
c++ -std=c++17 \  
-I examples/project-layout/include \  
examples/project-layout/src/main.cpp \  
examples/project-layout/src/counter.cpp \  
-o examples/project-layout/build/task_counter_app
```

Znaczenie elementów komendy:

- `-std=c++17` - wybiera standard języka,
- `-I .../include` - wskazuje katalog z własnymi nagłówkami,
- pliki `.cpp` - kod źródłowy do skompilowania,
- `-o ...` - nazwa programu wynikowego.

62.4.1 Ostrzeżenia kompilatora

Podczas nauki warto kompilować z ostrzeżeniami:

```
-Wall -Wextra -pedantic
```

Pełna komenda:

```
c++ -std=c++17 -Wall -Wextra -pedantic \  
-I examples/project-layout/include \  
examples/project-layout/src/main.cpp \  
examples/project-layout/src/counter.cpp \  
-o examples/project-layout/build/task_counter_app
```

Ostrzeżenia nie zawsze zatrzymują kompilację, ale często pokazują błąd, który później będzie trudny do znalezienia.

62.5 Przykład komendy: kompilacja i linkowanie

Budowanie programu można rozbić na dwa kroki.

Pierwszy krok tworzy pliki obiektowe .o:

```
c++ -std=c++17 -Wall -Wextra -pedantic \  
-I examples/project-layout/include \  
-c examples/project-layout/src/counter.cpp \  
-o examples/project-layout/build/counter.o  
  
c++ -std=c++17 -Wall -Wextra -pedantic \  
-I examples/project-layout/include \  
-c examples/project-layout/src/main.cpp \  
-o examples/project-layout/build/main.o
```

Drugi krok linkuje pliki obiektowe w program:

```
c++ examples/project-layout/build/main.o \  
examples/project-layout/build/counter.o \  
-o examples/project-layout/build/task_counter_app
```

Kompilacja sprawdza pojedyncze pliki źródłowe. Linkowanie łączy gotowe fragmenty w program wykonywalny.

62.6 Typowe błędy

62.6.1 Brak katalogu z nagłówkami

Jeśli zabraknie flagi `-I`, kompilator może zgłosić:

```
fatal error: 'task_counter/counter.h' file not found
```

62.6.2 Brak pliku implementacji

Jeśli podczas linkowania pominiemy `counter.cpp` albo `counter.o`, linker może zgłosić błąd podobny do:

```
undefined reference to `Counter::addTask()'
```

62.6.3 Artefakty w repozytorium

Programy wynikowe, pliki .o i tymczasowe katalogi build nie powinny trafiać do commita. Kod źródłowy powinien wystarczyć do odtworzenia tych plików.

62.7 Przykład komendy: prosty skrypt build

Dłuższe komendy warto zapisać w skrypcie:

```
sh examples/project-layout/build.sh
```

Skrypt w przykładzie:

- tworzy katalog `build`,
- kompiluje aplikację,
- kompiluje testy,
- uruchamia testy.

Domyślnie zapisuje wyniki w `examples/project-layout/build`. Dla czystej weryfikacji można wskazać inny katalog:

```
BUILD_DIR=/tmp/task-counter-build sh examples/project-layout/build.sh
```

62.8 Przykład referencyjny

Przykład znajduje się w katalogu `examples/project-layout`.

Uruchomienie pełnego buildu:

```
BUILD_DIR=/tmp/task-counter-build sh examples/project-layout/build.sh
```

Uruchomienie aplikacji po buildzie:

```
/tmp/task-counter-build/task_counter_app
```

62.9 Zadania do wykonania

1. Zbuduj aplikację jedną komendą `c++`.
2. Zbuduj aplikację w dwóch krokach: pliki `.o`, a potem linkowanie.
3. Uruchom `build.sh` z domyślnym katalogiem `build`.
4. Uruchom `build.sh` z katalogiem ustawionym przez `BUILD_DIR`.
5. Celowo usuń z komendy plik `counter.cpp` i przeczytaj komunikat linkera.

62.10 Kryteria zaliczenia

Student zalicza lekcję, jeśli potrafi:

- skompilować projekt z wieloma plikami `.cpp`,
- użyć flag `-std`, `-Wall`, `-Wextra`, `-pedantic` i `-I`,
- odróżnić kompilację od linkowania,
- utworzyć plik obiektowy `.o`,
- zlinkować pliki obiektowe w program,
- uruchomić prosty skrypt `build`,
- nie commitować plików wynikowych.

Rozdział 63

05 - Sprawdzanie składni

63.1 Cel lekcji

Celem lekcji jest nauczenie się szybkiego sprawdzania kodu C++ bez tworzenia programu wynikowego. Student powinien umieć użyć kompilatora do kontroli składni, włączyć ostrzeżenia i odróżnić błąd kompilacji od błędu linkowania.

63.2 Wymagania wstępne

Przed rozpoczęciem lekcji warto znać:

- kompilację pojedynczego pliku `.cpp`,
- kompilację projektu z kilkoma plikami,
- podstawowe flagi `-std`, `-Wall`, `-Wextra` i `-I`.

63.3 Krótka teoria

63.3.1 Po co sprawdzać składnię osobno

Pełna kompilacja tworzy program wykonywalny. Czasem chcemy szybciej sprawdzić, czy plik jest poprawny składniowo, ale jeszcze nie potrzebujemy programu.

Do tego służy flaga:

`-fsyntax-only`

Kompilator analizuje plik i zgłasza błędy, ale nie tworzy pliku `.o` ani programu wynikowego.

63.4 Przykład komendy: sprawdzanie składni

Dla pojedynczego pliku:

```
c++ -std=c++17 -fsyntax-only examples/syntax-check/student_report.cpp
```

Warto od razu dodać ostrzeżenia:

```
c++ -std=c++17 -Wall -Wextra -pedantic -fsyntax-only \
    examples/syntax-check/student_report.cpp
```

Jeśli komenda nie wypisuje błędów, składnia pliku jest poprawna.

63.4.1 Ostrzeżenia

Ostrzeżenie nie zawsze zatrzymuje kompilację, ale często wskazuje problem:

- nieużywana zmienna,

- brak wartości zwracanej z funkcji,
- porównanie różnych typów,
- podejrzana konwersja.

Podstawowy zestaw flag:

```
-Wall -Wextra -pedantic
```

Na późniejszym etapie można potraktować ostrzeżenia jak błędy:

```
-Werror
```

Przykład:

```
c++ -std=c++17 -Wall -Wextra -pedantic -Werror -fsyntax-only \
    examples/syntax-check/student_report.cpp
```

63.4.2 Błąd kompilacji a błąd linkowania

Sprawdzanie składni wykrywa błędy w pojedynczym pliku:

- brak średnika,
- literówkę w nazwie typu,
- niepoprawną składnię instrukcji,
- brakujący nagłówki.

Nie wykrywa wszystkich problemów linkowania. Jeśli funkcja jest tylko zadeklarowana, ale nie ma implementacji, `-fsyntax-only` może przejść poprawnie, a pełne linkowanie zakończy się błędem.

63.5 Przykład komendy: sprawdzanie projektu z katalogami

Dla projektu z własnymi nagłówkami nadal trzeba podać ścieżkę `-I`:

```
c++ -std=c++17 -Wall -Wextra -pedantic -fsyntax-only \
    -I examples/project-layout/include \
    examples/project-layout/src/main.cpp
```

Sprawdzenie składni nie zastępuje pełnego buildu. Jest dodatkowym, szybkim krokiem kontroli.

63.6 Przykład referencyjny

Przykład znajduje się w pliku `examples/syntax-check/student_report.cpp`.

Sprawdzenie składni:

```
c++ -std=c++17 -Wall -Wextra -pedantic -fsyntax-only \
    examples/syntax-check/student_report.cpp
```

Pełna kompilacja:

```
c++ -std=c++17 -Wall -Wextra -pedantic \
    examples/syntax-check/student_report.cpp \
    -o student_report
```

Uruchomienie:

```
./student_report
```

63.7 Zadania do wykonania

1. Sprawdź składnię przykładu za pomocą `-fsyntax-only`.
2. Skompiluj przykład pełną komendą i uruchom program.
3. Usuń średnik po jednej instrukcji i sprawdź komunikat kompilatora.
4. Dodaj nieużywaną zmienną i sprawdź, czy pojawi się ostrzeżenie.
5. Dodaj flagę `-Werror` i zaobserwuj, jak ostrzeżenie staje się błędem.

63.8 Kryteria zaliczenia

Student zalicza lekcję, jeśli potrafi:

- użyć `-fsyntax-only`,
- włączyć ostrzeżenia kompilatora,
- wyjaśnić sens flagi `-Werror`,
- odróżnić błąd składni od błędu linkowania,
- sprawdzić składnię pliku korzystającego z własnych nagłówków,
- wykonać pełną kompilację po sprawdzeniu składni.

Rozdział 64

06 - Proste testy

64.1 Cel lekcji

Celem lekcji jest pokazanie, jak sprawdzać działanie funkcji i klas bez frameworka testowego. Student powinien umieć napisać osobny program testujący, uruchomić go po kompilacji i interpretować kod zakończenia programu.

64.2 Wymagania wstępne

Przed rozpoczęciem lekcji warto znać:

- podział programu na pliki,
- strukturę katalogów `src`, `include` i `tests`,
- kompilację programu złożonego z kilku plików,
- podstawowe instrukcje warunkowe.

64.3 Krótka teoria

64.3.1 Po co pisać testy

Test to program, który sprawdza, czy kod zachowuje się zgodnie z oczekiwaniem. Najprostszy test:

1. przygotowuje dane,
2. wywołuje funkcję lub metodę,
3. porównuje wynik z oczekiwaniem,
4. zwraca 0, jeśli wszystko działa,
5. zwraca wartość różną od 0, jeśli test wykrył błąd.

Dzięki temu test można uruchamiać ręcznie albo w skrypcie build.

64.4 Przykład kodu: test jako osobny program

W przykładzie testy są w pliku:

```
examples/project-layout/tests/counter_tests.cpp
```

Program testujący korzysta z tej samej klasy `Counter`, co aplikacja:

```
#include "task_counter/counter.h"
```

Nie uruchamia `main.cpp` aplikacji. Ma własną funkcję `main`, która sprawdza wybrane przypadki.

64.4.1 Funkcja pomocnicza

Żeby testy były czytelniejsze, można napisać małą funkcję pomocniczą:

```
bool expectEqual(const std::string& name, int actual, int expected) {
    if (actual != expected) {
        std::cout << name << " failed: expected "
                    << expected << ", got " << actual << '\n';
        return false;
    }

    return true;
}
```

Taka funkcja zmniejsza powtarzanie kodu i wypisuje konkretny powód błędu.

64.4.2 Przypadki testowe

Dobry test sprawdza jeden scenariusz. Przykłady dla klasy Counter:

- dodanie zadania zwiększa liczbę otwartych zadań,
- zakończenie zadania zmniejsza liczbę otwartych zadań,
- zakończenie zadania zwiększa liczbę wykonanych zadań,
- próba zakończenia zadania przy pustej liście niczego nie psuje.

Nazwy funkcji testowych powinny opisywać zachowanie:

```
bool testCompletingTask();
bool testCompletingWithoutOpenTasks();
```

64.4.3 Kod zakończenia programu

Program powinien zwrócić:

- 0, jeśli testy przeszły,
- 1 albo inną wartość różną od zera, jeśli testy nie przeszły.

Skrypty build i narzędzia CI wykorzystują ten kod, aby przerwać proces po wykryciu błędu.

64.5 Przykład komendy: kompilacja testów

Testy kompilujemy jako osobny program:

```
c++ -std=c++17 -Wall -Wextra -pedantic \
-I examples/project-layout/include \
examples/project-layout/tests/counter_tests.cpp \
examples/project-layout/src/counter.cpp \
-o examples/project-layout/build/counter_tests
```

Uruchomienie:

```
./examples/project-layout/build/counter_tests
```

64.5.1 Testy w skrypcie build

Skrypt `examples/project-layout/build.sh` kompiluje aplikację, kompiluje testy i uruchamia testy. Jeśli testy zwrócą błąd, skrypt zakończy działanie z błędem.

Uruchomienie:

```
BUILD_DIR=/tmp/task-counter-build sh examples/project-layout/build.sh
```

64.6 Zadania do wykonania

1. Uruchom testy z `examples/project-layout`.
2. Dodaj test sprawdzający, że po utworzeniu `Counter` ma 0 otwartych zadań.
3. Dodaj metodę `reset` do klasy `Counter`.
4. Dopisz test metody `reset`.
5. Celowo zmień implementację `completeTask` tak, aby test się nie powiódł. Przeczytaj komunikat błędu i przywróć poprawną implementację.

64.7 Kryteria zaliczenia

Student zalicza lekcję, jeśli potrafi:

- napisać prosty test jako osobny program,
- przygotować dane wejściowe i sprawdzić wynik,
- wypisać czytelny komunikat błędu,
- zwrócić 0 dla sukcesu i 1 dla porażki,
- skompilować test razem z testowanym plikiem `.cpp`,
- uruchomić testy ze skryptu `build`.

Rozdział 65

07 - CI/CD

65.1 Cel lekcji

Celem lekcji jest zrozumienie, po co automatyzować kompilację i testy. Student powinien umieć opisać podstawową ideę CI/CD oraz przygotować prosty workflow, który po zmianie kodu uruchamia build i testy.

65.2 Wymagania wstępne

Przed rozpoczęciem lekcji warto znać:

- strukturę projektu C++,
- kompilację wielu plików,
- prosty skrypt build,
- testy uruchamiane jako osobny program.

65.3 Krótka teoria

65.3.1 Co oznacza CI

CI, czyli Continuous Integration, oznacza regularne sprawdzanie zmian w kodzie. W praktyce dla małego projektu C++ oznacza to najczęściej:

1. pobranie kodu z repozytorium,
2. przygotowanie środowiska,
3. kompilację programu,
4. uruchomienie testów,
5. zgłoszenie błędu, jeśli któryś krok się nie powiedzie.

CI nie zastępuje myślenia programisty, ale szybko wykrywa część problemów.

65.3.2 Co oznacza CD

CD może oznaczać Continuous Delivery albo Continuous Deployment. W tym kursie wystarczy rozumieć ogólną ideę: po przejściu testów projekt może być automatycznie przygotowany do wydania.

Dla repozytorium edukacyjnego zwykle wystarczy CI. Automatyczne wdrażanie nie jest potrzebne na tym etapie.

65.3.3 Dlaczego testy muszą zwracać kod błędu

Skrypt CI sprawdza kod zakończenia komendy:

- 0 oznacza sukces,
- wartość różna od 0 oznacza błąd.

Dlatego testy z lekcji 06-testy.md zwracają 1, jeśli któryś przypadek testowy nie przechodzi. Dzięki temu CI może zatrzymać proces.

65.4 Przykład komendy: lokalny odpowiednik CI

Zanim uruchomimy automatyzację na serwerze, warto mieć lokalną komendę, która robi to samo:

```
BUILD_DIR=/tmp/task-counter-build sh examples/project-layout/build.sh
```

Taki skrypt:

- kompiluje aplikację,
- kompiluje testy,
- uruchamia testy,
- kończy się błędem, jeśli testy się nie powiedą.

To jest najważniejsza część CI. Platforma, np. GitHub Actions, tylko uruchamia tę komendę automatycznie.

65.5 Przykład konfiguracji: GitHub Actions

Przykładowy workflow znajduje się w pliku `examples/ci-cd/github-actions-cpp.yml`.

Aby go użyć w prawdziwym repozytorium, należy skopiować go do:

```
.github/workflows/cpp.yml
```

Workflow uruchamia się dla `push` i `pull_request`, a następnie wykonuje skrypt `build` z przykładu.

65.5.1 Minimalny workflow

Najważniejsza część workflow to kroki:

```
- name: Checkout
  uses: actions/checkout@v4

- name: Build and test
  run: BUILD_DIR=/tmp/task-counter-build sh 08-projekt-build-testy/examples/project-layout/build.sh
```

Pierwszy krok pobiera kod. Drugi uruchamia `build` i testy.

65.5.2 Co powinno trafić do CI

Na tym etapie warto automatyzować:

- kompilację przykładów,
- uruchomienie testów,
- sprawdzanie składni najważniejszych plików,
- uruchomienie jednego skryptu, który zwraca poprawny kod zakończenia.

Nie warto zaczynać od zbyt złożonej konfiguracji. Najpierw trzeba mieć lokalny skrypt `build`, który działa przewidywalnie.

65.6 Typowe błędy

65.6.1 Workflow robi coś innego niż lokalny build

Jeśli CI używa innych komend niż programista lokalnie, trudniej odtworzyć błąd. Najlepiej, gdy CI uruchamia ten sam skrypt.

65.6.2 Testy wypisują błąd, ale zwracają 0

CI uzna taki wynik za sukces. Test powinien zwrócić wartość różną od zera, jeśli wykrył problem.

65.6.3 Build zapisuje artefakty w repozytorium

Pliki wynikowe powinny trafiać do katalogu tymczasowego albo do ignorowanego `build/`.

65.7 Zadania do wykonania

1. Uruchom lokalnie skrypt `examples/project-layout/build.sh`.
2. Sprawdź, jaki kod zakończenia zwraca skrypt po poprawnym buildzie.
3. Celowo zepsuj jeden test i sprawdź, czy skrypt kończy się błędem.
4. Przeczytaj plik `examples/ci-cd/github-actions-cpp.yml`.
5. Wyjaśnij, dlaczego workflow uruchamia ten sam skrypt, którego używamy lokalnie.

65.8 Kryteria zaliczenia

Student zalicza lekcję, jeśli potrafi:

- wyjaśnić podstawową ideę CI,
- odróżnić CI od CD,
- przygotować lokalny skrypt uruchamiany przez CI,
- wyjaśnić znaczenie kodu zakończenia programu,
- wskazać, gdzie w repozytorium umieszcza się workflow GitHub Actions,
- opisać, dlaczego CI powinno uruchamiać te same kroki co lokalny build.

Rozdział 66

08 - Zadania

66.1 Cel zestawu zadań

Ten plik zbiera zadania do segmentu 08 - Projekt, build i testy. Zadania sprawdzają preprocesor, podział programu na pliki, strukturę katalogów, kompilację, sprawdzanie składni, testy oraz podstawy CI/CD.

66.2 Zasady wykonania

Każde zadanie powinno być zapisane w osobnym katalogu albo osobnym pliku `.cpp`, jeśli zadanie jest krótkie. Program powinien kompilować się bez błędów.

Przykład kompilacji:

```
c++ -std=c++17 -Wall -Wextra -pedantic main.cpp -o program
./program
```

Dla projektów wieloplikowych używaj katalogów `include`, `src`, `tests` i `build`. Plików wynikowych nie dodawaj do commita.

66.3 Wymagania jakościowe

W zadaniach z tego segmentu zwracaj uwagę na:

- kompilację z flagami `-std=c++17 -Wall -Wextra -pedantic`,
- oddzielenie deklaracji w `.h` od implementacji w `.cpp`,
- uruchamianie testów jako osobnego programu,
- build zapisujący artefakty do katalogu `build` albo `/tmp`,
- brak plików wynikowych, `.o` i tymczasowych katalogów w commicie.
- dokumentowanie komend `build/test` w `README.md` projektu.

66.4 Poziom 1 - Preprocesor i nagłówki

66.4.1 Zadanie 1. Własny nagłówek

Utwórz pliki:

- `greeting.h`,
- `greeting.cpp`,
- `main.cpp`.

W nagłówku zadeklaruj funkcję `printGreeting`, a w pliku `.cpp` dodaj jej implementację.

Wymagania:

- `main.cpp` nie powinien znać szczegółów implementacji funkcji,

- kompilacja ma obejmować `main.cpp` i `greeting.cpp`,
- nagłówek nie powinien zawierać definicji funkcji, tylko deklarację.

66.4.2 Zadanie 2. Guard

Dodaj do `greeting.h` klasyczny guard:

```
#ifndef GREETING_H
#define GREETING_H

#endif
```

Dołącz nagłówek dwa razy w `main.cpp` i sprawdź, czy program nadal się kompiluje.

W komentarzu zapisz, jaki problem rozwiązuje guard.

66.4.3 Zadanie 3. Flaga debug

Dodaj fragment kodu, który wypisuje komunikat tylko wtedy, gdy program jest kompilowany z flagą `DEBUG_MODE`.

Kompilacja:

```
c++ -std=c++17 -Wall -Wextra -pedantic -DDEBUG_MODE main.cpp greeting.cpp -o app
```

Sprawdź program z flagą `DEBUG_MODE` i bez niej.

66.4.4 Zadanie 4. Informacja diagnostyczna

Wypisz `__FILE__` oraz `__LINE__` w wybranej funkcji. Sprawdź, jak zmienia się wynik po przeniesieniu instrukcji do innej linii.

Wynik powinien pomóc zrozumieć, z którego pliku i linii pochodzi komunikat.

66.5 Poziom 2 - Podział na pliki

66.5.1 Zadanie 5. Kalkulator

Utwórz moduł `calculator` z funkcjami:

- `add`,
- `subtract`,
- `multiply`,
- `divide`.

Podziel kod na pliki `calculator.h`, `calculator.cpp` i `main.cpp`.

Wymagania:

- `divide` ma obsługiwać dzielenie przez zero,
- nagłówek ma zawierać deklaracje wszystkich funkcji,
- `main.cpp` ma tylko demonstrować użycie modułu.

66.5.2 Zadanie 6. Moduł tekstowy

Utwórz moduł `text_utils` z funkcjami:

- `toUpper`,
- `repeat`,
- `startsWith`.

Przygotuj osobny `main.cpp`, który demonstruje działanie funkcji.

Scenariusz sprawdzenia:

```
toUpper("ala") -> "ALA"  
repeat("ha", 3) -> "hahaha"  
startsWith("program", "pro") -> true
```

66.5.3 Zadanie 7. Błąd linkera

Celowo pominij jeden plik `.cpp` w komendzie kompilacji. Zapisz w komentarzu, jaki komunikat pokazał linker i jak go naprawić.

Po zapisaniu obserwacji przywróć poprawną komendę kompilacji.

66.5.4 Zadanie 8. Interfejs i implementacja

Dla modułu `calculator` dopisz komentarz w nagłówku opisujący, co robi każda funkcja. Implementację zostaw w pliku `.cpp`.

Komentarz w nagłówku powinien opisywać zachowanie funkcji, a nie szczegóły jej implementacji.

66.6 Poziom 3 - Struktura projektu i build

66.6.1 Zadanie 9. Projekt notes

Utwórz projekt w strukturze:

```
notes/  
  include/notes/  
  src/  
  tests/  
  build/
```

Projekt powinien przechowywać liczbę notatek i umożliwiać dodanie nowej notatki.

Minimalne pliki:

```
notes/include/notes/notes.h  
notes/src/notes.cpp  
notes/src/main.cpp  
notes/tests/notes_tests.cpp  
notes/build/.gitignore
```

66.6.2 Zadanie 10. Kompilacja z `-I`

Skompiluj projekt `notes`, używając flagi `-I`. Program powinien korzystać z nagłówka w katalogu `include/notes`.

Przykład komendy:

```
c++ -std=c++17 -Wall -Wextra -pedantic \  
  -I notes/include \  
  notes/src/main.cpp notes/src/notes.cpp \  
  -o notes/build/notes
```

66.6.3 Zadanie 11. Pliki obiektowe

Zbuduj projekt `notes` w dwóch krokach:

1. utwórz pliki `.o`,
2. zlinkuj je w program.

Pliki `.o` zapisz w katalogu `build/`.

Wymagania:

- osobno skompiluj `notes.cpp`,
- osobno skompiluj `main.cpp`,

- linkowanie wykonaj jako ostatni krok.

66.6.4 Zadanie 12. Skrypt build

Dodaj skrypt `build.sh`, który:

- tworzy katalog `build`,
- kompiluje aplikację,
- uruchamia aplikację testowo.

Skrypt powinien zaczynać się od:

```
#!/bin/sh
set -eu
```

66.6.5 Zadanie 13. Ignorowanie artefaktów

Dodaj `.gitignore` w katalogu `build`, aby nie commitować programów wynikowych i plików `.o`.

Przykładowa zawartość:

```
*
!.gitignore
```

66.7 Poziom 4 - Sprawdzanie składni i testy

66.7.1 Zadanie 14. `-fsyntax-only`

Sprawdź składnię wybranego pliku `.cpp` bez tworzenia programu wynikowego.

Użyj:

```
c++ -std=c++17 -Wall -Wextra -pedantic -fsyntax-only plik.cpp
```

W komentarzu zapisz różnicę między sprawdzeniem składni a pełnym buildem.

66.7.2 Zadanie 15. Ostrzeżenie jako błąd

Dodaj do programu nieużywaną zmienną. Skompiluj program z `-Werror` i zapisz w komentarzu, co się stało.

Po wykonaniu obserwacji usuń nieużywaną zmienną, aby projekt znowu się kompilował.

66.7.3 Zadanie 16. Pierwszy test

Do projektu `notes` dodaj `tests/notes_tests.cpp`. Test powinien sprawdzić, czy nowy obiekt ma 0 notatek.

Test powinien zwrócić 0, jeśli przechodzi, i 1, jeśli wykryje błąd.

66.7.4 Zadanie 17. Test dodawania

Dodaj test sprawdzający, czy dodanie jednej notatki zwiększa licznik do 1.

Dodaj też test dodania dwóch notatek.

66.7.5 Zadanie 18. Funkcja `expectEqual`

Dodaj funkcję pomocniczą `expectEqual`, która wypisuje oczekiwaną i otrzymaną wartość, jeśli test nie przechodzi.

Funkcja powinna zwracać `bool`, aby można było składać kilka warunków testowych.

66.7.6 Zadanie 19. Testy w buildzie

Rozbuduj `build.sh`, aby kompilował i uruchamiał testy. Skrypt powinien kończyć się błędem, jeśli test nie przechodzi.

Wymagania:

- aplikacja i testy powinny być osobnymi programami wynikowymi,
- testy powinny uruchamiać się po kompilacji,
- jeśli test zwróci 1, skrypt ma zakończyć się błędem.

66.8 Poziom 5 - CI/CD i mini-projekt

66.8.1 Zadanie 20. Lokalny odpowiednik CI

Przygotuj jedną komendę, która uruchamia cały build i testy projektu `notes`. Zapisz ją w komentarzu w pliku `README.md` projektu.

Przykład:

```
sh build.sh
```

66.8.2 Zadanie 21. Przykładowy workflow

Przygotuj plik `github-actions-cpp.yml`, który:

- pobiera repozytorium,
- pokazuje wersję kompilatora,
- uruchamia `build.sh`.

Nie musi być aktywowany w `.github/workflows`.

Workflow powinien używać `ubuntu-latest` i kroku `actions/checkout`.

66.8.3 Zadanie 22. Analiza awarii

Celowo zepsuj jeden test i uruchom build. Zapisz w komentarzu:

- która komenda się nie powiodła,
- jaki był komunikat,
- jak naprawić problem.

Po analizie przywróć poprawny test.

66.8.4 Zadanie 23. Mini-projekt gradebook

Przygotuj projekt `gradebook` w strukturze:

```
gradebook/  
  include/gradebook/  
  src/  
  tests/  
  build/
```

Projekt powinien umożliwiać:

- dodanie oceny,
- policzenie średniej,
- sprawdzenie, czy wszystkie oceny są poprawne,
- uruchomienie testów przez `build.sh`.

Wymagania:

- logika ocen powinna być w osobnym module,
- `main.cpp` powinien tylko demonstrować użycie modułu,
- testy powinny sprawdzać pusty dziennik, poprawne oceny i błędne oceny.

66.8.5 Zadanie 24. Dokumentacja builda

Dodaj do mini-projektu `README.md`, który zawiera:

- opis struktury katalogów,
- komendę kompilacji aplikacji,
- komendę kompilacji testów,
- komendę uruchamiającą cały build,
- informację, czego nie commitować.

Jeśli chcesz przećwiczyć większy projekt wieloplikowy przed projektem zaliczeniowym, wykonaj 09-cwiczenie-kalkulator-rpn.md.

66.9 Wariant minimum

Do zaliczenia segmentu wykonaj co najmniej:

1. Zadanie 1 albo 2.
2. Zadanie 5 albo 6.
3. Zadanie 9.
4. Zadanie 10 albo 11.
5. Zadanie 16 albo 17.
6. Zadanie 19 albo 20.
7. Mini-sprawdzian segmentu z końca pliku.

Zadania 21-24 są rozszerzone albo projektowe. Możesz je robić po wykonaniu wariantu minimum.

66.10 Mini-sprawdzian segmentu

Przygotuj mały projekt `counter-app` w strukturze:

```
counter-app/  
  include/counter/counter.h  
  src/counter.cpp  
  src/main.cpp  
  tests/counter_tests.cpp  
  build/.gitignore  
  build.sh  
  README.md
```

Projekt powinien:

- mieć klasę albo moduł licznika z operacjami `increment`, `reset`, `value`,
- kompilować aplikację i testy przez `build.sh`,
- uruchamiać testy automatycznie w `build.sh`,
- zapisywać artefakty w `build`,
- mieć `README.md` z komendą uruchomienia pełnego builda.

Scenariusz sprawdzenia:

```
sh build.sh  
All tests passed
```

66.11 Kryteria zaliczenia segmentu

Student zalicza segment, jeśli potrafi samodzielnie:

- utworzyć nagłówek z `guardem`,
- rozdzielić program na deklaracje i implementacje,
- skompilować projekt z wieloma plikami `.cpp`,
- użyć katalogów `include`, `src`, `tests` i `build`,
- użyć flag `-I`, `-Wall`, `-Wextra`, `-pedantic`, `-fsyntax-only`,

- zbudować program przez pliki obiektowe,
- napisać prosty skrypt build,
- napisać test jako osobny program,
- sprawić, żeby test zwracał poprawny kod zakończenia,
- przygotować przykładowy workflow CI uruchamiający lokalny build.

66.12 Checklista oddania

Przed oddaniem rozwiązań sprawdź:

- ☐ Projekt kompiluje się z `-Wall -Wextra -pedantic`.
- ☐ Deklaracje są w `.h`, a implementacje w `.cpp`.
- ☐ Komenda build zawiera wszystkie potrzebne pliki `.cpp`.
- ☐ Nagłówki mają guard albo `#pragma once`.
- ☐ Testy są osobnym programem i zwracają kod błędu przy niepowodzeniu.
- ☐ `build.sh` kończy się błędem, jeśli kompilacja albo test się nie powiedzie.
- ☐ Artefakty build, `.o` i programy wynikowe nie są commitowane.
- ☐ `README.md` projektu opisuje kompilację, testy i strukturę katalogów.

66.13 Archiwum

Oryginalne materiały źródłowe tego segmentu są zachowane w katalogu archive.

Rozdział 67

09 - Ćwiczenie: kalkulator RPN

67.1 Cel ćwiczenia

Celem ćwiczenia jest przygotowanie małego projektu wieloplikowego, który oblicza wartość prostego wyrażenia matematycznego przez zamianę na odwrotną notację polską.

Ćwiczenie jest pomostem między segmentem build/testy a większym projektem Projekt 08 - Kalkulator. Tutaj zakres jest mniejszy: najważniejsze są podział na moduły, testy i czytelny build.

67.2 Wymagania wstępne

Przed wykonaniem ćwiczenia student powinien znać:

- podział programu na pliki `.h` i `.cpp`,
- strukturę katalogów `include`, `src`, `tests`, `build`,
- kompilację wielu plików jedną komendą,
- proste testy bez frameworka,
- podstawy pracy z `std::vector` i `std::string`.

67.3 Opis zadania

Napisz program konsolowy, który:

- wczytuje jedno wyrażenie tekstowe,
- obsługuje liczby rzeczywiste,
- obsługuje operatory `+`, `-`, `*`, `/`,
- respektuje priorytet mnożenia i dzielenia,
- obsługuje nawiasy okrągłe,
- zamienia wyrażenie na odwrotną notację polską,
- oblicza wynik ze stosu,
- zgłasza czytelny błąd dla niepoprawnego wyrażenia.

Przykłady:

```
2 + 2 * 2 = 6
(2 + 2) * 2 = 8
10 / 2 + 3 = 8
```

67.4 Proponowana struktura plików

```
rpn-calculator/
include/rpn/
token.h
```

```

tokenizer.h
rpn.h
evaluator.h
src/
  tokenizer.cpp
  rpn.cpp
  evaluator.cpp
  main.cpp
tests/
  rpn_tests.cpp
build/
  .gitignore
build.sh
README.md

```

67.5 Wymagania techniczne

Projekt powinien:

- być napisany w C++17,
- mieć funkcję tokenizującą wyrażenie,
- mieć funkcję zamieniającą tokeny na odwrotną notację polską,
- mieć funkcję obliczającą wynik z odwrotnej notacji polskiej,
- używać `std::vector` do listy tokenów,
- używać `std::vector` albo `std::stack` jako stosu,
- obsługiwać dzielenie przez zero,
- mieć testy dla tokenizacji albo obliczeń,
- mieć skrypt `build.sh`, który buduje aplikację i testy,
- kompilować się z flagami `-Wall -Wextra -pedantic`.

Przykładowe funkcje:

```

std::vector<Token> tokenize(const std::string& expression);
std::vector<Token> toReversePolishNotation(const std::vector<Token>& tokens);
double evaluateReversePolishNotation(const std::vector<Token>& tokens);

```

67.6 Wariant podstawowy

Minimalna wersja ćwiczenia powinna zawierać:

1. Wczytanie jednego wyrażenia od użytkownika.
2. Obsługę operatorów `+`, `-`, `*`, `/`.
3. Obsługę nawiasów.
4. Poprawną kolejność wykonywania działań.
5. Zamianę wyrażenia na odwrotną notację polską.
6. Obliczenie wyniku ze stosu.
7. Komunikat dla dzielenia przez zero.
8. Komunikat dla błędnych nawiasów.
9. Testy dla co najmniej trzech poprawnych wyrażeń.
10. Instrukcję kompilacji i uruchomienia.

67.7 Proponowany podział pracy

1. Utwórz strukturę katalogów projektu.
2. Dodaj `build.sh`, który kompiluje pusty program.
3. Zdefiniuj typ `Token`.
4. Napisz tokenizację liczb i operatorów.
5. Dodaj zamianę na odwrotną notację polską.
6. Dodaj obliczanie wyniku ze stosu.

7. Dodaj testy dla poprawnych wyrażeń.
8. Dodaj obsługę błędów i testy przypadków błędnych.

67.8 Zadania dodatkowe

Po wykonaniu wariantu podstawowego możesz dodać:

- liczby ujemne,
- operator potęgowania,
- stałe `pi` i `e`,
- tryb wielu obliczeń w pętli,
- historię wyników,
- testy automatyczne tokenizacji i obliczeń,
- porównanie rozwiązania z parserem rekurencyjnym.

Funkcje matematyczne, historia i zapis do pliku są częścią większego projektu Projekt 08 - Kalkulator, więc tutaj nie są wymagane w wariantcie podstawowym.

67.9 Pytania kontrolne

1. Który moduł odpowiada za tokenizację, a który za obliczenie wyniku?
2. Dlaczego tokenizacja powinna być oddzielona od obliczania?
3. Jak test zwraca informację o błędzie do skryptu `build.sh`?
4. Jakie pliki powinny trafić do commita, a jakie powinny zostać w `build/`?

67.10 Scenariusze sprawdzenia

1. Oblicz $2 + 2 * 2$ i sprawdź, czy wynik to 6.
2. Oblicz $(2 + 2) * 2$ i sprawdź, czy wynik to 8.
3. Oblicz $10 / 2 + 3$ i sprawdź, czy wynik to 8.
4. Spróbuj obliczyć $10 / 0$ i sprawdź komunikat błędu.
5. Spróbuj obliczyć $(2 + 3$ i sprawdź komunikat o nawiasach.
6. Spróbuj obliczyć $2 + * 3$ i sprawdź komunikat o składni.

67.11 Kryteria zaliczenia

Ćwiczenie jest zaliczone, jeśli:

- projekt kompiluje się bez błędów i ostrzeżeń,
- kod jest podzielony na pliki `.h` i `.cpp`,
- tokenizacja jest oddzielona od obliczania,
- program poprawnie liczy priorytety operatorów,
- program poprawnie obsługuje nawiasy,
- program wykrywa dzielenie przez zero,
- testy uruchamiają się z `build.sh`,
- `README.md` opisuje kompilację, testy i strukturę katalogów.

67.12 Źródło

Ćwiczenie powstało na podstawie starszego zadania z zadań pomocniczych TNT.

Rozdział 68

09 - Modern C++

Ten segment zbiera elementy współczesnego C++, które rozszerzają kurs podstawowy. Materiał pokazuje najważniejsze konstrukcje C++11 i nowszych standardów oraz tematy, które warto potraktować jako wprowadzenie do dalszej nauki.

68.1 Cele segmentu

Po zakończeniu segmentu student powinien umieć:

- rozpoznać wybrane elementy modern C++,
- używać `auto`, `nullptr`, inicjalizacji klamrowej i pętli zakresowej,
- stosować `enum class`, aliasy typów i `override`,
- używać prostych wyrażeń `lambda`,
- rozumieć podstawy semantyki przenoszenia,
- organizować nazwy za pomocą przestrzeni nazw,
- wyjaśnić, czym różni się moduł jako temat C++20 od klasycznego podziału na pliki,
- uruchomić prosty przykład z `std::thread`.

68.2 Kolejność lekcji

1. 01-nowosci-modern-cpp.md - najważniejsze elementy C++11 i nowszych standardów.
2. 02-auto-nullptr-inicjalizacja.md - `auto`, `nullptr`, inicjalizacja klamrowa.
3. 03-lambda-i-algorytmy.md - lambdy w praktycznym użyciu ze STL.
4. 04-enum-class-override-aliasy.md - typy silniejsze semantycznie i czytelniejsze API.
5. 05-move-semantics.md - podstawy referencji `rvalue` i przenoszenia.
6. 06-namespaces.md - przestrzenie nazw i operator zakresu.
7. 07-moduly-i-podzial-kodu.md - klasyczne pliki kontra moduły C++20.
8. 08-wielowatkowosc.md - podstawy `std::thread`.
9. 09-zadania.md - zadania podstawowe i projektowe.

68.3 Status porządkowania

Lekcje i zadania zostały uporządkowane do wspólnego formatu segmentów. Pierwotne materiały są zachowane w katalogu `archive`:

- `archive/modern-cpp.md`
- `archive/namespaces.md`
- `archive/modules.md`
- `archive/modules-linki.md`
- `archive/modules-ex1-main.cpp`
- `archive/wielowatkowosc.md`
- `archive/main1.cpp`

- `archive/main2.cpp`

68.4 Przykłady kodu

Przykłady w katalogu `examples` można kompilować osobno:

- `examples/modern_cpp_overview.cpp` - przegląd wybranych elementów modern C++.
- `examples/auto_nullptr_initialization.cpp` - `auto`, `nullptr` i inicjalizacja kłamrowa.
- `examples/lambda_algorithms.cpp` - lambdy z algorytmami STL.
- `examples/strong_types_api.cpp` - `enum class`, aliasy i `override`.
- `examples/move_semantics.cpp` - podstawy przenoszenia.
- `examples/namespaces.cpp` - przestrzenie nazw.
- `examples/code-organization` - podział kodu na moduł logiczny.
- `examples/thread_basics.cpp` - podstawy `std::thread`.

68.5 Testy

Katalog `tests` zawiera prosty test wybranych elementów modern C++ bez zewnętrznego frameworka. Test sprawdza pętlę zakresową z `auto`, `nullptr`, `enum class`, aliasy typów, lambdy z algorytmami STL oraz przenoszenie danych.

68.6 Format lekcji

Każda docelowa lekcja powinna mieć taki układ:

1. Cel lekcji.
2. Wymagania wstępne.
3. Krótka teoria.
4. Przykład kodu lub komend.
5. Zadania do wykonania.
6. Kryteria zaliczenia.

68.7 Katalogi pomocnicze

- `examples/` - kompilowalne przykłady `.cpp` dla tego segmentu.
- `tests/` - proste testy automatyczne dla elementów modern C++.
- `archive/` - pierwotne wersje materiałów zachowane do wglądu.

Rozdział 69

01 - Nowości modern C++

69.1 Cel lekcji

Celem lekcji jest uporządkowanie najważniejszych elementów współczesnego C++. Student powinien rozpoznać konstrukcje wprowadzone od C++11 wzwyż i rozumieć, które z nich poprawiają bezpieczeństwo, czytelność oraz organizację kodu.

69.2 Wymagania wstępne

Przed rozpoczęciem lekcji warto znać:

- funkcje i klasy,
- kontenery STL,
- podstawową kompilację programu C++,
- rozdzielanie kodu na mniejsze fragmenty.

69.3 Krótka teoria

69.3.1 Co oznacza modern C++

Modern C++ to styl pisania kodu korzystający z nowszych standardów języka, szczególnie C++11, C++14, C++17 i nowszych. Nie chodzi tylko o nowe słowa kluczowe. Ważniejsza jest zmiana stylu:

- mniej ręcznego zarządzania pamięcią,
- więcej typów i konstrukcji, które wyrażają intencję,
- krótszy i czytelniejszy kod,
- większe wykorzystanie biblioteki standardowej.

69.4 Przykład kodu: wybrane zmiany

69.4.1 Inicjalizacja klamrowa

```
int value{10};  
std::vector<int> numbers{1, 2, 3};
```

Klamry dają spójny zapis inicjalizacji dla prostych typów, obiektów i kontenerów.

69.4.2 auto

```
auto count = numbers.size();
```

auto pozwala kompilatorowi wywnioskować typ. Należy go używać tam, gdzie typ jest oczywisty z prawej strony przypisania albo bardzo długi.

69.4.3 nullptr

```
int* pointer = nullptr;
```

nullptr zastępuje starsze użycie NULL albo 0 jako pustego wskaźnika.

69.4.4 Pętla zakresowa

```
for (const auto& number : numbers) {  
    std::cout << number << '\n';  
}
```

Pętla zakresowa zmniejsza liczbę błędów z indeksami i dobrze pasuje do kontenerów STL.

69.4.5 enum class

```
enum class Status {  
    Active,  
    Inactive  
};
```

enum class ogranicza przypadkowe mieszanie wartości wyliczeniowych z liczbami całkowitymi.

69.4.6 Lambda

```
auto isEven = [](int value) {  
    return value % 2 == 0;  
};
```

Lambda pozwala zdefiniować krótką funkcję w miejscu użycia. Jest szczególnie przydatna z algorytmami STL.

69.4.7 Structured bindings

```
auto [name, score] = result;
```

Structured bindings z C++17 ułatwiają rozpakowanie par, krotek i prostych struktur.

69.4.8 Co zostanie omówione dalej

Ten segment rozwija wybrane tematy:

- auto, nullptr i inicjalizację,
- lambda i algorytmy,
- enum class, override i aliasy typów,
- podstawy semantyki przenoszenia,
- przestrzenie nazw,
- moduły jako temat C++20,
- podstawy wielowątkowości.

69.5 Przykład referencyjny

Przykład znajduje się w pliku examples/modern_cpp_overview.cpp.

Kompilacja:

```
c++ -std=c++17 -Wall -Wextra -pedantic examples/modern_cpp_overview.cpp -o modern_cpp_overview  
./modern_cpp_overview
```

69.6 Zadania do wykonania

1. Skompiluj i uruchom przykład.
2. Dodaj do wektora kolejną osobę i sprawdź wynik sortowania.
3. Zmień lambdę filtrującą tak, aby wybierała osoby z wynikiem co najmniej 70.
4. Dodaj nowe pole do struktury `Person` i wypisz je w raporcie.
5. Zastąp jeden jawny typ zmiennej przez `auto` tylko tam, gdzie nie pogarsza to czytelności.

69.7 Kryteria zaliczenia

Student zalicza lekcję, jeśli potrafi:

- wyjaśnić ogólnie, czym jest modern C++,
- wskazać przykłady konstrukcji z C++11 i C++17,
- użyć inicjalizacji klamrowej,
- zastosować pętlę zakresową,
- rozpoznać lambdę,
- skompilować przykład w standardzie C++17.

Rozdział 70

02 - auto, nullptr i inicjalizacja

70.1 Cel lekcji

Celem lekcji jest praktyczne użycie trzech częstych elementów modern C++: `auto`, `nullptr` oraz inicjalizacji klamrowej. Student powinien rozumieć, kiedy te konstrukcje poprawiają czytelność i bezpieczeństwo kodu.

70.2 Wymagania wstępne

Przed rozpoczęciem lekcji warto znać:

- typy podstawowe,
- wskaźniki,
- struktury,
- `std::vector`,
- pętlę zakresową.

70.3 Krótka teoria

70.3.1 auto

`auto` pozwala kompilatorowi wywnioskować typ zmiennej z wartości po prawej stronie przypisania.

```
auto age = 20;  
auto name = std::string{"Anna"};
```

`auto` jest przydatne, gdy typ jest oczywisty albo długi:

```
auto it = students.begin();
```

Nie należy używać `auto`, jeśli ukrycie typu utrudnia zrozumienie kodu:

```
auto value = loadData();
```

Jeśli nazwa funkcji nie mówi jasno, co zwraca, jawny typ może być lepszy.

70.3.2 auto i referencje

W pętli po większych obiektach często używamy referencji:

```
for (const auto& student : students) {  
    std::cout << student.name << '\n';  
}
```

`const auto&` oznacza:

- nie kopiuj elementu,

- nie pozwalaj go zmienić,
- dobierz typ automatycznie.

70.4 Przykład kodu: nullptr

nullptr oznacza pusty wskaźnik:

```
int* pointer = nullptr;
```

W modern C++ używamy nullptr zamiast NULL albo 0, ponieważ nullptr ma specjalny typ i lepiej wyraża intencję.

Przykład:

```
Student* selected = nullptr;
```

Taki zapis jasno mówi, że wskaźnik na razie nie wskazuje na żaden obiekt.

70.4.1 Sprawdzanie wskaźnika

Przed użyciem wskaźnika trzeba sprawdzić, czy nie jest pusty:

```
if (selected != nullptr) {
    std::cout << selected->name << '\n';
}
```

W prostych programach edukacyjnych wskaźniki nadal są użyteczne do pokazania mechaniki języka. W kodzie produkcyjnym często lepiej wybrać referencje albo inteligentne wskaźniki.

70.5 Przykład kodu: inicjalizacja klamrowa

Klamry pozwalają inicjalizować różne typy w spójny sposób:

```
int points{80};
std::string name{"Anna"};
std::vector<int> values{1, 2, 3};
```

Dla struktur zapis jest czytelny:

```
Student student{"Anna", 82};
```

Klamry pomagają ograniczyć część przypadkowych konwersji zawężających, np. przypisanie liczby zmiennej typu `int`.

70.6 Przykład referencyjny

Przykład znajduje się w pliku `examples/auto_nullptr_initialization.cpp`.

Kompilacja:

```
c++ -std=c++17 -Wall -Wextra -pedantic examples/auto_nullptr_initialization.cpp -o auto_nullptr_init.
./auto_nullptr_initialization
```

70.7 Zadania do wykonania

1. Skompiluj i uruchom przykład.
2. Dodaj kolejnego studenta do wektora.
3. Zmień warunek wyboru najlepszego studenta na wynik co najmniej 90.
4. Zamień jedną zmienną z jawnym typem na `auto`, jeśli nie pogorszy to czytelności.
5. Dodaj przykład wskaźnika ustawionego na `nullptr` i sprawdzenie przed użyciem.

70.8 Kryteria zaliczenia

Student zalicza lekcję, jeśli potrafi:

- użyć `auto` w prostym i czytelnym miejscu,
- wyjaśnić, kiedy `auto` może pogorszyć czytelność,
- użyć `const auto&` w pętli zakresowej,
- użyć `nullptr` jako pustego wskaźnika,
- sprawdzić wskaźnik przed dereferencją,
- zastosować inicjalizację klamrową dla typu prostego, struktury i wektora.

Rozdział 71

03 - Lambdy i algorytmy

71.1 Cel lekcji

Celem lekcji jest praktyczne użycie wyrażeń lambda z algorytmami STL. Student powinien umieć napisać prostą lambda, przekazać ją jako predykat oraz zrozumieć, czym jest przechwytywanie zmiennych.

71.2 Wymagania wstępne

Przed rozpoczęciem lekcji warto znać:

- `std::vector`,
- podstawowe algorytmy STL,
- funkcje,
- pętlę zakresową,
- podstawy `auto`.

71.3 Krótka teoria

71.3.1 Czym jest lambda

Lambda to krótka funkcja zapisana w miejscu użycia.

```
auto isEven = [](int value) {  
    return value % 2 == 0;  
};
```

Elementy lambda:

- `[]` - lista przechwytywania,
- `(int value)` - lista parametrów,
- `{ ... }` - ciało funkcji.

Lambdy są szczególnie wygodne z algorytmami STL, bo często potrzebujemy małego warunku tylko w jednym miejscu.

71.4 Przykład kodu: lambda jako predykat

Predykat to funkcja albo lambda zwracająca `bool`.

```
auto isPositive = [](int value) {  
    return value > 0;  
};
```

Predykaty pasują do algorytmów takich jak:

- `std::find_if`,
- `std::count_if`,
- `std::copy_if`,
- `std::remove_if`,
- `std::sort`.

71.5 Przykład kodu: sortowanie obiektów

Jeśli sortujemy obiekty lub struktury, trzeba podać kryterium:

```
std::sort(orders.begin(), orders.end(),
    [](const Order& left, const Order& right) {
        return left.total > right.total;
    });
```

Lambda zwraca `true`, jeśli `left` ma znaleźć się przed `right`.

71.5.1 Przechwytywanie zmiennych

Lambda może używać zmiennych z zewnętrznego zakresu.

Przechwycenie przez wartość:

```
double minimumTotal = 100.0;

auto isLargeOrder = [minimumTotal](const Order& order) {
    return order.total >= minimumTotal;
};
```

Przechwycenie przez wartość oznacza, że lambda dostaje kopię zmiennej.

Przechwycenie przez referencję:

```
int counter = 0;

auto countOrder = [&counter](const Order&) {
    ++counter;
};
```

Przechwytywanie przez referencję trzeba stosować ostrożnie, bo lambda pracuje na oryginalnej zmiennej.

71.5.2 Czytelność lambd

Lambda jest dobrym wyborem, gdy:

- warunek jest krótki,
- używamy go tylko w jednym miejscu,
- nazwa algorytmu jasno mówi, co się dzieje.

Jeśli warunek robi się długi, lepiej rozważyć osobną funkcję albo metodę.

71.6 Przykład referencyjny

Przykład znajduje się w pliku `examples/lambda_algorithms.cpp`.

Kompilacja:

```
c++ -std=c++17 -Wall -Wextra -pedantic examples/lambda_algorithms.cpp -o lambda_algorithms
./lambda_algorithms
```

71.7 Zadania do wykonania

1. Skompiluj i uruchom przykład.
2. Zmień próg `minimumTotal` i sprawdź wynik filtrowania.
3. Dodaj sortowanie alfabetyczne według nazwy klienta.
4. Dodaj `std::count_if`, które policzy zamówienia opłacone.
5. Zastąp jedną lambdę osobną funkcją i porównaj czytelność obu wersji.

71.8 Kryteria zaliczenia

Student zalicza lekcję, jeśli potrafi:

- napisać prostą lambdę,
- użyć lambdy jako predykatu,
- posortować obiekty według wskazanego pola,
- przechwycić zmienną przez wartość,
- wyjaśnić różnicę między przechwyceniem przez wartość i referencję,
- dobrać lambdę albo osobną funkcję w zależności od czytelności kodu.

Rozdział 72

04 - enum class, override i aliasy

72.1 Cel lekcji

Celem lekcji jest pokazanie konstrukcji, które pomagają pisać czytelniejsze i bezpieczniejsze API: `enum class`, aliasy typów, `override`, `= default` oraz `= delete`.

72.2 Wymagania wstępne

Przed rozpoczęciem lekcji warto znać:

- klasy,
- dziedziczenie,
- metody wirtualne,
- podstawy typów wyliczeniowych,
- konstruktory.

72.3 Krótka teoria

72.3.1 enum class

Zwykły `enum` łatwo miesza się z liczbami całkowitymi. `enum class` tworzy silniej oddzielony typ.

```
enum class Priority {  
    Low,  
    Normal,  
    High  
};
```

Wartości używamy z nazwą typu:

```
Priority priority = Priority::High;
```

Dzięki temu kod jest czytelniejszy i trudniej przypadkowo przekazać złą wartość.

72.3.2 Aliasy typów

Alias pozwala nadać czytelną nazwę istniejącemu typowi:

```
using UserId = std::string;  
using MessageCount = int;
```

Alias nie tworzy nowego, odrębnego typu. Poprawia jednak czytelność intencji.

```
void send(UserId userId);
```

Taki zapis jest bardziej opisowy niż samo `std::string`.

72.4 Przykład kodu: override

override informuje kompilator, że metoda ma nadpisywać metodę wirtualną z klasy bazowej.

```
class EmailNotifier : public Notifier {
public:
    void send(const Message& message) const override;
};
```

Jeśli popełnimy literówkę w nazwie metody albo zmienimy typ parametru, kompilator zgłosi błąd. Bez override łatwo przypadkowo utworzyć nową metodę zamiast nadpisać istniejącą.

72.4.1 = default

= default mówi, że chcemy domyślne zachowanie wygenerowane przez kompilator.

```
class Message {
public:
    Message() = default;
};
```

Taki zapis jest przydatny, gdy chcemy jawnie pokazać intencję.

72.4.2 = delete

= delete blokuje wybraną operację.

```
class Notifier {
public:
    Notifier(const Notifier&) = delete;
    Notifier& operator=(const Notifier&) = delete;
};
```

Można tak zabronić kopiowania obiektu, jeśli kopiowanie nie ma sensu albo mogłoby prowadzić do błędów.

72.5 Przykład referencyjny

Przykład znajduje się w pliku `examples/strong_types_api.cpp`.

Kompilacja:

```
c++ -std=c++17 -Wall -Wextra -pedantic examples/strong_types_api.cpp -o strong_types_api
./strong_types_api
```

72.6 Zadania do wykonania

1. Skompiluj i uruchom przykład.
2. Dodaj wartość `Critical` do `Priority` i obsłuż ją w funkcji wypisującej.
3. Dodaj alias `Subject` dla `std::string`.
4. Dodaj klasę `SmsNotifier`, która dziedziczy po `Notifier`.
5. Celowo usuń `override`, zmień sygnaturę metody i sprawdź, jak zmienia się zachowanie kompilatora po ponownym dodaniu `override`.

72.7 Kryteria zaliczenia

Student zalicza lekcję, jeśli potrafi:

- zdefiniować i użyć `enum class`,
- wyjaśnić przewagę `enum class` nad prostym `enum`,
- utworzyć alias typu przez `using`,
- użyć `override` przy metodzie wirtualnej,
- wyjaśnić sens `= default`,

- zablokować kopiowanie przez = `delete`.

Rozdział 73

05 - Move semantics

73.1 Cel lekcji

Celem lekcji jest zrozumienie podstaw semantyki przenoszenia. Student powinien umieć rozpoznać sytuację, w której obiekt jest kopiowany, oraz sytuację, w której zasoby mogą zostać przeniesione.

73.2 Wymagania wstępne

Przed rozpoczęciem lekcji warto znać:

- klasy i konstruktory,
- konstruktor kopiujący,
- `std::vector`,
- referencje,
- podstawy zarządzania zasobami.

73.3 Krótka teoria

73.3.1 Problem kopiowania

Kopiowanie obiektu oznacza utworzenie drugiego, niezależnego obiektu z taką samą zawartością.

```
Report copy = original;
```

Dla małych obiektów koszt jest niewielki. Dla obiektów przechowujących duże wektory, napisy albo uchwyty do zasobów kopiowanie może być kosztowne.

73.3.2 Przenoszenie

Przenoszenie pozwala przejąć zasoby z obiektu tymczasowego albo obiektu, którego już nie potrzebujemy.

```
Report moved = std::move(original);
```

Po `std::move(original)` obiekt `original` nadal istnieje, ale jego stan jest poprawny tylko w sensie technicznym. Nie należy zakładać, że zachował dawną zawartość.

73.4 Przykład kodu: `std::move`

`std::move` niczego samo nie przenosi. Ono tylko zamienia wyrażenie na takie, które może zostać obsłużone przez konstruktor przenoszący albo operator przenoszący.

Do `std::move` potrzebny jest nagłówek:

```
#include <utility>
```

73.4.1 Konstruktor przenoszący

Konstruktor przenoszący przyjmuje referencję rvalue:

```
Report (Report&& other) noexcept;
```

W praktyce często nie trzeba pisać go ręcznie, bo typy z biblioteki standardowej same dobrze obsługują przenoszenie. Własny konstruktor w przykładzie służy pokazaniu, kiedy przeniesienie następuje.

73.4.2 Kiedy używać przenoszenia

Przenoszenie jest przydatne, gdy:

- obiekt posiada duży zasób,
- obiekt tymczasowy ma trafić do kontenera,
- funkcja zwraca duży obiekt,
- nie potrzebujemy już starej zawartości obiektu.

Nie należy używać `std::move` mechanicznie. Jeśli obiekt ma być dalej używany z oczekiwaną zawartością, przenoszenie będzie błędem projektowym.

73.5 Przykład referencyjny

Przykład znajduje się w pliku `examples/move_semantics.cpp`.

Kompilacja:

```
c++ -std=c++17 -Wall -Wextra -pedantic examples/move_semantics.cpp -o move_semantics
./move_semantics
```

73.6 Zadania do wykonania

1. Skompiluj i uruchom przykład.
2. Usuń `std::move` i sprawdź, czy uruchamia się kopiowanie czy przenoszenie.
3. Dodaj operator przypisania przenoszącego.
4. Dodaj metodę `size`, która zwraca liczbę elementów raportu.
5. Po przeniesieniu obiektu wypisz jego rozmiar i porównaj z nowym obiektem.

73.7 Kryteria zaliczenia

Student zalicza lekcję, jeśli potrafi:

- wyjaśnić różnicę między kopiowaniem i przenoszeniem,
- użyć `std::move`,
- wyjaśnić, że `std::move` samo nie przenosi danych,
- rozpoznać konstruktor przenoszący,
- zachować ostrożność przy używaniu obiektu po przeniesieniu,
- skompilować przykład w standardzie C++17.

Rozdział 74

06 - Przestrzenie nazw

74.1 Cel lekcji

Celem lekcji jest zrozumienie, jak przestrzenie nazw pomagają organizować kod i unikać konfliktów nazw. Student powinien umieć zdefiniować własną przestrzeń nazw, użyć operatora zakresu `::` oraz świadomie korzystać z deklaracji `using`.

74.2 Wymagania wstępne

Przed rozpoczęciem lekcji warto znać:

- funkcje,
- klasy albo struktury,
- podział programu na moduły tematyczne,
- podstawowe użycie `std::`.

74.3 Krótka teoria

74.3.1 Po co są przestrzenie nazw

W większym programie różne części kodu mogą używać tych samych nazw funkcji, typów albo stałych. Przestrzeń nazw grupuje powiązane elementy i zmniejsza ryzyko konfliktu.

```
namespace math {  
    int add(int a, int b);  
}
```

Funkcję wywołujemy przez operator zakresu:

```
int result = math::add(2, 3);
```

74.4 Przykład kodu: operator `::`

Operator `::` wskazuje, z której przestrzeni nazw pochodzi element.

```
std::cout << "Text\n";
```

W tym przykładzie `cout` pochodzi z przestrzeni nazw `std`.

Własny przykład:

```
reporting::printSummary();
```

Taki zapis jest dłuższy, ale bardzo czytelny.

74.4.1 Zagnieżdżone przestrzenie nazw

W C++17 można zapisać zagnieżdżoną przestrzeń nazw krótko:

```
namespace app::reports {  
    void print();  
}
```

To odpowiada starszemu zapisowi:

```
namespace app {  
namespace reports {  
    void print();  
}  
}
```

74.4.2 using

Deklaracja `using` pozwala skrócić nazwę wybranego elementu:

```
using app::reports::print;  
  
print();
```

To jest zwykle bezpieczniejsze niż:

```
using namespace app::reports;
```

Szerokie `using namespace` może wprowadzić wiele nazw naraz i utrudnić rozpoznanie, skąd pochodzi dana funkcja.

74.4.3 using namespace std

W krótkich przykładach edukacyjnych czasem spotyka się:

```
using namespace std;
```

W materiałach tego kursu preferujemy zapis z `std::`, ponieważ uczy on, skąd pochodzą elementy biblioteki standardowej i zmniejsza ryzyko konfliktu nazw.

74.5 Przykład referencyjny

Przykład znajduje się w pliku `examples/namespaces.cpp`.

Kompilacja:

```
c++ -std=c++17 -Wall -Wextra -pedantic examples/namespaces.cpp -o namespaces  
./namespaces
```

74.6 Zadania do wykonania

1. Skompiluj i uruchom przykład.
2. Dodaj przestrzeń nazw `app::users` z funkcją `printUser`.
3. Dodaj funkcję `format` w dwóch różnych przestrzeniach nazw i wywołaj obie przez operator `::`.
4. Zastąp jedno długie wywołanie deklaracją `using`.
5. Usuń `std::` z jednego miejsca i sprawdź, jaki błąd zgłosi kompilator.

74.7 Kryteria zaliczenia

Student zalicza lekcję, jeśli potrafi:

- zdefiniować własną przestrzeń nazw,
- wywołać funkcję przez operator `::`,

- użyć zagnieżdżonej przestrzeni nazw,
- wyjaśnić, do czego służy `std::`,
- użyć deklaracji `using` dla pojedynczej nazwy,
- wyjaśnić ryzyko szerokiego `using namespace`.

Rozdział 75

07 - Moduły i podział kodu

75.1 Cel lekcji

Celem lekcji jest rozróżnienie dwóch pojęć: klasycznego podziału programu na pliki `.h` i `.cpp` oraz modułów C++20. Student powinien rozumieć, że w tym kursie stabilną podstawą pozostaje podział na nagłówki i pliki źródłowe, a moduły C++20 są tematem rozszerzającym.

75.2 Wymagania wstępne

Przed rozpoczęciem lekcji warto znać:

- pliki nagłówkowe,
- implementacje w plikach `.cpp`,
- guardy,
- kompilację wielu plików,
- przestrzenie nazw.

75.3 Krótka teoria

75.3.1 Klasyczny podział na pliki

W typowym projekcie C++ interfejs umieszczamy w pliku `.h`, a implementację w pliku `.cpp`.

```
message.h
message.cpp
main.cpp
```

Plik nagłówkowy mówi, co można wywołać:

```
std::string formatMessage(const Message& message);
```

Plik `.cpp` mówi, jak to działa:

```
std::string formatMessage(const Message& message) {
    return message.author + ": " + message.text;
}
```

To jest podstawowy i nadal bardzo powszechny sposób organizacji kodu C++.

75.3.2 „Moduł” jako część programu

W rozmowach o projekcie słowo „moduł” często oznacza logiczną część programu, np. moduł wiadomości, moduł użytkowników albo moduł raportów.

Taki moduł może być zwykłą parą plików:

```
message.h
message.cpp
```

Nie jest to jeszcze moduł C++20. To tylko organizacja kodu.

75.3.3 Moduły C++20

C++20 wprowadził językową funkcję `module`.

Przykładowy szkic:

```
export module messages;

export std::string formatMessage(const Message& message);
```

Moduły C++20 mają rozwiązywać część problemów nagłówków, np. wielokrotne dołączanie i koszt preprocesora. W praktyce ich obsługa zależy od kompilatora, wersji narzędzi i systemu build.

Dlatego w tym repozytorium przykłady kompilowalne trzymamy przy klasycznym podziale na `.h` i `.cpp`.

75.3.4 Co wybrać w tym kursie

Na tym etapie wybieraj:

- `.h` dla deklaracji i interfejsu,
- `.cpp` dla implementacji,
- `namespace` dla uporządkowania nazw,
- `examples/` dla małych programów demonstracyjnych.

Moduły C++20 warto znać jako kierunek rozwoju języka, ale nie są wymagane do zrozumienia większości projektów C++.

75.4 Przykład referencyjny

Przykład znajduje się w katalogu `examples/code-organization`.

Kompilacja:

```
c++ -std=c++17 -Wall -Wextra -pedantic \
  examples/code-organization/main.cpp \
  examples/code-organization/message.cpp \
  -o code_organization
./code_organization
```

75.5 Zadania do wykonania

1. Skompiluj i uruchom przykład.
2. Dodaj do `Message` pole `priority`.
3. Rozszerz `formatMessage`, aby wypisywała priorytet.
4. Dodaj drugi logiczny moduł `users.h` i `users.cpp`.
5. Napisz w komentarzu, czym różni się taki moduł logiczny od modułu C++20.

75.6 Kryteria zaliczenia

Student zalicza lekcję, jeśli potrafi:

- rozdzielić interfejs od implementacji,
- wskazać rolę pliku `.h` i `.cpp`,
- skompilować program złożony z kilku plików,
- wyjaśnić, że „moduł” może oznaczać logiczną część programu,
- odróżnić klasyczny podział kodu od modułów C++20,
- wskazać, dlaczego moduły C++20 zależą od narzędzi build i kompilatora.

Rozdział 76

08 - Wielowątkowość

76.1 Cel lekcji

Celem lekcji jest pierwsze, praktyczne użycie `std::thread`. Student powinien umieć uruchomić funkcję w osobnym wątku, poczekać na zakończenie przez `join` oraz rozumieć, dlaczego wspólne dane trzeba chronić.

76.2 Wymagania wstępne

Przed rozpoczęciem lekcji warto znać:

- funkcje,
- lambdy,
- referencje,
- podstawy klas i obiektów,
- kompilację programu C++17.

76.3 Krótka teoria

76.3.1 Czym jest wątek

Wątek to osobny tor wykonania w ramach jednego programu. Dzięki wątkom program może wykonywać kilka zadań pozornie równolegle, a na procesorach wielordzeniowych część pracy może faktycznie wykonywać się równolegle.

Do podstawowej obsługi wątków potrzebny jest nagłówek:

```
#include <thread>
```

76.4 Przykład kodu: uruchomienie wątku

Najprostszy przykład:

```
void work() {  
    std::cout << "Working\n";  
}
```

```
std::thread worker{work};  
worker.join();
```

`join` oznacza: poczekaj, aż wątek zakończy pracę.

76.4.1 Dlaczego join jest ważny

Jeśli utworzony wątek nie zostanie obsłużony przez `join` albo `detach`, program zakończy się błędem w czasie działania. Na tym etapie używamy `join`, bo jest łatwiejszy do zrozumienia i bezpieczniejszy.

76.5 Przykład kodu: wspólne dane

Jeśli kilka wątków modyfikuje tę samą zmienną, może dojść do wyścigu danych. Wynik programu staje się wtedy nieprzewidywalny.

Do ochrony wspólnych danych używamy `std::mutex`:

```
std::mutex mutex;

oraz std::lock_guard:

{
    std::lock_guard<std::mutex> lock{mutex};
    sharedValue += 1;
}
```

`lock_guard` blokuje `mutex` przy utworzeniu i automatycznie odblokowuje go przy wyjściu z zakresu.

76.5.1 Przekazywanie danych do wątku

Do wątku można przekazać funkcję, lambda albo obiekt funkcyjny.

```
std::thread worker{[&counter]() {
    counter.add(10);
}};
```

Jeśli lambda przechwytuje referencje, trzeba pilnować, aby obiekty istniały tak długo, jak działa wątek.

76.6 Przykład referencyjny

Przykład znajduje się w pliku `examples/thread_basics.cpp`.

Kompilacja:

```
c++ -std=c++17 -Wall -Wextra -pedantic examples/thread_basics.cpp -o thread_basics
./thread_basics
```

Na niektórych systemach Linux może być potrzebna flaga:

```
-pthread
```

76.7 Zadania do wykonania

1. Skompiluj i uruchom przykład.
2. Zmień liczbę iteracji w każdym wątku.
3. Dodaj trzeci wątek pracujący na tym samym liczniku.
4. Usuń tymczasowo `lock_guard` i zaobserwuj, dlaczego wynik może być niepewny. Następnie przywróć ochronę.
5. Jako zadanie rozszerzające zaprojektuj szkic rozwiązania problemu pięciu filozofów z użyciem `std::thread` i `std::mutex`.

76.8 Kryteria zaliczenia

Student zalicza lekcję, jeśli potrafi:

- utworzyć `std::thread`,
- wywołać `join`,
- wyjaśnić, czym jest wyścig danych,

- użyć `std::mutex`,
- użyć `std::lock_guard`,
- ostrożnie przekazać dane do lambdy uruchamianej w wątku.

Rozdział 77

09 - Zadania

77.1 Cel zestawu zadań

Ten plik zbiera zadania do segmentu 09 - Modern C++. Zadania sprawdzają konstrukcje C++11 i C++17, lambdy, silniejsze typy, semantykę przenoszenia, przestrzenie nazw, organizację kodu oraz podstawy wielowątkowości.

77.2 Zasady wykonania

Każde zadanie powinno być zapisane w osobnym pliku `.cpp` albo osobnym katalogu, jeśli wymaga kilku plików. Program powinien kompilować się bez błędów i ostrzeżeń.

Przykład kompilacji:

```
c++ -std=c++17 -Wall -Wextra -pedantic nazwa_pliku.cpp -o program
./program
```

Dla zadań z wątkami użyj dodatkowo `-pthread`, jeśli wymaga tego system.

77.3 Wymagania jakościowe

W zadaniach z tego segmentu zwracaj uwagę na:

- kompilację z flagami `-std=c++17 -Wall -Wextra -pedantic`,
- używanie `auto` tylko wtedy, gdy nie ukrywa znaczenia kodu,
- `enum class` zamiast luźnych liczb lub napisów opisujących stan,
- lambdy krótkie i czytelne, a dłuższą logikę przeniesioną do funkcji,
- wywołanie `join` dla każdego uruchomionego wątku.
- brak współdzielonych danych bez ochrony, jeśli używasz kilku wątków.

77.4 Poziom 1 - Podstawy modern C++

77.4.1 Zadanie 1. Inicjalizacja klamrowa

Utwórz zmienne typu `int`, `double`, `std::string` oraz `std::vector<int>` z użyciem inicjalizacji klamrowej. Wypisz ich wartości.

Wymagania:

- dołącz potrzebne nagłówki,
- pokaż inicjalizację pustego i niepustego wektora,
- w komentarzu zapisz, dlaczego inicjalizacja klamrowa jest czytelna.

77.4.2 Zadanie 2. auto w praktyce

Utwórz wektor napisów i przejdź po nim pętlą zakresową z `const auto&`. Następnie zapisz iterator z `begin()` do zmiennej z `auto`.

Wymagania:

- użyj `const auto&` przy wypisywaniu napisów,
- nie używaj `auto`, jeśli typ prosty jest bardziej czytelny, np. przy `int licznik`.

77.4.3 Zadanie 3. nullptr

Utwórz strukturę `Student` oraz funkcję zwracającą wskaźnik do najlepszego studenta albo `nullptr`, jeśli lista jest pusta.

Scenariusze sprawdzenia:

- pusta lista zwraca `nullptr`,
- lista z kilkoma studentami zwraca studenta z najwyższą liczbą punktów.

77.4.4 Zadanie 4. Pętla zakresowa

Przepisz program z klasyczną pętlą indeksową na pętlę zakresową. Zapisz w komentarzu, która wersja jest czytelniejsza i dlaczego.

Nie używaj pętli zakresowej, jeśli potrzebujesz modyfikować indeks. W komentarzu opisz tę różnicę.

77.4.5 Zadanie 5. Structured bindings

Utwórz `std::pair<std::string, int>` z nazwą i wynikiem. Rozpakuj parę przez structured bindings i wypisz wynik.

Przykład wyniku:

Anna: 82

77.5 Poziom 2 - Lambdy i algorytmy

77.5.1 Zadanie 6. Sortowanie przez lambdę

Utwórz wektor struktur `Product` z polami `name` i `price`. Posortuj produkty rosnąco według ceny za pomocą `std::sort` i lambdy.

Wymagania:

- lambda powinna przyjmować argumenty przez `const Product&`,
- po sortowaniu wypisz produkty, aby pokazać kolejność.

77.5.2 Zadanie 7. Filtrowanie z progiem

Utwórz zmienną `minimumPrice` i skopiuj do nowego wektora tylko produkty droższe lub równe tej wartości. Użyj `std::copy_if` i przechwycenia przez wartość.

Scenariusz sprawdzenia:

```
minimumPrice = 100
Produkty: 80, 100, 150
Wynik: 100, 150
```

77.5.3 Zadanie 8. Liczenie elementów

Policz produkty tańsze niż 100 za pomocą `std::count_if`.

Wynik wypisz jako pełne zdanie, np. Produkty tansze niz 100: 2.

77.5.4 Zadanie 9. `find_if`

Znajdź pierwszy produkt o podanej nazwie. Pamiętaj o sprawdzeniu wyniku przed dereferencją iteratora. Sprawdź przypadek znaleziony i niezaleziony.

77.5.5 Zadanie 10. Lambda czy funkcja

Przepisz jeden warunek z lambdy na osobną funkcję. Porównaj czytelność obu wersji w komentarzu. Jeśli warunek ma więcej niż jedną instrukcję albo jest używany w kilku miejscach, preferuj osobną funkcję.

77.6 Poziom 3 - Silniejsze typy i API

77.6.1 Zadanie 11. `enum class`

Utwórz `enum class` `OrderStatus` z wartościami:

- `New`,
- `Paid`,
- `Sent`,
- `Cancelled`.

Dodaj funkcję `statusToText`.

Wymagania:

- w `switch` obsłuż wszystkie wartości `OrderStatus`,
- nie używaj gołych liczb do reprezentowania statusu.

77.6.2 Zadanie 12. Aliasy typów

Dodaj aliasy:

```
using OrderId = std::string;
using Money = double;
```

Użyj ich w strukturze `Order`.

W komentarzu zapisz, czy alias zwiększył czytelność pól struktury.

77.6.3 Zadanie 13. `override`

Utwórz klasę bazową `Printer` z metodą wirtualną `print`. Następnie utwórz klasy `ConsolePrinter` i `SilentPrinter`, które nadpisują tę metodę z użyciem `override`.

Wymagania:

- destruktor klasy bazowej powinien być wirtualny,
- pokaż wywołanie przez referencję albo wskaźnik do klasy bazowej.

77.6.4 Zadanie 14. `= delete`

Utwórz klasę `Logger`, której nie da się kopiować. Użyj `= delete` dla konstruktora kopiującego i operatora przypisania.

W komentarzu pokaż przykład linii, która nie powinna się kompilować, ale nie zostawiaj jej aktywnej w kodzie.

77.6.5 Zadanie 15. `= default`

Dodaj do klasy `Config` jawny konstruktor domyślny przez `= default`. Zapisz w komentarzu, po co taki zapis może być czytelny.

Dodaj co najmniej jedno pole z wartością domyślną.

77.7 Poziom 4 - Przenoszenie i organizacja kodu

77.7.1 Zadanie 16. Kopiowanie i przenoszenie

Utwórz klasę `Buffer`, która przechowuje `std::vector<int>`. Dodaj konstruktor kopiujący i przenoszący, które wypisują odpowiedni komunikat.

Wymagania:

- konstruktor przenoszący powinien używać `std::move`,
- dołącz `<utility>`.

77.7.2 Zadanie 17. `std::move`

Utwórz obiekt `Buffer`, skopiuj go do drugiego obiektu, a następnie przenieś do trzeciego przez `std::move`.

Wynik programu powinien pokazać komunikat dla kopiowania i komunikat dla przeniesienia.

77.7.3 Zadanie 18. Stan po przeniesieniu

Po przeniesieniu wypisz rozmiar obiektu źródłowego i docelowego. Zapisz w komentarzu, dlaczego nie należy zakładać konkretnej zawartości obiektu po przeniesieniu.

Wymaganie: obiekt po przeniesieniu może być użyty tylko w poprawnym, ale nieokreślonym stanie. Nie pisz kodu zależnego od konkretnej zawartości.

77.7.4 Zadanie 19. Przestrzeń nazw

Utwórz przestrzeń nazw `app::reports` i funkcję `printSummary`. Wywołaj ją przez operator `::`.

Dodaj drugą funkcję o tej samej nazwie w innej przestrzeni nazw, aby pokazać, po co używa się kwalifikacji nazw.

77.7.5 Zadanie 20. Podział na pliki

Utwórz mały moduł logiczny `messages` z plikami:

- `message.h`,
- `message.cpp`,
- `main.cpp`.

W komentarzu wyjaśnij różnicę między takim modulem logicznym a modulem C++20.

Wymagania:

- `message.h` ma zawierać deklaracje,
- `message.cpp` ma zawierać implementację,
- `main.cpp` ma korzystać z interfejsu z nagłówka.

77.8 Poziom 5 - Wątki i mini-projekty

77.8.1 Zadanie 21. Pierwszy wątek

Utwórz funkcję `work`, która wypisuje komunikat. Uruchom ją w `std::thread` i zakończ program przez `join`.

Wymagania:

- dołącz `<thread>`,
- wywołaj `join` dokładnie raz dla uruchomionego wątku.

77.8.2 Zadanie 22. Dwa wątki

Uruchom dwa wątki, które wykonują tę samą funkcję z różnymi nazwami zadań. Zaobserwuj, że kolejność komunikatów może się zmieniać.

W komentarzu zapisz, dlaczego nie należy zakładać stałej kolejności wypisywania.

77.8.3 Zadanie 23. Bezpieczny licznik

Utwórz klasę `SafeCounter` z `std::mutex`, metodą `add` i metodą `getTotal`. Uruchom kilka wątków zwiększających licznik.

Wymagania:

- użyj `std::lock_guard<std::mutex>`,
- wszystkie wątki muszą zostać zakończone przez `join`,
- wynik końcowy powinien być zgodny z liczbą wykonanych inkrementacji.

77.8.4 Zadanie 24. Mini-projekt: monitor zadań

Przygotuj mini-projekt `task-monitor`, który:

- przechowuje listę zadań,
- używa `enum class` dla statusu zadania,
- sortuje zadania lambdą,
- ma przestrzeń nazw `app::tasks`,
- uruchamia dwa wątki zwiększające licznik wykonanych operacji,
- kompiluje się przez prostą komendę albo skrypt `build.sh`.

77.9 Wariant minimum

Do zaliczenia segmentu wykonaj co najmniej:

1. Zadanie 1 albo 2.
2. Zadanie 3 albo 5.
3. Zadanie 6 albo 7.
4. Zadanie 11 albo 13.
5. Zadanie 16 albo 17.
6. Zadanie 19 albo 20.
7. Zadanie 21 albo 23.

Zadanie 24 jest mini-projektem rozszerzonym. Możesz je robić po wykonaniu wariantu minimum.

77.10 Mini-sprawdzian segmentu

Napisz program `order-monitor`, który:

- używa `enum class` `OrderStatus`,
- przechowuje zamówienia w `std::vector`,
- filtruje zamówienia lambdą przez `std::copy_if`,
- sortuje zamówienia po wartości przez `std::sort`,
- używa aliasów `OrderId` i `Money`,
- ma przestrzeń nazw `app::orders`,
- uruchamia dwa wątki zwiększające bezpieczny licznik przetworzonych operacji.

Scenariusz sprawdzenia:

Zamowienia o statusie `Paid`: 2
Najdroższe zamówienie: `ORD-3`
Przetworzone operacje: 2000

77.11 Kryteria zaliczenia segmentu

Student zalicza segment, jeśli potrafi samodzielnie:

- użyć `auto`, `nullptr` i inicjalizacji klamrowej,
- napisać lambdę i użyć jej z algorytmem STL,
- przechwycić zmienną w lambdzie,
- zdefiniować `enum class`,
- użyć aliasu typu przez `using`,
- zastosować `override`, `= default` i `= delete`,
- wyjaśnić różnicę między kopiowaniem i przenoszeniem,
- użyć `std::move`,
- zorganizować kod w przestrzeni nazw,
- rozdzielić mały moduł logiczny na `.h` i `.cpp`,
- utworzyć `std::thread`, wywołać `join` i ochronić wspólne dane przez `std::mutex`.

77.12 Checklista oddania

Przed oddaniem rozwiązań sprawdź:

- ☐ Każde zadanie jest w osobnym pliku `.cpp` albo osobnym katalogu projektu.
- ☐ Programy kompilują się z `-Wall -Wextra -pedantic`.
- ☐ `auto` poprawia czytelność zamiast ją ukrywać.
- ☐ Lambdy są krótkie, a dłuższa logika jest w funkcjach.
- ☐ `enum class` zastępuje luźne liczby albo napisy statusów.
- ☐ Po `std::move` kod nie zakłada konkretnej zawartości obiektu źródłowego.
- ☐ Każdy uruchomiony wątek ma `join`.
- ☐ Wspólne dane używane przez wątki są chronione przez `std::mutex`.

77.13 Archiwum

Oryginalne materiały źródłowe tego segmentu są zachowane w katalogu `archive`.

Rozdział 78

10 - Projekty

Ten segment zbiera większe zadania projektowe i tematy zaliczeniowe. Projekty mają połączyć materiał z wcześniejszych segmentów: podstawy języka, STL, klasy, pliki, testy, organizację kodu i prostą dokumentację.

78.1 Cele segmentu

Po zakończeniu segmentu student powinien umieć:

- wybrać temat projektu i doprecyzować wymagania,
- podzielić rozwiązanie na mniejsze moduły,
- dobrać kontenery i klasy do problemu,
- przygotować kompilowalny projekt wieloplikowy,
- dodać podstawowe testy albo scenariusze sprawdzenia,
- przygotować krótką dokumentację użytkową i techniczną,
- opisać ograniczenia oraz możliwe rozszerzenia projektu.

78.2 Kategorie projektów

Jeśli nie wiesz, który temat wybrać, zacznij od przewodnika wyboru projektu. Dokument pomaga dobrać projekt do poziomu trudności, czasu i materiału, który chcesz przećwiczyć.

78.2.1 Narzędzia konsolowe

- Projekt 01 - Logger
- Projekt 08 - Kalkulator
- Projekt 09 - Kalkulator macierzy
- Projekt 11 - Parser plików CSV

78.2.2 Systemy i aplikacje użytkowe

- Projekt 02 - Kolejowanie zamówień
- Projekt 07 - Baza danych pojazdów
- Projekt 10 - System zarządzania zadaniami
- Projekt 12 - Symulator bankomatu

78.2.3 Gry

- Gra kółko i krzyżyk
- Projekt 04 - Tetris
- Projekt 05 - Snake
- Projekt 06 - Pac-Man

78.3 Status porządkowania

Opisy w docelowym formacie:

- `gra-kolko-krzyzyk.md`,
- `project-01-logger.md`,
- `project-02-kolejkowanie-zamowien.md`,
- `project-04-tetris.md`,
- `project-05-snake.md`,
- `project-06-pacman.md`,
- `project-07-baza-pojazdow.md`,
- `project-08-kalkulator.md`,
- `project-09-kalkulator-macierzy.md`,
- `project-10-system-zadan.md`,
- `project-11-parser-csv.md`,
- `project-12-symulator-bankomatu.md`.

Uporządkowane materiały pomocnicze:

- `tnt-tasks/`.

78.4 Dodatkowe zadania źródłowe

Katalog `tnt-tasks` zawiera uporządkowane zadania pomocnicze, które posłużyły jako źródło nowych ćwiczeń i projektu bankomatu. Aktualną mapę decyzji zawiera mapa przeniesienia zadań TNT.

78.5 Docelowy format opisu projektu

Każdy docelowy opis projektu powinien mieć taki układ:

1. Cel projektu.
2. Zakres funkcjonalny.
3. Wymagania techniczne.
4. Proponowana struktura plików.
5. Model danych albo model domeny, jeśli projekt tego wymaga.
6. Minimalny wariant zaliczeniowy.
7. Rozszerzenia dla chętnych.
8. Kryteria oceny.
9. Scenariusze sprawdzenia.

W opisach projektów nie usuwamy pierwotnego sensu zadania, tylko doprecyzowujemy go tak, aby student wiedział, co jest wersją minimalną, co jest rozszerzeniem i jak sprawdzić poprawność rozwiązania.

78.6 Wymagania dla rozwiązania

Projekt powinien zawierać:

- `README.md` z instrukcją kompilacji i uruchomienia,
- kod źródłowy podzielony na pliki,
- przykładowe dane wejściowe, jeśli są potrzebne,
- krótki opis decyzji projektowych,
- testy albo opis ręcznych scenariuszy sprawdzenia.

Przed oddaniem pracy student powinien przejść przez checklistę projektu. Checklistę zbiera wymagania techniczne, dokumentacyjne i demonstracyjne w jednym miejscu.

W trakcie pracy warto korzystać też z planu pracy nad projektem, który rozбивa projekt na małe etapy: start, model danych, wariant minimum, testy, dokumentację i pokaz. Do odbioru i wpisania punktacji służy karta oceny projektu.

78.7 Wariant minimum i rozszerzenie

Każdy projekt należy oddawać w jednym z dwóch wariantów:

- wariant minimum - spełnia sekcję **Minimalny wariant zaliczeniowy**, kompiluje się i obsługuje podstawowe przypadki użycia,
- wariant rozszerzony - spełnia wariant minimum oraz co najmniej dwa punkty z sekcji **Rozszerzenia dla chętnych**.

Wariant rozszerzony nie zwalnia z jakości kodu. Najpierw musi działać stabilna wersja minimalna, dopiero później warto dodawać funkcje dodatkowe.

78.8 Rubryka oceny projektu

Proponowana punktacja dla każdego projektu:

Obszar	Punkty	Co jest oceniane
Funkcjonalność minimum	30	Działają wszystkie wymagania z wariantu minimalnego.
Poprawność techniczna	20	Projekt kompiluje się z -Wall -Wextra -pedantic , nie wymaga plików wynikowych w repozytorium i obsługuje błędne dane.
Organizacja kodu	20	Kod jest podzielony na funkcje, klasy i pliki zgodnie z problemem. Nazwy są czytelne.
Testy lub scenariusze sprawdzenia	15	Autor potrafi pokazać przypadki poprawne, graniczne i błędne.
Dokumentacja	10	README.md wyjaśnia kompilację, uruchomienie, strukturę projektu i ograniczenia.
Rozszerzenia	5	Projekt zawiera sensowne funkcje dodatkowe bez psucia wersji minimalnej.

Ocena projektu powinna zaczynać się od uruchomienia wariantu minimalnego. Jeśli wariant minimalny nie działa, rozszerzenia nie powinny podnosić oceny ponad podstawowy próg zaliczenia.

78.9 Kryteria zaliczenia segmentu

Student zalicza segment, jeśli potrafi samodzielnie:

- uruchomić projekt z instrukcji,
- wyjaśnić strukturę plików,
- wskazać główne klasy i funkcje,
- pokazać działające podstawowe przypadki użycia,
- obsłużyć błędne dane wejściowe,
- opisać, co można dodać w kolejnej wersji.

78.10 Katalogi pomocnicze

- **examples/** - krótkie, kompilowalne szkice startowe do wybranych projektów.

- `szkice-klas-projektow.md` - przegląd projektów pod kątem przykładowych klas i modeli danych.
- `jak-wybrac-projekt.md` - przewodnik wyboru tematu i pierwszego planu pracy.
- `plan-pracy-nad-projektem.md` - harmonogram pracy od pierwszej wersji do pokazu.
- `checklista-projektu.md` - lista kontroli przed oddaniem i pokazem projektu.
- `karta-oceny-projektu.md` - karta odbioru, punktacji i poprawek.
- `tnt-tasks/` - starsze zadania źródłowe z mapą ich docelowego użycia.
- `archive/` - stare wersje plików po wchłonięciu ich treści do nowych opisów.

Rozdział 79

Jak wybrać projekt

Ten dokument pomaga dobrać temat projektu do poziomu zaawansowania, czasu i materiału, który student chce przećwiczyć. Najpierw wybierz stabilny wariant minimum, a dopiero potem planuj rozszerzenia.

79.1 1. Wybierz poziom trudności

79.1.1 Poziom podstawowy

Dla osób, które chcą przećwiczyć funkcje, pliki, proste klasy i menu konsolowe:

- Projekt 01 - Logger,
- Projekt 08 - Kalkulator,
- Projekt 11 - Parser plików CSV,
- Gra kółko i krzyżyk.

79.1.2 Poziom średni

Dla osób, które chcą użyć kilku klas, kontenerów STL i zapisu do pliku:

- Projekt 02 - Kolejowanie zamówień,
- Projekt 07 - Baza danych pojazdów,
- Projekt 10 - System zarządzania zadaniami,
- Projekt 12 - Symulator bankomatu,
- Projekt 09 - Kalkulator macierzy.

79.1.3 Poziom rozszerzony

Dla osób, które chcą połączyć logikę gry, pętlę programu, stan obiektów i więcej przypadków brzegowych:

- Projekt 04 - Tetris,
- Projekt 05 - Snake,
- Projekt 06 - Pac-Man.

79.2 2. Dopasuj projekt do materiału

Chcesz przećwiczyć	Wybierz
Pliki tekstowe i walidację	Logger, Parser CSV, System zadań
Klasy i enkapsulację	Baza pojazdów, System zadań, Kalkulator macierzy
STL i kolejki	Kolejkowanie zamówień, System zadań
Stan użytkownika i walidację	Symulator bankomatu, System zadań
Algorytmy i parsowanie	Kalkulator, Parser CSV
Pętlę gry i stan planszy	Kółko i krzyżyk, Snake, Tetris, Pac-Man

Chcesz przeciwzyć	Wybierz
Testy i strukturę projektu	Każdy projekt, szczególnie Logger albo Kalkulator

79.3 3. Sprawdź realny zakres

Przed zatwierdzeniem tematu odpowiedz na pytania:

- Czy umiesz opisać wariant minimum w pięciu punktach?
- Czy projekt da się uruchomić w terminalu bez dodatkowych bibliotek?
- Czy jesteś w stanie przygotować pierwszą działającą wersję w ciągu jednych lub dwóch zajęć?
- Czy wiesz, jakie dane program będzie przechowywał?
- Czy potrafisz wymienić trzy scenariusze sprawdzenia?

Jeśli odpowiedź na kilka pytań brzmi **nie**, wybierz prostszy temat albo zmniejsz zakres wariantu minimum.

79.4 4. Przygotuj pierwszy plan pracy

Dobry pierwszy plan ma małe kroki:

1. Utwórz strukturę katalogów projektu.
2. Dodaj minimalny `README.md` z komendą kompilacji.
3. Napisz najprostszy program, który się kompiluje.
4. Dodaj model danych albo pierwszą klasę.
5. Dodaj jedną funkcję z wariantu minimum.
6. Uruchom program i zapisz scenariusz sprawdzenia.
7. Dopiero potem dodawaj kolejne funkcje.

79.5 5. Nie zaczynaj od rozszerzeń

Rozszerzenia są oceniane dopiero wtedy, gdy wariant minimum działa stabilnie. Najczęstszy błąd to rozpoczęcie od funkcji dodatkowych, zanim program potrafi wykonać podstawowy przypadek użycia od początku do końca.

Przykład dobrej kolejności:

1. Kalkulator najpierw liczy $2 + 3$.
2. Potem obsługuje $+$, $-$, $*$, $/$.
3. Potem obsługuje błędy.
4. Dopiero później dodaje historię, funkcje matematyczne i eksport.

79.6 6. Przed oddaniem

Przed pokazem projektu przejdź przez:

- checklistę projektu,
- scenariusze sprawdzenia z opisu wybranego projektu,
- własny `README.md` projektu.

Rozdział 80

Plan pracy nad projektem

Ten dokument pomaga prowadzić projekt etapami. Celem nie jest napisanie całego programu naraz, tylko regularne dostarczanie małych, działających wersji.

80.1 1. Etap startowy

Na początku projektu przygotuj:

- wybrany temat i krótki opis celu,
- wariant minimum zapisany w kilku punktach,
- katalog projektu,
- pierwszy README.md,
- komendę kompilacji,
- najprostszy program, który się kompiluje i uruchamia.

Po tym etapie projekt nie musi jeszcze rozwiązywać problemu. Musi jednak mieć strukturę, którą da się dalej rozwijać.

80.2 2. Etap modelu danych

W drugim kroku ustal, jakie dane przechowuje program:

- jakie obiekty występują w problemie,
- jakie pola mają te obiekty,
- które dane są wpisywane przez użytkownika,
- które dane są liczone przez program,
- które dane trzeba zapisać do pliku.

Przykłady:

- system zadań przechowuje tytuł, opis, status i termin zadania,
- baza pojazdów przechowuje markę, model, rocznik i numer rejestracyjny,
- parser CSV przechowuje wiersze, kolumny i wynik walidacji.

80.3 3. Etap pierwszej funkcji

Pierwsza funkcja powinna być mała i widoczna dla użytkownika. Dobre przykłady:

- dodanie jednego zadania,
- wczytanie jednego rekordu z pliku,
- wykonanie jednego działania w kalkulatorze,
- wyświetlenie pustej planszy gry,
- zapis jednego wpisu do logu.

Po tym etapie program powinien mieć pierwszy scenariusz sprawdzenia.

80.4 4. Etap wariantu minimum

Wariant minimum jest gotowy, gdy program:

- realizuje wszystkie podstawowe wymagania z opisu projektu,
- obsługuje podstawowy przypadek użycia od początku do końca,
- obsługuje co najmniej jeden błąd użytkownika,
- nie wymaga ręcznego poprawiania plików po uruchomieniu,
- ma instrukcję kompilacji i uruchomienia.

Na tym etapie nie dodawaj jeszcze rozszerzeń, jeśli podstawowy przepływ działania nie jest stabilny.

80.5 5. Etap testów i scenariuszy

Przygotuj krótką listę sprawdzeń:

- przypadek poprawny,
- przypadek błędnych danych,
- przypadek pustych danych,
- przypadek graniczny,
- ponowne uruchomienie programu po zapisie danych.

Jeśli projekt ma testy automatyczne, opisz komendę ich uruchomienia. Jeśli ma testy ręczne, zapisz dokładne kroki i oczekiwany wynik.

80.6 6. Etap dokumentacji

README.md projektu powinien odpowiedzieć na pytania:

- co robi program,
- jak go skompilować,
- jak go uruchomić,
- jakie funkcje zawiera wariant minimum,
- jakie rozszerzenia zostały dodane,
- jakie są znane ograniczenia,
- jakie pliki są najważniejsze.

Dokumentacja nie musi być długa. Musi pozwolić innej osobie uruchomić projekt bez zgadywania.

80.7 7. Etap pokazu

Przed pokazem przygotuj kolejność demonstracji:

1. Uruchomienie programu.
2. Pokaz podstawowego przypadku użycia.
3. Pokaz obsługi błędnych danych.
4. Krótkie omówienie struktury plików.
5. Wskazanie najważniejszych funkcji albo klas.
6. Pokaz testów albo ręcznych scenariuszy.
7. Omówienie ograniczeń i możliwych rozszerzeń.

Pokaz powinien zaczynać się od wariantu minimum. Rozszerzenia pokazuj dopiero po udowodnieniu, że podstawowy program działa.

80.8 8. Proponowany harmonogram

Moment pracy	Co powinno być gotowe
Start	Temat, wariant minimum, katalog, pierwsze README.md

Moment pracy	Co powinno być gotowe
Po pierwszej iteracji	Program kompiluje się i wykonuje jedną prostą akcję
Po drugiej iteracji	Istnieje model danych i część funkcji minimum
Po trzeciej iteracji	Wariant minimum działa od początku do końca
Przed oddaniem	Dokumentacja, scenariusze sprawdzenia, obsługa błędów
Pokaz	Projekt uruchamia się na czystym stanowisku

80.9 9. Sygnały ostrzegawcze

Zatrzymaj się i uprość zakres, jeśli:

- program długo się nie kompiluje,
- nie da się pokazać żadnego działającego scenariusza,
- większość czasu zajmują rozszerzenia zamiast wariantu minimum,
- struktura plików jest niezrozumiała dla autora projektu,
- dane są przechowywane w wielu miejscach bez jasnej odpowiedzialności.

W takiej sytuacji lepiej zmniejszyć zakres i doprowadzić prostszą wersję do końca niż zostawić rozbudowany, ale niedziałający projekt.

Rozdział 81

Checklista projektu zaliczeniowego

Użyj tej listy przed oddaniem projektu i podczas krótkiego pokazu działania. Lista pomaga sprawdzić, czy projekt spełnia wariant minimum, czy da się go uruchomić na czystym stanowisku i czy student potrafi wyjaśnić własne decyzje.

81.1 1. Wariant minimum

- ☐ Projekt realizuje wszystkie punkty z sekcji **Minimalny wariant zaliczeniowy**.
- ☐ Menu albo główny sposób obsługi jest zrozumiały bez czytania kodu.
- ☐ Podstawowy przypadek użycia działa od początku do końca.
- ☐ Program obsługuje co najmniej jeden przypadek błędnych danych.
- ☐ Program nie kończy się awaryjnie po typowym błędzie użytkownika.

81.2 2. Kompilacja i uruchomienie

- ☐ W README.md projektu jest komenda kompilacji.
- ☐ W README.md projektu jest komenda uruchomienia.
- ☐ Projekt kompiluje się z flagami `-Wall -Wextra -pedantic`.
- ☐ Projekt nie wymaga plików wygenerowanych wcześniej ręcznie.
- ☐ W repozytorium nie ma artefaktów build, np. `.o`, `.exe`, binariów ani katalogu `build` z wynikiem kompilacji.

81.3 3. Organizacja kodu

- ☐ Kod jest podzielony na funkcje albo metody zgodnie z odpowiedzialnościami.
- ☐ W większym projekcie funkcja `main` uruchamia program, ale nie zawiera całej logiki.
- ☐ Pliki źródłowe mają czytelne nazwy.
- ☐ Jeśli projekt ma kilka modułów, deklaracje i implementacje są rozdzielone na `.h` i `.cpp`.
- ☐ Nazwy zmiennych, funkcji, klas i plików opisują ich rolę.

81.4 4. Dane i błędy

- ☐ Program sprawdza poprawność danych wpisanych przez użytkownika.
- ☐ Operacje na plikach sprawdzają, czy odczyt albo zapis się powiódł.
- ☐ Pusty zestaw danych jest obsługiwany w przewidywalny sposób.
- ☐ Przypadki graniczne są opisane albo przetestowane.
- ☐ Komunikaty błędów pomagają użytkownikowi poprawić dane.

81.5 5. Testy i scenariusze sprawdzenia

- ☐ Projekt ma testy automatyczne albo listę ręcznych scenariuszy sprawdzenia.
- ☐ Jest scenariusz dla poprawnych danych.
- ☐ Jest scenariusz dla danych błędnych.
- ☐ Jest scenariusz dla danych granicznych, np. pusta lista, zero, brak pliku.
- ☐ Student potrafi uruchomić te scenariusze podczas odbioru projektu.

81.6 6. Dokumentacja projektu

- ☐ README.md opisuje cel projektu.
- ☐ README.md opisuje strukturę katalogów i najważniejsze pliki.
- ☐ README.md opisuje wariant minimum.
- ☐ Rozszerzenia są oddzielone od wariantu minimum.
- ☐ Ograniczenia albo znane braki są opisane uczciwie.

81.7 7. Pokaz i omówienie

- ☐ Student potrafi uruchomić projekt bez pomocy prowadzącego.
- ☐ Student potrafi wskazać najważniejsze funkcje, klasy albo moduły.
- ☐ Student potrafi wyjaśnić, gdzie program przechowuje dane.
- ☐ Student potrafi pokazać obsługę błędnego wejścia.
- ☐ Student potrafi powiedzieć, co dodałby w kolejnej wersji.

81.8 8. Rozszerzenia

- ☐ Rozszerzenia działają dopiero po stabilnym wariantie minimum.
- ☐ Rozszerzenia są opisane w dokumentacji.
- ☐ Rozszerzenia nie psują podstawowych scenariuszy.
- ☐ Dla rozszerzeń istnieje krótki scenariusz sprawdzenia.

Rozdział 82

Karta oceny projektu

Ten dokument można skopiować do projektu studenta albo użyć podczas odbioru. Karta porządkuje ocenę, ale nie zastępuje rozmowy o kodzie i pokazu działania.

82.1 Dane projektu

Pole	Wartość
Student	
Temat projektu	
Wariant	minimum / rozszerzony
Data odbioru	
Link albo ścieżka do repozytorium	

82.2 Warunek wstępny

Przed przyznaniem punktów sprawdź:

- ☐ projekt kompiluje się z instrukcji,
- ☐ da się uruchomić program,
- ☐ student potrafi wskazać główne pliki,
- ☐ student potrafi pokazać podstawowy przypadek użycia.

Jeśli projekt nie kompiluje się albo nie uruchamia, najpierw opisz poprawkę. Pełną punktację warto przyznać dopiero po działającym wariantie minimum.

82.3 Punktacja

Obszar	Maks.	Punkty	Notatka
Funkcjonalność minimum	30		
Poprawność techniczna	20		
Organizacja kodu	20		
Testy lub scenariusze sprawdzenia	15		
Dokumentacja	10		
Rozszerzenia	5		
Razem	100		

82.4 Funkcjonalność minimum

Sprawdź, czy program realizuje wymagania z sekcji **Minimalny wariant zaliczeniowy** wybranego projektu.

- ☐ podstawowy przepływ działania działa od początku do końca,
- ☐ menu albo sposób obsługi jest zrozumiały,
- ☐ program zapisuje albo przetwarza dane zgodnie z tematem,
- ☐ program obsługuje co najmniej jeden błąd użytkownika,
- ☐ brak funkcji z minimum nie jest maskowany rozszerzeniami.

82.5 Poprawność techniczna

- ☐ projekt kompiluje się bez błędów,
- ☐ projekt kompiluje się z ostrzeżeniami włączonymi przez `-Wall -Wextra -pedantic`,
- ☐ repozytorium nie zawiera plików wynikowych,
- ☐ operacje na plikach sprawdzają błędy,
- ☐ program nie kończy się awaryjnie po typowych błędnych danych.

82.6 Organizacja kodu

- ☐ kod jest podzielony na funkcje albo metody,
- ☐ większe odpowiedzialności są rozdzielone na klasy albo moduły,
- ☐ `main` nie zawiera całej logiki projektu,
- ☐ nazwy plików, funkcji i klas są czytelne,
- ☐ student potrafi uzasadnić strukturę projektu.

82.7 Testy lub scenariusze sprawdzenia

- ☐ istnieje przypadek poprawny,
- ☐ istnieje przypadek błędnych danych,
- ☐ istnieje przypadek graniczny,
- ☐ student potrafi powtórzyć scenariusze podczas odbioru,
- ☐ oczekiwane wyniki są opisane.

82.8 Dokumentacja

- ☐ `README.md` opisuje cel projektu,
- ☐ `README.md` zawiera kompilację i uruchomienie,
- ☐ dokumentacja oddziela wariant minimum od rozszerzeń,
- ☐ opisuje strukturę katalogów albo najważniejsze pliki,
- ☐ opisuje ograniczenia lub znane braki.

82.9 Rozmowa techniczna

Podczas odbioru poproś studenta o krótkie wyjaśnienie:

- gdzie program wczytuje dane,
- gdzie program zapisuje dane,
- gdzie znajduje się najważniejsza logika,
- jak obsługiwany jest błąd użytkownika,
- co student zmieniłby w kolejnej wersji.

82.10 Decyzja

Wynik	Zaznacz
Projekt zaliczony	☐
Projekt zaliczony po drobnych poprawkach	☐
Projekt wymaga ponownego odbioru	☐

82.11 Poprawki

Jeśli projekt wymaga poprawy, wpisz konkretne zadania:

- 1.
- 2.
- 3.

Poprawka powinna być sprawdzalna. Zamiast pisać popraw kod, wpisz na przykład dodaj obsługę pustego pliku wejściowego albo opisz komendę uruchomienia w README.

82.12 Notatki z odbioru

Zapisz krótko, co działało dobrze i co wymaga dalszej pracy.

Rozdział 83

Gra kółko i krzyżyk

83.1 Cel projektu

Celem projektu jest przygotowanie gry w kółko i krzyżyk na planszy 3x3, w której człowiek gra przeciwko komputerowi. Projekt ma pokazać modelowanie planszy, sprawdzanie stanu gry, obsługę wejścia użytkownika oraz prostą sztuczną inteligencję.

Najważniejsze wymaganie: komputer ma grać inteligentnie i nigdy nie przegrywać. Dla klasycznej planszy 3x3 istnieje strategia gwarantująca komputerowi co najmniej remis.

83.2 Zakres funkcjonalny

Gra powinna umożliwiać:

- rozpoczęcie nowej gry,
- wybór znaku gracza, np. X albo O,
- wyświetlenie planszy w terminalu,
- wykonanie ruchu przez gracza,
- wykonanie ruchu przez komputer,
- odrzucenie ruchu na zajęte pole,
- sprawdzenie wygranej gracza,
- sprawdzenie wygranej komputera,
- wykrycie remisu,
- rozegranie kolejnej partii bez restartu programu.

Przykładowy widok planszy:

```
1 | 2 | 3
---+---+---
4 | 5 | 6
---+---+---
7 | 8 | 9
```

83.3 Wymagania techniczne

Projekt powinien:

- być napisany w C++17,
- używać klasy do reprezentowania planszy,
- używać `enum class` do reprezentowania pól planszy,
- oddzielać logikę gry od interfejsu terminalowego,
- walidować dane wejściowe użytkownika,
- implementować strategię komputera, która nie przegrywa,
- rozdzielać deklaracje i implementacje na pliki `.h` i `.cpp`,

- kompilować się z flagami `-Wall -Wextra -pedantic`.

83.4 Proponowana struktura plików

```
tic-tac-toe/
include/tic_tac_toe/
    board.h
    game.h
    player.h
    ai_player.h
    console_view.h
src/
    board.cpp
    game.cpp
    ai_player.cpp
    console_view.cpp
    main.cpp
tests/
    board_tests.cpp
    ai_player_tests.cpp
build/
```

Znaczenie elementów:

- Board - plansza i operacje na polach,
- Game - przebieg partii i sprawdzanie stanu gry,
- Player - informacja o graczu i jego znaku,
- AiPlayer - wybór ruchu komputera,
- ConsoleView - wyświetlanie planszy i komunikatów.

83.5 Model gry

Plansza powinna mieć dziewięć pól. Każde pole może być puste albo zajęte przez jednego z graczy.

Przykład:

```
enum class Cell {
    Empty,
    X,
    O
};

enum class GameState {
    InProgress,
    XWins,
    OWins,
    Draw
};

class Board {
public:
    bool makeMove(int position, Cell value);
    Cell at(int position) const;
    GameState state() const;

private:
    std::array<Cell, 9> cells_;
};
```

Pozycje mogą być numerowane od 1 do 9, ponieważ jest to wygodne dla użytkownika. Wewnątrz programu można je przeliczać na indeksy od 0 do 8.

83.6 Strategia komputera

Minimalna wersja strategii powinna:

1. Wygrać, jeśli komputer ma ruch kończący partię.
2. Zablokować wygraną gracza, jeśli gracz ma taki ruch.
3. Zająć środek, jeśli jest wolny.
4. Zająć róg, jeśli jest wolny.
5. Zająć dowolne wolne pole.

Wersja rekomendowana powinna użyć algorytmu minimax. Dla planszy 3x3 jest to proste do policzenia, a komputer może grać optymalnie w każdej sytuacji.

83.7 Minimalny wariant zaliczeniowy

Minimalna wersja projektu powinna zawierać:

1. Menu konsolowe z opcjami:
 - nowa gra,
 - zasady,
 - zakończ.
2. Planszę 3x3 wyświetlaną po każdym ruchu.
3. Walidację numeru pola.
4. Blokowanie ruchu na zajęte pole.
5. Sprawdzanie wszystkich linii wygranej:
 - trzy wiersze,
 - trzy kolumny,
 - dwie przekątne.
6. Wykrywanie remisu.
7. Komputer, który nie przegrywa.
8. Instrukcję kompilacji i uruchomienia w README.md projektu.

83.8 Rozszerzenia dla chętnych

Możliwe rozszerzenia:

- poziomy trudności,
- gracz zaczyna albo komputer zaczyna,
- tryb dwóch graczy,
- licznik zwycięstw, porażek i remisów,
- zapis statystyk do pliku,
- testy automatyczne algorytmu komputera,
- kolorowe znaki w terminalu,
- większa plansza jako osobny wariant, np. 4x4.

83.9 Kryteria oceny

Projekt jest zaliczony, jeśli:

- kompiluje się bez błędów i ostrzeżeń,
- poprawnie wyświetla planszę,
- przyjmuje poprawne ruchy gracza,
- odrzuca niepoprawne ruchy gracza,
- poprawnie wykrywa wygraną,
- poprawnie wykrywa remis,
- komputer blokuje natychmiastową wygraną gracza,

- komputer wykorzystuje własną natychmiastową wygraną,
- komputer nigdy nie przegrywa przy poprawnej strategii,
- kod jest podzielony na moduły,
- zawiera instrukcję uruchomienia,
- zawiera testy albo opis ręcznych scenariuszy sprawdzenia.

83.10 Scenariusze sprawdzenia

1. Rozpocznij grę i wykonaj ruch na wolne pole.
2. Spróbuj wykonać ruch na zajęte pole.
3. Ułóż sytuację, w której gracz ma dwa znaki w linii, i sprawdź, czy komputer blokuje trzeci znak.
4. Ułóż sytuację, w której komputer może wygrać jednym ruchem, i sprawdź, czy wybiera ruch wygrywający.
5. Rozegraj partię optymalnie po stronie gracza i sprawdź, czy wynik to remis.
6. Rozegraj kilka partii i sprawdź, czy program pozwala zacząć od nowa.

Rozdział 84

Projekt 01 - Logger C++

84.1 Cel projektu

Celem projektu jest przygotowanie prostego loggera w C++, który zastępuje proceduralny logger zapisujący dane przez funkcje z C. Projekt ma pokazać podział odpowiedzialności na klasy, obsługę plików, formatowanie wpisów oraz testowanie podstawowych scenariuszy.

Punktem wyjścia jest stary styl zapisu:

```
int ZapiszDoLog(char *sciezka, char *tryb)
{
    FILE *fp = fopen(sciezka, "a");
    if (fp == NULL)
    {
        return -1;
    }

    fprintf(fp, "LOG: %s", tryb);
    fclose(fp);
    return 0;
}
```

Nowa wersja powinna używać idiomów C++: klas, `std::string`, strumieni, enkapsulacji i czytelnej obsługi błędów.

84.2 Zakres funkcjonalny

Logger powinien umożliwiać:

- zapis wpisu tekstowego do pliku,
- zapis wpisu tekstowego do pamięci,
- dodanie poziomu logowania, np. `Info`, `Warning`, `Error`,
- automatyczne dodanie daty i czasu do wpisu,
- odczyt wpisów zapisanych w pamięci,
- obsługę błędów otwarcia pliku,
- czytelne formatowanie wpisu.

Przykład formatu:

```
[2026-06-12 14:30:05] [INFO] System started
```

84.3 Wymagania techniczne

Projekt powinien:

- być napisany w C++17,
- używać `std::ofstream` zamiast `FILE*`,
- używać `std::string` zamiast `char*`,
- mieć interfejs wspólny dla różnych miejsc zapisu,
- rozdzielać deklaracje i implementacje na pliki `.h` i `.cpp`,
- kompilować się z flagami `-Wall -Wextra -pedantic`.

Do formatowania czasu można użyć biblioteki standardowej:

- `<chrono>`,
- `<ctime>`,
- `<iomanip>`,
- `<sstream>`.

84.4 Proponowana struktura plików

```
logger/
include/logger/
    log_level.h
    log_sink.h
    file_sink.h
    memory_sink.h
    logger.h
src/
    file_sink.cpp
    memory_sink.cpp
    logger.cpp
    main.cpp
tests/
    logger_tests.cpp
build/
```

Znaczenie elementów:

- `LogLevel` - poziom wpisu,
- `LogSink` - abstrakcyjny interfejs miejsca zapisu,
- `FileSink` - zapis do pliku,
- `MemorySink` - zapis do wektora w pamięci,
- `Logger` - klasa przyjmująca wiadomość i przekazująca ją do wybranego sinka.

Kompilowalny szkic klas znajduje się w pliku `examples/logger_skeleton.cpp`. Przykład pokazuje wspólny interfejs `LogSink`, zapis do pamięci, zapis do pliku i klasę `Logger`.

84.5 Minimalny wariant zaliczeniowy

Minimalna wersja projektu powinna zawierać:

1. `enum class LogLevel` z wartościami `Info`, `Warning`, `Error`.
2. Klasę `Logger` z metodą:

```
void log(LogLevel level, const std::string& message);
```

3. Klasę `FileSink`, która dopisuje wpisy do pliku.
4. Klasę `MemorySink`, która przechowuje wpisy w `std::vector<std::string>`.
5. Program demonstracyjny zapisujący co najmniej trzy wpisy.
6. Instrukcję kompilacji i uruchomienia w `README.md` projektu.

84.6 Rozszerzenia dla chętnych

Możliwe rozszerzenia:

- filtrowanie wpisów poniżej minimalnego poziomu,
- osobne pliki dla różnych poziomów logowania,
- rotacja pliku logu po przekroczeniu rozmiaru,
- zapis do formatu CSV,
- zapis do bazy danych jako dodatkowy sink,
- testy automatyczne dla `MemorySink`,
- konfiguracja loggera z pliku tekstowego.

84.7 Kryteria oceny

Projekt jest zaliczony, jeśli:

- kompiluje się bez błędów i ostrzeżeń,
- zapisuje wpisy do pliku,
- zapisuje wpisy do pamięci,
- używa `enum class` dla poziomu logowania,
- ma wspólny interfejs dla różnych miejsc zapisu,
- obsługuje błąd otwarcia pliku,
- ma kod podzielony na pliki,
- zawiera instrukcję uruchomienia,
- zawiera testy albo opis ręcznych scenariuszy sprawdzenia.

84.8 Scenariusze sprawdzenia

1. Uruchom program z poprawną ścieżką pliku i sprawdź, czy plik zawiera wpisy.
2. Uruchom program ze ścieżką, której nie da się utworzyć, i sprawdź obsługę błędu.
3. Zapisz trzy wpisy do `MemorySink` i wypisz je na ekranie.
4. Sprawdź, czy poziomy `Info`, `Warning` i `Error` są widoczne w wyniku.
5. Sprawdź, czy każdy wpis ma datę i czas.

Rozdział 85

Projekt 02 - Kolejowanie zamówień w sklepie

85.1 Cel projektu

Celem projektu jest przygotowanie konsolowego systemu do obsługi kolejki zamówień w sklepie. Projekt ma pokazać praktyczne użycie kontenerów STL, klas, plików, walidacji danych oraz prostego zapisu stanu programu.

System powinien umożliwiać dodawanie zamówień, pobieranie ich w kolejności przyjęcia oraz zapis i odczyt kolejki między uruchomieniami programu.

85.2 Zakres funkcjonalny

System powinien umożliwiać:

- dodanie nowego zamówienia,
- pobranie najstarszego zamówienia z kolejki,
- wyświetlenie wszystkich oczekujących zamówień,
- wyszukanie zamówienia po identyfikatorze,
- zmianę statusu zamówienia,
- zapis kolejki do pliku,
- odczyt kolejki z pliku przy starcie programu,
- obsługę pustej kolejki i błędnych danych wejściowych.

Przykładowe statusy:

- `New`,
- `InProgress`,
- `Completed`,
- `Cancelled`.

85.3 Wymagania techniczne

Projekt powinien:

- być napisany w C++17,
- używać klas do modelowania zamówienia i systemu kolejowania,
- używać `std::queue` albo `std::deque` do obsługi kolejności FIFO,
- używać `std::map` albo `std::unordered_map` do szybkiego wyszukiwania po ID,
- używać `enum class` dla statusu zamówienia,
- zapisywać dane do pliku tekstowego, np. CSV,
- rozdzielać deklaracje i implementacje na pliki `.h` i `.cpp`,
- kompilować się z flagami `-Wall -Wextra -pedantic`.

Baza danych SQL albo NoSQL może być rozszerzeniem, ale minimalny wariant powinien działać bez zewnętrznych zależności.

85.4 Proponowana struktura plików

```
order-queue/  
  include/order_queue/  
    order.h  
    order_status.h  
    order_queue.h  
    order_storage.h  
  src/  
    order.cpp  
    order_queue.cpp  
    order_storage.cpp  
    main.cpp  
  tests/  
    order_queue_tests.cpp  
  data/  
    orders.csv  
  build/
```

Znaczenie elementów:

- `Order` - pojedyncze zamówienie,
- `OrderStatus` - status zamówienia,
- `OrderQueue` - logika dodawania i pobierania zamówień,
- `OrderStorage` - zapis i odczyt zamówień z pliku.

85.5 Model danych

Zamówienie powinno mieć co najmniej:

- `id`,
- `customerName`,
- `createdAt`,
- `totalAmount`,
- `status`.

Przykład:

```
struct Order {  
    std::string id;  
    std::string customerName;  
    double totalAmount;  
    OrderStatus status;  
};
```

85.6 Minimalny wariant zaliczeniowy

Minimalna wersja projektu powinna zawierać:

1. Menu konsolowe z opcjami:
 - dodaj zamówienie,
 - pobierz następne zamówienie,
 - wyświetl kolejkę,
 - zapisz dane,
 - wczytaj dane,
 - zakończ.
2. Klasę `OrderQueue`, która pilnuje kolejności FIFO.

3. Walidację pustej kolejki.
4. Zapis i odczyt danych z pliku CSV.
5. Przykładowy plik z danymi.
6. Instrukcję kompilacji i uruchomienia w `README.md` projektu.

85.7 Rozszerzenia dla chętnych

Możliwe rozszerzenia:

- filtrowanie zamówień po statusie,
- priorytety zamówień,
- historia zrealizowanych zamówień,
- zapis do SQLite,
- eksport raportu dziennego,
- testy automatyczne dla `OrderQueue`,
- osobny tryb demonstracyjny z przykładowymi zamówieniami,
- obsługa anulowania zamówienia po ID.

85.8 Kryteria oceny

Projekt jest zaliczony, jeśli:

- kompiluje się bez błędów i ostrzeżeń,
- poprawnie dodaje zamówienia,
- pobiera zamówienia w kolejności FIFO,
- obsługuje pustą kolejkę,
- zapisuje i odczytuje dane z pliku,
- używa klas i kontenerów STL,
- ma czytelne menu konsolowe,
- zawiera instrukcję uruchomienia,
- zawiera testy albo opis ręcznych scenariuszy sprawdzenia.

85.9 Scenariusze sprawdzenia

1. Dodaj trzy zamówienia i sprawdź, czy są pobierane w tej samej kolejności.
2. Spróbuj pobrać zamówienie z pustej kolejki i sprawdź komunikat błędu.
3. Zapisz kolejkę do pliku, uruchom program ponownie i wczytaj dane.
4. Wyszukaj zamówienie po poprawnym i niepoprawnym ID.
5. Zmień status zamówienia i sprawdź, czy zmiana jest widoczna po zapisie i odczycie.

Rozdział 86

Projekt 04 - Tetris

86.1 Cel projektu

Celem projektu jest przygotowanie gry Tetris działającej w terminalu. Projekt ma pokazać reprezentację planszy, modelowanie klocków, obsługę kolizji, pętlę gry, punktację oraz rozdzielenie logiki gry od wyświetlania.

Minimalna wersja może działać w terminalu tekstowym. Określenie “terminal graficzny” można potraktować jako czytelny interfejs z ramką planszy, kolorami ANSI albo prostymi znakami tekstowymi.

86.2 Zakres funkcjonalny

Gra powinna umożliwiać:

- rozpoczęcie nowej gry,
- wyświetlenie planszy,
- generowanie kolejnych klocków,
- przesuwanie klocka w lewo i prawo,
- obracanie klocka,
- przyspieszenie opadania klocka,
- wykrywanie kolizji ze ścianami i innymi klockami,
- zatrzymanie klocka po dotknięciu dna albo bloków,
- usuwanie pełnych linii,
- naliczanie punktów,
- zakończenie gry po wypełnieniu górnej części planszy.

Przykładowy widok planszy:

```
+-----+
|       |
|      □ |
|  □ □ □ |
|       |
| □ □ □ □ |
+-----+
Wynik: 1200
```

86.3 Wymagania techniczne

Projekt powinien:

- być napisany w C++17,
- używać klasy do reprezentowania planszy,
- używać klasy albo struktury do reprezentowania klocka,

- używać `enum class` do typów klocków,
- oddzielać logikę gry od interfejsu terminalowego,
- obsługiwać wejście użytkownika w pętli gry,
- walidować ruchy i obroty klocka przed ich wykonaniem,
- używać generatora liczb losowych z biblioteki standardowej,
- rozdzielać deklaracje i implementacje na pliki `.h` i `.cpp`,
- kompilować się z flagami `-Wall -Wextra -pedantic`.

86.4 Proponowana struktura plików

```
tetris/
  include/tetris/
    board.h
    tetromino.h
    game.h
    score.h
    console_view.h
    input.h
  src/
    board.cpp
    tetromino.cpp
    game.cpp
    score.cpp
    console_view.cpp
    input.cpp
    main.cpp
  tests/
    board_tests.cpp
    tetromino_tests.cpp
  build/
```

Znaczenie elementów:

- Board - plansza i zajęte pola,
- Tetromino - aktualny klocek i jego obroty,
- Game - pętla gry i reguły,
- Score - naliczanie punktów,
- ConsoleView - wyświetlanie planszy,
- Input - obsługa klawiszy albo komend gracza.

86.5 Model gry

Plansza powinna mieć stały rozmiar, np. 10x20. Każde pole może być puste albo zajęte przez zatrzymany klocek.

Przykład:

```
enum class TetrominoType {
    I,
    O,
    T,
    S,
    Z,
    J,
    L
};

struct Position {
    int row;
```

```

    int column;
};

class Board {
public:
    bool isInside(Position position) const;
    bool isOccupied(Position position) const;
    bool canPlace(const Tetromino& piece) const;
    int clearFullLines();

private:
    std::vector<std::vector<bool>> cells_;
};

```

Kłoczek można reprezentować jako zestaw czterech pozycji względem punktu odniesienia. Obrót zmienia te pozycje, ale powinien być zaakceptowany tylko wtedy, gdy nowy układ mieści się na planszy i nie nachodzi na zajęte pola.

86.6 Pętla gry

Minimalna pętla gry powinna wykonywać powtarzalnie:

1. Wyświetlenie planszy.
2. Odczyt komendy gracza.
3. Próbę przesunięcia albo obrotu klocka.
4. Przesunięcie klocka o jeden wiersz w dół.
5. Zatrzymanie klocka, jeśli nie może opaść niżej.
6. Usunięcie pełnych linii.
7. Wygenerowanie nowego klocka.
8. Sprawdzenie warunku końca gry.

W prostej wersji można sterować komendami tekstowymi, np. **a**, **d**, **w**, **s**, zamiast obsługiwać natychmiastowy odczyt klawiszy.

86.7 Minimalny wariant zaliczeniowy

Minimalna wersja projektu powinna zawierać:

1. Menu konsolowe z opcjami:
 - nowa gra,
 - zasady,
 - zakończ.
2. Planszę o wymiarach co najmniej 10x20.
3. Co najmniej cztery typy klocków.
4. Ruch w lewo, ruch w prawo i opadanie.
5. Obrót klocka.
6. Blokowanie ruchu poza planszę.
7. Blokowanie ruchu przez zajęte pola.
8. Usuwanie pełnej linii.
9. Punktację.
10. Warunek końca gry.
11. Instrukcję kompilacji i uruchomienia w `README.md` projektu.

86.8 Rozszerzenia dla chętnych

Możliwe rozszerzenia:

- wszystkie siedem klasycznych klocków,
- podgląd następnego klocka,

- poziomy trudności,
- rosnąca prędkość opadania,
- zapis najlepszego wyniku do pliku,
- pauza,
- kolory ANSI,
- testy automatyczne kolizji i usuwania linii,
- tryb demonstracyjny z ustalonymi klockami,
- obsługa natychmiastowego odczytu klawiszy.

86.9 Kryteria oceny

Projekt jest zaliczony, jeśli:

- kompiluje się bez błędów i ostrzeżeń,
- poprawnie wyświetla planszę,
- poprawnie generuje klocki,
- pozwala przesuwać i obracać klocek,
- nie pozwala wyjść klockiem poza planszę,
- nie pozwala przejść przez zajęte pola,
- zatrzymuje klocek po kolizji,
- usuwa pełne linie,
- nalicza punkty,
- kończy grę po wypełnieniu planszy,
- kod jest podzielony na moduły,
- zawiera instrukcję uruchomienia,
- zawiera testy albo opis ręcznych scenariuszy sprawdzenia.

86.10 Scenariusze sprawdzenia

1. Uruchom nową grę i sprawdź, czy pojawia się plansza oraz pierwszy klocek.
2. Przesuń klocek w lewo i prawo.
3. Spróbuj przesunąć klocek poza lewą albo prawą ścianę.
4. Obróć klocek przy wolnej przestrzeni.
5. Spróbuj obrócić klocek przy ścianie albo zajętych polach.
6. Ułóż pełną linię i sprawdź, czy zostaje usunięta.
7. Doprowadź do wypełnienia górnej części planszy i sprawdź koniec gry.

Rozdział 87

Projekt 05 - Snake

87.1 Cel projektu

Celem projektu jest przygotowanie gry Snake działającej w terminalu. Projekt ma pokazać reprezentację planszy, modelowanie ruchu w czasie, obsługę kolizji, generowanie losowych elementów, punktację oraz rozdzielenie logiki gry od wyświetlania.

Minimalna wersja może działać w terminalu tekstowym. Określenie “terminal graficzny” można potraktować jako czytelny interfejs z ramką planszy, znakami ASCII i opcjonalnymi kolorami ANSI.

87.2 Zakres funkcjonalny

Gra powinna umożliwiać:

- rozpoczęcie nowej gry,
- wyświetlenie planszy,
- sterowanie kierunkiem ruchu węża,
- automatyczny ruch węża w pętli gry,
- generowanie jedzenia w losowym wolnym polu,
- zwiększenie długości węża po zjedzeniu jedzenia,
- naliczanie punktów,
- wykrywanie kolizji ze ścianą,
- wykrywanie kolizji węża z samym sobą,
- zakończenie gry po kolizji,
- rozpoczęcie kolejnej gry bez restartu programu.

Przykładowy widok planszy:

```
+-----+
|       |
|   @*** |
|       * |
|  F  *  |
|       |
+-----+
Wynik: 40
```

87.3 Wymagania techniczne

Projekt powinien:

- być napisany w C++17,
- używać klasy do reprezentowania planszy,
- używać klasy albo struktury do reprezentowania węża,

- używać `enum class` do kierunku ruchu,
- oddzielać logikę gry od interfejsu terminalowego,
- walidować zmianę kierunku, aby nie zawrócić bezpośrednio w siebie,
- używać generatora liczb losowych z biblioteki standardowej,
- rozdzielać deklaracje i implementacje na pliki `.h` i `.cpp`,
- kompilować się z flagami `-Wall -Wextra -pedantic`.

87.4 Proponowana struktura plików

```
snake/
  include/snake/
    board.h
    snake.h
    food.h
    game.h
    score.h
    console_view.h
    input.h
  src/
    board.cpp
    snake.cpp
    food.cpp
    game.cpp
    score.cpp
    console_view.cpp
    input.cpp
    main.cpp
  tests/
    snake_tests.cpp
    board_tests.cpp
  build/
```

Znaczenie elementów:

- Board - rozmiar planszy i sprawdzanie granic,
- Snake - pozycje segmentów węża i zmiana kierunku,
- Food - pozycja jedzenia,
- Game - pętla gry i reguły,
- Score - naliczanie punktów,
- ConsoleView - wyświetlanie planszy,
- Input - obsługa komend gracza.

87.5 Model gry

Wąż może być reprezentowany jako lista pozycji. Pierwszy element listy to głowa, a kolejne elementy to segmenty ciała.

Przykład:

```
enum class Direction {
    Up,
    Down,
    Left,
    Right
};

struct Position {
    int row;
    int column;
```

```
};

class Snake {
public:
    Position head() const;
    void changeDirection(Direction direction);
    void move(bool grow);
    bool contains(Position position) const;

private:
    std::deque<Position> segments_;
    Direction direction_;
};
```

Do przechowywania segmentów dobrze pasuje `std::deque`, ponieważ w każdym kroku dodajemy nową głowę i zwykle usuwamy ostatni segment ogona.

87.6 Pętla gry

Minimalna pętla gry powinna wykonywać powtarzalnie:

1. Wyświetlenie planszy.
2. Odczyt komendy gracza.
3. Zmianę kierunku, jeśli jest poprawna.
4. Wyliczenie nowej pozycji głowy.
5. Sprawdzenie kolizji.
6. Sprawdzenie, czy wąż zjadł jedzenie.
7. Przesunięcie węża i ewentualne zwiększenie długości.
8. Wygenerowanie nowego jedzenia, jeśli poprzednie zostało zjedzone.
9. Aktualizację wyniku.

W prostej wersji można sterować komendami tekstowymi, np. **w**, **a**, **s**, **d**, zamiast obsługiwać natychmiastowy odczyt klawiszy.

87.7 Minimalny wariant zaliczeniowy

Minimalna wersja projektu powinna zawierać:

1. Menu konsolowe z opcjami:
 - nowa gra,
 - zasady,
 - zakończ.
2. Planszę o wymiarach co najmniej 20x10.
3. Węża złożonego z co najmniej trzech segmentów.
4. Sterowanie w czterech kierunkach.
5. Blokadę bezpośredniego zawracania.
6. Losowe generowanie jedzenia.
7. Wzrost węża po zjedzeniu jedzenia.
8. Kolizję ze ścianą.
9. Kolizję z własnym ciałem.
10. Punktację.
11. Warunek końca gry.
12. Instrukcję kompilacji i uruchomienia w `README.md` projektu.

87.8 Rozszerzenia dla chętnych

Możliwe rozszerzenia:

- rosnąca prędkość gry,

- zapis najlepszego wyniku do pliku,
- pauza,
- przeszkody na planszy,
- przechodzenie przez krawędzie planszy jako osobny tryb,
- kolory ANSI,
- poziomy trudności,
- testy automatyczne ruchu i kolizji,
- tryb demonstracyjny z ustalonymi pozycjami jedzenia,
- obsługa natychmiastowego odczytu klawiszy.

87.9 Kryteria oceny

Projekt jest zaliczony, jeśli:

- kompiluje się bez błędów i ostrzeżeń,
- poprawnie wyświetla planszę,
- poprawnie przesuwa węża,
- poprawnie zmienia kierunek ruchu,
- blokuje bezpośrednie zawracanie,
- generuje jedzenie na wolnym polu,
- zwiększa długość węża po zjedzeniu jedzenia,
- nalicza punkty,
- wykrywa kolizję ze ścianą,
- wykrywa kolizję z własnym ciałem,
- kończy grę po kolizji,
- kod jest podzielony na moduły,
- zawiera instrukcję uruchomienia,
- zawiera testy albo opis ręcznych scenariuszy sprawdzenia.

87.10 Scenariusze sprawdzenia

1. Uruchom nową grę i sprawdź, czy pojawia się plansza, wąż i jedzenie.
2. Zmień kierunek ruchu węża we wszystkich czterech kierunkach.
3. Spróbuj zawrócić bezpośrednio w przeciwną stronę.
4. Doprowadź węża do jedzenia i sprawdź wzrost długości oraz wynik.
5. Doprowadź węża do ściany i sprawdź koniec gry.
6. Doprowadź do kolizji z własnym ciałem i sprawdź koniec gry.
7. Rozpocznij nową grę po zakończeniu poprzedniej.

Rozdział 88

Projekt 06 - Pac-Man

88.1 Cel projektu

Celem projektu jest przygotowanie uproszczonej gry Pac-Man działającej w terminalu. Projekt ma pokazać reprezentację labiryntu, ruch postaci, zbieranie punktów, obsługę przeciwników, kolizje, punktację oraz rozdzielenie logiki gry od wyświetlania.

Minimalna wersja może działać w terminalu tekstowym. Określenie “terminal graficzny” można potraktować jako czytelny interfejs z labiryntem złożonym ze znaków ASCII i opcjonalnymi kolorami ANSI.

88.2 Zakres funkcjonalny

Gra powinna umożliwiać:

- rozpoczęcie nowej gry,
- wyświetlenie labiryntu,
- sterowanie graczem w czterech kierunkach,
- blokowanie przejścia przez ściany,
- zbieranie punktów z planszy,
- naliczanie wyniku,
- poruszanie przeciwników,
- wykrywanie kolizji z przeciwnikiem,
- wykrywanie wygranej po zebraniu wszystkich punktów,
- wykrywanie przegranej po utracie życia,
- rozpoczęcie kolejnej gry bez restartu programu.

Przykładowy widok labiryntu:

```
#####  
#P...#.....G#  
#.#.#.#####...#  
#...#.....#  
#####  
Wynik: 80  Życia: 3
```

88.3 Wymagania techniczne

Projekt powinien:

- być napisany w C++17,
- używać klasy do reprezentowania labiryntu,
- używać klasy albo struktury do reprezentowania postaci,
- używać `enum class` do typów pól planszy,
- oddzielać logikę gry od interfejsu terminalowego,

- walidować ruchy przed ich wykonaniem,
- mieć prostą strategię ruchu przeciwników,
- rozdzielać deklaracje i implementacje na pliki `.h` i `.cpp`,
- kompilować się z flagami `-Wall -Wextra -pedantic`.

88.4 Proponowana struktura plików

```
pacman/
  include/pacman/
    maze.h
    character.h
    player.h
    ghost.h
    game.h
    score.h
    console_view.h
    input.h
  src/
    maze.cpp
    character.cpp
    player.cpp
    ghost.cpp
    game.cpp
    score.cpp
    console_view.cpp
    input.cpp
    main.cpp
  tests/
    maze_tests.cpp
    game_tests.cpp
  data/
    level-01.txt
  build/
```

Znaczenie elementów:

- **Maze** - labirynt, ściany, punkty i puste pola,
- **Character** - wspólne dane postaci na planszy,
- **Player** - gracz sterowany z klawiatury,
- **Ghost** - przeciwnik poruszany przez program,
- **Game** - pętla gry i reguły,
- **Score** - naliczanie punktów,
- **ConsoleView** - wyświetlanie labiryntu,
- **Input** - obsługa komend gracza.

88.5 Model gry

Labirynt może być wczytywany z pliku tekstowego albo zdefiniowany w kodzie. Każdy znak oznacza inny typ pola.

Przykładowe znaczenie znaków:

- **#** - ściana,
- **.** - punkt do zebrania,
- **P** - pozycja startowa gracza,
- **G** - pozycja startowa przeciwnika,
- spacja - puste pole.

Przykład:

```

enum class Tile {
    Wall,
    Empty,
    Dot
};

struct Position {
    int row;
    int column;
};

class Maze {
public:
    bool isInside(Position position) const;
    bool isWall(Position position) const;
    bool hasDot(Position position) const;
    void collectDot(Position position);
    int remainingDots() const;

private:
    std::vector<std::vector<Tile>> tiles_;
};

```

88.6 Ruch przeciwników

Minimalna wersja może używać prostego ruchu przeciwników:

1. Przeciwnik próbuje iść w obecnym kierunku.
2. Jeśli trafia na ścianę, wybiera losowy dostępny kierunek.
3. Jeśli wejdzie na pole gracza, gracz traci życie albo gra się kończy.

Wersja rozszerzona może dodać prostą pogoń za graczem, np. wybór ruchu, który zmniejsza odległość Manhattan od gracza.

88.7 Pętla gry

Minimalna pętla gry powinna wykonywać powtarzalnie:

1. Wyświetlenie labiryntu.
2. Odczyt komendy gracza.
3. Próbę ruchu gracza.
4. Zebranie punktu, jeśli gracz wszedł na pole z punktem.
5. Ruch przeciwników.
6. Sprawdzenie kolizji z przeciwnikiem.
7. Sprawdzenie warunku wygranej.
8. Aktualizację wyniku i liczby żyć.

W prostej wersji można sterować komendami tekstowymi, np. **w**, **a**, **s**, **d**, zamiast obsługiwać natychmiastowy odczyt klawiszy.

88.8 Minimalny wariant zaliczeniowy

Minimalna wersja projektu powinna zawierać:

1. Menu konsolowe z opcjami:
 - nowa gra,
 - zasady,
 - zakończ.
2. Labirynt o wymiarach co najmniej 15x8.

3. Gracza poruszającego się w czterech kierunkach.
4. Blokowanie ruchu przez ściany.
5. Punkty do zebrania.
6. Co najmniej jednego przeciwnika.
7. Prosty ruch przeciwnika.
8. Kolizję z przeciwnikiem.
9. Punktację.
10. Warunek wygranej po zebraniu wszystkich punktów.
11. Warunek przegranej po kolizji albo utracie życia.
12. Instrukcję kompilacji i uruchomienia w `README.md` projektu.

88.9 Rozszerzenia dla chętnych

Możliwe rozszerzenia:

- kilka poziomów wczytywanych z plików,
- kilku przeciwników,
- różne strategie przeciwników,
- życia gracza,
- bonusowe punkty,
- czasowa możliwość zjadania przeciwników,
- zapis najlepszego wyniku do pliku,
- kolory ANSI,
- testy automatyczne ruchu i kolizji,
- prosty edytor poziomów w pliku tekstowym.

88.10 Kryteria oceny

Projekt jest zaliczony, jeśli:

- kompiluje się bez błędów i ostrzeżeń,
- poprawnie wyświetla labirynt,
- poprawnie porusza graczem,
- blokuje przechodzenie przez ściany,
- poprawnie zbiera punkty,
- nalicza wynik,
- porusza przeciwnikiem,
- wykrywa kolizję z przeciwnikiem,
- wykrywa wygraną po zebraniu punktów,
- wykrywa przegraną,
- kod jest podzielony na moduły,
- zawiera instrukcję uruchomienia,
- zawiera testy albo opis ręcznych scenariuszy sprawdzenia.

88.11 Scenariusze sprawdzenia

1. Uruchom nową grę i sprawdź, czy pojawia się labirynt, gracz i przeciwnik.
2. Spróbuj przejść graczem przez ścianę.
3. Przejdź przez kilka pól z punktami i sprawdź wynik.
4. Doprowadź do kolizji z przeciwnikiem i sprawdź przegraną albo utratę życia.
5. Zbierz wszystkie punkty i sprawdź warunek wygranej.
6. Rozpocznij nową grę po zakończeniu poprzedniej.

Rozdział 89

Projekt 07 - Baza danych pojazdów

89.1 Cel projektu

Celem projektu jest przygotowanie konsolowej aplikacji do zarządzania bazą pojazdów. Projekt ma pokazać modelowanie danych za pomocą klas, operacje CRUD, wyszukiwanie, walidację danych oraz zapis i odczyt bazy z pliku.

Minimalna wersja nie wymaga prawdziwej bazy danych. Wystarczy zapis do pliku tekstowego, np. CSV. Baza SQL albo NoSQL może być rozszerzeniem projektu.

89.2 Zakres funkcjonalny

System powinien umożliwiać:

- dodanie pojazdu,
- edycję danych pojazdu,
- usunięcie pojazdu,
- wyświetlenie danych pojedynczego pojazdu,
- wyświetlenie listy wszystkich pojazdów,
- wyszukiwanie po numerze rejestracyjnym,
- filtrowanie po typie pojazdu albo marce,
- zapis bazy do pliku,
- odczyt bazy z pliku przy starcie programu,
- obsługę błędnych danych wejściowych.

89.3 Wymagania techniczne

Projekt powinien:

- być napisany w C++17,
- używać klas do modelowania pojazdu i bazy pojazdów,
- używać `std::vector` do przechowywania pojazdów,
- używać `std::map` albo `std::unordered_map` jako indeksu po numerze rejestracyjnym,
- używać `enum class` dla typu pojazdu,
- zapisywać dane do pliku tekstowego, np. CSV,
- rozdzielać deklaracje i implementacje na pliki `.h` i `.cpp`,
- kompilować się z flagami `-Wall -Wextra -pedantic`.

89.4 Proponowana struktura plików

```
vehicle-database/  
  include/vehicle_database/
```

```

    vehicle.h
    vehicle_type.h
    vehicle_database.h
    vehicle_storage.h
src/
    vehicle.cpp
    vehicle_database.cpp
    vehicle_storage.cpp
    main.cpp
tests/
    vehicle_database_tests.cpp
data/
    vehicles.csv
build/

```

Znaczenie elementów:

- **Vehicle** - dane pojedynczego pojazdu,
- **VehicleType** - typ pojazdu,
- **VehicleDatabase** - operacje dodawania, edycji, usuwania i wyszukiwania,
- **VehicleStorage** - zapis i odczyt z pliku.

89.5 Model danych

Pojazd powinien mieć co najmniej:

- `registrationNumber`,
- `brand`,
- `model`,
- `productionYear`,
- `type`,
- `ownerName`.

Przykład:

```

enum class VehicleType {
    Car,
    Motorcycle,
    Truck,
    Bus
};

struct Vehicle {
    std::string registrationNumber;
    std::string brand;
    std::string model;
    int productionYear;
    VehicleType type;
    std::string ownerName;
};

```

89.6 Minimalny wariant zaliczeniowy

Minimalna wersja projektu powinna zawierać:

1. Menu konsolowe z opcjami:
 - dodaj pojazd,
 - edytuj pojazd,
 - usuń pojazd,
 - wyszukaj pojazd,

- wyświetl wszystkie pojazdy,
 - zapisz dane,
 - wczytaj dane,
 - zakończ.
2. Walidację unikalnego numeru rejestracyjnego.
 3. Walidację roku produkcji.
 4. Zapis i odczyt danych z pliku CSV.
 5. Przykładowy plik z danymi.
 6. Instrukcję kompilacji i uruchomienia w `README.md` projektu.

89.7 Rozszerzenia dla chętnych

Możliwe rozszerzenia:

- filtrowanie po właścicielu,
- sortowanie po marce, roczniku albo typie,
- historia zmian danych pojazdu,
- eksport raportu do pliku tekstowego,
- zapis do SQLite,
- import danych z innego pliku CSV,
- testy automatyczne dla `VehicleDatabase`,
- obsługa przeglądów technicznych i dat ubezpieczenia.

89.8 Kryteria oceny

Projekt jest zaliczony, jeśli:

- kompiluje się bez błędów i ostrzeżeń,
- poprawnie dodaje pojazdy,
- odrzuca duplikaty numeru rejestracyjnego,
- umożliwia edycję i usuwanie pojazdów,
- wyszukuje pojazd po numerze rejestracyjnym,
- zapisuje i odczytuje dane z pliku,
- używa klas i kontenerów STL,
- ma czytelne menu konsolowe,
- zawiera instrukcję uruchomienia,
- zawiera testy albo opis ręcznych scenariuszy sprawdzenia.

89.9 Scenariusze sprawdzenia

1. Dodaj trzy pojazdy i wyświetl całą bazę.
2. Spróbuj dodać drugi pojazd z tym samym numerem rejestracyjnym.
3. Edytuj właściciela wybranego pojazdu i sprawdź wynik wyszukiwania.
4. Usuń pojazd i sprawdź, czy nie pojawia się na liście.
5. Zapisz bazę do pliku, uruchom program ponownie i wczytaj dane.

Rozdział 90

Projekt 08 - Kalkulator

90.1 Cel projektu

Celem projektu jest przygotowanie kalkulatora działającego w terminalu. Projekt ma pokazać parsowanie tekstu, obsługę błędów, priorytety operatorów, nawiasy, funkcje matematyczne oraz podział programu na mniejsze moduły.

Minimalna wersja może działać jako zwykła aplikacja konsolowa. Określenie “terminal graficzny” można potraktować jako czytelny interfejs tekstowy: menu, historia obliczeń, wyróżnione komunikaty i uporządkowane wyniki.

90.2 Zakres funkcjonalny

Kalkulator powinien umożliwiać:

- wpisanie działania jako jednego wyrażenia tekstowego,
- obliczenie wyniku,
- obsługę działań $+$, $-$, $*$, $/$,
- obsługę nawiasów okrągłych,
- poprawną kolejność wykonywania działań,
- obsługę liczb zmiennoprzecinkowych,
- obsługę funkcji trygonometrycznych,
- wyświetlenie historii obliczeń,
- zapis historii do pliku,
- obsługę błędnych wyrażeń.

Przykładowe wyrażenia:

```
2 + 3 * 4
(2 + 3) * 4
sin(0)
cos(3.14159)
sqrt(16) + 2
```

90.3 Wymagania techniczne

Projekt powinien:

- być napisany w C++17,
- rozdzielać logikę kalkulatora od interfejsu użytkownika,
- mieć osobny moduł tokenizacji wyrażenia,
- mieć osobny moduł parsowania albo obliczania wyrażenia,
- obsługiwać błędy bez przerywania programu,
- używać wyjątków albo typu wyniku do zgłaszania błędów,

- używać `std::vector` do przechowywania tokenów i historii,
- używać funkcji z `<cmath>` dla operacji matematycznych,
- rozdzielać deklaracje i implementacje na pliki `.h` i `.cpp`,
- kompilować się z flagami `-Wall -Wextra -pedantic`.

90.4 Proponowana struktura plików

```
calculator/
  include/calculator/
    token.h
    tokenizer.h
    parser.h
    calculator.h
    history.h
  src/
    tokenizer.cpp
    parser.cpp
    calculator.cpp
    history.cpp
    main.cpp
  tests/
    calculator_tests.cpp
  data/
    history.txt
  build/
```

Znaczenie elementów:

- **Token** - reprezentacja liczby, operatora, nawiasu albo funkcji,
- **Tokenizer** - zamiana tekstu wejściowego na listę tokenów,
- **Parser** - sprawdzanie składni i obliczanie wartości wyrażenia,
- **Calculator** - główna logika kalkulatora,
- **History** - historia wykonanych obliczeń.

90.5 Obsługiwane operacje

Minimalny zestaw operacji:

- dodawanie: $a + b$,
- odejmowanie: $a - b$,
- mnożenie: $a * b$,
- dzielenie: a / b ,
- nawiasy: $(a + b) * c$,
- liczby ujemne: $-5 + 2$.

Funkcje matematyczne:

- `sin(x)`,
- `cos(x)`,
- `tan(x)`,
- `sqrt(x)`,
- `abs(x)`,
- `pow(a, b)` jako rozszerzenie.

Na początku warto przyjąć, że funkcje trygonometryczne używają radianów. Obsługa stopni może być rozszerzeniem projektu.

90.6 Algorytm obliczania

Do obliczania wyrażeń można użyć jednego z dwóch podejść:

1. Algorytm Shunting Yard:
 - zamiana wyrażenia na odwrotną notację polską,
 - obliczenie wyniku ze stosu.
2. Parser rekurencyjny:
 - osobne funkcje dla wyrażeń, składników i czynników,
 - naturalna obsługa priorytetów operatorów.

Dla tego projektu rekomendowany jest parser rekurencyjny, ponieważ łatwiej pokazać w kodzie kolejność działań i obsługę nawiasów.

90.7 Minimalny wariant zaliczeniowy

Minimalna wersja projektu powinna zawierać:

1. Menu konsolowe z opcjami:
 - oblicz wyrażenie,
 - pokaż historię,
 - zapisz historię,
 - wyczyść historię,
 - zakończ.
2. Obsługę operatorów $+$, $-$, $*$, $/$.
3. Obsługę nawiasów.
4. Obsługę co najmniej trzech funkcji matematycznych.
5. Komunikaty dla błędów:
 - dzielenie przez zero,
 - brakujący nawias,
 - nieznany znak,
 - puste wyrażenie.
6. Testy ręczne albo automatyczne dla przykładowych wyrażeń.
7. Instrukcję kompilacji i uruchomienia w `README.md` projektu.

90.8 Rozszerzenia dla chętnych

Możliwe rozszerzenia:

- zmienne, np. `x = 5`,
- stałe `pi` i `e`,
- tryb stopni i radianów,
- funkcja `pow(a, b)`,
- eksport historii do CSV,
- kolorowanie komunikatów w terminalu,
- testy automatyczne parsera,
- prosty tryb interaktywny REPL,
- obsługa skrótów poleceń, np. `:history`, `:clear`, `:quit`.

90.9 Kryteria oceny

Projekt jest zaliczony, jeśli:

- kompiluje się bez błędów i ostrzeżeń,
- poprawnie obsługuje priorytety operatorów,
- poprawnie obsługuje nawiasy,
- poprawnie liczy funkcje matematyczne,
- wykrywa błędne wyrażenia,
- nie kończy programu po błędnym wejściu,
- przechowuje historię obliczeń,
- zapisuje historię do pliku,
- ma czytelne menu konsolowe,
- zawiera instrukcję uruchomienia,

- zawiera testy albo opis ręcznych scenariuszy sprawdzenia.

90.10 Scenariusze sprawdzenia

1. Oblicz $2 + 3 * 4$ i sprawdź, czy wynik to 14.
2. Oblicz $(2 + 3) * 4$ i sprawdź, czy wynik to 20.
3. Oblicz $\sin(0)$ i sprawdź, czy wynik to 0.
4. Spróbuj obliczyć $10 / 0$ i sprawdź komunikat błędu.
5. Spróbuj obliczyć $(2 + 3$ i sprawdź komunikat o nawiasie.
6. Wykonaj kilka obliczeń, pokaż historię i zapisz ją do pliku.

Rozdział 91

Projekt 09 - Kalkulator macierzy

91.1 Cel projektu

Celem projektu jest przygotowanie kalkulatora macierzy działającego w terminalu. Projekt ma pokazać pracę z tablicami dwuwymiarowymi, klasami, walidacją danych, przeciążaniem operatorów oraz obsługą błędów matematycznych.

Minimalna wersja może działać jako zwykła aplikacja konsolowa. Określenie “terminal graficzny” można potraktować jako czytelny interfejs tekstowy: tabele, menu, historia operacji i uporządkowane komunikaty.

91.2 Zakres funkcjonalny

Kalkulator powinien umożliwiać:

- utworzenie macierzy o podanych wymiarach,
- ręczne wpisanie elementów macierzy,
- wyświetlenie macierzy w czytelnej formie,
- dodawanie macierzy,
- odejmowanie macierzy,
- mnożenie macierzy przez liczbę,
- mnożenie dwóch macierzy,
- transpozycję macierzy,
- obliczenie wyznacznika dla macierzy kwadratowej,
- zapis macierzy do pliku,
- odczyt macierzy z pliku,
- obsługę błędnych wymiarów i danych wejściowych.

Przykładowe operacje:

```
A + B
A - B
2 * A
A * B
transpose(A)
det(A)
```

91.3 Wymagania techniczne

Projekt powinien:

- być napisany w C++17,
- używać klasy `Matrix` do przechowywania danych macierzy,
- używać `std::vector<std::vector<double>>` albo jednowymiarowego `std::vector<double>` z przeliczaniem indeksów,

- walidować wymiary przed wykonaniem operacji,
- zgłaszać czytelne błędy dla niepoprawnych działań,
- rozdzielać logikę macierzy od interfejsu użytkownika,
- zapisywać i odczytywać macierze z pliku tekstowego,
- rozdzielać deklaracje i implementacje na pliki `.h` i `.cpp`,
- kompilować się z flagami `-Wall -Wextra -pedantic`.

91.4 Proponowana struktura plików

```
matrix-calculator/
  include/matrix_calculator/
    matrix.h
    matrix_io.h
    matrix_calculator.h
    menu.h
  src/
    matrix.cpp
    matrix_io.cpp
    matrix_calculator.cpp
    menu.cpp
    main.cpp
  tests/
    matrix_tests.cpp
  data/
    matrices.txt
  build/
```

Znaczenie elementów:

- **Matrix** - dane macierzy i podstawowe operacje matematyczne,
- **MatrixIo** - zapis i odczyt macierzy z pliku,
- **MatrixCalculator** - logika wyboru i wykonywania operacji,
- **Menu** - interfejs użytkownika w terminalu.

91.5 Model danych

Macierz powinna mieć co najmniej:

- liczbę wierszy,
- liczbę kolumn,
- wartości elementów.

Przykład:

```
class Matrix {
public:
    Matrix(std::size_t rows, std::size_t columns);

    std::size_t rows() const;
    std::size_t columns() const;

    double at(std::size_t row, std::size_t column) const;
    void set(std::size_t row, std::size_t column, double value);

private:
    std::size_t rows_;
    std::size_t columns_;
    std::vector<double> values_;
};
```

Jednowymiarowy wektor upraszcza przechowywanie danych i pozwala przeliczać indeks według wzoru:

```
index = row * columns + column
```

91.6 Obsługiwane operacje

Minimalny zestaw operacji:

- dodawanie macierzy o tych samych wymiarach,
- odejmowanie macierzy o tych samych wymiarach,
- mnożenie macierzy przez skalar,
- mnożenie macierzy, gdy liczba kolumn pierwszej macierzy jest równa liczbie wierszy drugiej macierzy,
- transpozycja dowolnej macierzy,
- wyznacznik macierzy 2×2 .

Rozszerzona wersja może obsługiwać:

- wyznacznik macierzy 3×3 ,
- wyznacznik metodą eliminacji Gaussa,
- macierz jednostkową,
- macierz odwrotną,
- potęgowanie macierzy kwadratowej.

91.7 Minimalny wariant zaliczeniowy

Minimalna wersja projektu powinna zawierać:

1. Menu konsolowe z opcjami:
 - utwórz macierz,
 - wyświetl macierz,
 - dodaj macierze,
 - odejmij macierze,
 - pomnóż macierz przez liczbę,
 - pomnóż macierze,
 - transponuj macierz,
 - oblicz wyznacznik,
 - zapisz macierze,
 - wczytaj macierze,
 - zakończ.
2. Walidację wymiarów dla każdej operacji.
3. Obsługę liczb zmiennoprzecinkowych.
4. Co najmniej dwie przechowywane macierze, np. A i B.
5. Zapis i odczyt macierzy z pliku tekstowego.
6. Testy ręczne albo automatyczne dla podstawowych operacji.
7. Instrukcję kompilacji i uruchomienia w `README.md` projektu.

91.8 Rozszerzenia dla chętnych

Możliwe rozszerzenia:

- dowolna liczba nazwanych macierzy,
- historia wykonanych operacji,
- import i eksport CSV,
- przeciążenie operatorów $+$, $-$, $*$,
- testy automatyczne klasy `Matrix`,
- generowanie macierzy losowej,
- wyznacznik dla większych macierzy,
- macierz odwrotna,
- rozwiązywanie układów równań liniowych,

- kolorowe formatowanie tabel w terminalu.

91.9 Kryteria oceny

Projekt jest zaliczony, jeśli:

- kompiluje się bez błędów i ostrzeżeń,
- poprawnie tworzy i wyświetla macierze,
- poprawnie dodaje i odejmuje macierze,
- poprawnie mnoży macierz przez skalar,
- poprawnie mnoży dwie macierze,
- poprawnie transponuje macierz,
- sprawdza zgodność wymiarów przed operacjami,
- zgłasza czytelne komunikaty błędów,
- zapisuje i odczytuje macierze z pliku,
- używa klasy `Matrix`,
- zawiera instrukcję uruchomienia,
- zawiera testy albo opis ręcznych scenariuszy sprawdzenia.

91.10 Scenariusze sprawdzenia

1. Utwórz macierz A o wymiarach 2×2 i wyświetl ją.
2. Utwórz macierz B o wymiarach 2×2 , dodaj $A + B$ i sprawdź wynik.
3. Pomnóż macierz A przez liczbę 2.
4. Pomnóż macierz 2×3 przez macierz 3×2 .
5. Spróbuj dodać macierze o różnych wymiarach i sprawdź komunikat błędu.
6. Oblicz wyznacznik macierzy 2×2 .
7. Zapisz macierze do pliku, uruchom program ponownie i wczytaj dane.

Rozdział 92

Projekt 10 - System zarządzania zadaniami

92.1 Cel projektu

Celem projektu jest przygotowanie konsolowego systemu do zarządzania zadaniami. Projekt ma pokazać modelowanie danych, priorytety, statusy, sortowanie, filtrowanie, zapis do pliku oraz podział programu na klasy.

Punktem wyjścia są zadania opisane polami:

- `UID` albo `PID`,
- `Name`,
- `Priority`,
- `StartTime`,
- `Function`.

W projekcie warto doprecyzować te pola i nadać im jasne znaczenie.

92.2 Zakres funkcjonalny

System powinien umożliwiać:

- dodanie nowego zadania,
- edycję zadania,
- usunięcie zadania,
- wyświetlenie listy zadań,
- wyszukanie zadania po identyfikatorze,
- zmianę statusu zadania,
- sortowanie po priorytecie albo czasie startu,
- filtrowanie po statusie,
- zapis zadań do pliku,
- odczyt zadań z pliku przy starcie programu.

Przykładowe statusy:

- `New`,
- `InProgress`,
- `Done`,
- `Cancelled`.

Przykładowe priorytety:

- `Low`,
- `Normal`,
- `High`,

- Critical.

92.3 Wymagania techniczne

Projekt powinien:

- być napisany w C++17,
- używać klas do modelowania zadania i systemu zadań,
- używać `enum class` dla statusu i priorytetu,
- używać `std::vector` do przechowywania zadań,
- używać `std::map` albo `std::unordered_map` do wyszukiwania po ID,
- używać algorytmów STL do sortowania i filtrowania,
- zapisywać dane do pliku tekstowego, np. CSV,
- rozdzielać deklaracje i implementacje na pliki `.h` i `.cpp`,
- kompilować się z flagami `-Wall -Wextra -pedantic`.

Pole `Function` można potraktować jako tekstowy opis akcji, np. `send_email`, `generate_report`, `backup_data`.

92.4 Proponowana struktura plików

```
task-system/
  include/task_system/
    task.h
    task_priority.h
    task_status.h
    task_manager.h
    task_storage.h
  src/
    task.cpp
    task_manager.cpp
    task_storage.cpp
    main.cpp
  tests/
    task_manager_tests.cpp
  data/
    tasks.csv
  build/
```

Znaczenie elementów:

- `Task` - dane pojedynczego zadania,
- `TaskPriority` - priorytet zadania,
- `TaskStatus` - status zadania,
- `TaskManager` - logika dodawania, usuwania, edycji i wyszukiwania,
- `TaskStorage` - zapis i odczyt zadań z pliku.

Kompilowalny szkic klasy zarządzającej znajduje się w pliku `examples/task_manager_skeleton.cpp`. Przykład pokazuje dodawanie zadań, odrzucenie duplikatu ID, zmianę statusu, sortowanie po priorytecie i filtrowanie po statusie.

92.5 Model danych

Zadanie powinno mieć co najmniej:

- `id`,
- `name`,
- `priority`,
- `status`,
- `startTime`,

- functionName.

Przykład:

```
enum class TaskPriority {
    Low,
    Normal,
    High,
    Critical
};

enum class TaskStatus {
    New,
    InProgress,
    Done,
    Cancelled
};

struct Task {
    std::string id;
    std::string name;
    TaskPriority priority;
    TaskStatus status;
    std::string startTime;
    std::string functionName;
};
```

92.6 Minimalny wariant zaliczeniowy

Minimalna wersja projektu powinna zawierać:

1. Menu konsolowe z opcjami:
 - dodaj zadanie,
 - edytuj zadanie,
 - usuń zadanie,
 - wyświetl zadania,
 - wyszukaj zadanie,
 - zmień status,
 - zapisz dane,
 - wczytaj dane,
 - zakończ.
2. Walidację unikalnego ID.
3. Sortowanie zadań po priorytecie.
4. Filtrowanie zadań po statusie.
5. Zapis i odczyt danych z pliku CSV.
6. Instrukcję kompilacji i uruchomienia w README.md projektu.

92.7 Rozszerzenia dla chętnych

Możliwe rozszerzenia:

- historia zmian statusu,
- data zakończenia zadania,
- przypisanie zadania do użytkownika,
- cykliczne zadania,
- eksport raportu do pliku tekstowego,
- testy automatyczne dla `TaskManager`,
- tryb demonstracyjny z przykładowymi zadaniami,
- zapis do SQLite.

92.8 Kryteria oceny

Projekt jest zaliczony, jeśli:

- kompiluje się bez błędów i ostrzeżeń,
- poprawnie dodaje zadania,
- odrzuca duplikaty ID,
- umożliwia edycję i usuwanie zadań,
- wyszukuje zadanie po ID,
- sortuje i filtruje zadania,
- zapisuje i odczytuje dane z pliku,
- używa klas, `enum class` i kontenerów STL,
- ma czytelne menu konsolowe,
- zawiera instrukcję uruchomienia,
- zawiera testy albo opis ręcznych scenariuszy sprawdzenia.

92.9 Scenariusze sprawdzenia

1. Dodaj trzy zadania z różnymi priorytetami i wyświetl listę.
2. Posortuj zadania po priorytecie.
3. Zmień status wybranego zadania na **Done** i przefiltruj zadania zakończone.
4. Spróbuj dodać drugie zadanie z tym samym ID.
5. Zapisz zadania do pliku, uruchom program ponownie i wczytaj dane.

Rozdział 93

Projekt 11 - Parser plików CSV

93.1 Cel projektu

Celem projektu jest przygotowanie aplikacji konsolowej do wczytywania, przetwarzania i zapisywania plików CSV. Projekt ma pokazać pracę z plikami, parsowanie tekstu, modelowanie danych tabelarycznych, filtrowanie, sortowanie oraz obsługę błędów wejścia.

CSV powinien być traktowany jako format tekstowy z rekordami i kolumnami, a nie tylko jako zwykłe dzielenie linii po przecinku. Warto uwzględnić wartości ujęte w cudzysłowy oraz puste pola.

93.2 Zakres funkcjonalny

Program powinien umożliwiać:

- wczytanie pliku CSV z dysku,
- rozpoznanie nagłówka kolumn,
- wyświetlenie danych w terminalu,
- wybór wybranych kolumn,
- filtrowanie rekordów po wartości w kolumnie,
- sortowanie rekordów po wskazanej kolumnie,
- zapis wyniku do nowego pliku CSV,
- wyświetlenie podstawowych informacji o pliku,
- obsługę pustych pól,
- obsługę błędnych ścieżek i niepoprawnych danych.

Przykładowe operacje:

```
load students.csv
show
select name,grade
filter grade >= 4.0
sort name asc
save result.csv
```

93.3 Wymagania techniczne

Projekt powinien:

- być napisany w C++17,
- mieć osobny moduł parsera CSV,
- mieć osobny model danych tabelarycznych,
- używać `std::vector` do przechowywania rekordów,
- używać `std::string` i strumieni plikowych,
- obsługiwać separator `,`,

- obsługiwać wartości w cudzysłowach,
- obsługiwać puste pola,
- używać algorytmów STL do sortowania i filtrowania,
- rozdzielać deklaracje i implementacje na pliki `.h` i `.cpp`,
- kompilować się z flagami `-Wall -Wextra -pedantic`.

93.4 Proponowana struktura plików

```
csv-parser/
  include/csv_parser/
    csv_parser.h
    csv_writer.h
    csv_table.h
    csv_record.h
    query.h
    menu.h
  src/
    csv_parser.cpp
    csv_writer.cpp
    csv_table.cpp
    query.cpp
    menu.cpp
    main.cpp
  tests/
    csv_parser_tests.cpp
  data/
    students.csv
    result.csv
  build/
```

Znaczenie elementów:

- `CsvParser` - zamiana pliku CSV na strukturę danych,
- `CsvWriter` - zapis danych do pliku CSV,
- `CsvTable` - tabela z nagłówkami i rekordami,
- `CsvRecord` - pojedynczy rekord,
- `Query` - wybieranie kolumn, filtrowanie i sortowanie,
- `Menu` - interfejs użytkownika w terminalu.

Kompilowalny szkic parsera znajduje się w pliku `examples/csv_parser_skeleton.cpp`. Przykład pokazuje podział linii CSV, obsługę pól w cudzysłowach, puste pola, model `CsvTable` oraz filtrowanie po wartości kolumny.

93.5 Model danych

Tabela powinna mieć co najmniej:

- listę nazw kolumn,
- listę rekordów,
- informację o separatorze,
- metody wyszukiwania indeksu kolumny po nazwie.

Przykład:

```
struct CsvRecord {
    std::vector<std::string> fields;
};

class CsvTable {
public:
```

```

const std::vector<std::string>& headers() const;
const std::vector<CsvRecord>& records() const;

std::optional<std::size_t> columnIndex(const std::string& name) const;

private:
    std::vector<std::string> headers_;
    std::vector<CsvRecord> records_;
};

```

93.6 Zasady parsowania CSV

Minimalna wersja parsera powinna obsługiwać:

- separator przecinka,
- koniec rekordu jako koniec linii,
- puste pola, np. Jan,,Kowalski,
- pola w cudzysłowach, np. "Kowalski, Jan",
- podwójny cudzysłów wewnątrz pola, np. "tekst ""w cudzysłowie""".

Nie trzeba od razu obsługiwać wszystkich wariantów CSV. Ważne jest jednak, aby jasno opisać obsługiwany format i poprawnie zgłaszać błędy dla danych, których program nie potrafi przetworzyć.

93.7 Minimalny wariant zaliczeniowy

Minimalna wersja projektu powinna zawierać:

1. Menu konsolowe z opcjami:
 - wczytaj plik,
 - pokaż nagłówki,
 - pokaż dane,
 - wybierz kolumny,
 - filtruj rekordy,
 - sortuj rekordy,
 - zapisz wynik,
 - zakończ.
2. Wczytanie pliku z nagłówkiem.
3. Wyświetlenie danych w terminalu.
4. Filtrowanie po równości tekstowej, np. `city == Warsaw`.
5. Sortowanie rosnące po wybranej kolumnie.
6. Zapis przefiltrowanych danych do nowego pliku CSV.
7. Przykładowy plik wejściowy w katalogu `data`.
8. Instrukcję kompilacji i uruchomienia w `README.md` projektu.

93.8 Rozszerzenia dla chętnych

Możliwe rozszerzenia:

- separator wybierany przez użytkownika,
- sortowanie malejące,
- filtrowanie liczbowe, np. `age > 18`,
- filtrowanie tekstowe typu `contains`,
- kilka filtrów połączonych operatorem `AND`,
- wykrywanie typu kolumny,
- paginacja wyników w terminalu,
- eksport tylko wybranych kolumn,
- testy automatyczne parsera,
- obsługa dużych plików bez wczytywania całości do pamięci.

93.9 Kryteria oceny

Projekt jest zaliczony, jeśli:

- kompiluje się bez błędów i ostrzeżeń,
- poprawnie wczytuje przykładowy plik CSV,
- poprawnie rozpoznaje nagłówki,
- poprawnie obsługuje puste pola,
- poprawnie obsługuje pola w cudzysłowach,
- pozwala wybrać kolumny,
- filtruje rekordy po wartości,
- sortuje rekordy po kolumnie,
- zapisuje wynik do nowego pliku,
- zgłasza czytelne błędy dla niepoprawnych danych,
- zawiera instrukcję uruchomienia,
- zawiera testy albo opis ręcznych scenariuszy sprawdzenia.

93.10 Scenariusze sprawdzenia

1. Wczytaj plik `students.csv` i wyświetl nagłówki.
2. Wyświetl pierwsze rekordy w terminalu.
3. Wybierz tylko kolumny `name` i `grade`.
4. Przefiltruj rekordy po warunku `city == Warsaw`.
5. Posortuj wynik po kolumnie `name`.
6. Zapisz wynik do pliku `result.csv`.
7. Wczytaj plik z polem "Kowalski, Jan" i sprawdź, czy przecinek nie dzieli pola na dwie kolumny.
8. Spróbuj wczytać nieistniejący plik i sprawdź komunikat błędu.

Rozdział 94

Projekt 12 - Symulator bankomatu

94.1 Cel projektu

Celem projektu jest przygotowanie aplikacji konsolowej symulującej działanie bankomatu. Projekt ma pokazać modelowanie prostego systemu użytkowego, walidację danych, obsługę menu, klasy, operacje na stanie konta oraz podstawową historię operacji.

Projekt jest dobrym wyborem dla osób, które chcą połączyć programowanie obiektowe, pracę ze stanem programu, proste uwierzytelnianie i scenariusze błędnych danych.

94.2 Zakres funkcjonalny

Program powinien umożliwiać:

- uruchomienie bankomatu w terminalu,
- zalogowanie użytkownika numerem karty albo identyfikatorem konta,
- sprawdzenie kodu PIN,
- wyświetlenie salda,
- wypłatę środków,
- wpłatę środków,
- zmianę PIN-u,
- wyświetlenie historii operacji,
- zakończenie sesji użytkownika,
- obsługę błędnych danych wejściowych.

Przykładowe menu:

1. Sprawdź saldo
2. Wpłac środki
3. Wypłac środki
4. Historia operacji
5. Zmien PIN
6. Wyloguj

94.3 Wymagania techniczne

Projekt powinien:

- być napisany w C++17,
- używać klasy `Account` do reprezentowania konta,
- używać klasy `Atm` albo `AtmSession` do obsługi sesji użytkownika,
- ukrywać saldo i PIN w polach prywatnych,
- walidować kwoty wpłaty i wypłaty,
- nie pozwalać wypłacić więcej niż wynosi saldo,

- ograniczać liczbę nieudanych prób podania PIN-u,
- używać `std::vector` do historii operacji,
- rozdzielać deklaracje i implementacje na pliki `.h` i `.cpp`,
- kompilować się z flagami `-Wall -Wextra -pedantic`.

94.4 Proponowana struktura plików

```
atm/
  include/atm/
    account.h
    atm.h
    transaction.h
    menu.h
  src/
    account.cpp
    atm.cpp
    transaction.cpp
    menu.cpp
    main.cpp
  tests/
    account_tests.cpp
  data/
    accounts.csv
  build/
```

Znaczenie elementów:

- **Account** - dane konta, saldo, PIN i operacje na środkach,
- **Atm** - logika logowania oraz obsługa sesji,
- **Transaction** - pojedyncza operacja w historii,
- **Menu** - komunikacja z użytkownikiem w terminalu.

Kompilowalny punkt startowy dla modelu konta znajduje się w pliku `examples/atm_account.cpp`. Przykład pokazuje klasę **Account**, historię operacji oraz podstawową walidację wpłaty, wypłaty i zmiany PIN-u.

94.5 Model danych

Konto powinno mieć co najmniej:

- identyfikator konta,
- PIN,
- saldo,
- informację, czy konto jest zablokowane,
- historię operacji.

Przykład:

```
enum class TransactionType {
    Deposit,
    Withdrawal,
    PinChange
};

struct Transaction {
    TransactionType type;
    double amount;
    std::string description;
};
```

```

class Account {
public:
    Account(std::string id, std::string pin, double balance);

    bool verifyPin(const std::string& pin) const;
    double balance() const;
    bool deposit(double amount);
    bool withdraw(double amount);
    bool changePin(const std::string& oldPin, const std::string& newPin);

private:
    std::string id_;
    std::string pin_;
    double balance_;
    std::vector<Transaction> history_;
};

```

94.6 Minimalny wariant zaliczeniowy

Minimalna wersja projektu powinna zawierać:

1. Menu startowe z logowaniem.
2. Jedno przykładowe konto zapisane w kodzie albo w pliku.
3. Sprawdzenie PIN-u.
4. Blokadę po trzech błędnych próbach PIN-u.
5. Wyświetlenie salda.
6. Wpłatę dodatknej kwoty.
7. Wypłatę dodatknej kwoty.
8. Odmowę wypłaty przy braku środków.
9. Historię operacji w trakcie działania programu.
10. Zakończenie sesji użytkownika.
11. Instrukcję kompilacji i uruchomienia.

94.7 Rozszerzenia dla chętnych

Możliwe rozszerzenia:

- wiele kont,
- zapis kont do pliku CSV,
- odczyt kont z pliku przy starcie programu,
- przelewy między kontami,
- limit dziennej wypłaty,
- data i godzina operacji,
- eksport historii operacji do pliku,
- testy automatyczne klasy `Account`,
- osobna rola administratora,
- prosta walidacja formatu PIN-u.

94.8 Kryteria oceny

Projekt jest zaliczony, jeśli:

- kompiluje się bez błędów i ostrzeżeń,
- poprawnie loguje użytkownika poprawnym PIN-em,
- odrzuca błędny PIN,
- blokuje konto albo sesję po zbyt wielu błędnych próbach,
- poprawnie pokazuje saldo,
- poprawnie obsługuje wpłatę,

- poprawnie obsługuje wypłatę,
- nie pozwala wypłacić więcej niż wynosi saldo,
- zapisuje operacje w historii,
- ma czytelne menu konsolowe,
- kod jest podzielony na klasy i moduły,
- zawiera instrukcję uruchomienia,
- zawiera testy albo opis ręcznych scenariuszy sprawdzenia.

94.9 Scenariusze sprawdzenia

1. Zaloguj się poprawnym PIN-em i sprawdź saldo.
2. Podaj trzy razy błędny PIN i sprawdź blokadę.
3. Wpłać 100 i sprawdź, czy saldo wzrosło.
4. Wypłać 50 i sprawdź, czy saldo spadło.
5. Spróbuj wypłacić kwotę większą niż saldo.
6. Spróbuj wpłacić albo wypłacić kwotę ujemną.
7. Wykonaj kilka operacji i wyświetl historię.
8. Zmień PIN i sprawdź logowanie nowym PIN-em.

94.10 Źródło

Projekt powstał na podstawie starszego zadania z zadań pomocniczych TNT.